Distinction and Continuation

Admissibility, Memory, and the Structure of Change

Flyxion
Independent Researcher

Working skeleton, October 2026

Contents

Division of Labour Between the Volumes **ix**

Preface **xi**

I Mathematical foundations of distinction **1**

1 Histories, Representations, and Three Obstructions **3**
1.1 Objects, descriptions, and observational questions . . . 3
1.2 A common model . . . 4
1.3 The obstructions in the running examples . . . 6
1.4 Plan of the volume . . . 7
1.5 Conventions and the recurring exercise sequence . . . 7
1.6 Counting the values of a joint description . . . 7
Exercises . . . 8

2 Equivalence Relations and Quotient Constructions **11**
2.1 Equivalence relations and partitions . . . 11
2.2 Factorization . . . 12
2.3 Refinement and the lattice of equivalence relations . . . 13
2.4 Families of questions . . . 14
2.5 A worked quotient: the account . . . 14
2.6 A second worked quotient: two questions about a monitor . . . 15
Exercises . . . 16

3 Orders of Informational Adequacy **17**
3.1 The adequacy preorder . . . 17
3.2 Combining and comparing: joint representations and common coarsenings . . . 18
3.3 Incomparable descriptions . . . 19
3.4 Adequacy relative to a family of questions . . . 20
3.5 Counting the maximal representations . . . 20
Exercises . . . 21

4 Histories and Continuation Languages **23**
4.1 Words, prefixes, and history sets . . . 23
4.2 Transition systems and the histories they generate . . . 24

4.3 Properties of states and properties of histories . . . 25
4.4 Safety and liveness . . . 26
4.5 Admissible extensions and the memory they require . . . 28
4.6 What finite histories can see of a behavioural property . . . 29
4.7 Why the extension clause cannot be dropped . . . 29
Exercises . . . 30

II Sufficient representations 31

5 Sufficiency for a Fixed Policy 33
5.1 Policies and sufficient representations . . . 33
5.2 No compression is free . . . 34
5.3 Dependence on the domain of cases . . . 35
5.4 How many thresholds a bounded representation can serve . . . 36
Exercises . . . 37

6 Sufficiency for Families of Policies 39
6.1 Families and joint sufficiency . . . 39
6.2 The cost of a growing family . . . 40
6.3 Structured families and general bounds . . . 41
6.4 How many policies does a representation support? . . . 42
6.5 The marginal cost of one more policy . . . 42
Exercises . . . 44

7 Incremental Sufficiency and Minimization 45
7.1 Maintaining an answer under updates . . . 45
7.2 The continuation congruence . . . 46
7.3 One-shot and maintained sufficiency . . . 47
7.4 Computing the minimal maintaining memory . . . 48
7.5 Families and the general history set . . . 49
7.6 A rate limit: one-shot against maintained memory . . . 50
Exercises . . . 50

8 Bisimulation, Products, and State Complexity 53
8.1 Behavioural equivalence and bisimulation . . . 53
8.2 Products of summaries . . . 54
8.3 When summaries compose, and when they do not . . . 55
8.3.1 Separability . . . 56
8.3.2 Composition of maintained answers . . . 56
8.4 Shared events and the limits of independence . . . 57
Exercises . . . 58

III Memory, compression, and verification 59

9 Exact, Approximate, and Probabilistic Preservation 61
9.1 Error of the best decision rule . . . 61
9.2 Information-theoretic bounds . . . 62
9.3 Approximate sufficiency is monotone and its errors add . . . 63
9.4 A computed Bayes error: binned balances . . . 64
Exercises . . . 65

10 Verification Relative to Retained Evidence 67
10.1 Verification relative to evidence . . . 67
10.2 Certificates . . . 68
10.3 The gap between a proof and a case . . . 69
10.4 A bounded-horizon computation of the verified region . . . 69
Exercises . . . 70

11 Provenance and Revision Lineage 73
11.1 Standard machinery . . . 73
11.2 Identity relative to audit questions . . . 74
11.3 A last-author audit . . . 74
11.4 Recency lists and a hierarchy of audit records . . . 75
Exercises . . . 76

12 Memory Architecture and Retrieval 79
12.1 Storage adequacy and retrieval adequacy . . . 79
12.2 Two lower bounds . . . 80
12.3 Adding derived cells . . . 80
12.4 Packing derived cells: how many are needed for depth one . . . 81
Exercises . . . 82

IV Admissibility and obligations 85

13 Admissible Transitions 87
13.1 Three conditions on histories . . . 87
13.2 The normative obstruction has four forms . . . 88
13.3 One-step admissibility is not maintainable . . . 89
13.4 Counting joint obstructions and minimal conflicts . . . 89
Exercises . . . 91

14 Obligation Semantics 93
14.1 Obligations and monitors . . . 93
14.2 Several obligations . . . 94
14.3 Data for the shared-clock problem . . . 96
Exercises . . . 98

15 Identity Through Continuation **99**
15.1 Promised operations and identity criteria . . . 99
15.2 Promises refine identity . . . 100
15.3 How an undo operation enlarges identity . . . 101
Exercises . . . 102

16 Disposition and Accountable Termination **105**
16.1 What cannot be done . . . 105
16.2 Assumptions that make exactly-once effects possible . . . 106
16.3 The memory cost of deduplication . . . 107
16.4 How often the non-atomic orders fail . . . 108
Exercises . . . 108

V Degradation and distributed systems **111**

17 Admissible Degradation **113**
17.1 Dependencies and shedding . . . 113
17.2 No scalar measure of surviving capacity . . . 114
17.3 Counting admissible shed sets and the error of scalar tests . . . 115
Exercises . . . 116

18 Compensation and Recovery **119**
18.1 Restoration and replacement . . . 119
18.2 Irreversible steps . . . 120
18.3 Sagas . . . 120
18.4 When the criterion is a congruence for free . . . 121
Exercises . . . 122

19 Distributed Continuation **123**
19.1 Evidence of a process . . . 123
19.2 What communication-free actions can add up to . . . 124
19.3 What fixed quotas cost when demand is skewed . . . 125
Exercises . . . 126

20 Interfaces and Delegated Authority **127**
20.1 Interface automata and compatibility . . . 127
20.2 Assume–guarantee composition . . . 128
20.3 Delegated authority . . . 129
20.4 Withholding a right: cuts in the permission graph . . . 129
Exercises . . . 130

VI Revision and synthesis **133**

21 The Revision Horizon and Revision Sufficiency **135**
21.1 The horizon of a representation . . . 135

21.2 Which cases lose their revised decision . . . 136
21.3 Approximate revision sufficiency . . . 136
21.4 How many values a threshold family requires . . . 137
Exercises . . . 138

22 The Cost of Preserving Alternatives 139
22.1 Selecting observations . . . 139
22.2 The cost of being updatable . . . 140
22.3 The price of a restricted vocabulary . . . 141
Exercises . . . 141

23 Worked Mathematical Systems 143
23.1 An obligation monitor . . . 143
23.2 A provenance-preserving document system . . . 144
23.3 A degrading service . . . 145
23.4 A claim ledger . . . 145
23.5 Counting what the service can lose . . . 146
Exercises . . . 146

24 Open Problems in Distinction and Continuation 149

A Proof Techniques 151
A.1 Induction on histories . . . 151
A.2 Fixed points . . . 151
A.3 Counting and the pigeonhole principle . . . 152
A.4 Adversary arguments . . . 153
A.5 Reductions . . . 153
A.6 Exchange arguments . . . 153
A.7 Constructing counterexamples . . . 154
Exercises . . . 154

B Category-Theoretic Language 155
B.1 Categories, functors, and the categories used here . . . 155
B.2 Universal constructions . . . 156
B.3 Galois connections . . . 156
B.4 Functors and the history construction . . . 157
Exercises . . . 157

C Automata and Temporal Logic 159
C.1 Deterministic automata and minimization . . . 159
C.2 Moore machines and outputs . . . 160
C.3 Temporal logic on finite traces . . . 160
C.4 Safety, bounded response, and monitors . . . 160
Exercises . . . 161

D Information Theory **163**
D.1 Entropy and mutual information . . . 163
D.2 Data processing and sufficiency . . . 163
D.3 Counting and entropy . . . 164
D.4 Fano's inequality and limits on recovery . . . 164
Exercises . . . 165

E Executable Examples **167**
E.1 Continuation classes by partition refinement . . . 167
E.2 The viability kernel by the counter algorithm . . . 169
E.3 A bounded-response monitor . . . 170
Exercises . . . 171

F Recap of Reachability and Viability **173**
F.1 Controlled systems and reachability . . . 173
F.2 Viability . . . 174
F.3 Connection to histories and obligations . . . 175
F.4 Robust viability . . . 175
Exercises . . . 175

G Hints and Solutions to Selected Exercises **177**

Division of Labour Between the Volumes

The governing rule is short. *Distinction and Continuation* defines and proves what a representation must preserve. *The Shape of Possibility* studies how actions transform reachable futures and what that transformation costs. When a topic appears in both volumes, the first volume owns its definitions and its sufficiency or impossibility theorems, and the second volume cites those definitions and adds only the dynamics, the optimization, and the computation of regions and costs. Each volume carries a self-contained appendix recapping the statements it needs from the other, so either can be read first.

Topic	**Distinction and Continuation**	**The Shape of Possibility**
Obligations	Semantics, monitors, and temporal specification of commitments that cannot be read from one state (Ch. 14).	Viability under evolving obligations: which regions allow continued compliance (Ch. 14).
Degradation	Characterization of admissible shedding from obligations and dependency structure (Ch. 17).	Computation of recovery and fallback regions and their cost (Ch. 16).
Revision	Revision sufficiency and the revision horizon (Ch. 21).	The cost of maintaining or acquiring revision sufficiency (Ch. 22).
History and state	Sufficient representations, incremental sufficiency, bisimulation (Ch. 5–8).	State augmentation and path-dependent feasibility (Ch. 7).
Interfaces	Delegated authority, boundary contracts, compatibility conditions (Ch. 20).	Action abstraction and simulation: when an interface preserves admissible reachability (Ch. 18).
Identity and provenance	Identity criteria relative to promised operations; lineage (Ch. 11, 15).	Cited only; no independent development.

Every result in either volume carries one of three statuses, displayed in its heading. An *established result* is attributed to its mathematical source. A *new result* is proved in full under assumptions stated beside it. A *proof obligation* is a proposed statement whose proof has not yet survived scrutiny; it is displayed as such and nothing downstream may cite it as a theorem. The companion ledger `proof-spine.tex` lists every load-bearing result with its status, assumptions, and dependencies.

Preface

This volume asks what a system must distinguish and retain for its subsequent operations to remain well defined and admissible. It proceeds from equivalence relations and quotient constructions, through automata and sufficient representations, to obligations, degradation, and distributed continuation. Established results are attributed to their mathematical sources. Results arising from the frameworks developed here are labelled as new and proved under stated assumptions, and statements whose proofs are not yet secured are labelled as proof obligations and are not cited as theorems. A recurring exercise sequence asks the reader to construct a representation, prove what it preserves, produce a counterexample to a stronger claim, and analyze what happens when the family of policies is revised.

Part I

Mathematical foundations of distinction

Chapter 1

Histories, Representations, and Three Obstructions

Question. What must a decision procedure be able to distinguish, and in how many different ways can a proposed continuation become unavailable?

Established sources. Factorization of a function through a quotient.

New results. Conservative gating removes admissible futures (Result 1.7).

Proof obligations. None.

Depends on. Nothing.

1.1 Objects, descriptions, and observational questions

Every mathematical model of a system begins with a decision about what the model will keep. A document under revision is, in the world, a succession of edits made by particular people at particular times; a model of it retains some of that succession and discards the rest, and everything that is later said about the document is said on the basis of what was retained. The purpose of this chapter is to fix a vocabulary in which that decision can be stated exactly, so that the question of what is lost by discarding can be asked and, in the cases that matter, answered.

Definition 1.1 (Cases, questions, representations)**.** Fix a nonempty set X of *cases*. An *observational question* on X is a function $q : X \to A$ into a set A of answers. A *representation* of X is a function $r : X \to R$ into a set R of descriptions. A representation r *answers* a question q if there is a function $g : r(X) \to A$ with $q = g \circ r$; any such g is a *reading* of q from r.

The definition separates an object from a representation of it. The value $r(x)$ is not the case x, and the equation $r(x) = r(x')$ records only that the representation cannot tell the two cases apart. Whether it matters that it cannot depends entirely on which questions are asked. A representation is therefore neither good nor bad in itself; it is adequate or inadequate for a family of questions, and the relation between a representation and the questions it answers is the subject of the next three chapters. In most of this volume the cases are histories, the finite records of what has happened to a system, and a question about a history is a question about the whole of what has happened and not only about where the system now stands.

Three families of examples recur throughout the book. Each is small enough to analyze completely and each isolates a distinct way in which a representation can fail.

Example 1.2 (Document revisions)**.** A document has m sections, each holding a value from a set C of contents, and every section initially holds a fixed value $c_0 \in C$. The events are the overwrites $\mathrm{set}(j, c)$ with $1 \le j \le m$ and $c \in C$, and the cases are all finite sequences of such events. Let $\tau(h) \in C^m$ be the *current text* after the history h, and let $\nu(h)$ record, for each section, the set of contents it has held at some point. The current text answers the question "what does section j contain now?" and does not answer "has section j ever contained c?". To see the second claim take $m = 1$, $C = \{0, 1\}$, $c_0 = 0$, and compare the empty history with $\mathrm{set}(1, 1)\,\mathrm{set}(1, 0)$: both have current text 0, but only the second has held the content 1. The held-content record ν answers the second question and fails the first in turn, since $\mathrm{set}(1, 1)$ and $\mathrm{set}(1, 1)\,\mathrm{set}(1, 0)$ have the same ν and different current texts. Neither representation can replace the other, and the pair (τ, ν) answers both questions.

Example 1.3 (Transaction histories)**.** The events are deposits $\mathrm{d}(n)$ and withdrawals $\mathrm{w}(n)$ for positive integers n, and the cases are all finite sequences of events. Overdraft is physically permitted. The *balance* $\beta(h)$ is the total deposited minus the total withdrawn in h, and $q_{\mathrm{neg}}(h) \in \{0, 1\}$ is 1 exactly when some prefix of h has a negative balance. The balance answers the question it names and does not answer q_{neg}: the empty history and $\mathrm{w}(1)\,\mathrm{d}(1)$ both have balance 0, but q_{neg} takes the values 0 and 1 on them. The pair $(\beta, q_{\mathrm{neg}})$ answers both questions, and Chapter 2 shows that it carries more than is needed to keep answering q_{neg} after every further event.

Example 1.4 (Monitored process)**.** A monitor observes the events tick, alarm, and reset. Let $c(h)$ be the number of alarms in h since the most recent reset, or since the start if there has been none, and let $\rho(h) = \min(c(h), 3)$. The representation ρ answers the question "are at least three alarms outstanding?" It identifies any two histories with the same capped count, and in particular it discards the number of ticks. That loss is harmless for every question about alarm counts and fatal for any question about elapsed time, such as how long ago the last alarm occurred. It also fails to answer "have three alarms ever been outstanding?", because a reset erases the count that would have recorded it.

1.2 A common model

Let E be a set of events and let $\mathcal{H} \subseteq \mathsf{E}^*$ be a prefix-closed set of finite words, the *histories*. Write $h{\cdot}w$ for the history obtained by appending the word w to h, and $\mathrm{Ext}(h) = \{w : h{\cdot}w \in \mathcal{H}\}$ for the extensions available after h. Each history determines three things.

- Its *physical state* $x(h) \in \mathcal{X}$, which fixes the set $\mathrm{Reach}(x(h)) \subseteq \mathrm{Ext}(h)$ of continuations that can physically be executed.
- Its *retained representation* $\sigma(h) \in \Sigma$, which is everything a decision procedure is permitted to consult.
- Its *obligations* $\mathrm{Obl}(h)$, a set of commitments that restrict which continuations count as acceptable.

A *proposed continuation* after h is a nonempty set $c \subseteq \mathrm{Ext}(h)$ of words, understood as a specification that any acceptable realization must meet. Its realizations are $\mathrm{Real}(h, c) = c \cap \mathrm{Reach}(x(h))$, and its admissible realizations are those realizations w for which every obligation in $\mathrm{Obl}(h)$ is satisfied along $h \cdot w$. We write $\mathrm{Adm}(h, c)$ for that set.

Definition 1.5 (Three obstructions)**.** A proposed continuation c after h is subject to
(i) the *physical obstruction* $\mathsf{O}_{\mathrm{phys}}(h, c)$ when $\mathrm{Real}(h, c) = \varnothing$;
(ii) the *obligation obstruction* $\mathsf{O}_{\mathrm{obl}}(h, c)$ when $\mathrm{Real}(h, c) \neq \varnothing$ and $\mathrm{Adm}(h, c) = \varnothing$;
(iii) the *representational obstruction* $\mathsf{O}_{\mathrm{rep}}(h, c)$ when there is a history h' with $\sigma(h') = \sigma(h)$ and $\mathbf{1}[\mathrm{Adm}(h', c) \neq \varnothing] \neq \mathbf{1}[\mathrm{Adm}(h, c) \neq \varnothing]$.

The first two are properties of the world and of the commitments in force. The third is a property of the pair $(\sigma, \text{decision})$: it records that the retained representation fails to separate histories that require different answers. The obstructions are analytically distinct but they interact, and Example 1.8 below shows one form of the interaction. They are therefore presented as distinct obstructions and not as mutually exclusive cases.

The representational obstruction

Let $P : \mathcal{H} \to \mathcal{D}$ assign to each history the decision it requires. For admissibility of a fixed c, take $\mathcal{D} = \{0, 1\}$ and $P(h) = \mathbf{1}[\mathrm{Adm}(h, c) \neq \varnothing]$.

Established result 1.6 (Factorization through a representation)**.** *There is a map $\delta : \sigma(\mathcal{H}) \to \mathcal{D}$ with $\delta \circ \sigma = P$ if and only if $\sigma(h_1) = \sigma(h_2)$ implies $P(h_1) = P(h_2)$ for all $h_1, h_2 \in \mathcal{H}$. In particular, if $\sigma(h_1) = \sigma(h_2)$ but $P(h_1) \neq P(h_2)$, then no decision procedure that receives only $\sigma(h)$ returns P correctly on both h_1 and h_2.*

Source: Standard factorization of a function through a quotient; developed in the chapter on equivalence relations and quotient constructions in the first volume.

Proof. If $\delta \circ \sigma = P$ and $\sigma(h_1) = \sigma(h_2)$, then $P(h_1) = \delta(\sigma(h_1)) = \delta(\sigma(h_2)) = P(h_2)$. Conversely, if equal representations force equal decisions, define $\delta(s) = P(h)$ for any h with $\sigma(h) = s$; the hypothesis makes this independent of the choice of h. □

This result establishes the impossibility of *uniformly correct adjudication* from σ. It does not by itself establish that any future becomes unavailable, because a procedure that errs on one of the two histories has not thereby foreclosed anything. Turning the impossibility into a claim about availability needs a further rule that says what the system does when admissibility cannot be established. The following proposition states the simplest such rule and what it yields.

New result 1.7 (Conservative gating removes admissible futures)**.** *Fix a proposed continuation c and call a map $g : \Sigma \to \{\textit{permit}, \textit{refuse}\}$ a* sound gate *if $g(\sigma(h)) = \textit{permit}$ implies $\mathrm{Adm}(h, c) \neq \varnothing$ for every $h \in \mathcal{H}$. If $\sigma(h_1) = \sigma(h_2)$, $\mathrm{Adm}(h_1, c) \neq \varnothing$ and $\mathrm{Adm}(h_2, c) = \varnothing$, then every sound gate refuses c after h_1.*

Assumptions: The gate consults $\sigma(h)$ alone, and soundness is required for all histories in $\mathcal{H}$, not only for those that have occurred.

Proof. Suppose a sound gate permitted c after h_1. Then $g(\sigma(h_1)) = \text{permit}$, so $g(\sigma(h_2)) = \text{permit}$ because $\sigma(h_1) = \sigma(h_2)$, and soundness gives $\mathrm{Adm}(h_2, c) \neq \varnothing$, a contradiction. □

The qualification matters. Proposition 1.7 says that a future which is physically executable and obligation-compliant at h_1 is nevertheless unavailable through any sound gate, and it says this *only* under the policy of refusing what cannot be established. A system that permitted such

transitions on a best-effort basis would keep the future available at the price of occasionally acting inadmissibly at h_2. Which trade is acceptable is a design question that the later chapters treat explicitly.

Example 1.8 (Representation concealing an obligation)**.** Let two histories reach the same physical state, so that $\mathrm{Reach}(x(h_1)) = \mathrm{Reach}(x(h_2))$. Suppose h_2 contains an unresolved consent request and h_1 does not, while the retained representation records only the physical state. Then $\sigma(h_1) = \sigma(h_2)$, the continuation c is admissible after h_1 and obstructed by $\mathsf{O}_{\mathrm{obl}}$ after h_2, and Proposition 1.7 shows that a sound gate refuses c after h_1 as well. The obstruction experienced at h_1 is $\mathsf{O}_{\mathrm{rep}}$, even though neither $\mathsf{O}_{\mathrm{phys}}$ nor $\mathsf{O}_{\mathrm{obl}}$ holds there. Here $\mathsf{O}_{\mathrm{rep}}$ is an inherited effect of $\mathsf{O}_{\mathrm{obl}}$ at a different history.

The division of the two volumes

The first volume develops what a representation must preserve for a given family of decisions to be well defined, and proves the corresponding sufficiency and impossibility results. The second volume studies how actions transform the set of reachable futures and what it costs to reach, keep, or restore them. A future can be lost because the physical means to reach it are gone, because obligations prohibit the path, or because the retained representation can no longer distinguish the circumstances in which the path would be admissible. Each of these obstructions recurs throughout both volumes, and the table in the following section records which volume owns which part of the mathematics.

1.3 The obstructions in the running examples

Definition 1.5 can be read off in each running example, and the readings show that the three obstructions are properties of different parts of a system. Consider an account with the transaction events of Example 1.3 together with a further event close and a further event report that files a regulatory report on past overdrafts. Suppose a bank holds a finite cash reserve, so that a withdrawal is physically executable only while the reserve covers it; this fixes $\mathrm{Reach}(x(h))$. Suppose in addition that closing the account is admissible only if no overdraft is unreported, meaning that a report has been filed after the last prefix of h with a negative balance. Write $p(h) = 1$ when some overdraft is unreported and $p(h) = 0$ otherwise; the rule fixes the obligations in $\mathrm{Obl}(h)$.

If the reserve is exhausted, the proposed continuation $\{\mathrm{w}(n)\}$ is subject to $\mathsf{O}_{\mathrm{phys}}$. If $p(h) = 1$, then $\{\mathrm{close}\}$ is physically executable but every realization violates the reporting obligation, and it is subject to $\mathsf{O}_{\mathrm{obl}}$. The third case is the delicate one. Suppose the bank retains only the balance, so that $\sigma = \beta$, and consider the empty history and $\mathrm{w}(1)\,\mathrm{d}(1)$ again. The first has $p = 0$ and closing is admissible; the second has $p = 1$ and closing is inadmissible; the representations coincide. By Result 1.7, every sound gate that consults the balance alone refuses $\{\mathrm{close}\}$ after the empty history, although closing is then both executable and compliant. The future has been lost to a decision about what to retain, and neither the reserve nor the obligation had any part in it.

The same example shows how a revision of the rules can reach back into the representation. The pair (β, p) determines admissibility of closure under the rule above, since p alone does. If the bank adds a rule that closure additionally requires that no deposit above a fixed amount

has occurred since the last report, the pair no longer determines admissibility, because the new rule depends on a distinction the pair never recorded. Whether the retained representation can support a given family of future rules is the revision question of Part VI, and it is already visible at this scale: a representation can be sufficient for every rule currently in force and insufficient for a rule that will be adopted later.

1.4 Plan of the volume

Part I develops the mathematics of distinction. Chapter 2 treats equivalence relations and quotient constructions and proves the factorization theorem on which everything else rests. Chapter 3 orders representations by the questions they answer and shows that, once a bound on size is imposed, no representation is universally best. Chapter 4 turns from sets of cases to histories and separates properties of the state a system occupies from properties of the history that led there. Part II asks which representation is sufficient for a given policy, a family of policies, or the maintenance of an answer under arbitrary future events, and it brings in the theory of minimal automata and bisimulation. Part III adds approximation, verification, provenance, and the architecture of retrieval. Part IV states obligations and admissible transitions precisely and studies identity and disposition. Part V treats degradation, compensation, distribution, and delegated authority. Part VI collects the consequences for revision, the cost of keeping alternatives open, and a set of complete worked systems, and it closes with the questions that remain open.

1.5 Conventions and the recurring exercise sequence

Functions are written on the left, $\ker f = \{(x, x') : f(x) = f(x')\}$ is the kernel of a function f, and $\operatorname{im} f$ is its image. A result is labelled *established* when it is attributed to a source, *new* when it is proved in full under the assumptions displayed beside it, and as a *proof obligation* when it is a proposed statement that has not yet been proved. No later chapter cites a proof obligation as a theorem.

Each chapter ends with exercises, and most chapters contain a sequence of four that is repeated with different subject matter. The first asks the reader to construct a representation for a stated family of questions. The second asks for a proof of exactly what the construction preserves. The third asks for a counterexample to a stronger claim that a reader might be tempted to make about it. The fourth asks what happens to the construction when the family of questions or policies is revised. The sequence is the book's working method in miniature: a representation is a claim about what suffices, and the claim has content only when its boundary is located.

1.6 Counting the values of a joint description

The pair (τ, ν) of Example 1.2 answers both of its questions, and it is natural to ask how many values it needs. The product $C^m \times \mathcal{P}(C)^m$ is far larger than the set of pairs that actually occur, because the current content of a section must belong to the set of contents the section has held. The count is exact and small enough to be checked by hand for $m = 1$.

New result 1.9 (Values of the joint document description)**.** *Let $|C| = k \geq 2$. The pair (τ, ν) takes exactly $\left(2^{k-2}(k+1)\right)^m$ values on the histories of Example 1.2, and the set of its values is the product over sections of the set of pairs (c, H) with $c \in H \subseteq C$ and $c_0 \in H$. In particular, for $k = 2$ and $m = 1$ there are three values, not the four of the product $C \times \mathcal{P}(C)$ restricted to $c_0 \in H$.*

Assumptions: Events are the overwrites $\mathrm{set}(j, c)$; every section starts at the fixed content c_0; $\nu_j(h)$ is the set of contents section j has held, including c_0.

Proof. Each event touches one section, so the pair of values for section j depends only on the subsequence of events addressed to j, and subsequences for different sections can be chosen independently; the set of values of (τ, ν) is therefore the product of the sets of values for single sections. For a single section, every value (c, H) satisfies $c \in H$ and $c_0 \in H$, because the current content is one the section has held and the initial content was held. Conversely, given such a pair, enumerate $H \setminus \{c_0\}$ in any order with c last if $c \neq c_0$, and overwrite in that order; if $c = c_0$ finish with $\mathrm{set}(j, c_0)$ after the other elements. The history has current content c and held set H. It remains to count the pairs. If $c = c_0$ then H ranges over the subsets of C containing c_0, giving 2^{k-1} pairs. If $c \neq c_0$ there are $k-1$ choices of c and 2^{k-2} sets H containing both c_0 and c. The total is $2^{k-1} + (k-1)2^{k-2} = 2^{k-2}(k+1)$. □

For $k = 2$ the three values are $(0, \{0\})$, $(0, \{0, 1\})$ and $(1, \{0, 1\})$: the fourth combination, a current content of 1 with 1 never held, is impossible. The count matters because it separates two ideas that the word "independent" tends to blur. The questions "what is the text now?" and "was c ever held?" are incomparable as representations, yet they are not independent as information, since the answer to the first constrains the answer to the second. The smallest representation answering both has three values and not four, and for $k = 3$ it has 8 values and not the $3 \cdot 4 = 12$ of $C \times \mathcal{P}(C \setminus \{c_0\})$ on one section. The figures 3, 8, 9 (for $k = 2, m = 2$) and 20 (for $k = 4$, $m = 1$) have been confirmed by enumerating all histories up to a length at which no new value appears.

Exercises

Exercise 1.10. Construct a representation r of the cases in Example 1.2 that answers both "what does section j contain now?" and "has section j ever contained c?" for all j and c, and whose set of values is as small as you can make it when $m = 1$ and $C = \{0, 1\}$.

Exercise 1.11. For the representation of the previous exercise, prove that it answers each of the two families of questions. Then prove that the empty history and $\mathrm{set}(1, 1)\,\mathrm{set}(1, 0)$ are separated by r.

Exercise 1.12. Show that the following stronger claim is false: "any representation that answers every question about the current text and every question about whether a content was ever held also answers every question about the order in which the contents were assumed." Give two histories for $m = 1$ and $C = \{0, 1, 2\}$ that exhibit the failure.

Exercise 1.13. The monitor of Example 1.4 is revised so that the rule becomes: five alarms outstanding, or three alarms outstanding with a tick in between each pair. Decide whether ρ still answers the revised question, and if it does not, exhibit a pair of histories that ρ identifies and the revised question separates.

Exercise 1.14. With p as in Section 1.3, show that the pair (β, p) determines whether $\{\text{close}\}$ is admissible after h, and that the pair (β, q_{neg}) does not, by exhibiting two histories with the same value of (β, q_{neg}) and different admissibility of closure. Then identify the step of your first argument that fails under the revised rule of that section.

Exercise 1.15. Extend Result 1.9 to the case in which the initial contents of the m sections differ, $c_0^{(j)}$ depending on j, and show that the count is unchanged. Then explain why adding an event $\text{clear}(j)$ that erases the held set of section j down to the current content changes the set of values of ν, and recompute the count for $m = 1$ and $k = 2$.

Exercise 1.16. Prove that the set of values of (τ, ν) for m sections is the product of the sets for single sections, and deduce that the number of values is multiplicative in m. Then show that the representation recording only the number of sections whose current content differs from c_0 is a coarsening of τ, and count its values.

Exercise 1.17. A representation $\kappa(h)$ records, for $k = 3$ and $m = 1$, the current content and the number of distinct contents held. Decide whether κ answers "has c ever been held?" for each c, and give the number of values of κ. Show that κ has 7 values, compare with the 8 of Result 1.9, and give two histories that κ identifies and that differ on a question about a held content.

Chapter 2

Equivalence Relations and Quotient Constructions

Question. When does an observation factor through a compressed representation?

Established sources. Partitions, quotient maps, refinement, and the universal property of the quotient; the lattice of equivalence relations.

New results. A complete computation of the continuation quotient for the account of Example 1.3.

Proof obligations. None.

Depends on. Chapter 1.

A representation identifies cases, and an identification is exactly an equivalence relation. This chapter develops the elementary theory of equivalence relations with the single purpose of making the following statement precise and complete: a question can be answered from a representation if and only if the representation identifies only cases that the question also identifies. Everything in this chapter is standard. It is presented in full because the later chapters apply it in settings where the cases are histories and the representations are memory, and each application is only as secure as the statement it rests on.

2.1 Equivalence relations and partitions

Definition 2.1 (Equivalence relation, class, partition)**.** An *equivalence relation* on a set X is a relation $\sim \subseteq X \times X$ that is reflexive, symmetric, and transitive. The *class* of x is $[x] = \{y : y \sim x\}$, and the set of classes is the *quotient* $X/\!\sim$. A *partition* of X is a set of nonempty, pairwise disjoint subsets of X whose union is X. The *quotient map* is $\pi_\sim : X \to X/\!\sim$, $x \mapsto [x]$.

Established result 2.2 (Equivalence relations are partitions)**.** *For every equivalence relation $\sim$ on X the set $X/\!\sim$ is a partition of X. For every partition P of X the relation $x \sim_P y$ if and only if x and y lie in the same block is an equivalence relation, and the two constructions are mutually inverse.*

Source: Standard; see any text on elementary set theory or algebra.

Proof. Each class is nonempty because $x \in [x]$ by reflexivity, and the classes cover X for the same reason. If $z \in [x] \cap [y]$ then $x \sim z$ and $z \sim y$, so $x \sim y$ by symmetry and transitivity, and

then $[x] = [y]$ because $w \sim x$ implies $w \sim y$ and conversely. Hence distinct classes are disjoint. In the other direction, "lying in the same block" is reflexive, symmetric, and transitive because equality of blocks is. The classes of $\sim_P$ are the blocks of P, and the relation $\sim_P$ built from the classes of $\sim$ is $\sim$ itself, which shows the constructions are inverse. □

Definition 2.3 (Kernel)**.** The *kernel* of a function $f : X \to Y$ is $\ker f = \{(x, x') \in X \times X : f(x) = f(x')\}$.

Established result 2.4 (Kernels)**.** *The kernel of every function is an equivalence relation, and every equivalence relation $\sim$ is the kernel of its quotient map,* $\ker \pi_\sim = \sim$.

Source: Standard.

Proof. Equality in Y is reflexive, symmetric, and transitive, and these properties transfer to $\ker f$. For the second claim, $\pi_\sim(x) = \pi_\sim(x')$ if and only if $[x] = [x']$ if and only if $x \sim x'$. □

By Result 2.4 nothing is lost in passing between equivalence relations and representations: every representation determines an equivalence relation on the cases, namely the set of identifications it makes, and every equivalence relation is realized by a representation.

2.2 Factorization

The following theorem is the technical center of the volume. It was already used in Result 1.6 for the pair (σ, P) and it is restated here in its general form.

Established result 2.5 (Factorization through a representation)**.** *Let X be nonempty, let $r : X \to R$ be a representation, and let $f : X \to Y$ be a question. The following are equivalent.*

(i) There is a function $g : r(X) \to Y$ with $f = g \circ r$.

(ii) $\ker r \subseteq \ker f$.

When they hold, g is unique.

Source: Standard; this is the universal property of the quotient by a kernel.

Proof. Suppose (i) holds and $(x, x') \in \ker r$. Then $f(x) = g(r(x)) = g(r(x')) = f(x')$, so $(x, x') \in \ker f$. Suppose (ii) holds. Define $g(r(x)) = f(x)$. If $r(x) = r(x')$ then $(x, x') \in \ker r \subseteq \ker f$, so $f(x) = f(x')$, and the definition does not depend on the choice of x with the given value $r(x)$. Hence g is a well-defined function on $r(X)$ with $f = g \circ r$. It is unique because its value at each element of $r(X)$ is forced. □

The theorem says that a question can be answered from a representation exactly when the representation never merges two cases that the question distinguishes. The condition is about the representation and the question and nothing else: it does not depend on how r is computed, stored, or named. An extension of g from $r(X)$ to all of R is always possible when Y is nonempty and plays no role in what follows.

Established result 2.6 (Canonical factorization)**.** *Every function $f : X \to Y$ factors as $f = \bar{f} \circ \pi_{\ker f}$ where $\bar{f} : X/\ker f \to Y$, $[x] \mapsto f(x)$, is injective with image $f(X)$.*

Source: Standard.

Proof. Result 2.5 applied to $r = \pi_{\ker f}$, whose kernel is $\ker f$ by Result 2.4, gives $\bar{f}$. If $\bar{f}([x]) = \bar{f}([x'])$ then $f(x) = f(x')$, so $[x] = [x']$, and injectivity follows. The image is $f(X)$ by construction. □

2.3 Refinement and the lattice of equivalence relations

Definition 2.7 (Refinement)**.** An equivalence relation $\sim$ *refines* $\approx$ if $\sim \subseteq \approx$. We call $\sim$ *finer* and $\approx$ *coarser*. Write $\mathrm{Eq}(X)$ for the set of equivalence relations on X, partially ordered by inclusion. Its least element is the diagonal $\{(x, x)\}$, which identifies nothing, and its greatest element is $X \times X$, which identifies everything.

Established result 2.8 (Lattice of equivalence relations)**.** $\mathrm{Eq}(X)$ *is a complete lattice. The meet of a family* $\{\sim_i\}_{i \in I}$ *is the intersection* $\bigcap_i \sim_i$*, and its join is the transitive closure of the union* $\bigcup_i \sim_i$*.*

Source: Ore, "Theory of equivalence relations," *Duke Mathematical Journal* 9 (1942); see also Birkhoff, *Lattice Theory.*

Proof. Take a nonempty family. Reflexivity, symmetry, and transitivity are each preserved by intersection, so $\bigcap_i \sim_i$ is an equivalence relation, and it is contained in each $\sim_i$ and contains every equivalence relation that is contained in all of them; it is therefore the greatest lower bound. For the join, let $U = \bigcup_i \sim_i$ and let U^+ be its transitive closure. The relation U is reflexive and symmetric, and these properties are inherited by U^+, which is transitive by construction. Any equivalence relation containing every $\sim_i$ contains U and hence U^+, so U^+ is the least upper bound. The empty family has meet $X \times X$ and join the diagonal. □

Example 2.9 (Three cases)**.** Let $X = \{1, 2, 3\}$. Then $\mathrm{Eq}(X)$ has five elements: the diagonal, the three relations each identifying exactly one pair, namely $\{12 \,|\, 3\}$, $\{13 \,|\, 2\}$, and $\{23 \,|\, 1\}$, and $X \times X$. Any two of the three pair relations intersect in the diagonal, and the transitive closure of the union of any two of them is $X \times X$. The lattice is therefore the five-element lattice consisting of a bottom, a top, and three pairwise incomparable elements between them. It is the standard five-element non-distributive lattice, so $\mathrm{Eq}(X)$ is not distributive whenever X has at least three elements.

Established result 2.10 (Coarsening is a function)**.** *If* $\sim$ *refines* $\approx$ *then there is a unique function* $c : X/{\sim} \to X/{\approx}$ *with* $c \circ \pi_\sim = \pi_\approx$*, and it is surjective. If a function* $d : X/{\approx} \to X/{\sim}$ *with* $d \circ \pi_\approx = \pi_\sim$ *exists, then* $\sim = \approx$*.*

Source: Standard.

Proof. Apply Result 2.5 with $r = \pi_\sim$ and $f = \pi_\approx$: the hypothesis $\ker \pi_\sim = \sim \subseteq \approx = \ker \pi_\approx$ gives c, which is surjective because $\pi_\approx$ is. If d exists then the same theorem gives $\approx \subseteq \sim$, so the relations are equal. □

The second statement of Result 2.10 is the mathematical content of the remark that discarded distinctions cannot be recovered. A coarser description is a function of a finer one, but the converse fails unless nothing was discarded.

Established result 2.11 (No recovery by computation)**.** *Let $r : X \to R$ be a representation and let $r' = u \circ r$ for some function u. Then every question answered by r' is answered by r. Conversely, if every question answered by r is answered by r', then* $\ker r = \ker r'$.

Source: Standard consequence of Result 2.5.

Proof. If $q = g \circ r'$ then $q = (g \circ u) \circ r$. For the converse, r answers the question $q = r$, so by hypothesis r' answers r, which by Result 2.5 means $\ker r' \subseteq \ker r$. The relation $\ker r \subseteq \ker r'$ holds because $r' = u \circ r$, and the two inclusions give equality. □

2.4 Families of questions

Questions are usually asked in families, and a representation is asked to serve all members at once.

Established result 2.12 (Joint kernel)**.** *Let $\{f_i : X \to Y_i\}_{i \in I}$ be a family of questions and let $f_I : X \to \prod_i Y_i$ be the function $x \mapsto (f_i(x))_{i \in I}$. Then* $\ker f_I = \bigcap_i \ker f_i$*, and a representation r answers every f_i if and only if* $\ker r \subseteq \bigcap_i \ker f_i$.

Source: Standard.

Proof. Two cases have the same image under f_I exactly when they have the same image under every f_i, which gives the first claim. By Result 2.5, r answers f_i if and only if $\ker r \subseteq \ker f_i$, and $\ker r$ is contained in every $\ker f_i$ if and only if it is contained in their intersection. □

Consequently the coarsest equivalence relation through which all of the f_i factor is $\bigcap_i \ker f_i$, realized by the quotient map onto $X/\bigcap_i \ker f_i$. This is the object that Chapters 5 and 6 study under the names of the minimal sufficient representation for a policy and for a family of policies.

2.5 A worked quotient: the account

The notion of answering a question about a history is not the notion of answering it about every possible future of the history, and the account of Example 1.3 shows the difference with exact numbers. Let $X = \{\mathrm{d}(n), \mathrm{w}(n)\}^*$ and let $q = q_{\mathrm{neg}}$. For a history h and a word w write hw for the extended history. The *continuation question* of q is the function $q^+ : X \to \{0,1\}^X$ with $q^+(h) = (w \mapsto q(hw))$. It asks of a history not only what q says now but what q would say after every further sequence of events.

New result 2.13 (Continuation quotient of the account)**.** *Let $m(h)$ denote the least balance among the prefixes of h, including the empty prefix, so that $m(h) \leq 0$ and $q(h) = 1$ if and only if $m(h) < 0$. Then* $\ker q^+$ *has the following description: $(h, h') \in \ker q^+$ if and only if either $q(h) = q(h') = 1$, or $q(h) = q(h') = 0$ and $\beta(h) = \beta(h')$. In particular $X/\ker q^+$ is in bijection with $\mathbb{N} \cup \{\bot\}$, where $\bot$ is the class of all histories with $q = 1$ and $n \in \mathbb{N}$ is the class of histories with $q = 0$ and balance n.*

Assumptions: Events are the deposits and withdrawals of Example 1.3, with positive integer amounts; overdraft is permitted; the cases are all finite sequences.

Proof. For histories h and words w, the prefixes of hw are the prefixes of h together with the histories hu for u a prefix of w, and $\beta(hu) = \beta(h) + \beta(u)$. Hence $m(hw) = \min\big(m(h),\, \beta(h) + m(w)\big)$, and therefore $q(hw) = 1$ if and only if $m(h) < 0$ or $\beta(h) + m(w) < 0$.

If $q(h) = q(h') = 1$ then $q(hw) = q(h'w) = 1$ for every w, so $q^+(h) = q^+(h')$. If $q(h) = 1$ and $q(h') = 0$, taking w empty separates them. Suppose now $q(h) = q(h') = 0$. Since $\beta(h)$ is the balance of a prefix of h, we have $\beta(h) \geq m(h) \geq 0$, and similarly $\beta(h') \geq 0$. Write $b = \beta(h)$ and $b' = \beta(h')$. For every w we have $q(hw) = 1$ if and only if $b + m(w) < 0$ and $q(h'w) = 1$ if and only if $b' + m(w) < 0$. If $b = b'$ the two functions of w coincide. If $b < b'$ then $b' \geq 1$ and the one-event word $w = \mathrm{w}(b')$ has $m(w) = -b'$, so that $q(hw) = 1$ because $b - b' < 0$, while $q(h'w) = 0$ because $b' - b' = 0$. By symmetry the case $b > b'$ is the same, and so $q^+(h) \neq q^+(h')$ when $b \neq b'$.

Every $n \in \mathbb{N}$ occurs as a balance with $q = 0$, realized by $\mathrm{d}(n)$ for $n \geq 1$ and by the empty history for $n = 0$, and $\perp$ occurs, realized by $\mathrm{w}(1)$. □

The comparison with the question asked once is immediate. The question q has two classes, so a single bit suffices to answer it about the present history. Keeping the answer available after every further event requires the whole of $\mathbb{N} \cup \{\perp\}$, and it requires no more: after an overdraft the balance can be forgotten, because every later answer is already 1. The pair (β, q) over-records in exactly this region, since it distinguishes histories with different balances and $q = 1$ that the future can never separate. Chapter 7 develops this phenomenon in general.

2.6 A second worked quotient: two questions about a monitor

The account has one infinite family of classes and one absorbing class. The monitor of Example 1.4 is finite, and it shows how the continuation quotient of a joint question differs from the product of the quotients of its parts. Take the events tick, alarm and reset, let $c(h)$ be the number of alarms since the last reset, let $q_{\mathrm{now}}(h) = 1$ when $c(h) \geq 3$, and let $q_{\mathrm{ever}}(h) = 1$ when some prefix h' of h has $c(h') \geq 3$. The continuation question of each is defined as in the previous section.

New result 2.14 (Continuation quotients of the monitor)**.** *The relation* $\ker q^+_{\mathrm{now}}$ *has four classes, determined by* $\min(c(h), 3)$*. The relation* $\ker q^+_{\mathrm{ever}}$ *has four classes: the histories with* $q_{\mathrm{ever}} = 1$*, which form one class, and the three classes of histories with* $q_{\mathrm{ever}} = 0$ *and* $c(h) = 0, 1, 2$*. The joint question* $(q_{\mathrm{now}}, q_{\mathrm{ever}})^+$ *has seven classes, and* $\ker(q_{\mathrm{now}}, q_{\mathrm{ever}})^+ = \ker q^+_{\mathrm{now}} \cap \ker q^+_{\mathrm{ever}}$ *is not the product relation, which would have* $4 \cdot 4 = 16$ *classes.*

Assumptions: Events tick, alarm, reset; c counts alarms since the last reset; ticks change nothing relevant.

Proof. Write $\rho = \min(c, 3)$. Ticks leave ρ and q_{ever} unchanged; an alarm sends ρ to $\min(\rho+1, 3)$; a reset sends ρ to 0; and q_{ever} becomes 1 exactly when ρ reaches 3 and stays 1 afterwards. So the pair $(\rho, q_{\mathrm{ever}})$ evolves as a function of itself and the event, and its reachable values are $(0,0), (1,0), (2,0)$ and $(0,1), (1,1), (2,1), (3,1)$; the value $(3,0)$ is impossible because $\rho = 3$ implies $q_{\mathrm{ever}} = 1$. For q_{now}: the future alarm$^{3-\rho}$ makes $q_{\mathrm{now}} = 1$ and no shorter run of alarms does, so histories with different ρ are separated, and equal ρ gives equal futures. For q_{ever}: once $q_{\mathrm{ever}} = 1$ every future answer is 1, so all such histories are identified. If $q_{\mathrm{ever}} = 0$ then $\rho \in \{0, 1, 2\}$, and the future alarm$^{3-\rho}$ makes $q_{\mathrm{ever}} = 1$ while no shorter run does, so different

ρ are separated and equal ρ are identified. For the joint question the seven reachable values of (ρ, q_{ever}) are pairwise separated: two values with different q_{ever} are separated by the empty future, and two with $q_{\text{ever}} = 1$ and different ρ are separated by q_{now} as above. The kernel of the joint question is the intersection by Result 2.12. □

The numbers carry the lesson. Each question alone needs four classes; the two together need seven, not sixteen and not even eight, since the combination $\rho = 3$ with $q_{\text{ever}} = 0$ cannot occur. A designer who sizes the memory as the product of the sizes for the parts overbuilds by a factor of more than two, and one who sizes it as the larger of the two underbuilds, because the two quotients are cut along different lines. The seven values have been confirmed by comparing the answers of all futures up to length four from each reachable state.

Exercises

Exercise 2.15. For the cases of Example 1.2 with $m = 1$ and $C = \{0, 1\}$, compute the join in $\mathrm{Eq}(X)$ of $\ker \tau$ and $\ker \nu$. Conclude that the only questions answered by both τ and ν are the constant ones, and explain in words why the two representations share no information even though neither is trivial.

Exercise 2.16. Prove that the join in Result 2.8 of two equivalence relations $\sim$ and $\approx$ can be described as follows: x and y are related if and only if there is a finite sequence $x = z_0, z_1, \ldots, z_k = y$ such that consecutive elements are related by $\sim$ or by $\approx$. Show by example that the union of two equivalence relations is in general not transitive.

Exercise 2.17. Show that if a representation $r : X \to R$ with $|r(X)| = k$ answers a question q, then $\ker q$ has at most k classes. Show by a small example that the converse fails: a question whose kernel has at most k classes need not be answered by r.

Exercise 2.18. List the seven histories of length at most two over the events $\mathrm{d}(1)$ and $\mathrm{w}(1)$ together with their values of (β, q), and use Result 2.13 to determine which of them lie in the same class of $\ker q^{+}$. Explain why the empty history and $\mathrm{d}(1)\,\mathrm{w}(1)$ are identified although they differ as histories.

Exercise 2.19. Revise the question of Example 1.3 to $q_k(h) = 1$ if and only if some prefix of h has balance below $-k$, where $k \geq 0$ is a fixed integer. Show that $X/\ker q_k^{+}$ is in bijection with $\{\perp\} \cup \{-k, -k+1, \ldots\}$, in the style of Result 2.13. Then show that for $k < k'$ neither of $\ker q_k^{+}$ and $\ker q_{k'}^{+}$ contains the other, so that revising the threshold in either direction can invalidate a representation that was sufficient before. (The histories $\mathrm{w}(k+1)$ and $\mathrm{w}(k+1)\,\mathrm{d}(1)$ and the histories $\mathrm{w}(k+1)\,\mathrm{d}(k+1)$ and the empty history are useful.)

Exercise 2.20. Replace the threshold 3 by t in Result 2.14 and compute the three class counts as functions of t. Check that the joint count is $2t + 1$.

Exercise 2.21. Prove that if $\ker f$ has a classes and $\ker g$ has b classes, then $\ker(f, g)$ has at least $\max(a, b)$ and at most ab classes, and exhibit questions on a set of six cases for which each extreme is attained. Which extreme does Result 2.14 resemble, and why is it strictly between?

Exercise 2.22. For the monitor of Result 2.14 prove that no representation with six values answers both q_{now}^{+} and q_{ever}^{+}, and exhibit a representation with seven values that does.

Chapter 3

Orders of Informational Adequacy

Question. Why can two representations preserve different useful distinctions without either being universally superior?

Established sources. Preorders on representations; the refinement lattice; joint representations and common coarsenings.

New results. Absence of a most informative representation under a size bound (Result 3.9).

Proof obligations. None.

Depends on. Chapter 2.

Chapter 2 showed that a question is answered by a representation exactly when the representation identifies only cases the question identifies. This chapter compares representations with one another. The comparison is made through the questions they answer, and the main outcome is negative: for any set of cases large enough to be interesting, two representations of the same size can each answer a question the other cannot, and no representation of bounded size answers everything that any other representation of that size answers. A claim that a description is the best one is therefore always relative to a stated family of questions or to an unbounded budget.

3.1 The adequacy preorder

Definition 3.1 (Adequacy preorder)**.** For representations $r_1 : X \to R_1$ and $r_2 : X \to R_2$ of the same set of cases, write $r_1 \sqsubseteq r_2$ if r_1 is answered by r_2, that is, if there is a function $f : r_2(X) \to R_1$ with $r_1 = f \circ r_2$. We say that r_1 is a *coarsening* of r_2 and that r_2 is *at least as informative* as r_1.

Established result 3.2 (Kernel description of adequacy)**.** *For representations r_1 and r_2 of X:*

(a) $r_1 \sqsubseteq r_2$ *if and only if* $\ker r_2 \subseteq \ker r_1$*;*

(b) $\sqsubseteq$ *is a preorder, and* $r_1 \sqsubseteq r_2 \sqsubseteq r_1$ *if and only if* $\ker r_1 = \ker r_2$*, which holds if and only if there is a bijection* $b : r_1(X) \to r_2(X)$ *with* $r_2 = b \circ r_1$*;*

(c) $r_1 \sqsubseteq r_2$ *if and only if every question answered by* r_1 *is answered by* r_2*.*

Source: Elementary consequences of Result 2.5.

Proof. (a) is Result 2.5 with $r = r_2$ and $f = r_1$. For (b), reflexivity and transitivity follow from the inclusions in (a). If the kernels are equal, define $b(r_1(x)) = r_2(x)$; this is well defined and injective because the kernels agree, and it is onto $r_2(X)$. Conversely a bijection b with $r_2 = b \circ r_1$ gives equal kernels. For (c), suppose $r_1 \sqsubseteq r_2$ and $q = g \circ r_1$; then $q = (g \circ f) \circ r_2$ and r_2 answers q. Conversely, r_1 answers the question $q = r_1$, so if r_2 answers every question answered by r_1 it answers r_1, which is the statement $r_1 \sqsubseteq r_2$. □

By (b), representations that are equivalent in this preorder differ only by a relabelling of their values, and the preorder on representations modulo this equivalence is the set $\mathrm{Eq}(X)$ ordered by reverse inclusion: the most informative representations are the injective ones, with the diagonal as kernel, and the least informative are the constant ones. By (c), the preorder is not an arbitrary choice of ranking. It is exactly the relation of one representation being able to serve every purpose that the other can serve.

3.2 Combining and comparing: joint representations and common coarsenings

Established result 3.3 (Joint representation)**.** *For representations r_1 and r_2 of X, the pairing $\langle r_1, r_2\rangle : x \mapsto (r_1(x), r_2(x))$ is a least upper bound of r_1 and r_2 for $\sqsubseteq$: it is at least as informative as each, and any representation at least as informative as both is at least as informative as it.*

Source: Elementary; this is the join of Result 2.8 read in the reverse order.

Proof. We have $\ker\langle r_1, r_2\rangle = \ker r_1 \cap \ker r_2$. A representation r is an upper bound of r_1 and r_2 if and only if $\ker r \subseteq \ker r_1$ and $\ker r \subseteq \ker r_2$, that is, if and only if $\ker r \subseteq \ker\langle r_1, r_2\rangle$, which says $\langle r_1, r_2\rangle \sqsubseteq r$. □

Established result 3.4 (Common coarsening)**.** *For representations r_1 and r_2 of X, the quotient map $\pi : X \to X/(\ker r_1 \vee \ker r_2)$ onto the join of Result 2.8 is a greatest lower bound of r_1 and r_2 for $\sqsubseteq$: it is a coarsening of both, and every common coarsening of r_1 and r_2 is a coarsening of it.*

Source: Elementary; Result 2.8.

Proof. A representation r is a lower bound of r_1 and r_2 if and only if $\ker r_1 \subseteq \ker r$ and $\ker r_2 \subseteq \ker r$, that is, if and only if $\ker r_1 \vee \ker r_2 \subseteq \ker r$, which says $r \sqsubseteq \pi$. □

The representation π records exactly the information on which r_1 and r_2 agree: it identifies two cases when one can be carried to the other by a chain of steps, each of which is invisible to r_1 or invisible to r_2.

Example 3.5 (A common coarsening)**.** Let $X = \{1, 2, 3, 4\}$ and let r_1 identify 1 with 2 and leave 3 and 4 separate, while r_2 identifies 2 with 3 and leaves 1 and 4 separate. The join of the kernels identifies $1, 2, 3$ and leaves 4 alone, so the only question answered by both is whether a case is 4. The pair $\langle r_1, r_2\rangle$ is injective on X, since $1, 2, 3, 4$ receive four distinct pairs of values, and so it answers every question.

3.3 Incomparable descriptions

Established result 3.6 (Adequacy and answerable questions)**.** *Two representations r_1 and r_2 are incomparable for $\sqsubseteq$ if and only if there is a question answered by r_1 and not by r_2, and a question answered by r_2 and not by r_1.*

Source: Elementary consequence of Result 3.2(c).

Proof. If they are incomparable then $r_1 \not\sqsubseteq r_2$ and $r_2 \not\sqsubseteq r_1$. By Result 3.2(c) the first failure supplies a question answered by r_1 and not by r_2, and the second supplies one in the other direction. Conversely, a question answered by r_1 and not by r_2 rules out $r_1 \sqsubseteq r_2$, and the other question rules out $r_2 \sqsubseteq r_1$. □

Example 3.7 (Document: current text against held contents)**.** Let $m = 1$ and $C = \{0, 1\}$ in Example 1.2. The representations τ and ν are incomparable. The empty history and $\mathrm{set}(1,1)\,\mathrm{set}(1,0)$ have the same current text and different held sets, so τ does not answer ν, and the histories $\mathrm{set}(1,1)$ and $\mathrm{set}(1,1)\,\mathrm{set}(1,0)$ have the same held set and different current texts, so ν does not answer τ.

Example 3.8 (Account: balance against overdraft flag)**.** In Example 1.3 the representations β and q_{neg} are incomparable. The empty history and $\mathrm{w}(1)\,\mathrm{d}(1)$ have equal balances and different overdraft flags, and the empty history and $\mathrm{d}(1)$ have equal overdraft flags and different balances. Neither representation is a coarsening of the other, and the pair $\langle \beta, q_{\mathrm{neg}} \rangle$ is strictly above both.

Incomparability of this kind is not a pathology of particular examples. For any set of cases with at least three elements, $\mathrm{Eq}(X)$ is not a chain: Example 2.9 exhibits three pairwise incomparable relations. What matters for design is the next result, which shows that incomparability is forced as soon as the size of the representation is bounded.

New result 3.9 (No best representation under a size bound)**.** *Let X be a finite set with n elements and let $2 \le k < n$. Let $\mathcal{R}_k$ be the set of representations r of X with $|r(X)| \le k$, preordered by $\sqsubseteq$. Then $\mathcal{R}_k$ has no greatest element, and it has at least two maximal elements, up to equivalence, which are incomparable.*

Assumptions: X is finite; the size of a representation is the number of values it takes; $2 \le k < n$.

Proof. A greatest element would be a representation $g \in \mathcal{R}_k$ with $r \sqsubseteq g$ for all $r \in \mathcal{R}_k$, that is, with $\ker g \subseteq \ker r$ for every such r. Let $x \neq y$ be elements of X. The representation that maps x to one value and every other element to a second value lies in $\mathcal{R}_k$ because $k \ge 2$, and its kernel does not contain (x, y). Hence $\ker g$ does not contain (x, y) either. Since x and y were arbitrary distinct elements, $\ker g$ is the diagonal, so g is injective and $|g(X)| = n > k$, contradicting $g \in \mathcal{R}_k$. For the second claim, the quotient of $\mathcal{R}_k$ by equivalence is a finite nonempty partially ordered set, so every element lies below a maximal element. If there were only one maximal element, it would lie above every element of the finite poset and would be greatest, which has just been excluded. Two distinct maximal elements are incomparable, because a comparison would make the smaller one fail to be maximal. □

The result says that choosing a memory of bounded size is a choice among incomparable alternatives, each of which preserves some distinctions at the expense of others. Which one is correct is determined by the questions that will actually be asked, and by nothing internal to

the representations themselves. It will be sharpened to a statement about the sizes required for specified families of questions in Chapters 5 and 6.

3.4 Adequacy relative to a family of questions

When the family of questions is fixed, the incomparability disappears.

Established result 3.10 (Least adequate representation for a family)**.** *Let $\mathcal{F} = \{f_i\}_{i \in I}$ be a family of questions on X and let $\kappa = \bigcap_i \ker f_i$. A representation r answers every member of $\mathcal{F}$ if and only if $\ker r \subseteq \kappa$. Consequently the quotient map π_κ is, up to equivalence, the least informative representation answering all of $\mathcal{F}$, and every representation answering all of $\mathcal{F}$ takes at least as many values as π_κ, in the sense that $|r(X)| \geq |X/\kappa|$ for cardinal numbers.*

Source: Elementary; Results 2.12 and 2.10.

Proof. The characterization is Result 2.12. By Result 3.2(a), $\pi_\kappa \sqsubseteq r$ for every such r and π_κ answers $\mathcal{F}$, so it is a least element among answering representations. Since $\ker r \subseteq \kappa$, Result 2.10 gives a surjection from $X/\ker r$ onto X/κ, and $X/\ker r$ is in bijection with $r(X)$ by Result 2.6. □

Remark 3.11. Results 3.9 and 3.10 locate the problem exactly. For a fixed family there is a unique cheapest adequate representation up to relabelling, and the work of choosing it is a calculation of κ. For a fixed bound on size and an unspecified family there is no best choice. The interesting situations lie between these two: the family is not fixed but constrained, for instance by a stated set of policies that may later be revised, and the cost of an adequate representation changes as the family changes. That is the subject of Chapter 6 and, in its dynamic form, of Chapters 7 and 21.

3.5 Counting the maximal representations

Result 3.9 says that bounded representations have no greatest element and at least two maximal ones. The structure can be described exactly. A representation with at most k values corresponds, up to relabelling, to a partition of X into at most k blocks, and the adequacy order is refinement of partitions.

New result 3.12 (Maximal bounded representations)**.** *Let $|X| = n$ and $2 \leq k < n$. The maximal elements of $\mathcal{R}_k$, up to equivalence, are exactly the partitions of X into precisely k blocks, and there are $S(n, k)$ of them, where S denotes the Stirling number of the second kind. Any two are incomparable. Every question with at most k answers is answered by some maximal element of $\mathcal{R}_k$.*

Assumptions: X finite; representations of size at most k; the adequacy order of Definition 3.1.

Proof. A partition P with fewer than k blocks has a block with at least two elements, because $k < n$ forces P to have fewer blocks than elements. Splitting that block into two nonempty parts gives a strictly finer partition with at most k blocks, which is strictly above P in the adequacy order, so P is not maximal. A partition P with exactly k blocks is maximal: a strictly finer partition has strictly more blocks, hence more than k, and does not lie in $\mathcal{R}_k$.

Two different partitions with k blocks are incomparable, since a refinement with the same number of blocks is equal. For the last claim, a question f with at most k answers has a partition $\ker f$ with at most k blocks; by the first paragraph it is refined by a partition with exactly k blocks, obtained by splitting blocks one at a time, and f is answered by the finer partition by Result 3.2. □

The count is large even for modest sets. For $n = 4$ and $k = 2$ there are $S(4, 2) = 7$ maximal two-valued descriptions; for $n = 5$ and $k = 2$, fifteen; for $n = 6$ and $k = 3$ there are 90 among the 203 partitions of a six-element set. Each is a distinct way of spending a budget of three values, and none dominates another. The last assertion of the result is the useful one for design: any single question with a small answer set can be served by a maximal description, but a maximal description serving that question also answers, for free, some further questions, and which further questions it answers is an accident of how the surplus blocks were cut. Choosing among the 90 is therefore a choice about which further questions to favour, and the representation itself offers no guidance.

Exercises

Exercise 3.13. Let $X = \{1, 2, 3, 4, 5, 6\}$. Give two representations with three values each that are incomparable for $\sqsubseteq$, and a third with three values that is incomparable to both. Compute their pairwise joins and common coarsenings.

Exercise 3.14. Prove that $\sqsubseteq$ is a preorder but not a partial order on representations, and that its quotient by equivalence is order isomorphic to $\mathrm{Eq}(X)$ with the reverse of inclusion. Describe the most informative and least informative classes.

Exercise 3.15. Show that Result 3.9 fails when $k = 1$ and when $k \geq n$, and identify in each case what the greatest element is.

Exercise 3.16. For the account of Example 1.3, determine for each of β and q_{neg} whether the representation $\rho(h) = \max(\beta(h), 0)$ is comparable with it, and give a separating pair of histories for each direction of comparison that fails.

Exercise 3.17. A family $\mathcal{F}$ is revised to $\mathcal{F}' = \mathcal{F} \cup \{g\}$ for a new question g. Show that the least adequate representation for $\mathcal{F}'$ is at least as informative as that for $\mathcal{F}$, and that the two are equivalent if and only if $\kappa \subseteq \ker g$, where κ is the joint kernel of $\mathcal{F}$. Describe what this says about when a revision can be absorbed without changing the memory.

Exercise 3.18. For $n = 5$ and $k = 2$ list the fifteen maximal elements of $\mathcal{R}_2$ by the size of their smaller block, and count how many of them answer a given question that separates one named case from the other four.

Exercise 3.19. Prove the formula $S(n, k) = kS(n - 1, k) + S(n - 1, k - 1)$ by classifying partitions into k blocks according to where the last element lies, and use it to recompute $S(6, 3) = 90$.

Exercise 3.20. Give a family of two questions on a set of six cases, each with three answers, such that no single maximal element of $\mathcal{R}_3$ answers both, and a family of two questions such that exactly one does. State what this says about the sufficiency of a bounded memory for a revised family of questions.

Chapter 4

Histories and Continuation Languages

Question. Which properties belong to a current state, and which belong to the history that produced it?

Established sources. Finite words and the prefix order; deterministic transition systems; the Myhill–Nerode construction; safety and liveness (Alpern and Schneider).

New results. The set of admissible extensions of the account determines its balance on viable histories (Result 4.21).

Proof obligations. None.

Depends on. Chapters 1 and 2.

Up to this point the cases have been arbitrary. From now on they are histories: finite sequences of events, ordered by the relation of one being the beginning of another. This chapter sets out the elementary structure of such sets of histories, introduces the transition systems that generate them, and draws the distinction on which the rest of the volume depends. A property that can be read from the state a system occupies is a property of a particular representation of its history, and whether a given property can be read from a given state is exactly the factorization question of Chapter 2. The chapter closes with the classical separation of properties into those whose violation is detected on a finite history and those that remain open at every finite history, which is the form in which obligations will appear later.

4.1 Words, prefixes, and history sets

Definition 4.1 (Words and the prefix order)**.** Let E be a nonempty set of events. E^* denotes the set of finite words over E with concatenation, empty word ε, and length $|w|$. Write $u \preceq v$ if $v = uw$ for some word w.

Established result 4.2 (Structure of the prefix order)**.** *The relation $\preceq$ is a partial order on E^*. The prefixes of a word v form a chain with exactly $|v|+1$ elements, one of each length from 0 to $|v|$, and every nonempty word has a unique immediate predecessor, obtained by deleting its last event.*

Source: Standard.

Proof. Reflexivity holds with $w = \varepsilon$ and transitivity by associativity of concatenation. If $u \preceq v$ and $v \preceq u$ then $|u| = |v|$ and $v = uw$ forces $w = \varepsilon$, so $u = v$. A prefix of v of length ℓ is determined by ℓ and consists of the first ℓ events of v, which gives the chain. The last statement is the case $\ell = |v| - 1$. □

Definition 4.3 (History set and extensions)**.** A *history set* is a nonempty set $\mathcal{H} \subseteq \mathsf{E}^*$ that is closed under prefixes. For $h \in \mathcal{H}$, the set of *extensions* of h is $\mathrm{Ext}(h) = \{w \in \mathsf{E}^* : hw \in \mathcal{H}\}$.

Established result 4.4 (Extensions)**.** *For $h \in \mathcal{H}$ the set $\mathrm{Ext}(h)$ is itself a history set. If $u \in \mathrm{Ext}(h)$, then $\mathrm{Ext}(hu) = \{w : uw \in \mathrm{Ext}(h)\}$.*

Source: Standard.

Proof. We have $\varepsilon \in \mathrm{Ext}(h)$ since $h \in \mathcal{H}$. If $w_1 w_2 \in \mathrm{Ext}(h)$ then $hw_1 w_2 \in \mathcal{H}$, so $hw_1 \in \mathcal{H}$ by closure under prefixes, and $w_1 \in \mathrm{Ext}(h)$. For the second statement, $w \in \mathrm{Ext}(hu)$ means $huw \in \mathcal{H}$, which means $uw \in \mathrm{Ext}(h)$. □

A history set is the same thing as a rooted tree whose edges are labelled by events, with no two edges leaving a node carrying the same label: nodes are histories, the root is ε, and h is joined to ha whenever both lie in $\mathcal{H}$. The set $\mathrm{Ext}(h)$ is the subtree rooted at h. The constraints of a system, whether physical or normative, show up in this picture as missing branches.

4.2 Transition systems and the histories they generate

Definition 4.5 (Deterministic transition system)**.** A *deterministic labelled transition system* is a tuple $T = (S, s_0, \mathsf{E}, \delta)$ with a set S of states, an initial state $s_0 \in S$, and a partial function $\delta : S \times \mathsf{E} \rightharpoonup S$. Extend δ to words by $\delta^*(s, \varepsilon) = s$ and $\delta^*(s, wa) = \delta(\delta^*(s, w), a)$ whenever both sides are defined, and leave δ^* undefined otherwise. The set of histories generated by T is $\mathcal{H}(T) = \{w : \delta^*(s_0, w) \text{ is defined}\}$, and the *state map* is $s_T : \mathcal{H}(T) \to S$, $h \mapsto \delta^*(s_0, h)$.

Established result 4.6 (Histories and transition systems)**.** *(a) $\mathcal{H}(T)$ is a history set for every T.*

(b) Every history set $\mathcal{H}$ is generated by the tree system *$T_{\mathcal{H}} = (\mathcal{H}, \varepsilon, \mathsf{E}, \delta)$ with $\delta(h, a) = ha$ when $ha \in \mathcal{H}$.*

(c) $\mathcal{H}$ is a regular language if and only if $\mathcal{H} = \mathcal{H}(T)$ for some T with finitely many states.

Source: Standard automata theory; see Hopcroft and Ullman, *Introduction to Automata Theory, Languages, and Computation*.

Proof. (a) If $\delta^*(s_0, wa)$ is defined then $\delta^*(s_0, w)$ is defined, so $\mathcal{H}(T)$ is prefix-closed, and it contains ε. (b) By induction on length, $\delta^*(\varepsilon, w) = w$ exactly when $w \in \mathcal{H}$. (c) A finite T becomes a complete deterministic automaton on $S \cup \{\bot\}$ by sending every undefined transition to a new sink $\bot$ and declaring every state of S accepting; this accepts $\mathcal{H}(T)$. Conversely let $\mathcal{H}$ be regular and let M be its minimal complete deterministic automaton. A state of M reached by a word of $\mathcal{H}$ is accepting. A state reached by a word outside $\mathcal{H}$ has empty residual language, because $\mathcal{H}$ is prefix-closed, so all such states coincide with a single sink. Deleting the sink leaves a finite partial system in which every state is reached by a word of $\mathcal{H}$ and which generates $\mathcal{H}$. □

The tree system is the largest and least informative way of presenting a history set, in that its states are the histories themselves. Other systems that generate the same history set carry less, and the whole question of this volume is which of them carries enough.

4.3 Properties of states and properties of histories

Definition 4.7 (Property of histories, state property)**.** A *property* of a history set $\mathcal{H}$ is a function $P : \mathcal{H} \to D$ into a set of verdicts. For a transition system T with $\mathcal{H}(T) = \mathcal{H}$, the property P is a *state property of* T if there is a function $p : s_T(\mathcal{H}) \to D$ with $P = p \circ s_T$.

Established result 4.8 (State properties are factorizations)**.** *Let T generate $\mathcal{H}$ and let P be a property of $\mathcal{H}$. Then P is a state property of T if and only if* $\ker s_T \subseteq \ker P$.

Source: Result 2.5 with $r = s_T$ and $f = P$.

Whether a property is a state property is therefore not a feature of the property alone. It is a relation between the property and a way of presenting the history set.

Example 4.9 (Two presentations of the account)**.** Let $\mathcal{H} = \mathsf{E}^*$ for the events of Example 1.3. Let T_1 have state set $\mathbb{Z}$, initial state 0, and transitions $\delta(n, \mathrm{d}(k)) = n+k$ and $\delta(n, \mathrm{w}(k)) = n-k$. Let T_2 have state set $\mathbb{Z} \times \{0, 1\}$, initial state $(0, 0)$, and transitions $\delta((n, f), \mathrm{d}(k)) = (n + k, f)$ and $\delta((n, f), \mathrm{w}(k)) = (n - k, \max(f, [n - k < 0]))$, where $[\cdot]$ is 1 when the condition holds and 0 otherwise. Then $s_{T_1} = \beta$ and $s_{T_2} = (\beta, q_{\mathrm{neg}})$. For T_1 this is immediate. For T_2 the claim follows by induction on length: it holds for ε since $q_{\mathrm{neg}}(\varepsilon) = 0$; a deposit changes the balance of the history to $\beta(h)+k$, and if that is negative then $\beta(h) < 0$, so $q_{\mathrm{neg}}(h) = 1$ already and the flag is unchanged; a withdrawal changes the balance to $\beta(h)-k$, and the new prefix contributes $[\beta(h) - k < 0]$ to the flag. By Result 4.8, the balance is a state property of both systems and q_{neg} is a state property of T_2 only, since the balance does not answer q_{neg}.

Example 4.10 (Document)**.** For the document of Example 1.2, the system whose state is the current text $\tau(h) \in C^m$ makes "what does section j contain now?" a state property and does not make "has section j ever contained c?" one. A system whose state is the entire history, the tree system, makes every property of histories a state property and is therefore the extreme case.

The word *state* in the common usage of a system is therefore relative to a presentation that someone chose, and a state description is a representation of histories in the sense of Chapter 1. The question of which presentation is sufficient is the sufficiency question that Part II treats in full.

Continuation equivalence

A state map has a further property that an arbitrary representation lacks: it is closed under the passage of time. Two histories that reach the same state remain at the same state after every common sequence of further events. The following construction isolates what this requires of a property.

Definition 4.11 (Residual and continuation equivalence)**.** For a property P of $\mathcal{H}$ and a history h, the *residual* is $P_h : \mathrm{Ext}(h) \to D$, $P_h(w) = P(hw)$. Histories h and h' are *continuation equivalent for* P, written $h \approx_P h'$, if $\mathrm{Ext}(h) = \mathrm{Ext}(h')$ and $P_h = P_{h'}$.

Established result 4.12 (The Myhill–Nerode construction)**.** *Let P be a property of $\mathcal{H}$.*

(a) $\approx_P$ is an equivalence relation contained in $\ker P$*.*

(b) If $h \approx_P h'$ and $u \in \mathrm{Ext}(h)$, then $hu \approx_P h'u$.

(c) If $\sim$ is an equivalence relation on $\mathcal{H}$ contained in $\ker P$ *such that $h \sim h'$ implies* $\mathrm{Ext}(h) = \mathrm{Ext}(h')$ *and $hu \sim h'u$ for all $u \in \mathrm{Ext}(h)$, then $\sim \subseteq \approx_P$.*

Source: Myhill (1957) and Nerode (1958), in the formulation for properties of histories.

Proof. (a) Reflexivity, symmetry, and transitivity hold because equality of residuals does. Taking $w = \varepsilon$ in $P_h = P_{h'}$ gives $P(h) = P(h')$. (b) By Result 4.4, $\mathrm{Ext}(hu) = \{w : uw \in \mathrm{Ext}(h)\} = \mathrm{Ext}(h'u)$, and for such w we have $P_{hu}(w) = P(huw) = P_h(uw) = P_{h'}(uw) = P_{h'u}(w)$. (c) Let $h \sim h'$ and $w \in \mathrm{Ext}(h)$. Then $hw \sim h'w$, so $P(hw) = P(h'w)$ because $\sim \subseteq \ker P$. Together with $\mathrm{Ext}(h) = \mathrm{Ext}(h')$ this gives $P_h = P_{h'}$. □

Established result 4.13 (Sufficiency of state, and minimality of the Nerode system)**.** *Let P be a property of $\mathcal{H}$.*

(a) The system $T_P = (\mathcal{H}/\!\approx_P, [\varepsilon], \mathsf{E}, \delta)$ with $\delta([h], a) = [ha]$ for $ha \in \mathcal{H}$ is well defined, generates $\mathcal{H}$, and has P as a state property.

(b) If T is any transition system generating $\mathcal{H}$ for which P is a state property, then $\ker s_T \subseteq \approx_P$*. In particular T has at least $|\mathcal{H}/\!\approx_P|$ reachable states.*

Source: Myhill and Nerode; see Hopcroft and Ullman.

Proof. (a) The transition is well defined by Result 4.12(b) with $u = a$, and Ext is constant on classes by definition of $\approx_P$. By induction $s_{T_P}(h) = [h]$, so T_P generates $\mathcal{H}$, and P is a state property because $\approx_P \subseteq \ker P$. (b) Suppose $s_T(h) = s_T(h') = s$. Then $\mathrm{Ext}(h) = \{w : \delta^*(s, w) \text{ is defined}\} = \mathrm{Ext}(h')$, and for $w \in \mathrm{Ext}(h)$ we have $s_T(hw) = \delta^*(s, w) = s_T(h'w)$, so $P(hw) = P(h'w)$ because P is a state property of T. Hence $h \approx_P h'$. The count follows from Result 2.10. □

The residual construction expresses the difference between answering a question about the present history and answering it after every further event. A representation r answers P now if $\ker r \subseteq \ker P$. A representation that is the state map of a transition system and answers P must satisfy the much stronger condition $\ker r \subseteq \approx_P$. For the account with $P = q_{\mathrm{neg}}$, the relation $\approx_P$ is the kernel $\ker q^+$ of Result 2.13, the Nerode system has the states $\mathbb{N} \cup \{\bot\}$ with $[b] \xrightarrow{\mathrm{d}(k)} [b + k]$, $[b] \xrightarrow{\mathrm{w}(k)} [b - k]$ for $k \le b$, $[b] \xrightarrow{\mathrm{w}(k)} \bot$ for $k > b$, and $\bot$ absorbing. Chapter 2 showed that the single flag q_{neg} answers the question about the present history while the Nerode system needs the balance for as long as no overdraft has occurred. Chapter 7 takes up this contrast systematically.

4.4 Safety and liveness

Properties of histories arise in practice from properties of unbounded behaviour. A rule such as "the balance never goes negative" or "every request is eventually answered" speaks about all prefixes, or about the whole future, of a run. This section states, with proofs, the classical vocabulary for sorting such rules by what a finite history can reveal about them.

Let E^ω be the set of infinite words over E, and for the remainder of the section take the history set to be all of E^*. A *behavioural property* is a set $\Pi \subseteq \mathsf{E}^\omega$. For a finite word u write $\mathrm{sat}_\Pi(u)$ if some $\sigma \in \mathsf{E}^\omega$ has $u\sigma \in \Pi$, so that u is still compatible with satisfying Π.

Definition 4.14 (Safety and liveness)**.** A property Π is a *safety property* if for every $\sigma \notin \Pi$ some finite prefix u of σ has $\neg\,\mathrm{sat}_\Pi(u)$. It is a *liveness property* if $\mathrm{sat}_\Pi(u)$ holds for every finite word u.

A safety property is one whose violation is detected on a finite history that no continuation can repair; a liveness property is one for which no finite history is ever decisive.

Established result 4.15 (Closure characterization of safety)**.** *Let*

$$\Pi_{\mathrm{cl}} = \{\sigma \in \mathsf{E}^\omega : \mathrm{sat}_\Pi(u) \text{ for every finite prefix } u \text{ of } \sigma\}.$$

Then $\Pi \subseteq \Pi_{\mathrm{cl}}$ *for every* Π*, and* Π *is a safety property if and only if* $\Pi = \Pi_{\mathrm{cl}}$*.*

Source: Alpern and Schneider, "Defining liveness," *Information Processing Letters* 21 (1985).

Proof. If $\sigma \in \Pi$ then every prefix u of σ satisfies $\mathrm{sat}_\Pi(u)$, as shown by σ itself, so $\Pi \subseteq \Pi_{\mathrm{cl}}$. If $\Pi = \Pi_{\mathrm{cl}}$ and $\sigma \notin \Pi$, some prefix u fails sat_Π, so Π is safe. If Π is safe and $\sigma \in \Pi_{\mathrm{cl}} \setminus \Pi$, the definition of safety gives a prefix u with $\neg\,\mathrm{sat}_\Pi(u)$, contradicting $\sigma \in \Pi_{\mathrm{cl}}$. □

Established result 4.16 (Decomposition)**.** *Every property* Π *is the intersection of a safety property and a liveness property.*

Source: Alpern and Schneider (1985).

Proof. Take $\Pi_S = \Pi_{\mathrm{cl}}$ and $\Pi_L = \Pi \cup (\mathsf{E}^\omega \setminus \Pi_{\mathrm{cl}})$. The set Π_{cl} is safe: if $\sigma \notin \Pi_{\mathrm{cl}}$ then some prefix u has $\neg\,\mathrm{sat}_\Pi(u)$, and no infinite extension of u lies in Π_{cl}, since each such extension has u as a prefix. The set Π_L is live: if $\mathrm{sat}_\Pi(u)$ then some extension of u lies in $\Pi \subseteq \Pi_L$, and if $\neg\,\mathrm{sat}_\Pi(u)$ then every extension of u lies outside Π_{cl} and so inside Π_L. Finally $\Pi_S \cap \Pi_L = (\Pi_{\mathrm{cl}} \cap \Pi) \cup \emptyset = \Pi$, using $\Pi \subseteq \Pi_{\mathrm{cl}}$. □

The set of finite histories compatible with a property is a history set in its own right.

Established result 4.17 (Viable histories)**.** *For nonempty* Π *let* $\mathrm{Viable}(\Pi) = \{u \in \mathsf{E}^* : \mathrm{sat}_\Pi(u)\}$*. Then* $\mathrm{Viable}(\Pi)$ *is a history set which is* pruned*: every* $u \in \mathrm{Viable}(\Pi)$ *has some* $a \in \mathsf{E}$ *with* $ua \in \mathrm{Viable}(\Pi)$*. A word outside* $\mathrm{Viable}(\Pi)$ *has all its extensions outside* $\mathrm{Viable}(\Pi)$*.*

Source: Standard.

Proof. Nonemptiness of Π gives $\varepsilon \in \mathrm{Viable}(\Pi)$. If uv is viable, with extension σ, then u is viable, with extension $v\sigma$, which shows prefix-closure. If u is viable via $\sigma = a\sigma'$ then ua is viable via σ', which shows pruning. The final claim is the contrapositive of prefix-closure. □

Established result 4.18 (Safety properties are pruned history sets)**.** *For a pruned history set* V *let* $\Pi_V = \{\sigma : \text{every finite prefix of } \sigma \text{ lies in } V\}$*. The maps* $\Pi \mapsto \mathrm{Viable}(\Pi)$ *and* $V \mapsto \Pi_V$ *are mutually inverse bijections between nonempty safety properties and pruned history sets.*

Source: The correspondence between closed subsets of E^ω and pruned trees; see Kechris, *Classical Descriptive Set Theory.*

Proof. Let V be pruned and $u \in V$. Starting from $u_0 = u$, pruning gives events a_i with $u_{i+1} = u_i a_i \in V$ for all i (by dependent choice), and the limit σ has all of its prefixes in V because every prefix is a prefix of u or equals some u_i. Thus $\sigma \in \Pi_V$ extends u and $u \in \mathrm{Viable}(\Pi_V)$. If $u \notin V$ then no extension of u lies in Π_V, so $u \notin \mathrm{Viable}(\Pi_V)$. Hence $\mathrm{Viable}(\Pi_V) = V$. This also shows Π_V is safe, because $(\Pi_V)_{\mathrm{cl}}$ consists of the words all of whose prefixes lie in $\mathrm{Viable}(\Pi_V) = V$, which is Π_V itself, and Result 4.15 applies. For a nonempty safety property Π, Result 4.17 shows $\mathrm{Viable}(\Pi)$ is a pruned history set, and $\Pi_{\mathrm{Viable}(\Pi)} = \Pi_{\mathrm{cl}} = \Pi$ by Result 4.15. □

Thus a safety property is nothing other than a pruned history set read as a permission to continue: the property holds of a run exactly when the run never leaves the set. The histories in the set are the ones from which the property can still be met, and this is the finite-history form of the viability condition studied in the second volume. A liveness property has $\mathrm{Viable}(\Pi) = \mathsf{E}^*$, since by definition every finite word is compatible with it. Violating it is a feature of the infinite run and never of a finite history.

4.5 Admissible extensions and the memory they require

Definition 4.19 (Admissible extensions)**.** For a nonempty property Π and a viable history h, the set of *admissible extensions* of h is $\mathrm{Ext}_\Pi(h) = \{w \in \mathsf{E}^* : hw \in \mathrm{Viable}(\Pi)\}$.

Example 4.20 (The account that must not be overdrawn)**.** In the account of Example 1.3 let Π_{neg} be the set of infinite runs all of whose finite prefixes have nonnegative balance. The set $V = \{h : q_{\mathrm{neg}}(h) = 0\}$ is a history set, since the prefixes of a history with no negative prefix have none, and it is pruned, since a deposit of 1 cannot create a negative prefix. Then $\Pi_{\mathrm{neg}} = \Pi_V$ and $\mathrm{Viable}(\Pi_{\mathrm{neg}}) = V$ by Result 4.18. For $h \in V$ the admissible extensions are $\mathrm{Ext}_\Pi(h) = \{w : \beta(h) + m(w) \geq 0\}$, where $m(w) \leq 0$ is the least balance among the prefixes of w.

New result 4.21 (Admissible extensions determine the balance)**.** *For $h, h' \in V$ in Example 4.20, $\mathrm{Ext}_\Pi(h) = \mathrm{Ext}_\Pi(h')$ if and only if $\beta(h) = \beta(h')$.*

Assumptions: Events, balances, and the property Π_{neg} are as in Example 4.20; amounts are positive integers.

Proof. If $\beta(h) = \beta(h')$ the descriptions of the two sets in Example 4.20 coincide. Suppose $\beta(h) = b < b' = \beta(h')$. The one-event word $w = \mathrm{w}(b')$ has $m(w) = -b'$ and $b' \geq 1$. Then $b' + m(w) = 0 \geq 0$, so $w \in \mathrm{Ext}_\Pi(h')$, while $b + m(w) = b - b' < 0$, so $w \notin \mathrm{Ext}_\Pi(h)$. □

The result says that on viable histories the information needed to decide which transitions are admissible is the balance and nothing coarser, and combined with Result 2.13 it identifies the least memory that keeps admissibility computable: the balance on the viable region and a single absorbing mark beyond it. This is the first instance of a pattern that organizes Part IV: the representation required to adjudicate admissibility is the continuation quotient of the property that defines it.

4.6 What finite histories can see of a behavioural property

The decomposition into safety and liveness has a consequence for retention that is easy to miss. A decision procedure that consults only finite histories cannot tell a behavioural property from its safety closure, and the part of a requirement that is lost in this way is exactly the liveness component.

New result 4.22 (Finite histories see only the closure)**.** *For every nonempty* $\Pi \subseteq \mathsf{E}^\omega$*,* $\mathrm{Viable}(\Pi) = \mathrm{Viable}(\Pi_{\mathrm{cl}})$ *and* $\Pi_{\mathrm{cl}} = \Pi_{\mathrm{Viable}(\Pi)}$*. Consequently every function of finite histories that is determined by viability, in particular the sets* $\mathrm{Ext}_\Pi(h)$ *of admissible extensions, is the same for* Π *and for its closure.*

Assumptions: Π nonempty; the history set is all of E^*.

Proof. Since $\Pi \subseteq \Pi_{\mathrm{cl}}$, every word compatible with Π is compatible with Π_{cl}. Conversely, if $\sigma \in \Pi_{\mathrm{cl}}$ extends u, then u is a prefix of σ and $\mathrm{sat}_\Pi(u)$ holds by the definition of Π_{cl}. So $\mathrm{Viable}(\Pi) = \mathrm{Viable}(\Pi_{\mathrm{cl}})$. The second equation is the definition of Π_{cl} read through Viable: an infinite word belongs to Π_{cl} exactly when all of its prefixes are viable. □

Example 4.23 (A requirement with a liveness component)**.** For the monitor of Example 1.4 let Π be the set of runs in which no prefix has three alarms outstanding and in which reset occurs infinitely often. The closure Π_{cl} is the set of runs with no prefix having three alarms outstanding: a finite word with that property extends to a member of Π by appending resets forever, so it is compatible with Π, and a word with a prefix of three outstanding alarms is compatible with nothing. Hence $\mathrm{Viable}(\Pi)$ is the set of finite histories with $q_{\mathrm{ever}} = 0$, and the decomposition of Result 4.16 is $\Pi = \Pi_{\mathrm{cl}} \cap \Pi_L$ with $\Pi_L = \Pi \cup (\mathsf{E}^\omega \setminus \Pi_{\mathrm{cl}})$. The property Π is neither safe nor live. The finite history $\mathrm{alarm}\,\mathrm{alarm}\,\mathrm{tick}^n$ is viable for every n, however long the system goes without a reset, and no finite history ever shows the requirement of infinitely many resets to have failed. A memory sized to adjudicate Π on finite histories is the Nerode system of q_{ever}, with four states; the infinitary requirement costs nothing in memory because it cannot be adjudicated by memory at all.

The practical reading is that obligations of the form "eventually" are not retained as state. They constrain what the system must still do, and Chapter 14 converts them into bounded-response obligations whose pending deadlines are state, which is a different object.

4.7 Why the extension clause cannot be dropped

Definition 4.11 requires $\mathrm{Ext}(h) = \mathrm{Ext}(h')$ in addition to $P_h = P_{h'}$. The clause looks like a technicality, and a short example shows that it is not.

Example 4.24 (Histories that agree where both are defined)**.** Let $\mathsf{E} = \{a, b\}$ and $\mathcal{H} = \{\varepsilon, a, b, bb\}$, and let P be constant. Suppose we declared h and h' equivalent whenever P_h and $P_{h'}$ agree on $\mathrm{Ext}(h) \cap \mathrm{Ext}(h')$. Then all four histories would be equivalent, and the resulting one-state system would have to allow a and b at its only state, hence generate a larger history set than $\mathcal{H}$. With the clause the classes are $\{\varepsilon\}$, $\{b\}$ and $\{a, bb\}$, since $\mathrm{Ext}(\varepsilon) = \{\varepsilon, a, b, bb\}$, $\mathrm{Ext}(b) = \{\varepsilon, b\}$ and $\mathrm{Ext}(a) = \mathrm{Ext}(bb) = \{\varepsilon\}$. The Nerode system has three states, an initial state with transitions a to the end state and b to a middle state, and a middle state with a

transition b to the end state, and it generates $\mathcal{H}$ exactly. No system generating $\mathcal{H}$ has fewer than three reachable states, by Result 4.13(b).

The example makes a point about what a property can be said to distinguish. A constant property distinguishes nothing about verdicts, yet the histories a, b and ε differ in how much future they have, and a system that is to generate the history set must retain that difference. In the common model this is the physical component: $\mathrm{Reach}(x(h))$ is part of what must be retained whatever questions are asked, and the obligations come on top of it.

Exercises

Exercise 4.25. Construct a transition system with four states that generates all words over $\{\mathrm{tick}, \mathrm{alarm}, \mathrm{reset}\}$ and for which "at least three alarms are outstanding" (the question of Example 1.4) is a state property.

Exercise 4.26. For your system, prove that $s_T = \rho$, and use Result 4.8 to conclude that $\ker \rho$ is contained in the kernel of the question. Then use Result 4.13 to show that no transition system generating the same history set and having the question as a state property has fewer than four reachable states.

Exercise 4.27. Show that "three alarms have ever been outstanding" is not a state property of your system, by exhibiting two histories with the same state and different verdicts. Then compute the continuation equivalence for this second property and show that its Nerode system has exactly four states, namely the capped counts $0, 1, 2$ together with one absorbing state, and compare this with the four states of the first property.

Exercise 4.28. The threshold of the question is revised from three alarms to j alarms. Compute $\approx_P$ for the revised question and show that the Nerode system has exactly $j + 1$ states. Describe in words what is lost if the memory is kept at ρ with cap 3 and the threshold is revised to 5.

Exercise 4.29. Classify the following behavioural properties of the monitor as safety, liveness, or neither: "no prefix of the run has three alarms outstanding" and "some prefix of the run has three alarms outstanding". For each, describe $\mathrm{Viable}(\Pi)$ and compute the decomposition of Result 4.16. Explain why the two properties are complements of one another and yet fall on opposite sides of the classification.

Exercise 4.30. Prove that for Π a safety property and h a viable history, $\mathrm{Ext}_\Pi(h)$ is a pruned history set, and that $\mathrm{Ext}_\Pi(hu) = \{w : uw \in \mathrm{Ext}_\Pi(h)\}$ for $u \in \mathrm{Ext}_\Pi(h)$. Use this to show that the map $h \mapsto \mathrm{Ext}_\Pi(h)$ on viable histories is constant on the classes of the continuation equivalence for the indicator of $\mathrm{Viable}(\Pi)$.

Exercise 4.31. For the property Π of Example 4.23, compute Π_{cl} and Π_L explicitly, verify that Π_L is live, and exhibit a run in $\Pi_L \setminus \Pi$ that lies in Π_{cl} and one that lies outside Π_{cl}.

Exercise 4.32. Prove that Π is a liveness property if and only if $\Pi_{\mathrm{cl}} = \mathsf{E}^\omega$. Deduce that a property that is both safe and live is all of E^ω.

Exercise 4.33. Change the history set of Example 4.24 to $\{\varepsilon, a, b, bb, bbb\}$ with $P(h) = 1$ exactly when h contains an a. Compute the classes of $\approx_P$ and show that the Nerode system has five states, so that it coincides with the tree system. Explain which histories the extension clause separates that the verdicts alone would not.

Part II

Sufficient representations

Chapter 5

Sufficiency for a Fixed Policy

Question. What is the coarsest representation that still decides a specified policy, and how far does that answer depend on the policy and on the cases that can occur?

Established sources. The kernel of a policy is the coarsest equivalence through which it factors.

New results. Compression is policy-independent only for injective representations (Result 5.3); minimal representations are sensitive to a change at a single case (Result 5.4); sufficiency depends on the domain of cases (Result 5.7).

Proof obligations. None.

Depends on. Chapters 1, 2, and 3.

The preceding chapters treated questions and representations as two kinds of function and studied the relation between them. This chapter assigns that relation a purpose. A system acts, and what it does at a given case is determined by a policy. The retained representation is what the system consults when it acts, and a representation is sufficient for a policy when the consultation yields the action that the policy prescribes. The mathematics is the factorization theorem of Chapter 2 read in this setting. The substance of the chapter lies in three consequences: the minimal representation for a policy is determined by the policy and by nothing else, no non-injective representation is safe for every policy, and the answer depends both on the policy and on the set of cases that can in fact occur.

5.1 Policies and sufficient representations

Definition 5.1 (Policy, decision rule, sufficiency)**.** A *policy* on a nonempty set X of cases with values in a set A of actions is a function $\pi : X \to A$. A representation $r : X \to R$ is *sufficient for* π if there is a function $d : r(X) \to A$, a *decision rule*, with $\pi = d \circ r$.

A policy says what ought to be done at each case. A decision rule says what is done on the basis of the representation alone. Sufficiency is the requirement that these agree everywhere, so that consulting the representation instead of the case costs nothing. It is a property of the pair (r, π) and is exactly the answering relation of Definition 1.1.

Established result 5.2 (Sufficiency for a fixed policy)**.** *A representation r is sufficient for π if and only if* $\ker r \subseteq \ker \pi$. *The decision rule is then unique. The representation r is sufficient*

and has the fewest possible values among sufficient representations if and only if $\ker r = \ker \pi$*, so the minimal sufficient representations are, up to relabelling, the quotient map* $\pi_{\ker \pi}$*, and each has exactly* $|\pi(X)|$ *values.*

Source: Results 2.5, 2.6, and 3.10.

Proof. The first two statements are Result 2.5. By Result 3.10 applied to the one-element family $\{\pi\}$, every sufficient representation takes at least $|X/\ker \pi|$ values, with equality exactly when $\ker r = \ker \pi$. Result 2.6 identifies $X/\ker \pi$ with $\pi(X)$. □

The theorem has a conclusion that should be stated plainly because it governs how the rest of the volume is organized. For a single policy on a fixed set of cases, the policy itself is a minimal sufficient representation: it suffices to store the action to be taken. The number of bits required is $\lceil \log_2 |\pi(X)| \rceil$. Nothing in the theory of this chapter is deeper than that statement, and the difficulty of the subject lies elsewhere. A system that stores only the action it must take now cannot adopt a different policy later, which is the topic of Chapter 6; it cannot in general update the stored action when a further event occurs without consulting information it has discarded, which is the topic of Chapter 7; and it cannot be assumed to face every case in X, which is taken up in the last section of this chapter.

5.2 No compression is free

A common informal claim holds that a description which is much smaller than the thing described has lost nothing of importance. The following results show how that claim fails once policies are made explicit.

New result 5.3 (Compression is policy-independent only when it is injective)**.** *A representation* r *of* X *is sufficient for every policy* $X \to \{0, 1\}$ *if and only if* r *is injective. Consequently, if* r *is not injective then there is a two-action policy for which* r *is not sufficient, and the failure is exhibited by every pair of cases that* r *identifies.*

Assumptions: No assumption on X beyond nonemptiness.

Proof. If r is injective then $r^{-1} : r(X) \to X$ exists, and every policy satisfies $\pi = (\pi \circ r^{-1}) \circ r$. If r is not injective, pick $x \neq x'$ with $r(x) = r(x')$ and let π be the indicator of $\{x\}$. Then $(x, x') \in \ker r$ and $(x, x') \notin \ker \pi$, so $\ker r \not\subseteq \ker \pi$ and r is not sufficient by Result 5.2. □

The result is trivial to prove and widely ignored in practice. Any scheme that discards cases, whether by hashing, summarizing, truncating, or averaging, is safe only relative to the policies that are known not to depend on what was discarded.

The next result shows how little it takes for the identity of the best representation to change.

New result 5.4 (Sensitivity to a change at one case)**.** *Let* $A \subseteq X$ *with* $|A| \geq 2$ *and* $A \neq X$*, let* $x_0 \in A$*, and let* $\pi = \mathbf{1}_A$ *and* $\pi' = \mathbf{1}_{A \setminus \{x_0\}}$ *be the two-action policies that disagree only at* x_0*. Then* $\ker \pi$ *and* $\ker \pi'$ *are incomparable, so the minimal sufficient representations of* π *and* π' *are incomparable for* $\sqsubseteq$*, and every representation sufficient for both takes at least three values.*

Assumptions: X has at least three elements, as the hypotheses on A imply.

Proof. Choose $a \in A \setminus \{x_0\}$ and $c \in X \setminus A$. The pair (a, x_0) lies in $\ker \pi$ and not in $\ker \pi'$, since $\pi'(a) = 1$ and $\pi'(x_0) = 0$. The pair (x_0, c) lies in $\ker \pi'$ and not in $\ker \pi$, since $\pi'(x_0) = \pi'(c) = 0$ while $\pi(x_0) = 1$ and $\pi(c) = 0$. Hence neither kernel contains the other, and Result 3.2 gives incomparability of the minimal sufficient representations. A representation sufficient for both has kernel inside $\ker \pi \cap \ker \pi'$, which separates the three nonempty sets $A \setminus \{x_0\}$, $\{x_0\}$, and $X \setminus A$ from one another, so it has at least three classes. □

Example 5.5 (Withdrawal approval at one threshold and at two)**.** In the account of Example 1.3, the policy $\pi_k(h) = [\beta(h) \geq k]$ approves a withdrawal of k without overdraft. By Result 5.2 the minimal sufficient representation of π_k is a single bit, namely whether the balance is at least k. A system that must approve withdrawals of k and of $k' > k$ needs the joint kernel of π_k and $\pi_{k'}$, which has three classes: $\beta < k$, $k \leq \beta < k'$, and $\beta \geq k'$. Neither bit alone is sufficient for the pair of policies. The two bits taken together allow four combinations but only three occur, since $\beta \geq k'$ implies $\beta \geq k$, and the joint representation has exactly three values.

5.3 Dependence on the domain of cases

A set X of cases is rarely the set of all conceivable ones. Physical constraints rule out some histories and obligations rule out others, and the representation need only serve the histories that can occur. The following notion records this.

Definition 5.6 (Sufficiency on a domain)**.** Let $Y \subseteq X$ be nonempty. A representation r of X is *sufficient for π on Y* if the restriction $\pi|_Y$ factors through $r|_Y$, equivalently if $\ker r \cap (Y \times Y) \subseteq \ker \pi$.

New result 5.7 (Domain dependence)**.** *Sufficiency on a domain is monotone and not transferable: if r is sufficient for π on Y and $Y' \subseteq Y$ then r is sufficient for π on Y'; but sufficiency on Y does not in general extend to a larger domain. In particular the least informative representation sufficient for π on Y is $Y/(\ker \pi \cap (Y \times Y))$ and may be strictly smaller than the least informative representation sufficient on X.*

Assumptions: The set Y is fixed in advance and the representation is evaluated only on Y.

Proof. Monotonicity follows from the inclusion $\ker r \cap (Y' \times Y') \subseteq \ker r \cap (Y \times Y)$. For nontransferability, take $X = \{1, 2, 3\}$, $Y = \{1, 2\}$, $\pi = \mathbf{1}_{\{3\}}$, and r the representation identifying 2 with 3 and leaving 1 alone. Then $\pi|_Y$ is constant and r is sufficient on Y, but $(2, 3) \in \ker r \setminus \ker \pi$, so r is not sufficient on X. The least informative representation sufficient on Y is obtained by applying Result 5.2 to the restricted policy on Y, and in the example it is constant on Y while any sufficient representation on X takes two values. □

Example 5.8 (Executable histories need less memory)**.** Let $Y = V = \{h : q_{\mathrm{neg}}(h) = 0\}$ be the histories of the account that never go negative. On V the policy q_{neg} is constant, so the constant representation is sufficient on V, while on all of E^* the constant representation is not sufficient. The saving is the whole of the flag. It is obtained at a price: the system must be certain that no history outside V can reach it. If that certainty rests on a physical guarantee the saving is secure; if it rests on a policy that a person may override, the representation has silently become dependent on the override never occurring.

The example connects this chapter with the obstructions of Definition 1.5. Physical and normative constraints shrink the set of cases and so shrink the memory required, and a representation chosen for the shrunken set is insufficient for any revision that readmits the excluded cases. The sufficiency question is therefore always a question about a triple: a representation, a policy, and the set of cases to which both apply.

5.4 How many thresholds a bounded representation can serve

Result 5.4 shows that neighbouring policies need incomparable representations. The family of threshold policies on an ordered set of cases makes the cost of flexibility exact. Let $X = \{0, 1, \dots, n-1\}$ be ordered, as when the cases are balances, counts or ages, and for $1 \le t \le n-1$ let $\pi_t(x) = [x \ge t]$. A representation is said to *serve* the threshold t when it is sufficient for π_t.

New result 5.9 (Thresholds served by a bounded representation)**.** *A representation r of $X = \{0, \dots, n-1\}$ with k values serves at most $k-1$ thresholds, and for every $k \le n$ the bound is attained by the representation that sends x to the index of the block of k contiguous intervals containing it, serving exactly the $k-1$ thresholds at the block boundaries.*

Assumptions: The cases are the integers $0, \dots, n-1$ with the usual order; thresholds are $\pi_t = [x \ge t]$ for $1 \le t \le n-1$.

Proof. By Result 5.2, r serves t if and only if every block of $\ker r$ lies entirely in $\{0, \dots, t-1\}$ or entirely in $\{t, \dots, n-1\}$. Suppose r serves the thresholds $t_1 < \cdots < t_s$ and set $t_0 = 0$, $t_{s+1} = n$. Each interval $I_j = \{t_j, \dots, t_{j+1} - 1\}$ for $0 \le j \le s$ is nonempty, and every block of $\ker r$ is contained in a single I_j, because a block meeting two of the intervals would straddle some t_i. So the $s+1$ intervals are unions of blocks and r takes at least $s+1$ values, that is $k \ge s+1$. For attainment, cut X into k intervals; every cut point is served because no block crosses it. □

For a balance that runs from 0 to 5 and a memory of three values, at most two of the five possible approval thresholds can be served, and the enumeration of all 203 partitions of the six cases confirms that no partition with three blocks serves more than two. The consequence for design is that flexibility of policy is paid for linearly in memory: each additional threshold that the system must be able to honour costs one further value, and the cost cannot be shared between thresholds by any cleverness in the choice of blocks. This is the finite form of the remark of Chapter 3 that bounded memory is a choice among incomparable alternatives. It also locates the weakness of the common practice of quantizing a quantity into a few coarse bins: the bins serve exactly the policies whose thresholds coincide with bin boundaries, and a revised threshold that falls inside a bin cannot be honoured without recovering information the quantization discarded.

Example 5.10 (A policy-count view)**.** A representation with k values is sufficient for exactly $|A|^k$ of the $|A|^n$ policies from a set of n cases to a set A of actions, because a sufficient policy is determined by its decision rule, which is an arbitrary function on the k values. For $n = 6$, $k = 3$ and two actions there are 8 of 64 policies served by any one three-valued representation. The thresholds are two of those 8 for the best choice of blocks, and the remaining six served policies, such as the parity of the block index, are not thresholds at all.

Exercises

Exercise 5.11. Construct a minimal sufficient representation for the escalation policy "escalate if at least three alarms are outstanding" of Example 1.4, as a function on histories, and state its number of values. Compute the number of bits.

Exercise 5.12. Prove exactly what your representation preserves: show that it is sufficient for the escalation policy and for every policy that is a function of the escalation policy, and that it is not sufficient for any policy that separates two histories with the same escalation verdict.

Exercise 5.13. Show that the claim "a representation sufficient for π is sufficient for every policy that agrees with π on all but one case" is false, using Result 5.4. Determine for which pairs of policies agreeing on all but one case the claim happens to hold.

Exercise 5.14. The escalation policy is revised so that it escalates at two alarms instead of three. Compute the minimal sufficient representation for the revised policy, compare it with the representation of the first exercise, and determine whether the original representation is sufficient for the revised policy. Then determine the least representation sufficient for both policies.

Exercise 5.15. Let $Y \subseteq X$ be the set of cases that a particular system can reach. Show that if Y is a proper subset then a representation may be sufficient on Y and not on X, and give a concrete instance from the document of Example 1.2 in which only edits to section 1 are ever permitted.

Exercise 5.16. Prove the count of Example 5.10: a representation with k values is sufficient for exactly $|A|^k$ policies into A. Use it to show that, for a fixed pair of three-valued representations of six cases, the number of policies served by both is 2^c, where c is the number of blocks of the common coarsening, and compute it for two representations of your choice.

Exercise 5.17. Replace thresholds by window policies $w_{s,t}(x) = [s \le x < t]$ on $\{0, \dots, n-1\}$ with $1 \le s < t \le n-1$. Show that a representation whose classes are intervals of the line and which has k values serves at most $\binom{k-1}{2}$ of them, and that this number is attained. Then show that, once the classes are allowed to be non-intervals, a three-valued representation can serve three window policies, whereas an interval representation with three values serves only one.

Exercise 5.18. Show that Result 5.9 depends on the linear order of the cases: for the cases $\{0,1\}^3$ and the policies $\pi_i(x) = x_i$ for $i = 1, 2, 3$, determine how many of the three can be served by a representation with k values for each k, and explain why the answer is not $k-1$.

Chapter 6

Sufficiency for Families of Policies

Question. How much finer must a representation become as the family of supported policies grows, and how many policies does a given representation support?

Established sources. Joint kernels of families; the Sauer–Shelah lemma and Assouad's bound on the dual VC dimension; the count of equivalence relations by Bell numbers.

New results. Fully separating families force injective representations (Result 6.4); monotone growth of the minimal representation with the family and exact costs for chains and independent families; the number of policies a representation supports (Result 6.12).

Proof obligations. None. Sharp bounds beyond the separating case are posed as Research problem 6.9.

Depends on. Chapters 2, 3, and 5.

A single policy requires only that its own verdict be recoverable. The reason for storing anything richer is that the policy may change. The set of policies that a system is expected to be able to implement is its policy family, and the richer the family the finer the representation that supports it. This chapter makes that relation exact. It shows that the cost of a family is determined by the joint kernel of its members, gives general bounds and exact values for the structured families that occur in practice, and turns the relation around by asking how many policies a representation of a given size can support at all.

6.1 Families and joint sufficiency

Definition 6.1 (Policy family)**.** A *policy family* on X is a set $\Phi = \{\pi_i : X \to A_i\}_{i \in I}$ of policies. A representation r is *sufficient for* Φ if it is sufficient for each π_i. Write $\kappa(\Phi) = \bigcap_i \ker \pi_i$ for the *joint kernel*.

Established result 6.2 (Joint sufficiency)**.** *A representation r is sufficient for Φ if and only if* $\ker r \subseteq \kappa(\Phi)$*. The minimal sufficient representations are, up to relabelling, the quotient map onto $X/\kappa(\Phi)$, whose number of values is the number of distinct* profiles $(\pi_i(x))_{i \in I}$ *taken by the policies on X.*

Source: Results 2.12 and 3.10.

Proof. The first two claims are Result 3.10. Two cases lie in the same class of $\kappa(\Phi)$ exactly when every policy takes the same value on them, so the classes are the fibres of the profile map $x \mapsto (\pi_i(x))_i$, and their number is the size of its image. □

Definition 6.3 (Separating family)**.** A family Φ is *separating* if $\kappa(\Phi)$ is the diagonal: for all $x \neq x'$ some π_i has $\pi_i(x) \neq \pi_i(x')$.

New result 6.4 (Separating families force injectivity)**.** *A family Φ is separating if and only if every sufficient representation for Φ is injective. When X is finite with n elements and Φ is separating, every sufficient representation takes at least n values and therefore needs at least $\lceil \log_2 n \rceil$ bits. If the members of Φ take at most m values each, then $|\Phi| \geq \lceil \log_m n \rceil$.*

Assumptions: For the quantitative statements, X is finite and $m \geq 2$.

Proof. If Φ is separating then $\kappa(\Phi)$ is the diagonal, so $\ker r$ is the diagonal for every sufficient r, and r is injective. Conversely if $\kappa(\Phi)$ contains a pair $x \neq x'$, the quotient map onto $X/\kappa(\Phi)$ is sufficient and not injective. The bound on values follows because an injective function on n points has n values. For the last claim, the profile map is injective on X when Φ separates, and takes values in a set of size at most $m^{|\Phi|}$, so $n \leq m^{|\Phi|}$. □

A separating family is the extreme case of the theory. If a system is to be ready to implement a family that tells every pair of cases apart, then no representation short of the case itself will do, and compression is impossible in the sense of Result 5.3.

6.2 The cost of a growing family

Established result 6.5 (Monotonicity and bounds)**.** *Let $\Phi \subseteq \Phi'$ be families on X. Then $\kappa(\Phi') \subseteq \kappa(\Phi)$, so every representation sufficient for Φ' is sufficient for Φ and $|X/\kappa(\Phi')| \geq |X/\kappa(\Phi)|$. For every finite family $\Phi = \{\pi_1, \ldots, \pi_k\}$,*

$$\max_i |\pi_i(X)| \;\leq\; |X/\kappa(\Phi)| \;\leq\; \min\Big(|X|,\; \prod_i |\pi_i(X)|\Big),$$

and both bounds can be attained.

Source: Elementary consequences of Results 2.10 and 6.2.

Proof. Intersecting more kernels yields a smaller relation, which gives the first claim and the monotonicity of the number of classes by Result 2.10. Since $\kappa(\Phi) \subseteq \ker \pi_i$ there is a surjection from $X/\kappa(\Phi)$ onto $X/\ker \pi_i \cong \pi_i(X)$, which gives the lower bound. The profile map injects $X/\kappa(\Phi)$ into $\prod_i \pi_i(X)$, and trivially $|X/\kappa(\Phi)| \leq |X|$. For attainment of the upper bound take $X = \{0,1\}^k$ and π_i the ith coordinate, so that $|X/\kappa(\Phi)| = 2^k = \prod_i |\pi_i(X)|$. For attainment of the lower bound take $\pi_i = g_i \circ \pi$ for a single policy π and functions g_i with $g_1 = \mathrm{id}$, so that $\ker \pi_1 = \kappa(\Phi)$ and the number of classes equals $|\pi_1(X)|$. □

The two extreme cases correspond to two situations that occur constantly. In the first the policies depend on unrelated aspects of the case, so memory grows multiplicatively with the family. In the second they all depend on one underlying quantity, and memory is determined by that quantity alone. The thresholds of the account are of the second kind.

New result 6.6 (Chains of thresholds)**.** *Let $\varphi : X \to \mathbb{Z}$ be a function and let $K \geq 1$. The family $\Phi_K = \{\mathbf{1}[\varphi \geq k] : k = 1, \ldots, K\}$ has $|X/\kappa(\Phi_K)| = |\{\min(\max(\varphi(x), 0), K) : x \in X\}|$, which is at most $K + 1$, and is exactly $K + 1$ if $\varphi(X) \supseteq \{0, 1, \ldots, K\}$. Thus the cost of supporting K nested thresholds is linear in K and not exponential in K.*

Assumptions: φ is an arbitrary integer-valued function; the thresholds are consecutive integers.

Proof. Write $c(x) = \min(\max(\varphi(x), 0), K) \in \{0, \dots, K\}$. The profile of x is $(\mathbf{1}[\varphi(x) \geq 1], \dots, \mathbf{1}[\varphi(x) \geq K])$, which is a vector with a leading block of ones of length $c(x)$ followed by zeros. The profile determines $c(x)$ and is determined by it, so profiles correspond to values of c, and c takes at most $K+1$ values and exactly $K+1$ when every integer in $\{0, \dots, K\}$ occurs among the values of φ. □

Example 6.7 (The account and the monitor)**.** For the account of Example 1.3 with $\varphi = \beta$, the family Φ_K of withdrawal approvals at thresholds $1, \dots, K$ has joint kernel with exactly $K+1$ classes, namely $\beta \leq 0$, $\beta = 1, \dots, K-1$, and $\beta \geq K$. For the monitor of Example 1.4 with $\varphi = c$, the family of escalation policies at $1, \dots, K$ has joint kernel with $K+1$ classes, the capped alarm count. In both cases the minimal sufficient representation for the family is the original quantity clamped to a window whose width equals the largest threshold supported.

6.3 Structured families and general bounds

The bound in Result 6.5 is attained by independent families, but exponential growth is not the typical situation. For families with bounded combinatorial complexity the number of classes grows only polynomially in the number of policies, and the classical lemma of Sauer and Shelah gives the exponent.

Let $\mathcal{S}$ be a set system on X, that is, a set of subsets of X. A subset $B \subseteq X$ is *shattered* by $\mathcal{S}$ if every subset of B is of the form $B \cap S$ with $S \in \mathcal{S}$, and the *VC dimension* of $\mathcal{S}$ is the largest size of a shattered set. The *dual system* $\mathcal{S}^*$ is the system on $\mathcal{S}$ consisting, for each $x \in X$, of the set $\{S \in \mathcal{S} : x \in S\}$. A two-action policy π corresponds to the set $\pi^{-1}(1)$.

Established result 6.8 (Polynomial growth for bounded VC dimension)**.** *Let $\mathcal{S}$ be a set system on X of VC dimension d, and let Φ be a family of N two-action policies whose one-sets lie in $\mathcal{S}$. Then the number of classes of $\kappa(\Phi)$ is at most $\sum_{i=0}^{d^*} \binom{N}{i}$, where $d^* < 2^{d+1}$ is the VC dimension of the dual system. In particular it is at most $(eN/d^*)^{d^*}$ for $N \geq d^*$.*

Source: Sauer (1972) and Shelah (1972) for the counting lemma; Assouad (1983) for the bound on the dual dimension.

Proof. Let $A_1, \dots, A_N$ be the one-sets. The class of x in $\kappa(\Phi)$ is determined by the pattern $\{j : x \in A_j\} \subseteq \{1, \dots, N\}$, which is the trace on $\{A_1, \dots, A_N\}$ of the set $\{S \in \mathcal{S} : x \in S\}$ of the dual system. By the Sauer–Shelah lemma, a set system of VC dimension d^* has at most $\sum_{i=0}^{d^*} \binom{N}{i}$ distinct traces on an N-element set. Assouad's theorem gives $d^* < 2^{d+1}$ for the dual of a system of VC dimension d. The closing bound is the standard estimate $\sum_{i \leq d^*} \binom{N}{i} \leq (eN/d^*)^{d^*}$. □

The theorem converts a combinatorial hypothesis on what the policies can express into a bound on the memory needed to support N of them. A family drawn from a class of bounded VC dimension has joint kernel with polynomially many classes in N, whereas an unrestricted family of N two-action policies can have 2^N classes, as the coordinate policies on $\{0,1\}^N$ show. The bound is a bound and not an exact count. For families of halfspace policies $\mathbf{1}[\langle w_j, x \rangle > t_j]$ on $X = \mathbb{R}^d$ the joint kernel is governed by the faces of an arrangement of N hyperplanes, whose number is $O(N^d)$, and this order of growth is attained by hyperplanes in general position, which cut $\mathbb{R}^d$ into $\sum_{i \leq d} \binom{N}{i}$ regions.

Research problem 6.9. Determine, for the families of policies induced by the states of an obligation monitor with n states (Chapter 14), the growth of $|X/\kappa(\Phi)|$ when a single admissible revision is added to the family, in terms of the transition structure of the monitor and not only of n. Characterize the monitors for which the growth is bounded independently of n.

6.4 How many policies does a representation support?

The relation can be read in the opposite direction. A representation σ fixes the set of policies that can be implemented from it.

Definition 6.10 (Implementable policies)**.** For a representation σ of X, the set of *implementable policies* is $\mathrm{Pol}(\sigma) = \{\pi : X \to A \text{ for some } A,\ \ker\sigma \subseteq \ker\pi\}$.

Established result 6.11 (Forgetting contracts the implementable family)**.** *If $\sigma' = u \circ \sigma$ for some function u, then $\mathrm{Pol}(\sigma') \subseteq \mathrm{Pol}(\sigma)$. If moreover $\mathrm{Pol}(\sigma) \subseteq \mathrm{Pol}(\sigma')$ then $\ker\sigma = \ker\sigma'$.*

Source: Elementary; Result 2.11.

Proof. If $\ker\sigma' \subseteq \ker\pi$ then $\ker\sigma \subseteq \ker\sigma' \subseteq \ker\pi$, so $\pi \in \mathrm{Pol}(\sigma)$. If $\mathrm{Pol}(\sigma) \subseteq \mathrm{Pol}(\sigma')$ then, since $\sigma \in \mathrm{Pol}(\sigma)$ trivially, we have $\sigma \in \mathrm{Pol}(\sigma')$, so $\ker\sigma' \subseteq \ker\sigma$, and the two inclusions give equality. □

The number of policies that a finite representation supports is given by a classical sequence.

Established result 6.12 (Counting implementable policies)**.** *Let σ take exactly k values. Up to relabelling of the actions, the policies in $\mathrm{Pol}(\sigma)$ are in bijection with the equivalence relations on a k-element set, and their number is the Bell number B_k. In particular, a one-bit representation supports two policies up to relabelling, a two-bit representation with four values supports fifteen, and B_k grows faster than c^k for every constant c.*

Source: Bell numbers; see, for example, Stanley, *Enumerative Combinatorics*, Volume 1.

Proof. A policy π is determined up to relabelling of its actions by $\ker\pi$, and $\ker\pi$ ranges over the equivalence relations on X containing $\ker\sigma$. These correspond to the equivalence relations on $X/\ker\sigma$, a k-element set. The values $B_2 = 2$ and $B_4 = 15$ are standard, and the superexponential growth of B_k is classical. □

The count quantifies the dependence of the revision horizon on memory. A system with k values of memory is prepared for B_k different policies, but the preparedness is of a particular kind: the policies it supports are exactly those obtainable by merging values, and no others.

6.5 The marginal cost of one more policy

The bounds of Result 6.5 compare whole families. A designer more often asks what a single further policy will cost on top of a family already supported, and the answer is exact and computable class by class.

New result 6.13 (Marginal cost of a policy)**.** *Let Φ be a family on a finite set X and let π be a further policy. For a class $c \in X/\kappa(\Phi)$ write $\pi(c)$ for the set of values taken by π on c. Then*

$$|X/\kappa(\Phi \cup \{\pi\})| = \sum_{c \in X/\kappa(\Phi)} |\pi(c)|,$$

so the marginal cost is $\sum_c (|\pi(c)| - 1)$. It is zero if and only if π is sufficiently represented by the profile map of Φ, that is, if π is a function of the profiles of the policies in Φ, and it is at most $(|\pi(X)| - 1)\,|X/\kappa(\Phi)|$.

Assumptions: X finite; $\kappa(\Phi)$ is the joint kernel of Definition 6.1.

Proof. The classes of $\kappa(\Phi \cup \{\pi\}) = \kappa(\Phi) \cap \ker \pi$ are the nonempty sets $c \cap \pi^{-1}(a)$ for c a class of $\kappa(\Phi)$ and a an action, and the nonempty ones for a fixed c correspond to the values $a \in \pi(c)$. Summing over c gives the formula. The marginal cost vanishes exactly when $|\pi(c)| = 1$ for every c, which says that π is constant on each class of $\kappa(\Phi)$, that is, $\kappa(\Phi) \subseteq \ker \pi$, which by Result 5.2 says that the quotient onto $X/\kappa(\Phi)$ is sufficient for π. The upper bound follows from $|\pi(c)| \leq |\pi(X)|$. □

The formula makes three cases visible. A policy that is a function of policies already supported costs nothing, however complicated it looks. A policy that refines only one class costs the number of extra values taken within that class. An independent policy, constant on no class, costs the full product. A computed example shows all three in a small setting.

Example 6.14 (An account with an alarm count)**.** Let the cases be pairs (b, a) of a balance $b \in \{0, \ldots, 5\}$ and an alarm count $a \in \{0, \ldots, 3\}$, twenty-four in all, and let $\Phi = \{[b \geq 3],\ [a \geq 2]\}$, whose joint kernel has four classes. Adding the policy $\pi = [b + a \geq 4]$ gives

class of $\kappa(\Phi)$	values of $b + a$ in the class	$\lvert\pi(c)\rvert$
$b < 3,\ a < 2$	$0, \ldots, 3$	1
$b \geq 3,\ a < 2$	$3, \ldots, 6$	2
$b < 3,\ a \geq 2$	$2, \ldots, 5$	2
$b \geq 3,\ a \geq 2$	$5, \ldots, 8$	1

so the family grows from four classes to six, a marginal cost of two. In the same setting, adding the threshold $[b \geq 4]$ to the chain $\{[b \geq 1], [b \geq 2], [b \geq 3]\}$, which has four classes, gives five: the class $\{b \geq 3\}$ is the only one the new policy splits. And adding $[a \geq 2]$ to $\{[b \geq 3]\}$ doubles two classes to four. In each case the count has been confirmed by enumerating the twenty-four cases.

Two lessons follow. The cost of a revision depends on the family that is already in place, and the same policy can be free for one memory and expensive for another; the bill is presented class by class, and it is the classes that the new policy cuts across that pay. And a policy that depends on a combination of quantities, as $b + a \geq 4$ does, is cheap only if the combination is already recoverable from what is retained, which for sums of independently thresholded quantities it is not.

Exercises

Exercise 6.15. Construct a family of policies on the histories of Example 1.2 with $m = 2$ and $C = \{0, 1\}$ whose joint kernel has exactly four classes, and a different family on the same cases whose joint kernel has exactly sixteen.

Exercise 6.16. Prove Result 6.6 in full for the family of escalation policies at $1, \ldots, K$ on the monitor, and determine $|X/\kappa(\Phi_K)|$ for the monitor with the histories restricted to words over $\{\text{alarm}\}$ alone.

Exercise 6.17. Show that the joint-kernel bound $|X/\kappa(\Phi)| \leq \prod_i |\pi_i(X)|$ can be strict for dependent policies, and give a family of three policies on a six-element set for which the three bounds of Result 6.5 take three different values.

Exercise 6.18. For the account with thresholds $1, \ldots, K$, the family is revised by replacing the threshold set $\{1, \ldots, K\}$ with $\{2, 4, \ldots, 2K\}$. Determine the joint kernel of the revised family, and decide whether a representation sufficient for the original is sufficient for the revised family. Determine the least representation sufficient for both.

Exercise 6.19. Use Result 6.12 to determine the number of policies with at most two values, up to relabelling, supported by a representation with k values, and show that this number is 2^{k-1}, so that of the B_k policies supported, those with at most two values form a small fraction for large k.

Exercise 6.20. In Example 6.14 compute the marginal cost of $[b + a \geq 4]$ to the family $\{[b \geq 1], [b \geq 2], [b \geq 3], [b \geq 4], [b \geq 5], [a \geq 1], [a \geq 2], [a \geq 3]\}$, and explain the answer in terms of Result 6.13.

Exercise 6.21. Show that the marginal cost of a policy is not monotone in the family. For $X = \{0, 1\}^2$ and $\pi(x) = x_1 \oplus x_2$ compute the marginal cost of π to the empty family, to $\{x_1\}$ and to $\{x_1, x_2\}$, and explain why a policy can become more expensive to add after a related policy has been supported and then free once enough is supported.

Exercise 6.22. Let $X = \{0, \ldots, 11\}$ and $\Phi = \{x \bmod 2,\ x \bmod 3\}$. Compute $|X/\kappa(\Phi)|$ and the marginal cost of $x \bmod 4$ and of $x \bmod 6$. State the general rule for the marginal cost of $x \bmod p$ in terms of the moduli already present.

Chapter 7

Incremental Sufficiency and Minimization

Question. Why can answering a question once require less memory than maintaining the answer under arbitrary future updates, and how is the minimal maintaining memory computed?

Established sources. The Myhill–Nerode theorem and the minimal deterministic machine; Moore's partition-refinement algorithm and Hopcroft's improvement.

New results. A comparison of one-shot and update-maintainable sufficiency that is exact for every family of future events, with unbounded gaps (Result 7.5).

Proof obligations. None.

Depends on. Chapters 2, 4, and 5.

The previous chapters ask whether a representation can answer a question about a history that has already occurred. A system that is operating does not hold a finished history. It holds a representation and receives one event at a time, and it must produce the next representation from the current one and the event, without returning to the history. Whether a representation can be kept up to date in this way is a different property from whether it answers the question, and the two properties can separate sharply. A single bit may answer a question about every history and nevertheless be impossible to maintain with any finite memory. This chapter makes the difference exact, proves that it can be arbitrarily large, and describes the procedure that computes the least memory that does maintain a given answer.

Throughout this chapter the history set is E^*, so that every sequence of events can occur. Section 7.5 remarks on the general case.

7.1 Maintaining an answer under updates

Definition 7.1 (Maintainable representation)**.** A representation $r : \mathsf{E}^* \to R$ is *maintainable* if there is a function $u : R \times \mathsf{E} \to R$, an *update function*, with $r(ha) = u(r(h), a)$ for every history h and event a. For a set $F \subseteq \mathsf{E}^*$ of words closed under concatenation and containing ε (a *submonoid*), the representation is *F-maintainable* if $r(hw)$ depends only on $r(h)$ and w for every $w \in F$, that is, if $r(h) = r(h')$ implies $r(hw) = r(h'w)$ for all $w \in F$.

The update function reads the new event and the current representation and nothing else. An F-maintainable representation is the same requirement when the only updates that will ever occur are the words of F; for example, if only deposits will ever occur, F is the set of finite sequences of deposits.

Established result 7.2 (Maintainability is a congruence property)**.** *A representation r is maintainable if and only if* $\ker r$ *is a right congruence, that is, $h \ker r\ h'$ implies $ha \ker r\ h'a$ for every event a. It is then E^*-maintainable in the sense of Definition 7.1.*

Source: Standard; Nerode (1958).

Proof. If $r(ha) = u(r(h), a)$ and $r(h) = r(h')$ then $r(ha) = u(r(h'), a) = r(h'a)$. Conversely, if $\ker r$ is a right congruence then $u(r(h), a) = r(ha)$ is well defined on $r(\mathsf{E}^*) \times \mathsf{E}$, and extends arbitrarily to $R \times \mathsf{E}$. Invariance under a word $w = a_1 \cdots a_k$ follows by applying the congruence property k times. □

7.2 The continuation congruence

For a property $P : \mathsf{E}^* \to D$ and a submonoid F, define

$$h \approx_P^F h' \quad \Longleftrightarrow \quad P(hw) = P(h'w) \ \text{ for all } w \in F.$$

When $F = \mathsf{E}^*$ this is the continuation equivalence $\approx_P$ of Definition 4.11, since every history has every word as an extension.

New result 7.3 (Myhill–Nerode for families of future events)**.** *Let $F \subseteq \mathsf{E}^*$ be a submonoid and P a property.*

(a) $\approx_P^F$ is an equivalence relation contained in $\ker P$*, and it is F-invariant.*

(b) Every F-invariant equivalence relation contained in $\ker P$ *is contained in $\approx_P^F$.*

(c) A representation is F-maintainable and answers P if and only if $\ker r$ *is an F-invariant equivalence contained in* $\ker P$*. The least number of values of such a representation is $|\mathsf{E}^*/{\approx_P^F}|$, attained by the quotient map.*

(d) If $F \subseteq F'$ then $\approx_P^{F'} \subseteq \approx_P^F \subseteq \ker P$, so the least memory grows as the family of future events grows.

Assumptions: F is a submonoid of E^*. For $F = \mathsf{E}^*$ the statement is the Myhill–Nerode theorem for properties of histories.

Proof. (a) Reflexivity, symmetry, and transitivity hold because they hold for equality in D. Taking $w = \varepsilon \in F$ shows $\approx_P^F \subseteq \ker P$. If $h \approx_P^F h'$ and $u \in F$, then for every $w \in F$ we have $uw \in F$ and so $P(huw) = P(h'uw)$, which says $hu \approx_P^F h'u$. (b) Let $\sim$ be F-invariant with $\sim \subseteq \ker P$ and let $h \sim h'$. For each $w \in F$ we have $hw \sim h'w$, so $P(hw) = P(h'w)$. Hence $h \approx_P^F h'$. (c) The first statement is the definition of F-maintainability together with Result 5.2. If r is F-maintainable and answers P then $\ker r \subseteq \approx_P^F$ by (b), so $|r(\mathsf{E}^*)| \geq |\mathsf{E}^*/{\approx_P^F}|$ by Result 2.10; and by (a) the quotient map of $\approx_P^F$ attains this. (d) A larger set F' imposes more conditions in the definition of the relation. □

7.3 One-shot and maintained sufficiency

Definition 7.4 (One-shot and maintained size)**.** For a property P and a submonoid F, let $s_{\text{one}}(P) = |\mathsf{E}^*/\ker P|$, the least number of values of any representation that answers P, and let $s_F(P) = |\mathsf{E}^*/{\approx_P^F}|$, the least number of values of an F-maintainable representation that answers P. Write $s_{\text{inc}}(P) = s_{\mathsf{E}^*}(P)$.

By Result 7.3 we have $s_{\text{one}}(P) \le s_F(P) \le s_{\text{inc}}(P)$ for every submonoid F. The following theorem states when the inequalities are equalities and shows that they can be as strict as one pleases.

New result 7.5 (The gap between answering once and maintaining)**.** *Let P be a property with $s_{\text{one}}(P)$ finite.*

(a) For a submonoid F, $s_F(P) = s_{\text{one}}(P)$ if and only if $\ker P$ is F-invariant, that is, if and only if $P(h) = P(h')$ implies $P(hw) = P(h'w)$ for all $w \in F$.

(b) For every $n \ge 1$ there is a property P_n over a three-event alphabet with $s_{\text{one}}(P_n) = 2$ and $s_{\text{inc}}(P_n) = n + 1$.

(c) There is a property over a two-event alphabet with $s_{\text{one}} = 2$ and $s_{\text{inc}} = \infty$.

Assumptions: (a) requires $s_{\text{one}}(P) < \infty$. For (b), the alphabet is $\{\text{tick}, \text{alarm}, \text{reset}\}$; for (c), $\{\text{d}, \text{w}\}$ with unit amounts.

Proof. (a) If $\ker P$ is F-invariant then it is an F-invariant equivalence contained in $\ker P$, so $\ker P \subseteq {\approx_P^F} \subseteq \ker P$ by Result 7.3, and the two quotients coincide. Conversely, suppose $s_F(P) = s_{\text{one}}(P)$. Since ${\approx_P^F} \subseteq \ker P$ the quotient by ${\approx_P^F}$ maps onto the quotient by $\ker P$, and both are finite of the same size, so the map is a bijection and ${\approx_P^F} = \ker P$. By Result 7.3(a), $\ker P$ is F-invariant.

(b) Let $c(h)$ be the number of alarms in h since the last reset, and let $P_n(h) = \mathbf{1}[c(h) \ge n]$. Both verdicts occur, so $s_{\text{one}}(P_n) = 2$. For a word w with no reset, $c(hw) = c(h) + a(w)$ with $a(w)$ the number of alarms in w, and for a word w containing a reset, $c(hw)$ does not depend on h. Since $c(h) + a \ge n$ if and only if $\min(c(h), n) + a \ge n$ for $a \ge 0$, the residual of h is determined by $\min(c(h), n)$. Conversely suppose $j = \min(c(h), n) < j' = \min(c(h'), n)$, so $j < n$. If $j' = n$ the empty word separates h from h'. Otherwise $j' = c(h') < n$ and the word $w = \text{alarm}^{\,n-j'}$ gives $c(hw) = j + n - j' < n$ and $c(h'w) = n$, so $P_n(hw) = 0 \ne 1 = P_n(h'w)$. Therefore ${\approx_{P_n}}$ has exactly the classes of $\min(c, n) \in \{0, \dots, n\}$, and $s_{\text{inc}}(P_n) = n + 1$.

(c) Over events d and w with unit amounts let $P = q_{\text{neg}}$. As in Result 2.13, $h \approx_P h'$ if and only if $q_{\text{neg}}(h) = q_{\text{neg}}(h') = 1$ or both are 0 with equal balance, because the word $\text{w}^{\,b'}$ plays the role of the withdrawal of b'. There are infinitely many classes, one for each $b \in \mathbb{N}$ together with $\perp$, so $s_{\text{inc}}(P) = \infty$ while $s_{\text{one}}(P) = 2$. □

The theorem reduces the comparison to a single condition. A one-bit answer is maintainable with one bit exactly when its verdicts are preserved by the future events in F, in the sense that two histories with the same verdict continue to receive the same verdict. Otherwise the memory must record whatever distinguishes the histories in some future. In the account, the verdict "never overdrawn" is preserved by deposits and not by withdrawals, so the single flag is maintainable if only deposits will ever occur and fails to be maintainable otherwise.

Example 7.6 (Restricting the updates)**.** Let $F = \text{D}^*$ be the finite sequences of deposits in the account of Example 1.3. A deposit cannot create a negative prefix, since $m(w) = 0$ for

$w \in \mathrm{D}^*$, so $q_{\mathrm{neg}}(hw) = q_{\mathrm{neg}}(h)$ for $w \in F$ and $h \approx^F_{q_{\mathrm{neg}}} h'$ if and only if $q_{\mathrm{neg}}(h) = q_{\mathrm{neg}}(h')$. Thus $s_F = 2 = s_{\mathrm{one}}$: a single bit is maintainable for the family of deposit-only futures, while $s_{\mathrm{inc}} = \infty$ for the family of all futures.

The two possibilities correspond to two questions a designer can ask. If the answer is only ever needed at the present, one-shot sufficiency is the right measure, and the memory can be as small as the number of verdicts. If the answer must be available after each of a stream of events, the maintained size is the right measure, and it is determined by the family of events that the system must be able to absorb. The difference is the cost of readiness for futures that have not occurred.

Established result 7.7 (Finite maintenance and regularity)**.** *Let* E *be finite and let* $P : \mathsf{E}^* \to D$ *have finitely many values. The following are equivalent: (i)* $s_{\mathrm{inc}}(P) < \infty$*; (ii)* P *is computed by a finite machine with an output at each state, that is,* $P = \mathrm{out} \circ \delta^*(s_0, \cdot)$ *for a finite complete deterministic* $(S, s_0, \mathsf{E}, \delta)$ *and* $\mathrm{out} : S \to D$*; (iii)* $P^{-1}(d)$ *is a regular language for every* $d \in D$*.*

Source: Myhill (1957), Nerode (1958); see Hopcroft and Ullman.

Proof. (i)⇒(ii): the machine whose states are the classes of $\approx_P$, with transition $[h] \xrightarrow{a} [ha]$ and output $P(h)$, is well defined by Result 7.3. (ii)⇒(iii): each $P^{-1}(d)$ is accepted by the machine with accepting set $\mathrm{out}^{-1}(d)$. (iii)⇒(i): each $P^{-1}(d)$ has a Nerode congruence of finite index by the Myhill–Nerode theorem; the relation $\approx_P$ is the intersection of these finitely many congruences, since two histories have the same residual for P exactly when they have the same residual for each indicator $\mathbf{1}_{P^{-1}(d)}$, and a finite intersection of finite-index equivalences has finite index. □

Result 7.5(c) shows in this light that no finite machine over two events maintains the bit q_{neg}, although the bit is a function of the history with two values. The deficiency is not in the property but in the update structure: after a withdrawal the new verdict depends on a quantity, the balance, which the bit does not carry and which is unbounded.

7.4 Computing the minimal maintaining memory

When a finite machine for the property is given, the least memory is obtained by merging states that have the same behaviour. The following algorithm and its correctness are classical.

Definition 7.8 (Moore machine)**.** A *Moore machine* is a tuple $M = (S, s_0, \mathsf{E}, \delta, \mathrm{out})$ with a finite set S of states, a total transition function $\delta : S \times \mathsf{E} \to S$, and an output function $\mathrm{out} : S \to D$. The *behaviour* of a state s is the function $b_s : \mathsf{E}^* \to D$ with $b_s(w) = \mathrm{out}(\delta^*(s, w))$.

Established result 7.9 (Moore's partition refinement)**.** *Let* M *be a Moore machine with* n *states. Define* $\equiv_0 = \ker \mathrm{out}$ *and, for* $i \geq 0$*,* $s \equiv_{i+1} t$ *if and only if* $s \equiv_i t$ *and* $\delta(s, a) \equiv_i \delta(t, a)$ *for every* $a \in \mathsf{E}$*.*

(a) $s \equiv_i t$ *if and only if* $b_s(w) = b_t(w)$ *for every word* w *of length at most* i*.*

(b) The sequence stabilizes at some $i \leq n - 1$*, and the stable relation* $\equiv$ *satisfies* $s \equiv t$ *if and only if* $b_s = b_t$*.*

(c) The quotient of the reachable part of M *by* $\equiv$ *is a Moore machine computing the same function* $P_M = \mathrm{out} \circ \delta^*(s_0, \cdot)$ *with exactly* $|\mathsf{E}^*/\approx_{P_M}|$ *states, which is the least possible number of states of any machine computing* P_M*.*

(d) Each round can be carried out in time $O(n|\mathsf{E}|)$ by sorting transition signatures, so the minimal machine is obtained in time $O(n^2|\mathsf{E}|)$. Hopcroft's algorithm achieves $O(n|\mathsf{E}|\log n)$.

Source: Moore, "Gedanken-experiments on sequential machines" (1956); Hopcroft, "An $n \log n$ algorithm for minimizing states in a finite automaton" (1971).

Proof. (a) For $i = 0$ the statement is the definition. If it holds for i, then $s \equiv_{i+1} t$ says that the behaviours agree on words of length at most i and that the successors $\delta(s,a)$ and $\delta(t,a)$ have behaviours that agree on words of length at most i for every a, which together say that b_s and b_t agree on all words of length at most $i+1$. (b) The relations are decreasing, $\equiv_{i+1} \subseteq \equiv_i$. If $\equiv_{i+1} = \equiv_i$ then $\equiv_{i+2} = \equiv_{i+1}$, because each is obtained from the previous by the same rule, so the sequence is constant from that point. Before stabilization each step strictly increases the number of classes, which lies between 1 and n, so stabilization occurs by index $n-1$. By (a), the stable relation is the intersection over all i, which is equality of behaviours. (c) For a reachable state $s = \delta^*(s_0, h)$ the behaviour b_s is the residual $(P_M)_h$, so $h \approx_{P_M} h'$ if and only if their states have equal behaviour, that is, if and only if their states are $\equiv$-equivalent. The classes of $\approx_{P_M}$ therefore correspond to the classes of $\equiv$ on reachable states, and minimality is Result 4.13(b). (d) Each round recomputes classes by comparing the pair consisting of the current class and the tuple of successor classes, which can be done by one pass of radix sorting; the bound on the number of rounds is (b). □

Example 7.10 (Merging states)**.** Let $\mathsf{E} = \{a\}$ and let M have states $0, \dots, 5$ with $\delta(i,a) = \min(i+1, 5)$, initial state 0, and $\mathrm{out}(i) = \mathbf{1}[i \geq 3]$. Then $\equiv_0$ has classes $\{0,1,2\}$ and $\{3,4,5\}$. In the next round 2 separates from 0 and 1 because $\delta(2,a) = 3$ lies in the other class, giving $\{0,1\}, \{2\}, \{3,4,5\}$, and in the round after, 1 separates from 0. The relation then stabilizes at $\{0\}, \{1\}, \{2\}, \{3,4,5\}$, so the minimal machine has four states, the same as $\approx_{P_n}$ for $n = 3$ restricted to words over {alarm}.

7.5 Families and the general history set

Established result 7.11 (Maintaining a family)**.** *For a family $\Phi = \{\pi_i\}_{i \in I}$ of properties of E^*, the least number of values of a maintainable representation sufficient for Φ is $|\mathsf{E}^* / \bigcap_i \approx_{\pi_i}|$. The bounds of Result 6.5 hold with $\approx_{\pi_i}$ in place of* $\ker \pi_i$.

Source: Results 7.3 and 6.5.

Proof. Apply Result 7.3 to the property $x \mapsto (\pi_i(x))_i$. The residual of the tuple at h is the tuple of residuals, so $\approx_{(\pi_i)_i} = \bigcap_i \approx_{\pi_i}$. □

When the history set $\mathcal{H}$ is not all of E^*, a representation must also determine which events are possible, since otherwise it cannot know which updates it must perform. This is the requirement $\ker r \subseteq \{(h,h') : \mathrm{Ext}(h) = \mathrm{Ext}(h')\}$ built into the definition of continuation equivalence in Definition 4.11, and Results 4.12 and 4.13 give the same conclusions in that setting. Physical constraints therefore enter the memory requirement twice: they remove futures that need not be accommodated, and they impose that the representation encode what remains possible.

7.6 A rate limit: one-shot against maintained memory

A rate limit is the simplest rule in which the answer to the present question is a single bit and the memory needed to keep that bit correct under further events grows without bound in a parameter of the rule. Take the events use and idle and the history set $\mathcal{H}_w$ of words in which any two uses are at least w steps apart, that is, at most one use occurs in any w consecutive steps. The set is prefix-closed, and the question asked of a history is whether a further use is permitted, $Q(h) = [h\,\text{use} \in \mathcal{H}_w]$. Only two verdicts occur, so $s_{\text{one}}(Q) = 2$.

New result 7.12 (A single-use rate limit)**.** *For every $w \geq 2$ the continuation equivalence of the history set $\mathcal{H}_w$ has exactly w classes, given by the number $\ell(h)$ of events since the last use, capped at $w-1$, with the value $w-1$ also assigned to histories with no use. The one-shot size of Q is 2 and the maintained size is w, so the ratio is unbounded as w grows. In particular no maintainable representation of Q over $\mathcal{H}_w$ has fewer than w values, and the counter $\min(\ell, w-1)$ attains the bound.*

Assumptions: The history set is $\mathcal{H}_w$, so the representation must also determine which extensions are possible (Definition 4.11); $w \geq 2$.

Proof. A use after h is possible exactly when its position exceeds the position of the last use by at least w, that is, when $\ell(h) \geq w-1$, where $\ell(h)$ is the number of events after the last use (and $\ell = \infty$ if there is none). After a further idle event ℓ increases by one, and after a use it becomes 0. So $\text{Ext}(h)$ depends only on $c(h) = \min(\ell(h), w-1)$, and c is updated by $c \mapsto \min(c+1, w-1)$ for idle and $c \mapsto 0$ for use; the equivalence given by c is therefore a right congruence contained in the continuation equivalence, with at most w classes. To see that the classes are separated, let $c(h) = j < c(h') = j'$ with $j \leq w-2$. The word $\text{idle}^{\,w-2-j}\,\text{use}$ is an extension of h', because after the idles the counter of h' is at least $j' + w - 2 - j \geq w - 1$, and it is not an extension of h, because after the idles the counter of h equals $w-2$. Hence $\text{Ext}(h) \neq \text{Ext}(h')$. Finally Q is a function of Ext, so the equivalence is the continuation equivalence of Q on $\mathcal{H}_w$ and $s_{\text{inc}}(Q) = w$. □

The result puts numbers on the remark of Chapter 4 that the minimal system must retain what remains possible. A rate limit of w steps requires $\log_2 w$ bits of clock even though every question the system is asked is a yes or no. The clock cannot be replaced by any smaller description that remains correct under all further events; it can be replaced by a smaller one only if the future events are restricted, for instance to sequences with no idles, in the sense of Result 7.5(a). For a limit of k uses in w steps the count of classes is $\binom{w}{k}$ for $1 \leq k < w$, a figure that has been confirmed here by exhaustive computation for $w \leq 6$ and that is proved, with the normal form that realizes it, in Chapter 7 of the second volume. The case $k = 1$ above is the part that belongs to the theory of what must be retained, since it needs nothing beyond the continuation equivalence, and the general case, which is a question about the structure of the dynamics, is left to that chapter.

Exercises

Exercise 7.13. Construct a maintainable representation of the monitor of Example 1.4 that answers "at least n alarms are outstanding", and give its update function explicitly. State its number of values.

Exercise 7.14. Prove that your representation answers the question and that no maintainable representation answering it has fewer values, using Result 7.3.

Exercise 7.15. Show that the one-bit representation q_{neg} of the account answers the question of whether an overdraft has occurred and is not maintainable, by exhibiting histories h, h' and an event a with $q_{\mathrm{neg}}(h) = q_{\mathrm{neg}}(h')$ and $q_{\mathrm{neg}}(ha) \neq q_{\mathrm{neg}}(h'a)$. Determine the smallest event alphabet over which this failure occurs.

Exercise 7.16. The threshold of the monitor is revised from n to $n' > n$. Determine the least number of values of a maintainable representation sufficient for both thresholds, using Result 7.11, and compare it with the least number for each threshold alone. Describe in words what the memory of the first system lacks when the revision arrives.

Exercise 7.17. Run Moore's algorithm on the machine with states $\{0, \ldots, 6\}$, events $\{a, b\}$, $\delta(i, a) = \min(i + 1, 6)$, $\delta(i, b) = 0$, and output $\mathbf{1}[i \geq 4]$. List the relations $\equiv_0, \equiv_1, \ldots$ until stabilization and give the number of states of the minimal machine. Explain why the answer agrees with $s_{\mathrm{inc}}(P_4)$.

Exercise 7.18. In the account with all deposit and withdrawal amounts, let $F = \mathrm{D}^*$ and let $P(h) = \mathbf{1}[\beta(h) \geq k]$. Compute $\approx_P^F$ and $s_F(P)$, and show that $s_F(P)$ is infinite although $s_{\mathrm{one}}(P) = 2$. Explain why this does not contradict Example 7.6.

Exercise 7.19. For $w = 4$ list the four classes of Result 7.12 by a representative history each, and give for every pair of classes a shortest separating extension. Run Moore's algorithm of Result 7.9 on the machine whose states are the counter values and confirm that it does not merge any states.

Exercise 7.20. Replace the rule by a lifetime cap of m uses with no clock. Compute the continuation equivalence of the resulting history set and its number of classes, and show that the maintained size is $m+1$ while the one-shot size of the question "is another use permitted?" remains 2.

Exercise 7.21. Combine the single-use rate limit of window w with the lifetime cap of m uses. Show that the number of classes is $(m - 1)w + 2$ for $m \geq 1$, and confirm the value 5 for $w = 3$, $m = 2$ by enumerating histories. Explain why the combination is cheaper than the product $w(m + 1)$ of the two separate counts.

Chapter 8

Bisimulation, Products, and State Complexity

Question. When do separate summaries compose efficiently, and when does their interaction produce a large state space?

Established sources. Bisimulation (Park, Milner); the product construction; the state complexity of intersection (Yu, Zhuang, and Salomaa).

New results. Compositional sufficiency under separability: the minimal memory for independent properties is the product (Result 8.11); sufficiency composes through functions but minimality does not (Result 8.12).

Proof obligations. None.

Depends on. Chapters 4 and 7.

The theory of the previous chapter computes the least memory that maintains a single property. A system is rarely asked to maintain one property. It maintains several, each with its own summary, and the combined requirement can be either the sum of the parts or something very different. This chapter gives the tools for deciding which. It begins with the notion of behavioural equivalence that underlies minimization, in the stronger form called bisimulation that remains meaningful when a representation is not deterministic. It then treats the product of summaries, states a separability condition under which the product is exactly the minimal combined memory, and shows by example how both the product bound and the lower bound can be attained and how interaction between properties destroys the efficiency of composing summaries.

8.1 Behavioural equivalence and bisimulation

Definition 8.1 (Bisimulation of Moore machines)**.** Let $M_1 = (S_1, s_1, \mathsf{E}, \delta_1, \mathrm{out}_1)$ and $M_2 = (S_2, s_2, \mathsf{E}, \delta_2, \mathrm{out}_2)$ be Moore machines with the same output set. A relation $B \subseteq S_1 \times S_2$ is a *bisimulation* if $(s,t) \in B$ implies $\mathrm{out}_1(s) = \mathrm{out}_2(t)$ and $(\delta_1(s,a), \delta_2(t,a)) \in B$ for every $a \in \mathsf{E}$. States s and t are *bisimilar* if some bisimulation contains (s,t).

Established result 8.2 (For deterministic machines, bisimilarity is behavioural equivalence)**.** *States $s \in S_1$ and $t \in S_2$ are bisimilar if and only if $b_s = b_t$.*

Source: Park, "Concurrency and automata on infinite sequences" (1981); Milner, *Communication and Concurrency* (1989).

Proof. If $(s,t) \in B$ for a bisimulation B, then by induction on $|w|$ we have $(\delta_1^*(s,w), \delta_2^*(t,w)) \in B$, so the outputs after w agree and $b_s(w) = b_t(w)$. Conversely the relation $B = \{(s,t) : b_s = b_t\}$ is a bisimulation. If $b_s = b_t$ then $\mathrm{out}_1(s) = b_s(\varepsilon) = b_t(\varepsilon) = \mathrm{out}_2(t)$, and $b_{\delta_1(s,a)}(w) = b_s(aw) = b_t(aw) = b_{\delta_2(t,a)}(w)$ for all w. It contains (s,t). □

Together with Result 7.9 this shows that the stable partition computed by Moore's algorithm is the greatest bisimulation of the machine with itself, and that the minimal machine for a property is the quotient of any machine computing it by bisimilarity. For deterministic representations nothing finer than the continuation equivalence of Chapter 4 is available.

The distinction between behavioural equivalence and bisimulation appears when a representation does not determine the next state. A *labelled transition system* is a triple $(S, \mathsf{E}, \to)$ with $\to \subseteq S \times \mathsf{E} \times S$. A relation B on its states is a bisimulation if whenever $(s,t) \in B$ and $s \xrightarrow{a} s'$ there is t' with $t \xrightarrow{a} t'$ and $(s',t') \in B$, and symmetrically for moves of t. The *traces* of a state are the words the system can perform from it.

Example 8.3 (Same traces, different commitments)**.** Let $p_0 \xrightarrow{a} p_1$, $p_1 \xrightarrow{b} p_2$, $p_1 \xrightarrow{c} p_3$, and let $q_0 \xrightarrow{a} q_1$, $q_0 \xrightarrow{a} q_2$, $q_1 \xrightarrow{b} q_3$, $q_2 \xrightarrow{c} q_4$. Both p_0 and q_0 have the traces ε, a, ab, and ac. They are not bisimilar: the move $q_0 \xrightarrow{a} q_1$ must be matched by $p_0 \xrightarrow{a} p_1$ with p_1 bisimilar to q_1, but p_1 can perform c and q_1 cannot. The first system defers the choice between b and c until after a, and the second has committed to one of them when a occurs.

The example matters for representation because a retained representation that does not determine the future behaviour of a system is in the position of the second system: it has resolved something that the first has not. Two representations can agree on every sequence of observations and still differ in whether they permit the same later choices. Chapter 19 takes this up for distributed systems. In the deterministic setting of the rest of this chapter the issue does not arise.

8.2 Products of summaries

Definition 8.4 (Product machine)**.** For Moore machines M_1 and M_2 over the same alphabet, the *product* is $M_1 \times M_2 = (S_1 \times S_2, (s_1,s_2), \mathsf{E}, \delta, \mathrm{out})$ with $\delta((s,t),a) = (\delta_1(s,a), \delta_2(t,a))$ and $\mathrm{out}(s,t) = (\mathrm{out}_1(s), \mathrm{out}_2(t))$.

Established result 8.5 (The product computes the pair)**.** *The product computes the property $h \mapsto (P_{M_1}(h), P_{M_2}(h))$. If the minimal machines for P_{M_1} and P_{M_2} have m and n states, then the minimal machine for the pair has at least $\max(m,n)$ and at most mn states. The same bound mn applies to the minimal machine for any property of the form $g(P_{M_1}, P_{M_2})$.*

Source: Standard product construction; Yu, Zhuang, and Salomaa, "The state complexities of some basic operations on regular languages," *Theoretical Computer Science* 125 (1994), on the operation of intersection.

Proof. By induction on length the product reaches $(\delta_1^*(s_1,h), \delta_2^*(s_2,h))$ after h, so it computes the pair. The bounds on the minimal machine follow from Results 7.11 and 6.5, applied to the

minimal machines. For $g(P_1, P_2)$, the continuation equivalence of the pair is contained in that of $g(P_1, P_2)$, as shown in Result 8.6 below, and the pair has at most mn classes. □

Established result 8.6 (Post-processing never increases memory)**.** *For any property P and function g, $\approx_P \subseteq \approx_{g\circ P}$. Consequently $s_{\mathrm{inc}}(g \circ P) \leq s_{\mathrm{inc}}(P)$, and a maintainable representation sufficient for P is sufficient for $g \circ P$.*

Source: Elementary.

Proof. If $P(hw) = P(h'w)$ for all w then $g(P(hw)) = g(P(h'w))$ for all w. The consequence follows from Result 7.3, together with Result 2.10. □

A lower bound for the number of classes of $\approx_P$ is obtained by exhibiting histories that the future separates.

Established result 8.7 (Distinguishing sets)**.** *A set $\{h_1, \ldots, h_k\}$ of histories is* pairwise distinguishable *for P if for all $i \neq j$ there is a word w with $P(h_i w) \neq P(h_j w)$. Then $|\mathsf{E}^*/\approx_P| \geq k$, and $|\mathsf{E}^*/\approx_P|$ is the supremum of the sizes of pairwise distinguishable sets.*

Source: Elementary; the Myhill–Nerode lower bound technique.

Proof. Two histories in the same class of $\approx_P$ cannot be distinguished by any word, so the elements of a pairwise distinguishable set lie in distinct classes. A set containing one history from each class is pairwise distinguishable. □

8.3 When summaries compose, and when they do not

Example 8.8 (Independent counters: the product is necessary)**.** Let $\mathsf{E} = \{a, b\}$, let $P_1(h) = \mathbf{1}[\#_a(h) \equiv 0 \bmod m]$, and let $P_2(h) = \mathbf{1}[\#_b(h) \equiv 0 \bmod n]$, with $\#_a$ and $\#_b$ the numbers of occurrences. Each has m and n classes respectively. The conjunction $P = P_1 \wedge P_2$ is a single bit, yet $|\mathsf{E}^*/\approx_P| = mn$. Indeed let $h = a^x b^y$ and $h' = a^{x'} b^{y'}$ with $(x, y) \neq (x', y')$ modulo (m, n), and let $w = a^{(-x) \bmod m} b^{(-y) \bmod n}$. Then hw satisfies P, and $h'w$ satisfies P only if $x' \equiv x$ and $y' \equiv y$. So the mn histories $a^x b^y$ are pairwise distinguishable and Result 8.7 gives the lower bound; the product construction gives the matching upper bound. The one-bit answer costs mn states to maintain.

Example 8.9 (Nested properties: no growth)**.** If $P_2 = g \circ P_1$ then by Result 8.6 the pair (P_1, P_2) has the same continuation equivalence as P_1, so the combined memory equals that of P_1 alone. Escalation at 2 and at 3 outstanding alarms in the monitor of Example 1.4 are not of this form, but both are functions of the capped count $\min(c, 3)$, and the pair needs the four values of that quantity, not the twelve of the product of the two separate machines.

Example 8.10 (Interaction: the combined question is cheaper than separate summaries)**.** Let $\mathsf{E} = \{a, b\}$ and $P(h) = \mathbf{1}[\#_a(h) \equiv \#_b(h) \bmod m]$. The class of h is determined by $(\#_a(h) - \#_b(h)) \bmod m$: if $\delta = (\#_a - \#_b) \bmod m$ and δ' are different, the word $a^{(-\delta) \bmod m}$ separates h from h', so there are exactly m classes. The separate summaries of the two counts, $\#_a \bmod m$ and $\#_b \bmod m$, each have m values, and their pair has m^2. Of those m^2 values the question uses only the difference.

The three examples display the range. Composition is free when one property is a function of another, multiplicative when the properties depend on unrelated parts of the history, and wasteful when the combined question depends on only part of what the separate summaries record. A designer who builds one summary per property and combines them pays the product regardless; the true requirement is the continuation equivalence of the combined question.

8.3.1 Separability

The multiplicative case can be characterized exactly.

New result 8.11 (Independent properties: the product is exactly the minimal memory)**.** *Let* $\mathsf{E} = \mathsf{E}_1 \sqcup \mathsf{E}_2$ *be a partition of the alphabet and let* $\mathrm{proj}_i : \mathsf{E}^* \to \mathsf{E}_i^*$ *be the map that erases the events outside* E_i*. Suppose* $P_i = Q_i \circ \mathrm{proj}_i$ *for properties* Q_i *of* E_i^**. Then*

$$\left|\mathsf{E}^*/{\approx_{(P_1,P_2)}}\right| = \left|\mathsf{E}_1^*/{\approx_{Q_1}}\right| \cdot \left|\mathsf{E}_2^*/{\approx_{Q_2}}\right|,$$

and the map $h \mapsto ([\mathrm{proj}_1 h], [\mathrm{proj}_2 h])$ *is a minimal maintainable representation of the pair.*

Assumptions: The alphabet is split into two parts and each property depends only on the subsequence of events from its own part. The cardinalities may be infinite.

Proof. Since proj_1 is a monoid homomorphism, $P_1(hw) = Q_1(\mathrm{proj}_1(h)\,\mathrm{proj}_1(w))$, and as w ranges over E^* the word $\mathrm{proj}_1(w)$ ranges over all of E_1^*. Hence $h \approx_{P_1} h'$ if and only if $\mathrm{proj}_1(h) \approx_{Q_1} \mathrm{proj}_1(h')$, and likewise for the second property. By Result 7.11, $h \approx_{(P_1,P_2)} h'$ if and only if both conditions hold. The map $\Theta : [h] \mapsto ([\mathrm{proj}_1 h], [\mathrm{proj}_2 h])$ is therefore well defined and injective. It is surjective: given classes c_1 and c_2 choose $v_1 \in c_1$ and $v_2 \in c_2$, and the concatenation $h = v_1 v_2$ satisfies $\mathrm{proj}_1(h) = v_1$ and $\mathrm{proj}_2(h) = v_2$ because v_2 contains no event of E_1 and v_1 none of E_2. Hence Θ is a bijection and the cardinalities multiply. The map is a maintainable representation by Result 7.11, and minimal by the count. □

Example 8.8 is the case $\mathsf{E}_1 = \{a\}$, $\mathsf{E}_2 = \{b\}$. Example 8.10 violates the hypothesis, since P depends on both counts jointly, and the multiplicative formula would give the wrong answer m^2. The hypothesis is sufficient and not necessary: the product formula can hold for properties that are not separable, but without separability it is no longer guaranteed, and in general only the bounds of Result 8.5 are available.

8.3.2 Composition of maintained answers

New result 8.12 (Sufficiency composes through functions; minimality does not)**.** *(a) If* r_1 *and* r_2 *are maintainable representations sufficient for* P_1 *and* P_2*, then the pair* $\langle r_1, r_2 \rangle$ *is maintainable and sufficient for* $g(P_1, P_2)$*, for every function* g*, with at most* $|r_1(\mathsf{E}^*)| \cdot |r_2(\mathsf{E}^*)|$ *values.*

(b) For every $m \geq 2$ *there are properties* P_1, P_2 *and a function* g *such that* P_1 *and* P_2 *each have exactly* m *continuation classes, the pair of their minimal maintainable representations has* m^2 *values, and the minimal maintainable representation of* $g(P_1, P_2)$ *has* m *values.*

Assumptions: (a) concerns arbitrary properties of E^*. (b) is over $\mathsf{E} = \{a, b\}$.

Proof. (a) The kernel of the pair is $\ker r_1 \cap \ker r_2$, an intersection of right congruences and hence a right congruence, so the pair is maintainable by Result 7.2. It is contained in $\ker P_1 \cap \ker P_2$, so it is sufficient for the pair (P_1, P_2) and therefore for $g(P_1, P_2)$. The bound on values is the cardinality of a product. (b) Let $P_1(h) = \#_a(h) \bmod m$, $P_2(h) = \#_b(h) \bmod m$, and $g(x, y) = \mathbf{1}[x = y]$. Then $\approx_{P_1}$ has the m classes of $\#_a \bmod m$, separated by words a^i, and likewise for P_2. By Example 8.8 and Result 8.11, the pair has m^2 classes. The property $g(P_1, P_2)$ is the property of Example 8.10, which has m classes. □

Part (b) is the formal statement that maintaining a combined question by maintaining its parts is, in general, strictly more expensive than maintaining the combined question directly. It is the first instance of a theme that recurs in Part IV: the representation required for an obligation is the continuation quotient of the obligation, which in general is not recoverable from the quotients of its constituents.

8.4 Shared events and the limits of independence

Result 8.11 requires that each property read only its own part of the alphabet. Real systems share events: a single reset, a single clock tick, a single audit touches several counters at once. The product formula survives such sharing under a condition on reachability, and the condition separates the architectures that look alike on paper.

New result 8.13 (Products with shared events)**.** *Let $\mathsf{E} = \mathsf{E}_1 \sqcup \mathsf{E}_2 \sqcup R$ and let M_1, M_2 be minimal Moore machines for properties P_1, P_2, where M_i reads the letters of $\mathsf{E}_i \cup R$ and ignores (acts as the identity on) the letters of the other part. Suppose that every state of M_i is reached from its initial state by a word over E_i alone. Then the pair (P_1, P_2) has exactly $|S_1| \cdot |S_2|$ continuation classes, where S_i is the state set of M_i.*

Assumptions: The two machines are minimal; letters of E_{3-i} act as the identity on M_i; every state of each machine is reachable by letters of its own part.

Proof. The product machine reaches the pair (s, t) after the history uv, where $u \in \mathsf{E}_1^*$ reaches s in M_1 and $v \in \mathsf{E}_2^*$ reaches t in M_2: the word u leaves M_2 at its initial state and v leaves M_1 where u put it. So all $|S_1||S_2|$ pairs are reachable. Two different pairs are separated: if $s \neq s'$ there is a word w over $\mathsf{E}_1 \cup R$ on which the outputs of s and s' in M_1 differ, because M_1 is minimal; the outputs of the pair machine on w differ in the first coordinate. The same holds in the second coordinate. By Result 7.11 the number of classes of the pair is the number of pairwise distinguishable reachable pairs, which is $|S_1||S_2|$. □

The shared set R may include resets, and a reset that clears both components does not reduce the count, because the pairs are reached before the reset occurs. What the result does not permit is a shared letter in the reachability words. The monitor shows the contrast with exact numbers. Let $P_1 = [c_a \geq 2]$ and $P_2 = [c_b \geq 3]$, where c_a and c_b count alarms of two kinds since the last reset. With separate alarm events a and b and no reset, with a reset that clears both counters, and with a reset that clears only c_a, the pair has $3 \cdot 4 = 12$ classes in each case, as the result predicts; the values have been confirmed by exhaustive comparison of futures of length up to seven. If instead the same alarm event feeds both counters, so that $P_1 = [c \geq 2]$ and $P_2 = [c \geq 3]$ with c a single count, then the pair has only 4 classes, since both

are thresholds of one capped count in the sense of Result 6.6. The architecture, which letters feed which counters, determines the memory, and the properties alone do not.

Example 8.14 (Reachability can fail)**.** Let a single letter step advance both of two modular counters modulo 2 and modulo 4 at once, so that $\mathsf{E}_1 = \mathsf{E}_2 = \varnothing$ and $R = \{\text{step}\}$. The hypothesis of Result 8.13 fails, since the state 1 of the modulo-2 counter is not reached by a word over the empty part, and the pair has $\operatorname{lcm}(2, 4) = 4$ classes, not $2 \cdot 4 = 8$. Sharing the driving event correlates the counters, and the correlation is arithmetic.

Exercises

Exercise 8.15. Construct the product machine of the monitor machines for "at least 2 alarms outstanding" and "at least 3 alarms outstanding" (three and four states respectively). Count its reachable states, minimize it using Moore's algorithm, and compare with the bound mn of Result 8.5.

Exercise 8.16. Using Result 8.7, show that the property "$\#_a + \#_b$ is even" over $\{a, b\}$ has exactly two classes although the separate parity summaries of $\#_a$ and $\#_b$ have four joint values, and identify which function of the pair of summaries computes it.

Exercise 8.17. Let $P_1(h) = \#_a(h) \bmod 2$ and $P_2(h) = (\#_a(h) + \#_b(h)) \bmod 3$ over $\{a, b\}$. Show that the hypothesis of Result 8.11 fails for this pair, and nevertheless that $|\mathsf{E}^*/{\approx_{(P_1,P_2)}}| = 6 = 2 \cdot 3$. Explain why a sufficient condition may hold without being necessary, and find the structure that makes the product formula correct here.

Exercise 8.18. Prove that the two nondeterministic systems of Example 8.3 have the same traces and are not bisimilar. Show that every deterministic system with the same traces has a single state after the event a, and that this state can perform both b and c, and conclude that determinization identifies p_0 with q_0.

Exercise 8.19. A system maintains the pair of questions "at least 3 alarms outstanding" and "the alarm count since reset is a multiple of 4" on the monitor. Compute the continuation classes of each question, the continuation classes of the pair, and the minimal memory of the conjunction. Compare the four numbers with the bounds of Result 8.5.

Exercise 8.20. The family of questions of the previous exercise is revised by adding the question "at least 5 ticks have occurred since the last alarm". Use Result 8.11 to determine whether the new question is independent of the first two under a suitable partition of the alphabet, and compute the memory of the revised family.

Exercise 8.21. Verify Result 8.13 for the counters of the monitor with thresholds 2 and 3 by listing the twelve reachable pairs, one representative history each, and a separating extension for each pair of the form (s, t) against (s', t) with $s \neq s'$.

Exercise 8.22. Show that Result 8.13 fails if the machines are not required to be minimal, by exhibiting non-minimal machines for the same properties whose pair machine has more reachable states than there are classes. State which part of the proof uses minimality.

Exercise 8.23. In Example 8.14 replace the moduli by 3 and 4 and by 4 and 6. Compute the number of classes of the pair in each case, and compare with the product of the moduli.

Part III

Memory, compression, and verification

Chapter 9

Exact, Approximate, and Probabilistic Preservation

Question. How does exact logical sufficiency differ from high average predictive accuracy?

Established sources. Fano's inequality and the data-processing inequality (Cover and Thomas); Shannon's rate–distortion function; Bayes decision rules.

New results. Exact sufficiency is the only notion of preservation that holds under every distribution of cases (Result 9.2); approximate sufficiency is monotone under coarsening and its errors add across policies (Result 9.8).

Proof obligations. None.

Depends on. Chapters 5 and 6.

Chapters 5 and 6 asked for representations from which a policy can be recovered exactly in every case. Practical compression rarely meets that standard. A summary is judged by how often the decision made from it agrees with the decision that the full case would have produced, and the judgement is made on the cases that happen to arrive. This chapter compares the two standards. The comparison shows that average accuracy and exact sufficiency are different kinds of property: the first is relative to a distribution of cases and the second is not, and a representation that is nearly perfect on one distribution can be useless on another that is supported on the cases it discarded.

9.1 Error of the best decision rule

Throughout this chapter X is finite, μ is a probability mass function on X, and $\pi : X \to A$ is a policy. For a representation $r : X \to R$ and a decision rule $d : r(X) \to A$, the *error* of d is $\mu\{x : d(r(x)) \neq \pi(x)\}$.

Definition 9.1 (Bayes error of a representation)**.** The *Bayes error* of r for π under μ is

$$\operatorname{err}_\mu(r, \pi) \;=\; \min_d \, \mu\{x : d(r(x)) \neq \pi(x)\} \;=\; 1 - \sum_{y \in r(X)} \max_{a \in A} \mu\big(r^{-1}(y) \cap \pi^{-1}(a)\big).$$

The representation is *ε-sufficient* for π under μ if $\operatorname{err}_\mu(r, \pi) \leq \varepsilon$.

The second expression is the Bayes decision rule: within each class of r the best rule chooses the action carrying the most probability mass. The minimum is attained because each class is handled independently. When $\mathrm{err}_\mu(r,\pi) = 0$ and μ charges every case, the best rule is correct everywhere, so r is sufficient in the sense of Definition 5.1. The next result records exactly how the exact and probabilistic standards relate.

New result 9.2 (Exact preservation versus average accuracy)**.** *Let r be a representation of a finite set X and let π be a policy.*

(a) If r is sufficient for π then $\mathrm{err}_\mu(r,\pi) = 0$ for every μ.
(b) If $\mathrm{err}_\mu(r,\pi) = 0$ for some μ with $\mu(x) > 0$ for all x, then r is sufficient for π.
(c) If r is not sufficient for π, then for every $\varepsilon > 0$ there is a μ charging every case with $0 < \mathrm{err}_\mu(r,\pi) < \varepsilon$.
(d) If r is not sufficient for π, then every decision rule d satisfies $\sup_\mu \mu\{d \circ r \neq \pi\} = 1$.

Assumptions: X is finite. Distributions are arbitrary in (a), (c), and (d).

Proof. (a) A decision rule d with $\pi = d \circ r$ has zero error under every μ. (b) If the best rule has error zero and every case has positive mass, then no case can be misclassified, so $\pi = d \circ r$. (c) Because r is not sufficient there are $x_0 \neq x_1$ with $r(x_0) = r(x_1)$ and $\pi(x_0) \neq \pi(x_1)$. Let $\mu(x_0) = 1 - \varepsilon/2$ and spread the remaining mass $\varepsilon/2$ over $X \setminus \{x_0\}$ with every case receiving a positive amount. The rule that answers $\pi(x_0)$ on the class of x_0, and the best rule on the other classes, errs only outside x_0, so the Bayes error is at most $\varepsilon/2 < \varepsilon$. It is positive by (b). (d) If $d \circ r$ agreed with π at every case then r would be sufficient. Hence there is a case x with $d(r(x)) \neq \pi(x)$, and the point mass at x gives error 1. □

Parts (c) and (d) express the gap between the two standards. A representation can be arbitrarily accurate on a distribution and still fail on a different distribution carried by the cases it discarded. The error that a deployed system sees depends on the distribution of arriving cases, which in the settings of this volume is itself shaped by the obligations and physical constraints that select which histories occur.

Example 9.3 (A rare case)**.** Let $X = \{0, 1, \ldots, N\}$, let π be the indicator of the single case N, and let r be constant. If $\mu(N) = p$ is small, the best rule always answers 0 and has error p. The same rule has error 1 under the point mass at N. A system that is audited only on typical cases cannot distinguish this representation from a sufficient one, and the audit has no bearing on the case in which the retained summary matters most.

9.2 Information-theoretic bounds

Information theory supplies the standard quantitative tools. Logarithms are to base 2. For random variables Z and Y with finite ranges, $H(Z \mid Y)$ is the conditional entropy and $I(Z;Y)$ is the mutual information.

Established result 9.4 (Fano's inequality)**.** *Let Z take values in a finite set A and let $\hat{Z} = g(Y)$ be any estimator of Z from Y. If $P_e = \Pr[\hat{Z} \neq Z]$ then*

$$H(Z \mid Y) \;\leq\; h(P_e) + P_e \log(|A| - 1), \qquad \textit{hence} \qquad P_e \;\geq\; \frac{H(Z \mid Y) - 1}{\log |A|},$$

where h is the binary entropy function.

Source: Fano (1961); Cover and Thomas, *Elements of Information Theory*, Theorem 2.10.1.

Established result 9.5 (Data-processing inequality)**.** *If Z, Y, W form a Markov chain $Z \to Y \to W$, in particular if $W = f(Y)$ is a function of Y, then $I(Z;W) \le I(Z;Y)$, equivalently $H(Z \mid W) \ge H(Z \mid Y)$.*

Source: Cover and Thomas, Theorem 2.8.1.

Applied to the present setting, take X distributed according to μ, $Z = \pi(X)$, and $Y = r(X)$. Then $\operatorname{err}_\mu(r,\pi) \ge (H(\pi(X) \mid r(X)) - 1)/\log|A|$. A representation whose conditional entropy of the policy is large cannot be accurate, and by the data-processing inequality coarsening can only increase that entropy. For the exact notion the relevant fact is the following.

New result 9.6 (Zero conditional entropy characterizes sufficiency)**.** *If μ charges every case, then r is sufficient for π if and only if $H(\pi(X) \mid r(X)) = 0$.*

Assumptions: X is finite and μ has full support.

Proof. $H(Z \mid Y) = \sum_y \Pr[Y = y]\, H(Z \mid Y = y)$, and each term vanishes if and only if Z is almost surely constant given $Y = y$. With full support, almost surely means for every case in the class, so π is constant on each class of r, which is the inclusion $\ker r \subseteq \ker \pi$. □

The result is a reminder that entropy-based measures are coarse versions of the kernel condition. They agree with exact sufficiency at zero and say nothing about which cases are affected when the value is positive but small.

Remark 9.7. Shannon's rate–distortion function $R(D) = \min I(X;\hat{X})$ over channels with expected distortion at most D is also an average-case quantity. It shows that representations below the size required for exact recovery exist whenever $D > 0$, and it does not provide a guarantee for any particular case. Result 9.2(d) shows why: the guarantee for a fixed decision rule disappears as soon as the distribution is allowed to move.

9.3 Approximate sufficiency is monotone and its errors add

New result 9.8 (Behaviour of approximate sufficiency)**.** *(a) If $r' = f \circ r$ then $\operatorname{err}_\mu(r',\pi) \ge \operatorname{err}_\mu(r,\pi)$ for every μ and π.*

(b) For policies π_1, π_2 and the pair policy $\pi = (\pi_1, \pi_2)$, $\operatorname{err}_\mu(r,\pi) \le \operatorname{err}_\mu(r,\pi_1) + \operatorname{err}_\mu(r,\pi_2)$, and $\operatorname{err}_\mu(r,\pi) \ge \max_i \operatorname{err}_\mu(r,\pi_i)$.

(c) The upper bound in (b) is attained: for every $0 < p < \frac{1}{4}$ there are X, μ, r, π_1, π_2 with $\operatorname{err}_\mu(r,\pi_i) = p$ and $\operatorname{err}_\mu(r,(\pi_1,\pi_2)) = 2p$.

Assumptions: X is finite and μ is a probability mass function on X.

Proof. (a) A rule d' on r' gives the rule $d' \circ f$ on r with the same error, so the minimum over rules on r is no larger. (b) Let d_i be optimal for π_i. The rule (d_1, d_2) errs for the pair only if at least one component errs, which has probability at most the sum. The lower bound holds because any rule for the pair yields a rule for each component, by projection, with no larger error. (c) Let $X = \{x_1, x_2, x_3\}$, $\mu(x_1) = \mu(x_2) = p$, $\mu(x_3) = 1 - 2p$, let r be constant, $\pi_1 = \mathbf{1}_{\{x_1\}}$, and $\pi_2 = \mathbf{1}_{\{x_2\}}$. Since $p < 1 - p$ the best rule for each component answers 0 and errs with probability p. For the pair the best rule must pick one of the four pairs of answers; answering $(0,0)$ is correct only at x_3, answering $(1,0)$ only at x_1, answering $(0,1)$ only at x_2, and $(1,1)$ at none. The largest correct mass is $1 - 2p > p$, so the error is $2p$. □

Part (a) is the quantitative form of the principle that discarded distinctions cannot be recovered by later computation (Result 2.11). Part (b) shows the practical consequence for families of policies: a representation that is acceptable for each policy separately may be unacceptable for the family, and the loss accumulates at the rate given by the union bound, which part (c) shows cannot be improved in general.

9.4 A computed Bayes error: binned balances

The exact criterion of Chapter 5 says that a binned balance serves a threshold policy only if the threshold falls on a bin boundary. The Bayes error says how badly it serves a threshold that does not. Take balances $X = \mathbb{N}$ with the geometric law $\mu(x) = (1-q)q^x$, the approval policy $\pi_t = [x \geq t]$, and the representation that sends x to the index $\lfloor x/B \rfloor$ of its bin of width B. Within every bin that does not contain the threshold the policy is constant, so those bins contribute no error, and the bin containing t contributes the smaller of the two masses it carries.

New result 9.9 (Bayes error of a binned threshold)**.** *Let $jB < t < (j+1)B$. Then*

$$\operatorname{err}_\mu(r, \pi_t) = q^{jB} \min\bigl(1 - q^{\,t-jB},\ q^{\,t-jB} - q^B\bigr),$$

and the error is zero if t is a multiple of B. In particular the error is not monotone in the distance from the boundary: with B fixed it rises and then falls as t moves across the bin, with its maximum where $q^{\,t-jB} = (1+q^B)/2$.

Assumptions: Geometric law on $\mathbb{N}$; policy $[x \geq t]$; bins of equal width B.

Proof. By Definition 9.1 the error is the sum over bins of the smaller of the masses of the two actions in the bin. A bin other than the jth lies entirely below or entirely above t, so one of its masses is zero. In the jth bin the masses are $\mu[jB, t) = q^{jB}(1 - q^{\,t-jB})$ and $\mu[t, (j+1)B) = q^{jB}(q^{\,t-jB} - q^B)$. The two expressions inside the minimum are respectively increasing and decreasing in $s = t - jB$, so the minimum is maximal where they are equal, which is the stated value of q^s. □

The numbers are instructive. For $q = 0.9$, $B = 8$ and $j = 0$, the thresholds $t = 1, \dots, 7$ have errors 0.100, 0.190, 0.271, 0.226, 0.160, 0.101 and 0.048. The worst threshold is $t = 3$, not the middle of the bin, because the geometric law concentrates its mass at low balances and the lower side of the bin carries the larger share. Under the flatter law $q = 0.5$ the same representation serves $t = 5$ with error 0.027, since almost all of the mass lies below the bin's upper part and the policy is nearly constant on what is likely. A designer who audits the representation on typical balances finds small errors for most thresholds, and the audit reveals nothing about the thresholds that are costly under a different law. The next result states how far that dependence can go.

New result 9.10 (Worst-case distribution for the best rule)**.** *Let $m = \max_{y \in r(X)} |\pi(r^{-1}(y))|$ be the largest number of distinct actions that π takes inside one class of r. Then*

$$\sup_\mu \operatorname{err}_\mu(r, \pi) = 1 - \frac{1}{m},$$

and the supremum is attained by the uniform distribution on one representative of each action within a class that realizes m.

Assumptions: X finite; the supremum is over all probability mass functions on X.

Proof. Within a class y with $m_y \le m$ actions, the largest action mass is at least $\mu(r^{-1}(y))/m_y$, so the error contribution of the class is at most $(1 - 1/m_y)\mu(r^{-1}(y)) \le (1 - 1/m)\mu(r^{-1}(y))$; summing over classes bounds the error by $1 - 1/m$. If a class realizes m actions, put mass $1/m$ on one case of each action in it; the best rule answers one action and errs on the other $m - 1$ of the m cases, so the error equals $1 - 1/m$. □

Result 9.2(d) states that every fixed rule can be forced to err with probability one; the present result shows that the best rule chosen after the distribution is known cannot be forced above $1 - 1/m$. The first is the statement about a deployed system whose rule was fixed before the distribution moved, the second about a system that is able to re-derive its rule. For a binary policy the ceiling is $\frac{1}{2}$, which no distribution on the cases can exceed however it is chosen, and which is reached as soon as one class contains both actions.

Exercises

Exercise 9.11. For the monitor of Example 1.4, take the representation that records only whether at least one alarm is outstanding, and the escalation policy of Chapter 5. Describe the Bayes decision rule on each class and write the Bayes error as a function of the probabilities of the classes of histories with exactly one, two, and three or more outstanding alarms.

Exercise 9.12. Construct a distribution under which the representation of the previous exercise has error zero, and explain why this does not make it sufficient on the full set of histories. Relate your answer to Result 5.7.

Exercise 9.13. Prove that if $\mathrm{err}_\mu(r, \pi) \le \varepsilon$ and μ' satisfies $\mu'(x) \le c\,\mu(x)$ for all x, then the rule that is optimal for μ has error at most $c\varepsilon$ under μ'. Conclude that approximate sufficiency transfers between distributions only with a multiplicative loss controlled by the density ratio, and show that no such bound holds when μ' is not dominated by a multiple of μ.

Exercise 9.14. Using Fano's inequality, show that a representation of a uniformly distributed n-bit string with $k < n$ bits cannot recover all n bits with error below $(n-k-1)/n$ in the sense of the bound $P_e \ge (H(Z \mid Y) - 1)/\log|A|$, taking Z to be the string itself. State carefully what the bound does and does not say about particular strings.

Exercise 9.15. Show that the inequality of Result 9.8(a) can be strict, and that it can be an equality even when f is not injective on $r(X)$. Give an example of each.

Exercise 9.16. Compute the table of errors in Result 9.9 for $q = 0.8$ and $B = 6$, and locate the worst threshold. Compare it with the value predicted by $q^s = (1 + q^B)/2$, and explain why the maximizing s is not an integer in general.

Exercise 9.17. Determine the width B and bin boundaries that minimize the sum of the Bayes errors for the two thresholds $t_1 = 3$ and $t_2 = 12$ under the geometric law with $q = 0.9$, among representations with three values whose blocks are intervals. Compare with the best three-valued representation whose blocks need not be intervals.

Exercise 9.18. Show that the supremum in Result 9.10 is not attained by any distribution charging every case, and that it is approached by such distributions. Deduce the value of $\sup_\mu \mathrm{err}_\mu(r, \pi)$ over the distributions of full support.

Chapter 10

Verification Relative to Retained Evidence

Question. What does verification establish about a formal object, and what additional evidence connects that object to the world?

Established sources. Certifying algorithms (McConnell, Mehlhorn, Näher, Schweitzer); proof-carrying code (Necula); proof objects and checkers.

New results. Verification defined relative to a retained evidence set, with soundness stated relative to that set (Definition 10.1); the verified region of a claim is its interior in the partition induced by the evidence (Result 10.2); a formal proof reaches the world only through evidence that settles its assumptions (Result 10.6).

Proof obligations. None.

Depends on. Chapter 5.

A formal proof establishes that a claim holds for every object satisfying a stated set of assumptions. A system that relies on the proof has to act on a particular case, and whether the claim holds for that case depends on whether the assumptions hold for it. The connection is made by evidence, and evidence is retained or it is not. This chapter defines verification as a relation between a claim and a retained evidence set, shows that the cases for which a claim is verified are determined by the partition the evidence induces, and uses that to state the gap between a proof about a model and a conclusion about a case.

10.1 Verification relative to evidence

Let W be a set of cases, which in later chapters are histories. A *claim* is a subset $\varphi \subseteq W$, the set of cases in which it holds. *Evidence* is a function $e : W \to V$, where $e(w)$ is the retained record of the case w. The evidence is a representation in the sense of Chapter 1, and the fibers $e^{-1}(v)$ are the sets of cases that the record does not separate.

Definition 10.1 (Verification relative to evidence)**.** A *verifier* for the evidence e is a function v that assigns to each pair (u, φ) of a record $u \in e(W)$ and a claim φ a value in $\{0, 1\}$. It is *sound relative to* e if $v(u, \varphi) = 1$ implies $e^{-1}(u) \subseteq \varphi$. It is *complete relative to* e if $e^{-1}(u) \subseteq \varphi$ implies $v(u, \varphi) = 1$. A claim φ is *verified at* w by v if $v(e(w), \varphi) = 1$.

Soundness says that the verifier never accepts a claim unless it holds in every case compatible with the record. Completeness says it accepts whenever the record entails the claim. Both conditions are relative to e: a verifier cannot be held to account for distinctions the record has discarded.

New result 10.2 (The verified region is the interior of the claim)**.** *For a claim $\varphi \subseteq W$ and evidence e, let $\mathrm{int}_e(\varphi) = \{w : e^{-1}(e(w)) \subseteq \varphi\}$. Then:*

(a) $\mathrm{int}_e(\varphi)$ is the largest union of fibers of e contained in φ.

(b) A verifier that is sound relative to e verifies φ only inside $\mathrm{int}_e(\varphi)$, and a verifier that is sound and complete relative to e verifies φ exactly on $\mathrm{int}_e(\varphi)$.

(c) $\mathrm{int}_e(\varphi) = \varphi$ if and only if $\mathbf{1}_\varphi$ factors through e, that is, if and only if the evidence settles the claim in every case.

(d) If $e' = g \circ e$ is a coarsening of e then $\mathrm{int}_{e'}(\varphi) \subseteq \mathrm{int}_e(\varphi)$.

Assumptions: No assumption on W or on the cardinality of V.

Proof. (a) A union of fibers is contained in φ exactly when each of its fibers is, and $\mathrm{int}_e(\varphi)$ is the union of all fibers contained in φ. (b) Soundness gives $e^{-1}(e(w)) \subseteq \varphi$ at every verified w, which is the definition of membership in the interior; completeness gives the reverse. (c) If $\mathrm{int}_e(\varphi) = \varphi$ then φ is a union of fibers, so $\ker e \subseteq \ker \mathbf{1}_\varphi$, which is factorization by Result 2.5; the converse is the same argument read backwards. (d) A fiber of e' is a union of fibers of e, so if it lies in φ then so does each fiber of e it contains. □

Part (d) is the verification form of the principle that coarsening is irreversible: a coarser record never verifies more. Part (c) identifies the claims that can be verified in all the cases where they hold: exactly those that the evidence answers in the sense of Definition 1.1.

Example 10.3 (The balance does not verify the absence of overdraft)**.** Take W to be all finite event sequences of the account of Example 1.3, and the claim $\varphi = \{q_{\mathrm{neg}} = 0\}$. For $e = \beta$ and any balance $b \geq 0$, the fiber of b contains the history $\mathrm{w}(1)\,\mathrm{d}(1)\,\mathrm{d}(b)$ for $b \geq 1$, or $\mathrm{w}(1)\,\mathrm{d}(1)$ for $b = 0$, which overdraws. For $b < 0$ every history in the fiber has overdrawn. Hence $\mathrm{int}_\beta(\varphi) = \varnothing$: no history is verified overdraft-free by its balance alone, even though most histories with a given balance are. For $e = (\beta, q_{\mathrm{neg}})$ the claim is settled and $\mathrm{int}_e(\varphi) = \varphi$.

10.2 Certificates

The standard device for retaining just enough evidence to verify a result is the certificate. A *certifying algorithm* returns, with its answer, an additional record from which the correctness of the answer can be checked by a procedure that is simpler than the algorithm and independent of it.

Established result 10.4 (Certifying algorithms and checkers)**.** *A certifying algorithm for a decision problem on instances I outputs a pair (y, c) with $y \in \{yes, no\}$ and c a certificate, together with a checker $\mathrm{Chk}(I, y, c) \in \{0, 1\}$ such that: if $\mathrm{Chk}(I, y, c) = 1$ then y is the correct answer for I, and the algorithm always outputs a pair that the checker accepts. Correctness of the algorithm then reduces to correctness of the checker.*

Source: McConnell, Mehlhorn, Näher, and Schweitzer, "Certifying algorithms," *Computer Science Review* 5 (2011); the same pattern appears in proof-carrying code, Necula, "Proof-carrying code," POPL 1997.

Example 10.5 (Bipartiteness)**.** For the question whether a graph is bipartite, a yes answer is certified by a proper two-colouring, which is checked edge by edge, and a no answer by an odd cycle, which is checked by following it. In the language of Definition 10.1, the retained record for an instance is the pair of the instance and the certificate, the claim is the correctness of y, and the checker is a sound verifier relative to that record. The record is far smaller than the algorithm's computation and the checker can be trusted without trusting the algorithm.

10.3 The gap between a proof and a case

A theorem of the form "every case satisfying the assumptions A has property φ" is the inclusion $A \subseteq \varphi$. The following result states what such a theorem delivers for an individual case, in terms of the retained evidence.

New result 10.6 (A proof reaches a case only through evidence that settles its assumptions)**.** *Let $A \subseteq \varphi \subseteq W$ be established by proof. Then:*

(a) The claim φ is verified at every $w \in \operatorname{int}_e(A)$ by a verifier whose only premise is the proof.

(b) If $w \notin \operatorname{int}_e(A)$, then no sound verifier relative to e accepts the claim A at w. Since A is the strongest claim the proof supports, the proof by itself verifies, for arbitrary $\varphi \supseteq A$, exactly the region $\operatorname{int}_e(A)$.

Assumptions: A is the set of cases satisfying the assumptions of the theorem. The evidence e is fixed in advance.

Proof. (a) If $w \in \operatorname{int}_e(A)$ then every case sharing the record of w lies in A, hence in φ. (b) For a record $u = e(w)$ with $e^{-1}(u) \not\subseteq A$ the sound-verifier condition for the claim A fails, which is Result 10.2(b) for $\varphi = A$. A case outside $\operatorname{int}_e(A)$ is therefore unverified for the claim A, and the proof supports no stronger claim. □

The content of the result is easiest to read from part (a). The assumptions of a theorem about a controller, a protocol, or a model are claims about the case, and the proof carries no information about whether they hold. Unless the retained evidence determines membership of A on the region of interest, the theorem verifies nothing there. The designer's task is therefore to choose evidence whose fibers lie inside A wherever the theorem is to be used, and the task is a question about the kernel of the evidence in the sense of Chapter 2.

10.4 A bounded-horizon computation of the verified region

Example 10.3 shows that the balance verifies nothing over all histories. The conclusion depends on the unboundedness of the domain, and the dependence can be seen exactly by restricting the domain to histories of length at most L over the two events $\mathrm{d} = \mathrm{d}(1)$ and $\mathrm{w} = \mathrm{w}(1)$. Write W_L for that set, which has $2^{L+1} - 1$ elements, and $\varphi = \{q_{\mathrm{neg}} = 0\} \cap W_L$ for the claim that no prefix is overdrawn.

New result 10.7 (Verified region of the never-overdrawn claim under the balance)**.** *For $L \geq 2$ the interior of φ in W_L under the evidence $e = \beta$ consists of exactly two histories,* d^{L-1} *and* d^{L}*. Hence the fraction of W_L verified by the balance is $2/(2^{L+1}-1)$, although the claim holds in φ for a constantly growing share of cases, and under the evidence $e = (\beta, q_{\mathrm{neg}})$ the whole of φ is verified.*

Assumptions: The domain is W_L, the set of words of length at most L over $\{\mathrm{d}, \mathrm{w}\}$; amounts are one unit; the evidence is the balance.

Proof. Since the number of deposits minus withdrawals is the balance b and the number of events is their sum, the length of a history of balance b is congruent to b modulo 2 and is at least $|b|$. For $b \geq 0$ the fibre of b in W_L contains an overdrawing history if and only if it contains one of length at most L; the shortest overdrawing history of balance b is $\mathrm{w}\,\mathrm{d}^{\,b+1}$, of length $b+2$, so the fibre contains an overdrawing history if and only if $b+2 \leq L$. For $b < 0$ every history of the fibre is overdrawn. Hence the fibre of b lies in φ if and only if $b \geq L-1$, and the only balances in $[L-1, L]$ present in W_L are $L-1$ and L, each realized by the histories whose length is congruent to b and at most L: for $b = L$ this is d^{L} alone, and for $b = L-1$ it is d^{L-1} alone, because the next admissible length $L+1$ exceeds L. The last statement is Result 10.2(c). □

The table gives the counts for small horizons, confirmed by enumeration.

L	$\lvert W_L \rvert$	never overdrawn	verified by β
2	7	4	2
3	15	7	2
4	31	13	2
5	63	23	2
6	127	43	2

The number of histories that satisfy the claim grows almost as fast as 2^L, the ratio of consecutive entries being near 1.9, while the number that the balance verifies stays at two. The verified histories are the ones whose balance is so high that no overdraft could have occurred within the horizon, so the balance verifies the claim only where the horizon itself rules out the counterexample. This is the quantitative form of the point made after Result 10.6: the evidence has to settle the assumptions on the region of use, and a record that settles them only by virtue of an unstated bound on how long the system has run provides assurance that is lost as the system keeps operating.

Exercises

Exercise 10.8. For the account, compute $\mathrm{int}_e(\varphi)$ for $\varphi = \{\beta \geq 0\} \cap \{q_{\mathrm{neg}} = 0\}$ and for each of $e = \beta$, $e = q_{\mathrm{neg}}$, and $e = (\beta, q_{\mathrm{neg}})$. Which of the three verify the largest region, and what does the comparison show?

Exercise 10.9. Prove that for two evidence functions e_1, e_2, $\mathrm{int}_{(e_1,e_2)}(\varphi) \supseteq \mathrm{int}_{e_1}(\varphi) \cup \mathrm{int}_{e_2}(\varphi)$ and give an example where the inclusion is strict.

Exercise 10.10. Describe the certificate for the question "is there a path from s to t in this directed graph?" for each of the answers yes and no. State the checker and argue that its soundness does not depend on the algorithm that produced the certificate.

Exercise 10.11. A controller is proved safe provided its sensor error is at most δ. The retained evidence records the sensor reading and a bit stating whether a self-test passed. Describe the set A and the sets of cases in the interior of A under the evidence consisting of the reading alone, and under the reading together with the self-test bit. State what further property of the self-test is needed for the second evidence to verify safety.

Exercise 10.12. Show that $\mathrm{int}_e(\varphi \cap \psi) = \mathrm{int}_e(\varphi) \cap \mathrm{int}_e(\psi)$ and that $\mathrm{int}_e(\varphi \cup \psi) \supseteq \mathrm{int}_e(\varphi) \cup \mathrm{int}_e(\psi)$ with strict inclusion possible. Explain what the second statement means for a system that verifies a disjunction of claims.

Exercise 10.13. Compute the interior of φ in W_L under the evidence $e = (\beta, |h|)$, the balance together with the length, and under $e = (\#_{\mathrm{d}}, \#_{\mathrm{w}})$. Show that they coincide and that the interior is $\{\mathrm{d}^k : k \le L\}$, and compare its size with the two histories of Result 10.7.

Exercise 10.14. Show that the evidence $(\beta, q_{\mathrm{neg}})$ takes exactly 12 values on W_4, list them, and explain why this is fewer than the $2 \cdot 9 = 18$ pairs of a balance in $\{-4, \ldots, 4\}$ and a flag. Which of the twelve values lie in the interior of φ?

Exercise 10.15. Let the claim be $\psi = \{\beta \ge 0\} \cap W_L$, the claim that the present balance is nonnegative. Show that ψ is settled by β, that its interior is ψ, and explain why verifying ψ and verifying φ differ in the way the evidence must be maintained as the history grows.

Chapter 11

Provenance and Revision Lineage

Question. What fixes the identity of a revised object?

Established sources. Content addressing and hash-linked records (Merkle); the happens-before relation and vector clocks (Lamport; Fidge; Mattern); provenance semirings (Green, Karvounarakis, and Tannen).

New results. Identity relative to a family of audit questions (Definition 11.4); minimal lineage records for last-author audits and their cost (Result 11.5); content identity does not determine lineage (Result 11.6).

Proof obligations. None.

Depends on. Chapters 4 and 5.

An object that has been revised carries a history. Whether two revised objects are the same object, and whether the history a system keeps is adequate, are questions about what the history has to answer. This chapter states the standard machinery for recording lineage, content-addressed chains, partial orders of events, and annotated query results, and then applies the kernel analysis of Part I to ask how much of the lineage a given family of audit questions requires.

11.1 Standard machinery

Established result 11.1 (Hash-linked records determine the chain)**.** *Let H be a function that is injective on the set of pairs (p, u) of a payload and an identifier, with $\perp$ not in its range. Define for a sequence $c = (p_1, \ldots, p_n)$ of payloads the identifiers $u_0 = \perp$ and $u_i = H(p_i, u_{i-1})$. Then the head u_n determines the whole sequence: if two sequences of the same or different length have the same head then they are equal.*

Source: Merkle (1980) for hash trees; hash-linked records as used in version control and ledgers. The statement is the idealization in which collisions do not occur; for a collision-resistant function it holds except with negligible probability.

Proof. Suppose $u_n = u'_m$. By injectivity $(p_n, u_{n-1}) = (p'_m, u'_{m-1})$. Applying the argument to $u_{n-1} = u'_{m-1}$ and descending, the sequences agree entry by entry; the process stops at $\perp$ on both sides at once, because $\perp$ is not in the range of H, so $n = m$. □

In the terminology of Chapter 5 the head is an injective representation of the chain. It is therefore sufficient for every question about the chain and, by Result 5.3, it is the only kind of representation that is sufficient for all of them.

Established result 11.2 (Happens-before and vector clocks)**.** *In a message-passing system of n processes, let $e \to e'$ denote Lamport's happens-before relation. Vector timestamps $V(e) \in \mathbb{N}^n$, in which each process increments its own coordinate at each event and takes componentwise maxima on receipt of a message, satisfy $e \to e'$ if and only if $V(e) < V(e')$ in the componentwise order with $V(e) \neq V(e')$.*

Source: Lamport, "Time, clocks, and the ordering of events in a distributed system" (1978); Fidge (1988); Mattern (1989).

Established result 11.3 (Provenance semirings)**.** *Positive relational algebra queries evaluated on relations whose tuples are annotated by elements of a commutative semiring K, with union and projection interpreted by addition and join by multiplication, give annotated results. The semiring $\mathbb{N}[X]$ of polynomials over the tuple identifiers is universal: for every commutative semiring K and valuation of the identifiers in K, evaluating in $\mathbb{N}[X]$ and then applying the induced homomorphism equals evaluating directly in K.*

Source: Green, Karvounarakis, and Tannen, "Provenance semirings," PODS 2007.

The semiring result is a statement that a single detailed record (the polynomial) is sufficient for a whole family of coarser audit questions (set provenance, multiplicity, access levels), each obtained by a homomorphism. It is an instance of the principle of Chapter 2 that coarser answers are functions of finer ones.

11.2 Identity relative to audit questions

Definition 11.4 (Lineage and identity relative to a family of audit questions)**.** A *lineage* is a finite sequence of revision events from a set E, that is, a history in the sense of Chapter 4. An *audit question* is a function q on lineages. For a family $\mathcal{Q}$ of audit questions, two lineages are *indistinguishable by* $\mathcal{Q}$ if $q(h) = q(h')$ for all $q \in \mathcal{Q}$. A *record* is adequate for $\mathcal{Q}$ if it is sufficient in the sense of Definition 6.1.

By Result 6.2 the least adequate record is the quotient of the lineages by indistinguishability. Two lineages that are indistinguishable receive the same record, so the notion of when two revision histories describe the same object for the purposes of audit is determined entirely by the family. Chapter 15 takes up the use of this relation as an identity criterion.

11.3 A last-author audit

Consider a document with K sections edited by a finite set U of authors. The events are pairs (u, i) with $u \in U$ and $i \in \{1, \dots, K\}$, read as author u revises section i. Let $\lambda_i(h) \in U \cup \{\bot\}$ be the author of the last revision to section i in the lineage h, and $\bot$ if the section has never been revised. Write $\Lambda(h) = (\lambda_1(h), \dots, \lambda_K(h))$.

New result 11.5 (Minimal records for last-author audits)**.** *Let $\mathcal{Q} = \{\lambda_1, \dots, \lambda_K\}$.*

(a) Λ *is a maintainable representation that is adequate for* $\mathcal{Q}$*, and* $\ker \Lambda$ *is the joint kernel of* $\mathcal{Q}$*.*
(b) Every representation adequate for $\mathcal{Q}$ *takes at least* $(|U|+1)^K$ *values, and* Λ *takes exactly this many.*
(c) Let $\mathcal{Q}^+ = \mathcal{Q} \cup \{S_1, \ldots, S_K\}$ *where* $S_i(h) \subseteq U$ *is the set of authors who have revised section* i *in* h*. Then the least adequate record for* $\mathcal{Q}^+$ *has* $\left(1 + |U|\, 2^{|U|-1}\right)^K$ *values.*

Assumptions: U and K are finite. Lineages are all finite sequences of events.

Proof. (a) Appending (u, i) replaces the ith coordinate of Λ by u and leaves the others unchanged, so the value of Λ after an event depends only on its value before, and $\ker \Lambda$ is a right congruence. The joint kernel of $\mathcal{Q}$ is by definition the kernel of Λ. (b) Every vector in $(U \cup \{\perp\})^K$ occurs: for the target $(a_1, \ldots, a_K)$ perform the event (a_i, i) for each i with $a_i \neq \perp$ in turn. Two different vectors differ in some coordinate i, where the corresponding λ_i separates them. So the joint kernel has $(|U|+1)^K$ classes and, by Result 3.10, every adequate representation has at least that many values. (c) Fix a section i. The pair (λ_i, S_i) takes the value $(\perp, \varnothing)$, or a value (a, S) with S nonempty and $a \in S$. Every such value occurs, by revising section i successively by the members of S with a last. The number of pairs (a, S) is $\sum_{S \neq \varnothing} |S| = |U| 2^{|U|-1}$, so each section has $1 + |U| 2^{|U|-1}$ possible values. Words acting on different sections can be concatenated in any order without changing the pair values of other sections, so all combinations occur, and the joint kernel has the stated number of classes. □

The count in (c) makes the cost of enlarging an audit family concrete. For two authors and one section the record grows from three values to five, and with K sections the multiplicative effect of Result 8.11 appears: every section pays the same factor.

New result 11.6 (Content identity does not determine lineage)**.** *Let* c *be the function that maps a lineage to the current text of the document, and let* $\mathcal{Q}$ *be any family containing a question that depends on the author of a revision. If two authors can produce the same text by a revision, then* c *is not adequate for* $\mathcal{Q}$*.*

Assumptions: There is a text t, two authors $a \neq b$, and a section i such that the revisions (a, i) and (b, i) applied to some lineage produce the same text t. The question $\lambda_i \in \mathcal{Q}$ is the last-author question.

Proof. Let h_a and h_b be the lineages obtained by applying the two revisions. They have the same text, so $c(h_a) = c(h_b)$, and $\lambda_i(h_a) = a \neq b = \lambda_i(h_b)$. Thus $\ker c \not\subseteq \ker \lambda_i$, and Result 2.5 shows c is not adequate. □

The result separates two things that are often both called identity. A content address identifies the text. Whether the text is the product of a given sequence of revisions is a different question, and whether a system can answer it depends on whether the lineage record is fine enough for the audit questions asked of it.

11.4 Recency lists and a hierarchy of audit records

The last-author audit has a natural refinement that records not only who revised a section most recently but the order in which the authors last touched it. This is the information a

reviewer needs to ask who edited the section before the last editor, and it gives a third level in a hierarchy of records of increasing cost.

For a section i and a lineage h, the *recency list* $R_i(h)$ is the sequence of the distinct authors who have revised i, ordered by the time of their most recent revision, newest first. Appending (u, i) moves u to the front of R_i, inserting it if absent; the other lists are unchanged.

New result 11.7 (Recency lists)**.** *Let $|U| = n$. The recency list is maintainable, determines λ_i (its first element, or $\perp$ if empty) and S_i (its set of entries), and takes exactly $\sum_{j=0}^{n} n!/(n-j)!$ values on one section, namely the number of sequences of distinct authors of length $0, 1, \ldots, n$. This equals the $1 + n2^{n-1}$ values of the pair (λ_i, S_i) if and only if $n \leq 2$; for $n = 3$ it has 16 values against 13, for $n = 4$ it has 65 against 33, and it is the least record adequate for the question family that asks for every pair of authors of i which of them revised i most recently.*

Assumptions: Lineages are all finite sequences of events (u, i); U finite.

Proof. Maintainability is the move-to-front rule above, which defines the new list from the old list and the event. The first element and the set of entries are read off the list. Every sequence of distinct authors is realized by applying the events in the reverse of the list order, so all $\sum_j n!/(n-j)!$ sequences occur, and they are pairwise separated by the questions "which of u and u' revised i more recently" together with membership. For the comparison, the pair (λ_i, S_i) has $1 + n2^{n-1}$ values by Result 11.5(c). The two counts agree at $n = 1$ (2 and 2) and $n = 2$ (5 and 5), and differ from $n = 3$ onward because $S = \{a, b, c\}$ with last author a leaves two possible orders of b and c in the list, which the pair does not separate. Minimality for the stated family holds because separating every ordered pair of authors determines the list. □

The hierarchy has three levels. The last-author record costs $(n+1)^K$ values, the last author with the set of contributors costs $(1 + n2^{n-1})^K$, and the recency list costs $\left(\sum_j n!/(n-j)!\right)^K$, which for three authors and a ten-section document is $16^{10} \approx 1.1 \times 10^{12}$ against $13^{10} \approx 1.4 \times 10^{11}$ and $4^{10} \approx 10^6$. The factor of roughly 10^6 between the cheapest and the most informative level, for a small team and a short document, is the price of the ability to answer questions about the second author from the end. Whether that ability is worth the price depends on whether such questions are ever asked, and the family of audit questions, not the document, is what settles it.

Exercises

Exercise 11.8. For the document of this chapter with $|U| = 2$ and $K = 2$, list the values of Λ and count them. List the additional distinctions required to also answer the question "has more than one author revised section 1?" and compute the number of values of the least adequate record.

Exercise 11.9. Show that the representation Λ remains adequate for the question "which sections have been revised?", and find a coarsening of Λ that answers it with 2^K values. Prove that the coarsening is maintainable.

Exercise 11.10. Hash-linked records are injective and therefore adequate for every audit family. Explain why this does not make them minimal, and compute the ratio between the number of lineages of length at most n and the number of values of Λ for $|U| = 2$ and $K = 2$ as n grows.

Exercise 11.11. Two replicas apply revisions concurrently. Using vector clocks, define the lineage as the set of events together with the happens-before order and state which last-author questions are well defined for concurrent revisions to the same section. Propose a representation that records the set of maximal concurrent revisions and compute its size in terms of $|U|$.

Exercise 11.12. In the semiring of Result 11.3, give a homomorphism from $\mathbb{N}[X]$ to the Boolean semiring and one to $(\mathbb{N}, +, \times)$, and state the audit question that each answers. Show that the first is a coarsening of the second.

Exercise 11.13. For $|U| = 3$ list the 16 recency lists of a single section and the map to the values of (λ, S). Show that exactly three values of (λ, S) have two preimages each and the rest have one, and deduce the counts 13 and 16.

Exercise 11.14. Add the first-author question ϕ_i (who made the first revision of section i) to the family $\{\lambda_i\}$ and compute the number of values of the least adequate record for one section. Show that it is $|U|^2 + 1$, and that the record is maintainable. Explain why the same count does not hold for the family consisting of ϕ_i and S_i.

Exercise 11.15. Show that the recency list of Result 11.7 is not adequate for the question “how many times has the section been revised?”. Give the smallest family of additional distinctions that makes the record adequate for it, and show that the augmented record is infinite.

Chapter 12

Memory Architecture and Retrieval

Question. When does a system retain evidence that it can no longer effectively recover?

Established sources. The cell-probe model (Yao); indexing and caching models.

New results. Retrieval adequacy defined separately from storage adequacy (Definition 12.2); lower bounds on probes for parity and for locating a single marked cell (Result 12.3); adding cells that are functions of the content leaves storage adequacy unchanged and can change retrieval adequacy completely (Result 12.4).

Proof obligations. None.

Depends on. Chapters 5 to 7.

Chapters 5 to 7 asked whether the information needed for a decision is present in a representation. That is a question about the kernel and it ignores the cost of reading. A system can store everything it needs and still be unable to act on it in the time it has, because finding the relevant part requires reading most of the store. This chapter separates the two notions, storage adequacy and retrieval adequacy, and shows by elementary lower bounds that they come apart.

12.1 Storage adequacy and retrieval adequacy

A stored object is a function $M : X \to \Sigma^N$ from cases to arrays of N cells, each cell holding an element of a finite alphabet Σ. A *reader* for a question $q : X \to B$ is a deterministic decision tree: it chooses a cell to read, sees its content, chooses the next cell as a function of what it has seen, and after at most t reads outputs an answer. This is the cell-probe model.

Established result 12.1 (Cell-probe model)**.** *In the cell-probe model, the cost of a computation on stored data is the number of cells read; computation between reads is free.*

Source: Yao, "Should tables be sorted?" *Journal of the ACM* 28 (1981).

Definition 12.2 (Storage adequacy and retrieval adequacy)**.** The stored object M is *storage adequate* for q if q factors through M, that is $\ker M \subseteq \ker q$. The *retrieval depth* $D(M, q)$ is the least t such that some reader of depth at most t outputs $q(x)$ for every x when run on $M(x)$. The stored object is *t-retrieval adequate* for q if $D(M, q) \leq t$.

Storage adequacy is the condition of Definition 1.1. If M is storage adequate then $D(M, q) \leq N$, because reading all cells determines $M(x)$ and hence $q(x)$; and if M is not storage adequate then no reader can output q correctly, because the reader sees only $M(x)$. Retrieval adequacy therefore refines storage adequacy by a quantitative measure that lies between 0 and N.

12.2 Two lower bounds

New result 12.3 (Reading cost of parity and of a single marked cell)**.** *Let $X = \{0,1\}^n$ and let M be the identity with $N = n$ cells, one bit each.*

(a) For q = parity of x, $D(M, q) = n$.

(b) Let $X' \subseteq \{0,1\}^n$ be the strings with exactly one bit equal to 1 and let $q'(x)$ be the position of that bit. Then $D(M, q') = n - 1$.

Assumptions: Readers are deterministic and are required to be correct on every x in the domain.

Proof. (a) Suppose a reader of depth less than n is correct. Run it on the string 0^n and let j be a cell that it does not read; such a cell exists. Changing x_j does not alter any answer the reader sees, so its trajectory and output are the same on 0^n and on the string with $x_j = 1$, but the parities differ. Hence depth n is necessary, and reading all cells suffices. (b) Run a reader of depth at most $n - 2$ on the adversary's answers, which return 0 to every query. At the end at least two cells remain unread, and the strings with the single 1 at either of them are consistent with every answer given, so the reader errs on one of them. Hence depth at least $n - 1$ is needed, and reading $n - 1$ cells suffices, since if all of them are 0 the remaining cell holds the 1. □

12.3 Adding derived cells

The remedy for expensive retrieval is an index: additional cells that are computed from the content. The next result records what an index does and does not change.

New result 12.4 (Derived cells leave storage adequacy unchanged)**.** *Let $M : X \to \Sigma^N$ and let $M^+(x) = (M(x), I(x))$, where $I(x)$ is a function of $M(x)$.*

(a) $\ker M^+ = \ker M$, so M^+ is storage adequate for exactly the same questions as M.

(b) $D(M^+, q) \leq D(M, q)$ for every q.

(c) The inequality in (b) can be as strong as possible: in the setting of Result 12.3(a), adding a single cell holding the parity gives $D(M^+, q) = 1$, while $D(M, q) = n$. In the setting of Result 12.3(b), adding a single cell holding the position of the 1 gives $D(M^+, q') = 1$, while $D(M, q') = n - 1$.

Assumptions: The index is computed from the stored content, which is the situation in which an index is a pure acceleration structure.

Proof. (a) Since I is a function of M, two cases with equal M have equal M^+, and M^+ determines M. (b) A reader for M is also a reader for M^+ that ignores the added cells. (c) A reader that reads the added cell outputs the answer. □

The result states formally why the adequacy notion of the earlier chapters is blind to indexing. An index adds nothing to the kernel, so a criterion that depends only on kernels cannot tell a store with a good index from one without. A system's ability to use what it keeps is therefore a property of the pair of the stored object and the access structure, and a design that argues from sufficiency alone concludes only that the information is present. The question of whether it can be found within a given time has to be asked separately.

The index has a price. The added cells enlarge the stored object, they must be maintained as the content changes, and an index tuned for one family of questions does nothing for another. Parity is answered by the first added cell and not by the second; the position of the marked bit is answered by the second and not by the first. The cost of keeping an index for every anticipated family of questions is of the same kind as the cost of keeping a representation sufficient for every anticipated family of policies, which Chapter 6 bounds, and Volume 2 returns to it as a question of the cost of maintaining a capacity to answer.

Remark 12.5. A cache is a different kind of derived cell: it holds copies of content that can be read cheaply. Like an index it adds no information, and it improves retrieval depth only for the questions whose answers depend on cached cells. It also can fail by returning stale content when the content changes, which is a failure of the maintainability studied in Chapter 7 and not of storage.

12.4 Packing derived cells: how many are needed for depth one

Result 12.4 shows that a single derived cell can give depth one for a single question. For a family of questions the number of derived cells needed depends on how much each cell can hold, and the count is exact. Let the stored object be m blocks of b bits, $X = \{0,1\}^{mb}$, with the original cells $M(x) = x$, and let the family be the m block parities $\mathcal{Q} = \{p_1, \dots, p_m\}$, $p_k(x) = \bigoplus_{i \in \text{block } k} x_i$. Derived cells may take values in an alphabet of size $s \geq 2$.

New result 12.6 (Derived cells needed for depth one)**.** *For $b \geq 2$, the family $\mathcal{Q}$ has retrieval depth b for each question on the raw store, and a store $M^+ = (M, I_1, \dots, I_c)$ with derived cells over an alphabet of size s has retrieval depth one for every member of $\mathcal{Q}$ if and only if $c \geq \lceil m / \lfloor \log_2 s \rfloor \rceil$. The bound is attained by packing $\lfloor \log_2 s \rfloor$ block parities into each derived cell.*

Assumptions: Readers are deterministic; derived cells are functions of the stored content; the original cells are single bits and $b \geq 2$, so that no original cell determines a block parity.

Proof. A reader of depth one reads a cell fixed in advance and outputs a function of its content, so the question p_k is served at depth one exactly when some cell γ has $\ker \gamma \subseteq \ker p_k$. An original cell is a single bit x_i with $b \geq 2$, so it is constant on no set that determines a parity, and it serves nothing. A derived cell γ with values in an alphabet of size s serves a set T of block parities exactly when its kernel is contained in the joint kernel of $\{p_k : k \in T\}$; the block parities are independent uniformly distributed bits on X, so the joint kernel has $2^{|T|}$ classes, and γ takes at least $2^{|T|}$ values, whence $2^{|T|} \leq s$ and $|T| \leq \lfloor \log_2 s \rfloor$. The m questions must each be served by some cell, so $c \lfloor \log_2 s \rfloor \geq m$. For attainment partition the questions into groups of $\lfloor \log_2 s \rfloor$ and let each derived cell store the tuple of the block parities of its group,

which has at most $2^{\lfloor \log_2 s \rfloor} \leq s$ values. The depth on the raw store is b by the argument of Result 12.3(a) applied to one block. □

For $m = 4$ blocks of $b = 3$ bits the raw store has depth 3, and with binary derived cells four cells are needed, with four-valued cells two, and with sixteen-valued cells one. The intermediate case is the practical one: a derived cell is typically a word, and a word can carry several summaries, so the index adds cells in proportion to the number of independent questions divided by the width of a word, and not in proportion to the number of questions. The result is the retrieval analogue of Result 6.4, in which a family that separates every pair of cases forces an injective representation: a family of independent questions forces an index whose total information is at least one bit per question, however the bits are packed.

Exercises

Exercise 12.7. Prove that the retrieval depth of the majority function on n one-bit cells, with n odd, is n. Adapt the adversary argument of Result 12.3(a) by answering the first $(n-1)/2$ reads with 0 and the next $(n-1)/2$ with 1.

Exercise 12.8. For the account of Example 1.3, suppose the stored object is the full sequence of events, of fixed length N, in N cells. State the retrieval depth for the question "what is the current balance?" and for "has an overdraft ever occurred?" Describe the derived cells that reduce each depth to 1 and count the cells added.

Exercise 12.9. Show that if $M^+ = (M, I)$ with I a function of M then $D(M^+, q) \leq D(M, q)$ and construct an example where equality holds for a nonconstant question. Explain why such a question cannot be sped up by any derived cell.

Exercise 12.10. Randomized readers are allowed to err with small probability. Argue that the lower bound of Result 12.3(b) fails for readers that err with probability $\delta > 0$, and compute the number of reads needed to find the marked cell with probability at least $1 - \delta$ by reading in random order. Relate the answer to the difference between exact and probabilistic preservation in Chapter 9.

Exercise 12.11. A store holds n records and answers the question "is record k present?" for a variable k. Compare the retrieval depth of an unsorted array, a sorted array, and an idealized hash table with a single probe. Show that each is storage adequate for the question, and say why their kernels as stored objects nevertheless differ.

Exercise 12.12. Show that Result 12.6 fails for the family of parities of overlapping blocks. Take $x \in \{0,1\}^4$ and the three questions $x_1 \oplus x_2$, $x_2 \oplus x_3$, $x_1 \oplus x_3$, and show that three binary derived cells are needed for depth one while a single four-valued cell suffices. Explain how the dependence among the questions, which makes the third a function of the other two, enters the count and why the bound $\lfloor \log_2 s \rfloor$ per cell no longer holds.

Exercise 12.13. Prove the lower bound on the raw retrieval depth for one block parity in Result 12.6, namely that every reader that is correct on all of $\{0,1\}^{mb}$ reads all b cells of the block, and give the number of cells read on average by the best reader that is allowed to err with probability δ.

Exercise 12.14. Suppose derived cells must be maintained: each write to an original cell must update every derived cell that depends on it. Compute the number of derived-cell updates per write for the packed index of Result 12.6 with $s = 4$, and compare with the unpacked index. Explain why packing reduces the number of cells without reducing the update work.

Part IV

Admissibility and obligations

Chapter 13

Admissible Transitions

Question. How are physical executability, authorization, and satisfaction of obligations separated?

Established sources. Guarded and labelled transition systems.

New results. A three-way decomposition of admissibility, consistent with Definition 1.5 (Result 13.2); a four-way refinement of the normative obstruction in which responsibility is not additive (Result 13.3); one-step admissibility is not maintainable (Example 13.5).

Proof obligations. None.

Depends on. Chapters 1 and 4.

Chapter 1 defined admissibility through two sets, the continuations that can physically be executed and the realizations that satisfy the obligations in force, and distinguished the obstructions that arise when either is empty. Practical systems distinguish a third condition. A continuation can be physically possible and compatible with every commitment and still be one that the actor has no standing to perform. This chapter separates the three conditions, shows that the separation is consistent with the earlier definition, and shows that the separation is not merely bookkeeping: when two normative restrictions act together, neither can be held responsible for an obstruction that they jointly produce.

13.1 Three conditions on histories

Let $\mathcal{H} \subseteq \mathsf{E}^*$ be a prefix-closed set of histories as in Chapter 1.

Definition 13.1 (Executable, authorized, and compliant histories)**.** Three prefix-closed sets $\mathcal{P}, \mathcal{U}, \mathcal{O} \subseteq \mathcal{H}$ are given.

- $\mathcal{P}$ is the set of *physically executable* histories: $h \in \mathcal{P}$ if every step of h could be carried out. The set $\mathrm{Reach}(x(h))$ of Chapter 1 is $\{w : h \cdot w \in \mathcal{P}\}$ for $h \in \mathcal{P}$.
- $\mathcal{U}$ is the set of *authorized* histories: $h \in \mathcal{U}$ if the actor was entitled to perform every step of h.
- $\mathcal{O}$ is the set of *obligation-compliant* histories: $h \in \mathcal{O}$ if no obligation incurred along h has been irrevocably violated.

The *admissible* histories are $\mathcal{A} = \mathcal{P} \cap \mathcal{U} \cap \mathcal{O}$. A word w is an *admissible realization* after $h \in \mathcal{A}$ if $h \cdot w \in \mathcal{A}$.

Prefix closure of $\mathcal{O}$ reflects the form in which obligations enter the formal development: an obligation whose breach is detected on a finite history is a safety property in the sense of Definition 4.14, and $\mathcal{O}$ is its set of non-violating histories. An obligation that is a liveness property cannot be violated by any finite history, and Result 4.17 says to replace $\mathcal{O}$ by the viable histories, those from which the obligation can still be met.

New result 13.2 (Admissibility is stepwise, and each condition is independent)**.** *(a) $\mathcal{A}$ is prefix closed, and for $h \in \mathcal{A}$ an event a satisfies $h \cdot a \in \mathcal{A}$ if and only if $a \in \mathrm{Adm}_1(h) := P_1(h) \cap U_1(h) \cap O_1(h)$, where $P_1(h) = \{a : ha \in \mathcal{P}\}$, $U_1(h) = \{a : ha \in \mathcal{U}\}$, and $O_1(h) = \{a : ha \in \mathcal{O}\}$.*

(b) A word w is an admissible realization after $h \in \mathcal{A}$ if and only if every event of w is admissible at the history reached just before it.

(c) Each of the three conditions can be the only one that fails: for every nonempty alphabet that contains at least one event there are systems in which exactly $\mathcal{P}$, exactly $\mathcal{U}$, or exactly $\mathcal{O}$ excludes an event that the other two permit.

Assumptions: $\mathcal{P}, \mathcal{U}, \mathcal{O}$ are prefix closed and contain the empty history.

Proof. (a) An intersection of prefix-closed sets is prefix closed. For $h \in \mathcal{A}$ we have $h \in \mathcal{P} \cap \mathcal{U} \cap \mathcal{O}$, so $ha \in \mathcal{A}$ if and only if ha is in each of the three sets, which is the stated condition. (b) By induction on $|w|$ using (a). (c) Take $\mathcal{P} = \mathcal{U} = \mathcal{O} = \mathcal{H}$ except for removing the single history a and all its extensions from one of the three sets. In each case a is excluded by exactly that set. □

The decomposition is consistent with Definition 1.5. Authorization is a normative restriction on the same footing as obligation: the admissible realizations of Chapter 1, those satisfying every restriction in force, are the realizations that lie in $\mathcal{U} \cap \mathcal{O}$. What the decomposition adds is the ability to say which restriction an obstruction comes from.

13.2 The normative obstruction has four forms

Fix $h \in \mathcal{A}$ and a proposed continuation c. Let $R = \mathrm{Real}(h, c)$ be the set of realizations in c that are physically executable, and let $A = \{w \in R : h \cdot w \in \mathcal{U}\}$ and $B = \{w \in R : h \cdot w \in \mathcal{O}\}$. Then $\mathrm{Adm}(h, c) = A \cap B$. The obstruction $\mathsf{O}_{\mathrm{obl}}$ of Definition 1.5 holds when $R \neq \varnothing$ and $A \cap B = \varnothing$.

New result 13.3 (Four forms of the normative obstruction)**.** *Suppose $R \neq \varnothing$ and $A \cap B = \varnothing$. Exactly one of the following holds.*

(i) $A = \varnothing$ and $B \neq \varnothing$ (an authorization obstruction);

(ii) $A \neq \varnothing$ and $B = \varnothing$ (an obligation obstruction proper);

(iii) $A = \varnothing$ and $B = \varnothing$ (both restrictions exclude every realization);

(iv) $A \neq \varnothing$, $B \neq \varnothing$, and $A \cap B = \varnothing$ (a joint *obstruction).*

All four occur. In case (iv), removing either restriction alone makes an admissible continuation available, and neither restriction taken by itself leaves the proposed continuation obstructed. Consequently the pair of bits $(\mathbf{1}[A \neq \varnothing], \mathbf{1}[B \neq \varnothing])$ is not sufficient for the question whether $\mathrm{Adm}(h, c) \neq \varnothing$.

Assumptions: R is finite, or at least the sets A and B are given as subsets of R.

Proof. The four cases are the four combinations of the two emptiness conditions, with the combination $A \neq \varnothing \neq B$, $A \cap B \neq \varnothing$ excluded by hypothesis. They occur: let $c = \{w_1, w_2\}$ with both realizations physically executable and choose A, B as follows: $(\varnothing, \{w_1\})$ for (i); $(\{w_1\}, \varnothing)$ for (ii); $(\varnothing, \varnothing)$ for (iii); $(\{w_1\}, \{w_2\})$ for (iv). In case (iv), dropping the authorization requirement leaves $B = \{w_2\} \neq \varnothing$, and dropping the obligation requirement leaves $A = \{w_1\} \neq \varnothing$. For the last statement compare case (iv) with the situation $A = B = \{w_1\}$: both give the bit pair $(1, 1)$, yet $\mathrm{Adm}(h, c)$ is empty in the first and not in the second. □

Example 13.4 (Releasing a record)**.** A clinic holds a patient record. The proposed continuation is to release it, either to an auditor (w_1) or to the patient's representative (w_2). Only the auditor is authorized to receive it under the clinic's role assignments, and a legal hold on the record permits release only to the representative. Each restriction alone leaves a recipient available; together they leave none. The clinic cannot answer "is there an admissible release?" by consulting an authorization service that reports "yes, someone may receive it" and a compliance service that reports "yes, a recipient is permitted by the hold". The two answers are correct and the conjunction is false. The information that resolves the question is the set of recipients allowed by each, and not whether the sets are empty.

The result is the first instance, in the normative setting, of a pattern from Part II: separate summaries that each answer their own question cannot in general answer the combined question (Result 8.12), and a representation for the combination must retain the finer structure of the parts.

13.3 One-step admissibility is not maintainable

The sets $\mathrm{Adm}_1(h)$ describe what may be done next. One might hope that they form a state: that the pair of the history and its admissible next events suffices to continue. They do not.

Example 13.5 (Same next events, different futures)**.** Let $\mathsf{E} = \{s, t, a, c, d\}$ and let $\mathcal{A}$ be the prefix closure of $\{sac, tad\}$. Then $\mathrm{Adm}_1(s) = \mathrm{Adm}_1(t) = \{a\}$, so the map $h \mapsto \mathrm{Adm}_1(h)$ does not separate s from t. After the event a the sets are $\mathrm{Adm}_1(sa) = \{c\}$ and $\mathrm{Adm}_1(ta) = \{d\}$. Hence $\ker \mathrm{Adm}_1$ is not a right congruence, and Adm_1 is not a maintainable representation in the sense of Definition 7.1. The coarsest maintainable representation that determines admissibility of every continuation is the Nerode quotient of $\mathcal{A}$ from Result 4.13, which separates s from t.

The example explains why Chapter 14 turns to monitors. A monitor is a state that is maintainable, in the sense that its next value is a function of its current value and the event, and from which admissibility of all continuations can be read. The next-event sets are only one projection of it.

13.4 Counting joint obstructions and minimal conflicts

Result 13.3 shows that the joint obstruction exists. Two further facts show how much of the normative obstruction it accounts for and how far the failure of additive responsibility extends when more than two restrictions are in force.

New result 13.6 (Counting the forms of the obstruction)**.** *Let $|R| = r$ and consider all pairs (A, B) of subsets of R with $A \cap B = \varnothing$, which are the configurations in which a proposed continuation with r physically executable realizations is obstructed. There are 3^r such pairs, of which $3^r - 2^{r+1} + 1$ are joint obstructions, case (iv) of Result 13.3. The proportion of joint obstructions tends to 1 as r grows. Among the $(2^r - 1)^2$ configurations in which both restrictions leave something permitted, so that the bit pair is $(1, 1)$, exactly $3^r - 2^{r+1} + 1$ are obstructed.*

Assumptions: R finite with r elements; A and B are arbitrary subsets of R, taken to be the sets permitted by the authorization and by the obligations.

Proof. A disjoint pair assigns to each element of R one of three states, in A, in B, or in neither, so there are 3^r pairs. The pairs with $A = \varnothing$ number 2^r (the choices of B), those with $B = \varnothing$ number 2^r, and the pair with both empty is counted twice, so the number with at least one set empty is $2^{r+1} - 1$ and the number of joint obstructions is $3^r - 2^{r+1} + 1$. The ratio to 3^r tends to 1 since $2^{r+1}/3^r \to 0$. The last statement counts the pairs of nonempty sets: there are $(2^r - 1)^2$ of them and the disjoint ones are exactly the joint obstructions. □

For $r = 4$ there are 81 disjoint pairs, of which 50 are joint obstructions, and 225 configurations with both sets nonempty, so a bit pair $(1, 1)$ hides a joint obstruction in 50 of 225 configurations, slightly more than one in five. For $r = 10$ the proportion of joint obstructions among the obstructed configurations is $1 - 2^{11}/3^{10} + 3^{-10} \approx 0.965$. In a system with many alternative realizations, the typical way for a continuation to be blocked is not that one restriction excludes everything but that the restrictions together have no common permitted realization.

New result 13.7 (Minimal conflicts of arbitrary size)**.** *For restrictions with permitted sets $A_1, \ldots, A_k \subseteq R$ call a set $J \subseteq \{1, \ldots, k\}$ a* conflict *if $\bigcap_{i \in J} A_i = \varnothing$, and a* minimal conflict *if no proper subset is a conflict. For every $k \geq 2$ there is a configuration with $|R| = k$ whose only minimal conflict is $\{1, \ldots, k\}$. The minimal conflicts of any configuration form an antichain, so there are at most $\binom{k}{\lfloor k/2 \rfloor}$ of them.*

Assumptions: R and the index set are finite; the sets A_i are the realizations permitted by restriction i.

Proof. Let $R = \{1, \ldots, k\}$ and $A_i = R \setminus \{i\}$. Any $k - 1$ of the sets omit a single element, and the element omitted by the missing set is in all of them, so they intersect; the whole family has empty intersection because each element of R is omitted by one set. So the only conflict is the full index set, which is therefore the only minimal conflict. Minimal conflicts form an antichain by definition, and Sperner's theorem bounds the size of an antichain of subsets of a k-element set by $\binom{k}{\lfloor k/2 \rfloor}$. □

The second result says that the obstruction can be attributable to no proper subset of the restrictions: every restriction is necessary to the conflict, and none is individually responsible. For the clinic of the earlier example, add a third restriction, a patient consent, and let the permitted recipients be the auditor and the representative (authorization), the representative and the court (hold), and the auditor and the court (consent). Each pair of restrictions leaves a recipient, and the three together leave none. A designer who tests the restrictions two at a time finds all the pairs satisfiable and concludes that the system is consistent. The failure of additivity of responsibility is thus not a feature of pairs; it extends to every size.

Exercises

Exercise 13.8. For the account of Example 1.3, with a rule that withdrawals above a limit require a second approval, identify the sets $\mathcal{P}$, $\mathcal{U}$, and $\mathcal{O}$, state which of them excludes a withdrawal exceeding the balance, a withdrawal exceeding the limit without approval, and a withdrawal in a month in which a report is pending, and compute Adm_1 after the history of Example 1.8.

Exercise 13.9. Give a system in which case (iv) of Result 13.3 occurs with $|R| = 4$ and with A and B each of size 2. Show that when $|R| = 3$ no pair of two-element sets $A, B \subseteq R$ is disjoint, and find the largest $|A| + |B|$ that case (iv) allows for a given $|R|$.

Exercise 13.10. Prove that if A and B are required to be nested ($A \subseteq B$ or $B \subseteq A$) then case (iv) cannot occur. Give a natural reason why authorization and compliance might be nested in a particular institution, and a reason they would not be.

Exercise 13.11. Show that $\mathcal{A}$ need not be the set of viable histories of Result 4.17 even when $\mathcal{P}$, $\mathcal{U}$, and $\mathcal{O}$ are each pruned. Construct an example in which an admissible history has no admissible extension.

Exercise 13.12. For the system of Example 13.5 compute the Nerode system of $\mathcal{A}$ and count its states. Then add the event b with $\mathrm{Adm}_1(h) \ni b$ for all h, leading to a history from which nothing is admissible, and recompute.

Exercise 13.13. Verify the counts of Result 13.6 for $r = 3$ by listing the 27 disjoint pairs, the 15 with at least one empty set, and the 12 joint obstructions.

Exercise 13.14. Construct a configuration of four restrictions on $R = \{1, \ldots, 4\}$ whose minimal conflicts are exactly $\{1, 2\}$ and $\{3, 4\}$, and one whose minimal conflicts are exactly $\{1, 2, 3\}$ and $\{2, 3, 4\}$. Determine in each case which single restrictions can be removed to restore an admissible realization.

Exercise 13.15. Prove that the antichain bound of Result 13.7 is attained: for $k = 4$ exhibit a configuration with $\binom{4}{2} = 6$ minimal conflicts, each of size two.

Chapter 14

Obligation Semantics

Question. How are obligations that cannot be assessed from a single instantaneous state specified and monitored?

Established sources. Linear temporal logic (Pnueli); monitorability and runtime verification (Bauer, Leucker, and Schallhart).

New results. The exact size of the minimal monitor for a fragment of bounded-response obligations over disjoint alphabets, with a matching lower bound (Result 14.6).

Proof obligations. The shared-clock case is open (Research problem 14.9).

Depends on. Chapters 4, 7, and 13.

An obligation that is created by an event and discharged by a later one cannot be assessed from the instantaneous state of the world. Whether a payment is overdue depends on when the invoice arrived; whether a report is owed depends on whether one has been sent since the incident. The specification must say what the obligation is, and the monitor must keep exactly the information about the past that the specification makes relevant. This chapter fixes a fragment, bounded-response obligations, in which both can be stated precisely, and proves the size of the smallest monitor.

14.1 Obligations and monitors

Established result 14.1 (Linear temporal logic and bad prefixes)**.** *Linear temporal logic over finite and infinite words was introduced by Pnueli as a specification language for reactive systems. A property is a* safety property *when every violation is detected on a finite prefix, in the sense of Definition 4.14, and a* monitor *for it is a deterministic machine that reads the history event by event and signals as soon as a bad prefix has been read. Safety properties are monitorable in this sense.*

Source: Pnueli, "The temporal logic of programs" (1977); Alpern and Schneider (1985); Bauer, Leucker, and Schallhart, "Runtime verification for LTL and TLTL," *ACM Transactions on Software Engineering and Methodology* 20 (2011).

Definition 14.2 (Bounded-response obligation)**.** Let $\Sigma = \{a, b, e\}$ be a three-letter alphabet and $k \geq 1$. The *bounded-response obligation* $\mathrm{Resp}(a, b, k)$ holds of a word $u \in \Sigma^*$ if for every position t with $u_t = a$ there is a position t' with $t < t' \leq t + k$ and $u_{t'} = b$, whenever $|u| \geq t + k$.

A word is *violating* if it fails this condition, that is, if some a at position t is followed by k further events none of which is b. In temporal logic the obligation is $\mathbf{G}\big(a \to Xb \vee X^2 b \vee \cdots \vee X^k b\big)$ read on finite prefixes.

The letter a creates an obligation and b discharges it; e stands for every other event of the component, and its role is to let time pass. A b discharges every obligation created before it, so only the oldest undischarged a matters.

Definition 14.3 (Monitor state)**.** For $u \in \Sigma^*$ define $\rho(u) \in \{0, 1, \ldots, k\} \cup \{\bot\}$ as follows. $\rho(\varepsilon) = 0$. From $\rho = 0$: the event a gives k; b and e give 0. From $\rho \in \{1, \ldots, k\}$: the event b gives 0; the events a and e give $\rho - 1$ if $\rho \geq 2$ and $\bot$ if $\rho = 1$. The value $\bot$ is absorbing. The value ρ is the number of further events during which the oldest pending a may remain undischarged.

New result 14.4 (Minimal monitor for one bounded-response obligation)**.** *u is violating if and only if $\rho(u) = \bot$, and ρ is a maintainable representation. Its kernel is the continuation equivalence of the property $P(u) = \mathbf{1}[u \text{ is not violating}]$, so ρ is minimal, with exactly $k + 2$ values.*

Assumptions: Alphabet $\{a, b, e\}$, $k \geq 1$, and all finite words.

Proof. That $\rho = \bot$ exactly on the violating words follows by induction on $|u|$ from the definition: ρ is the remaining allowance of the oldest pending a and reaches $\bot$ when that allowance is exhausted without b. The next value of ρ is a function of the current value and the event, so $\ker \rho$ is a right congruence. All values occur: 0 at ε, $r \in \{1, \ldots, k\}$ at $a\, e^{k-r}$, and $\bot$ at $a\, e^k$. They are pairwise distinguishable by the property P. The value $\bot$ is separated from all others by the empty word. For $r = 0 < r'$ the word $e^{r'}$ is violating from r' and not from 0. For $1 \leq r < r'$ the word e^r is violating from r and not from r', which is left with allowance $r' - r \geq 1$. Hence there are at least $k + 2$ classes of continuation equivalence, and ρ has exactly that many values, so $\ker \rho = \approx_P$. □

14.2 Several obligations

The general fragment places several such obligations side by side. Each obligation i lives in its own component alphabet, and time for obligation i is measured in events of that component.

Definition 14.5 (Disjoint bounded-response fragment)**.** Let $\mathsf{E} = \bigsqcup_{i=1}^{m} \Sigma_i$ with $\Sigma_i = \{a_i, b_i, e_i\}$. The obligation $\Phi = \bigwedge_i \mathrm{Resp}(a_i, b_i, k_i)$ is satisfied by a word $u \in \mathsf{E}^*$ if for every i the projection $\mathrm{proj}_i(u)$ is not violating in the sense of Definition 14.2. Let $\mathcal{O}_\Phi$ be the set of non-violating words.

New result 14.6 (Exact size of the minimal monitor)**.** *For Φ as in Definition 14.5 with all $k_i \geq 1$, the vector $\rho(u) = (\rho_1(\mathrm{proj}_1 u), \ldots, \rho_m(\mathrm{proj}_m u))$, with all coordinates collapsed to a single value $\bot$ when any coordinate is $\bot$, is a maintainable representation that determines whether u is violating. The minimal monitor for Φ, namely the Nerode system of $\mathcal{O}_\Phi$, has exactly*

$$\prod_{i=1}^{m} (k_i + 1) \;+\; 1$$

states, where the last state is the absorbing violation state.

Assumptions: The alphabets Σ_i are pairwise disjoint, and time for each obligation is counted in its own events. All finite words are histories.

Proof. Upper bound and correctness. Each coordinate evolves by Definition 14.3 on the events of its own component and is unchanged by events of other components. The word is violating if and only if some coordinate is $\perp$, and once any coordinate is $\perp$ the word and all its extensions are violating, so collapsing is consistent with the transitions. This gives a deterministic machine with $\prod_i (k_i+1)$ live states, indexed by the vectors in $\prod_i \{0, \dots, k_i\}$, together with one violation state.

Reachability. Every live vector is reached: for target $(r_1, \dots, r_m)$ concatenate the words $a_i\, e_i^{k_i - r_i}$ over the coordinates with $r_i \geq 1$. Events of one component do not affect another, so each coordinate acquires its target value. The violation state is reached by $a_1 e_1^{k_1}$.

Distinguishability. The violation state is separated from every live state by the empty word. Let two live vectors differ in coordinate i, with values $r \neq r'$, and suppose $r < r'$. If $r \geq 1$, apply e_i^r. This drives coordinate i of the first vector to $\perp$ and leaves the second at $r' - r \geq 1$, and it does not affect other coordinates, so the first is violating and the second is not. If $r = 0$ apply $e_i^{r'}$, which violates from r' and not from 0. Hence the live vectors are pairwise distinguishable by the property of being non-violating.

By Result 4.13 the Nerode system is minimal among transition systems for which the property is a state property, and it has exactly the number of classes of $\approx_P$. The classes are therefore the $\prod_i (k_i + 1) + 1$ states above. □

New result 14.7 (Any maintained representation of admissibility must carry the monitor)**.** *Every maintainable representation r of E^* for which $\mathcal{O}_\Phi$ is a state property takes at least $\prod_i (k_i + 1) + 1$ values. In particular a system that keeps an obligation-compliant history and must continue to know which continuations remain compliant needs, for m obligations with all $k_i \geq 1$, at least $2^m + 1$ states.*

Assumptions: As in Result 14.6.

Proof. By Result 4.13(b), any transition system that generates the histories and has $\mathcal{O}_\Phi$ as a state property has at least as many states as classes of $\approx_P$. A maintainable representation induces such a system. The count is Result 14.6, and $\prod_i (k_i + 1) \geq 2^m$. □

The state of the monitor is the obligation component of the retained representation σ of Chapter 1. The earlier example of a pending-report flag is the case $m = 1$, $k = 1$. A representation that records the balance of an account but not the age of the oldest unreported incident has a kernel that does not contain $\ker \rho$, and the representational obstruction of Chapter 1 follows.

Remark 14.8. In the fragment of Definition 14.5 every live monitor state is viable: from any state the discharging events b_i are always available and move all coordinates to 0. The set $\mathcal{O}_\Phi$ is therefore the pruned history set of Result 4.18, and the admissible next events at a state are exactly those that do not lead to the violation state. The one-step map Adm_1 of Chapter 13 is a function of the monitor state but not conversely.

Research problem 14.9. Replace the disjoint alphabets by a single shared alphabet in which every event advances every deadline. Live vectors then have distinct ages and a vector may be doomed, in the sense that no continuation avoids violation, because two obligations expire

at the same step and a single event cannot discharge both. Determine the number of states of the minimal monitor for m obligations $\mathrm{Resp}(a_i, b_i, k_i)$ on the alphabet $\{a_i, b_i\}_{i \le m} \cup \{e\}$, in particular the number of reachable states that are doomed and the effect of treating doomed states as violations (monitoring for inevitable violation) rather than as live (monitoring for realized violation).

14.3 Data for the shared-clock problem

Research problem 14.9 asks for the size of the minimal monitor when every event advances every deadline. Two obligations already show how different the answer is from the disjoint case. Take the alphabet $\{a_1, b_1, a_2, b_2, e\}$ and the following semantics, which extend Definition 14.3 coordinatewise: every event advances every pending obligation by one step, so that a pending coordinate with allowance 1 becomes the violation value unless the event is its own discharging letter; the event b_i sets coordinate i to 0; the event a_i sets coordinate i to k_i if it is 0 and otherwise acts as e on that coordinate.

New result 14.10 (Reachable states of two shared-clock obligations of equal length)**.** *For two obligations with the same deadline k on a shared clock, the live states reachable from $(0,0)$ are exactly the pairs $(r_1, r_2) \in \{0, \dots, k\}^2$ other than those with $r_1 = r_2 \neq 0$. There are $k^2 + k + 1$ of them, and none is doomed. By contrast the disjoint-alphabet monitor of Result 14.6 has $(k+1)^2$ live states.*

Assumptions: Two obligations, equal deadline k, shared clock as defined above; states are the allowance vectors.

Proof. The two obligations are created at different events, so when both are pending the earlier one has the strictly smaller allowance, and $r_1 \neq r_2$ whenever both are nonzero. Every pair with at most one nonzero coordinate is reached: $(r, 0)$ by $a_1 e^{k-r}$, $(0, r)$ by $a_2 e^{k-r}$, and $(0, 0)$ by the empty word. For a pair with $r_2 > r_1 \ge 1$ put $d = r_2 - r_1$ and apply $a_1 e^{d-1} a_2 e^{k-r_2}$: the first coordinate is k after a_1, equals $k - d$ after e^{d-1} and a_2, and r_1 at the end, while the second coordinate is k after a_2 and r_2 at the end; the first coordinate decreases to its final value $r_1 \ge 1$, so it never becomes the violation value on the way. The case $r_1 > r_2 \ge 1$ is symmetric. Counting, there are $(k+1)^2 - k = k^2 + k + 1$ pairs. No state is doomed: from a state with one pending coordinate the discharging letter clears it, and from a state with $r_1 < r_2$ both nonzero the letter b_1 gives $(0, r_2 - 1)$ with $r_2 - 1 \ge r_1 \ge 1$, and then b_2 clears the rest; the case $r_1 > r_2$ is symmetric. □

Minimization by partition refinement on the reachable machines gives the sizes below, which count the violation state as a state. The column "inevitable" monitors for inevitable violation, merging doomed states with the violation state, and the last column is the disjoint-alphabet size of Result 14.6.

deadlines	live reachable	minimal (realized)	minimal (inevitable)	doomed	disjoint
$(1,1)$	3	4	4	0	5
$(2,1)$	5	6	5	1	7
$(2,2)$	7	8	8	0	10
$(3,2)$	10	11	10	1	13
$(3,3)$	13	14	14	0	17
$(4,2)$	13	14	13	1	16
$(1,1,1)$	4	5	5	0	9
$(2,2,2)$	13	14	14	0	28
$(3,2,1)$	15	13	12	4	25

For two obligations the data are explained by the following result, which settles the count for equal deadlines.

New result 14.11 (No live states merge for two shared-clock obligations)**.** *For two obligations with deadlines $k_1, k_2 \geq 1$ on a shared clock, the live states reachable from $(0,0)$ are pairwise inequivalent in the continuation equivalence of the property of non-violation. Hence the minimal realized monitor has exactly one more state than there are reachable live states, the extra state being the violation sink. For equal deadlines k it has $k^2 + k + 2$ states.*

Assumptions: Two obligations on the shared alphabet $\{a_1, b_1, a_2, b_2, e\}$ with the semantics above; states are allowance vectors.

Proof. Let (x, y) be a live state. The word $b_1 e^n$ acts as follows: b_1 sets the first coordinate to 0 and advances the second coordinate, which becomes the violation value at once if $y = 1$ and becomes $y - 1$ if $y \geq 2$; thereafter e^n advances the second coordinate by one step each, and it violates exactly at $n = y - 1$. Therefore the shortest word of the form $b_1 e^n$ that violates from (x, y) has length y, and no such word violates if $y = 0$. In the same way the shortest violating word of the form $b_2 e^n$ has length x, and none exists if $x = 0$. Two distinct live states differ in x or in y and are therefore separated by one of these words. A live state is separated from the violation sink by the empty extension, which is already a violation at the sink and not at a live state. So no two states of the reachable machine are equivalent, and the minimal realized monitor is the reachable machine. For equal deadlines the count of live states is $k^2 + k + 1$ by Result 14.10. □

The argument uses that only two coordinates are present: with three obligations the word $b_1 e^n$ reveals only the larger of the two remaining coordinates, and merging does occur, as the table shows for $(3, 2, 1)$. The pattern in the table is suggestive. For two obligations a doomed state is reachable exactly when the deadlines differ; for $(3, 2, 1)$ there are four doomed states, and the realized monitor is smaller than the number of reachable states plus one, so merging does occur once three obligations compete; and the shared-clock monitors are consistently smaller than the disjoint ones, by a margin that grows with the number of obligations (from 5 to 4 for $(1, 1)$ and from 28 to 14 for $(2, 2, 2)$). The data are evidence about the shape of an answer and not an answer to the problem.

Exercises

Exercise 14.12. For $m = 2$, $k_1 = 2$, $k_2 = 1$ list the live monitor states of Result 14.6 and the transitions on each of the six letters. Verify that the state count is $(3)(2) + 1 = 7$.

Exercise 14.13. Show by an explicit pair of histories that a representation consisting only of the number of a_1 events not yet followed by b_1 is not sufficient for non-violation of $\mathrm{Resp}(a_1, b_1, 2)$, and describe what it fails to retain.

Exercise 14.14. Change the semantics so that each a creates its own obligation that only a later b within k events can discharge, and a b discharges only the oldest pending obligation. Compute the minimal monitor for $k = 2$ and compare its size with that of Result 14.4.

Exercise 14.15. Prove that if the same event b discharges two obligations with different deadlines, the disjointness hypothesis of Definition 14.5 fails. Describe how the product bound of Result 8.5 applies and why the exact count of Result 14.6 need not.

Exercise 14.16. An obligation of the form $\mathbf{G}(a \to \mathbf{F}b)$ is a liveness property. Show that its non-violating histories are all histories, and explain in terms of Result 4.17 why the monitor of this chapter does not apply and what a monitor for the viability of the obligation must record.

Exercise 14.17. List the 13 live states of the shared-clock monitor for three obligations of deadline 2 and verify by hand that the number of reachable states is smaller than the 27 of the disjoint case. Identify which vectors of the product are unreachable and give the invariant that excludes them.

Exercise 14.18. For deadlines $(2, 1)$ find the doomed state, a word that reaches it, and the shortest word that violates from it. Show that a monitor for inevitable violation signals strictly earlier than a monitor for realized violation on this word, and compute by how many events.

Exercise 14.19. Prove that for two obligations on a shared clock with deadlines $k_1 > k_2$ the live reachable states include every pair $(r_1, 0)$ with $1 \le r_1 \le k_1$ and every pair $(0, r_2)$ with $1 \le r_2 \le k_2$, and that a doomed state is reachable. Find the doomed state for $(3, 2)$.

Chapter 15

Identity Through Continuation

Question. Which identity criteria are appropriate relative to the operations a system promises to support?

Established sources. Observational equivalence relative to a signature; the Myhill–Nerode congruence.

New results. The coarsest identity criterion preserving a family of promised observations is their joint kernel, and the coarsest one that also survives every promised update is the continuation equivalence of the family (Result 15.2); enlarging the promises refines identity and cannot be undone (Result 15.4).

Proof obligations. None.

Depends on. Chapters 5, 7, and 11.

Two records are the same object when nothing a system promises to do can tell them apart. That sentence is a definition schema, and the content depends on what is promised. Promising to report a balance makes two accounts with the same balance the same, for that purpose. Promising to report whether the account was ever overdrawn makes them different if their histories differ in that respect. This chapter states the schema precisely, proves that it has a unique coarsest instance for each set of promises, and shows how identity responds when promises are added.

15.1 Promised operations and identity criteria

Work with histories $\mathcal{H} \subseteq \mathsf{E}^*$ as in Chapter 4. A system promises two kinds of operation. An *observation* is a function $q : \mathcal{H} \to B_q$ that reports something about the current history without changing it. An *update* is an event $a \in \mathsf{E}$ that extends the history. A *promise* is a pair $(\mathcal{Q}, F)$ of a family $\mathcal{Q}$ of observations and a submonoid $F \subseteq \mathsf{E}^*$ of promised update sequences, the *future the system must support*. The promise $(\mathcal{Q}, \{\varepsilon\})$ consists of observations alone.

Definition 15.1 (Identity criterion)**.** An *identity criterion* is an equivalence relation $\sim$ on $\mathcal{H}$. It *preserves the observations* $\mathcal{Q}$ if $h \sim h'$ implies $q(h) = q(h')$ for all $q \in \mathcal{Q}$. It *preserves the promise* $(\mathcal{Q}, F)$ if, in addition, $h \sim h'$ implies $hw \in \mathcal{H} \Leftrightarrow h'w \in \mathcal{H}$ and $hw \sim h'w$ for every $w \in F$ with $hw \in \mathcal{H}$.

An identity criterion that preserves the promise is one that the system can use to treat identified histories as one object: whatever is observed, now or after any promised sequence of updates, is the same for both.

New result 15.2 (The coarsest criterion preserving a promise)**.** *Let $\mathcal{Q}$ be a family of observations and $F \subseteq \mathsf{E}^*$.*

(a) The coarsest identity criterion preserving the observations $\mathcal{Q}$ is the joint kernel $\kappa_{\mathcal{Q}} = \bigcap_{q\in\mathcal{Q}} \ker q$.

(b) For $F = \mathsf{E}^*$, the coarsest identity criterion preserving the promise $(\mathcal{Q}, \mathsf{E}^*)$ is the continuation equivalence

$$h \approx_{\mathcal{Q}} h' \iff \mathrm{Ext}(h) = \mathrm{Ext}(h') \text{ and } \forall w \in \mathrm{Ext}(h),\ \forall q \in \mathcal{Q} :\ q(hw) = q(h'w),$$

which is the largest right congruence contained in $\kappa_{\mathcal{Q}}$.

(c) For a submonoid F, the coarsest criterion preserving $(\mathcal{Q}, F)$ is $h \approx^F_{\mathcal{Q}} h'$ if and only if, for all $w \in F$, $hw \in \mathcal{H} \Leftrightarrow h'w \in \mathcal{H}$ and, when $hw \in \mathcal{H}$, $q(hw) = q(h'w)$ for all $q \in \mathcal{Q}$. For $F = \{\varepsilon\}$ this is $\kappa_{\mathcal{Q}}$ and for $F = \mathsf{E}^*$ it is $\approx_{\mathcal{Q}}$.

Assumptions: The family $\mathcal{Q}$ may be infinite; $\mathcal{H}$ is prefix closed so that Ext is defined; F is a submonoid of E^*.

Proof. (a) An equivalence relation preserves $\mathcal{Q}$ exactly when it is contained in every $\ker q$, hence in $\kappa_{\mathcal{Q}}$, and $\kappa_{\mathcal{Q}}$ is itself such a relation. (b) The relation $\approx_{\mathcal{Q}}$ is an equivalence, is contained in $\kappa_{\mathcal{Q}}$ by taking $w = \varepsilon$, and is a right congruence: if $h \approx_{\mathcal{Q}} h'$ and $u \in \mathrm{Ext}(h)$ then $\mathrm{Ext}(hu) = \{w : uw \in \mathrm{Ext}(h)\} = \mathrm{Ext}(h'u)$ and $q(huw) = q(h'uw)$ for all such w, so $hu \approx_{\mathcal{Q}} h'u$. Hence it preserves the promise with $F = \mathsf{E}^*$. Conversely, a relation $\sim$ that preserves the promise satisfies, for $h \sim h'$ and $w \in \mathrm{Ext}(h)$, $hw \sim h'w$ and therefore $q(hw) = q(h'w)$, and has equal extensions; so $\sim \subseteq \approx_{\mathcal{Q}}$. This is Result 7.3 for the family $\mathcal{Q}$ and the full set of updates. (c) The relation $\approx^F_{\mathcal{Q}}$ is an equivalence and, taking $w = \varepsilon \in F$, preserves $\mathcal{Q}$. If $u \in F$ and $hu \in \mathcal{H}$, then for $w \in F$ we have $uw \in F$ because F is a submonoid, so $huw \in \mathcal{H} \Leftrightarrow h'uw \in \mathcal{H}$ and the observations agree; hence $hu \approx^F_{\mathcal{Q}} h'u$. Conversely any $\sim$ preserving the promise satisfies, for $h \sim h'$ and $w \in F$, the membership condition and $hw \sim h'w$, so $q(hw) = q(h'w)$. Thus $\sim \subseteq \approx^F_{\mathcal{Q}}$. This is the restricted-update theory of Chapter 7. □

The account illustrates the three levels.

Example 15.3 (Identity of accounts under three promises)**.** Take the histories of Example 1.3.

(i) If the system promises only to report the current balance, $\mathcal{Q} = \{\beta\}$ and $F = \{\varepsilon\}$, identity is $\ker \beta$: two accounts are the same if their balances agree. If the system also promises to support further deposits and withdrawals, $F = \mathsf{E}^*$, identity is still $\ker \beta$, because $\beta(hw) = \beta(h) + \beta(w)$ makes $\ker \beta$ a right congruence.

(ii) If the system promises to report whether the account has ever been overdrawn, $\mathcal{Q} = \{q_{\mathrm{neg}}\}$ and $F = \{\varepsilon\}$, identity is $\ker q_{\mathrm{neg}}$, which has two classes. With $F = \mathsf{E}^*$ identity becomes the continuation quotient of Result 2.13: balances matter for accounts not yet overdrawn, and all overdrawn accounts are identified.

(iii) With both promises the criterion is the continuation equivalence of the pair, which is strictly finer than either.

15.2 Promises refine identity

New result 15.4 (Enlarging the promise refines identity)**.** *(a) If $\mathcal{Q} \subseteq \mathcal{Q}'$ and $F \subseteq F'$ then $\approx^{F'}_{\mathcal{Q}'} \subseteq \approx^F_{\mathcal{Q}}$.*

(b) The inclusion in (a) can be strict at each of the three levels (observations, updates, and both together).

(c) If a system has identified two histories $h \sim h'$ under a promise and later adds an observation or update that separates them, no operation of the system on the identified object can recover which of the two it was.

Assumptions: As in Result 15.2.

Proof. (a) Preserving a larger promise is a stronger requirement, so every criterion preserving $(\mathcal{Q}', F')$ preserves $(\mathcal{Q}, F)$. (b) In Example 15.3, $\{\beta\} \subseteq \{\beta, q_{\text{neg}}\}$ with $F = \{\varepsilon\}$ gives strict refinement; $F = \{\varepsilon\} \subseteq F' = \mathsf{E}^*$ with $\mathcal{Q} = \{q_{\text{neg}}\}$ gives strict refinement from two classes to the continuation quotient; combining them gives the third case. (c) If $\sim$ is the criterion in use and the system's only retained information about the object is the class under $\sim$, then by Result 2.11 no function of that class separates h from h'. □

Part (c) is the identity analogue of the irreversibility of coarsening. A system that merges records by a coarse criterion has made a commitment about what it will never be asked, and widening the promise later is only possible for records that were kept apart.

Remark 15.5. The criterion that preserves every conceivable observation is the identity relation on histories, realized by the injective hash-linked representation of Result 11.1. Chapter 11 used the kernel of an audit family as an identity criterion for revision lineages; this chapter shows that it is the case $F = \{\varepsilon\}$ of the same schema, and that when audits are to be answerable after further revisions the criterion must be the continuation equivalence.

15.3 How an undo operation enlarges identity

The promise to report the current text of a document and to support further edits identifies two histories whenever they have the same text, since a later edit overwrites what came before. The promise changes if an operation is added that reaches back into the past. Let a single section take contents from a set C with $|C| = k \geq 2$ and initial content c_0, and let the events be the overwrites $\text{set}(c)$ together with undo, which restores the content held before the most recent overwrite and has no effect if no overwrite has occurred since the start or since the last undo.

New result 15.6 (Identity of documents with one-level undo)**.** *For the promise "report the current text" with F the submonoid of all sequences of events, the continuation equivalence has exactly k^2 classes. Without the undo event it has k classes. A class is determined by the current content c together with either the statement that undo would change nothing or the content $p \neq c$ that undo would restore.*

Assumptions: One section, k contents, undo of depth one: after an undo a further undo has no effect until the next overwrite.

Proof. Track the pair (c, p) of current content and previous content, with $p = \perp$ before any overwrite and after an undo. The event $\text{set}(c')$ sends (c, p) to (c', c); the event undo sends (c, p) to $(p, \perp)$ if $p \neq \perp$ and leaves it unchanged otherwise. The pairs $(c, \perp)$ and (c, c) behave identically: $\text{set}(c')$ sends both to (c', c), and undo sends the first to itself and the second to $(c, \perp)$, which has the same behaviour, so they are identified. Every other pair (c, p) with

$p \notin \{\bot, c\}$ is reached by $\mathrm{set}(p)\,\mathrm{set}(c)$, and the pairs $(c, \bot)$ are reached for every c. This gives k classes with trivial previous content and $k(k-1)$ with a distinct previous content, in all k^2 candidates. They are pairwise separated: different current contents are separated by the empty word; with equal current content c and previous contents $p \neq p'$, at least one of them, say p, is a content different from c, and the word undo gives the text p in one case and either $p' \neq p$ or $c \neq p$ in the other. Without the undo event the previous content is never observed, and the classes are the k current contents. □

For two contents the four classes are the text 0 with nothing to undo, the text 1 with nothing to undo, the text 0 with 1 to restore, and the text 1 with 0 to restore, and the table of futures confirms that no two have the same behaviour; for three contents there are nine, a figure also confirmed by comparing the texts after all event sequences of length at most five. The lesson is general. A promise about the past, here that undo will restore an earlier text, converts information that an overwriting update discards into information that identity must retain, and the amount retained grows from k to k^2 for a single level of undo. A criterion of identity chosen when only edits were promised is no longer valid when undo is added: two documents that were rightly the same become different, in the sense of Result 15.4(c), and the system cannot separate them afterwards unless the previous contents were kept.

Exercises

Exercise 15.7. In the document of Example 1.2, compute the identity criterion for the promise "report the current text" with $F = \{\varepsilon\}$ and with F the submonoid of all sequences of edits. Decide whether two documents with the same text but different held contents are identified in each case.

Exercise 15.8. Add an undo operation to the document events, which restores the text before the last edit. Show that the continuation equivalence for the promise "report the current text" now separates histories that it identified without undo, and compute the minimal number of classes for documents of at most two edits.

Exercise 15.9. Prove that $\approx_Q^F$ is an equivalence relation and that it contains $\approx_Q$, and give an example where the containment is strict.

Exercise 15.10. Two replicas of a counter agree on every observation that either of them has been asked so far. Argue that this does not imply they are the same object for the promise to support future increments, and give the condition under which agreement on past observations suffices.

Exercise 15.11. A system identifies records by a content hash of the current text and later promises to report who last edited each section. Using Result 11.6, state exactly which pairs of records can no longer be re-separated and which could still be separated if the original lineages had been kept.

Exercise 15.12. Extend Result 15.6 to undo of depth two, in which the system keeps a stack of the two most recent previous contents and each undo pops it, and show that for $k = 2$ there are 8 classes. Identify the general pattern for undo of depth d and k contents.

Exercise 15.13. Add a redo event that reverses the most recent undo. Describe the continuation equivalence for $k = 2$ and show that it has 6 classes, more than the 4 of Result 15.6, listing a separating future for each pair.

Exercise 15.14. Show that the criterion of Result 15.6 is not a refinement of the criterion for the promise "report the current text" alone but is a refinement of the criterion for the promise "report the current text" with F consisting of overwrites only. Explain the difference.

Chapter 16

Disposition and Accountable Termination

Question. Under what assumptions can each unit of work be disposed of exactly once?

Established sources. Impossibility of coordinated action over unreliable channels (the two generals problem); idempotence and deduplication.

New results. Exactly-once application of effects under stated assumptions of identity, atomic recording, and fair channels (Result 16.3); a tight lower bound on deduplication memory (Result 16.5); failure of the guarantee when recording and applying are not atomic (Result 16.4).

Proof obligations. None.

Depends on. Chapters 7, 11, and 15.

A unit of work, a payment, a message to be acted on, a record to be deleted, ends in exactly one way when the system is accountable for it: it is done, and it is done once. Failures make this hard to guarantee. A message can be lost, a reply can be lost, a process can crash between acting and recording that it has acted. This chapter states the impossibility that governs the problem, the assumptions under which the guarantee can nonetheless be given, and how much memory the guarantee costs.

16.1 What cannot be done

Established result 16.1 (Coordinated action over an unreliable channel)**.** *There is no protocol by which two parties communicating over a channel that may lose any message can be certain to reach agreement on a joint action, in the sense that each acts if and only if the other acts, in every execution that may lose messages, unless one of them can act without the other's knowledge.*

Source: Akkoyunlu, Ekanadham, and Huber (1975); Gray, "Notes on data base operating systems" (1978); Halpern and Moses, "Knowledge and common knowledge in a distributed environment," *Journal of the ACM* 37 (1990).

The consequence for disposition is that no protocol over a channel that may drop messages can guarantee that the sender knows the effect occurred exactly when it occurred. The guarantees that follow therefore hold relative to assumptions that exclude arbitrary loss, and a claim of exactly-once disposition that does not state those assumptions is incomplete.

16.2 Assumptions that make exactly-once effects possible

The model has a sender, a receiver, and a channel. The sender holds units of work, each with an identifier from a set I. The channel may drop, duplicate, or reorder messages. The receiver is a deterministic machine with durable state; on each delivered message $\mathrm{msg}(u)$ it changes state and may produce an output $\mathrm{apply}(u)$, which is the effect to be disposed of, and an acknowledgement.

Definition 16.2 (Assumptions and the exactly-once property)**.** Four assumptions are made.

(A1) *Identity.* Each unit has a unique identifier $u \in I$ that is carried by every message about it.

(A2) *Record.* The receiver retains a set $D \subseteq I$ of identifiers whose effect it has applied.

(A3) *Atomicity.* On delivery of $\mathrm{msg}(u)$ the receiver, in a single step, tests $u \in D$, applies the effect and adds u to D if $u \notin D$, and acknowledges in either case.

(A4) *Fairness.* If the sender sends a message infinitely often, the channel delivers it infinitely often, in each direction; and the sender retransmits $\mathrm{msg}(u)$ until it receives an acknowledgement for u.

A system has the *exactly-once property* if in every execution consistent with the assumptions, each submitted unit u has $\mathrm{apply}(u)$ output exactly once, and the sender eventually stops retransmitting u.

The accounting boundary is part of the assumption: the exactly-once property concerns the effects the receiver itself applies in the single step of (A3). Effects outside that step, such as an email sent in the course of processing, are subject to the same limits as any other message over an unreliable channel.

New result 16.3 (Exactly-once effects under (A1) to (A4))**.** *Under (A1) to (A4) the protocol in which the sender retransmits until acknowledged and the receiver deduplicates against D has the exactly-once property.*

Assumptions: (A1) to (A4) of Definition 16.2. The receiver does not lose D.

Proof. At most once. By (A3) the effect for u is applied only when $u \notin D$, and u is then added to D in the same step. By (A2) D is retained. Hence after the first application every later delivery finds $u \in D$ and applies nothing.

At least once and termination. While the sender has no acknowledgement it sends $\mathrm{msg}(u)$ infinitely often, so by fairness the receiver receives it. If $u \notin D$ at that moment, the effect is applied; in either case the receiver acknowledges. Thus $u \in D$ after the first delivery, and the receiver sends an acknowledgement on every delivery. The sender keeps retransmitting until an acknowledgement arrives; each retransmission is a delivery producing an acknowledgement, and infinitely many acknowledgements are sent, so by fairness one arrives and the sender stops. □

New result 16.4 (Without atomicity the guarantee fails in one of two ways)**.** *Suppose (A3) is replaced by two steps, apply and record, in either order, between which the receiver may crash and restart with its durable state. If the receiver records first and then applies, there is an execution in which the effect for u is never applied. If it applies first and then records, there is an execution in which the effect for u is applied twice.*

Assumptions: Crashes may occur between the two steps and the retransmitting sender behaves as in (A4). The crash loses only what has not been written to durable state.

Proof. Record then apply: deliver $\mathrm{msg}(u)$, record u in D, crash before applying. After restart $u \in D$, so every retransmission is treated as a duplicate and the effect is never applied. Apply then record: deliver $\mathrm{msg}(u)$, apply the effect, crash before recording. After restart $u \notin D$, the retransmission applies the effect a second time. □

The two orders correspond to the familiar at-most-once and at-least-once disciplines. Exactly-once is obtained by making the application and the recording one step, which in practice means placing both inside one atomic transaction on the same store.

16.3 The memory cost of deduplication

The receiver in Theorem 16.3 keeps the set D. The next result shows that this is not a design choice but a necessity when the sender's retransmission is unbounded.

New result 16.5 (Deduplication memory)**.** *Let I be finite and let R be any deterministic receiver (a Mealy machine over input alphabet $\{\mathrm{msg}(u) : u \in I\}$ with output $\mathrm{apply}(u)$ or none) such that on every finite sequence of deliveries, with arbitrary duplication and reordering, each identifier that occurs is applied exactly once in total. Then R has at least $2^{|I|}$ reachable states, and the machine that tracks D has exactly $2^{|I|}$.*

Assumptions: Arbitrary duplication of any identifier at any later time is possible. The receiver sees only the sequence of deliveries.

Proof. For $S \subseteq I$ let q_S be the state reached by delivering the identifiers in S once each in a fixed order. Let $S \neq S'$ and choose $u \in S \setminus S'$ (the other case is symmetric). Deliver u once more after the sequence for S: the total number of applications of u must still be 1, and it was already 1, so R outputs nothing. Deliver u after the sequence for S': u has not yet occurred, so it must be applied and R outputs $\mathrm{apply}(u)$. The two states respond differently to the same input, hence $q_S \neq q_{S'}$. There are $2^{|I|}$ subsets, so R has at least that many states. The machine whose state is D and whose transition adds the delivered identifier has exactly $2^{|I|}$ states and meets the specification. □

The bound is exactly the lower bound for a record that distinguishes sets of identifiers by what a retransmission would do. It holds for unbounded duplication. If duplicates of an identifier can arrive only within a bounded window after its first delivery, the receiver can discard old entries, and the bound weakens accordingly (Exercise 16.8). The practical content is that garbage collection of the record is a statement about the sender: it is safe only if the sender can no longer retransmit, and the receiver can know this only if the protocol delivers that fact to it, which by Result 16.1 cannot be done with certainty over a channel that may lose messages. Every deployed system therefore either bounds retransmission by an assumption about time or keeps the record.

16.4 How often the non-atomic orders fail

Result 16.4 shows that each non-atomic order fails in some execution. A design decision is better informed by the rate of failure under an assumed crash model, and the rates are exact under a simple one. Suppose that on each delivery that reaches the two-step handler the receiver crashes between the steps independently with probability $c \in (0, 1)$, that a crash loses nothing already written to durable state, and that the sender keeps retransmitting until an acknowledgement is received, so that the unit is delivered again after every crash.

New result 16.6 (Failure rates of the two orders)**.** *Under the crash model above, with the order apply-then-record the number N of applications of the effect for a unit satisfies $\Pr[N = n] = c^{n-1}(1 - c)$ for $n \geq 1$, so that $\Pr[N \geq 2] = c$ and $\mathbb{E}[N] = 1/(1 - c)$, and the expected number of surplus applications is $c/(1-c)$. With the order record-then-apply, the unit is never applied with probability exactly c and applied exactly once with probability $1 - c$. With the atomic step of (A3), $N = 1$ with probability one.*

Assumptions: Independent crash probability c at each delivery of the two-step handler; the sender retransmits after every crash; the channel delivers each retransmission.

Proof. With apply-then-record, a delivery with $u \notin D$ applies the effect and then crashes with probability c before recording, in which case the state is unchanged and the next retransmission faces the same situation; the delivery that does not crash records u, after which all deliveries are treated as duplicates. So the number of applications is the number of deliveries up to and including the first non-crashing one, which is geometric with success probability $1 - c$. With record-then-apply, the first delivery records u; if the crash occurs before the application, which has probability c, every later delivery is a duplicate and nothing is applied, and otherwise the effect is applied once and all later deliveries are duplicates. □

For a crash probability of 0.01 per handled message, the apply-first order applies the effect twice or more for one unit in a hundred, and the expected number of surplus applications is 0.0101 per unit, about ten thousand per million units; the record-first order drops one unit in a hundred. For $c = 0.001$ the two figures become one in a thousand, and for $c = 0.1$ one in ten, with expected surplus 0.111 per unit in the apply-first order. The apply-first order is the usual choice because a duplicate can be made harmless by making the effect idempotent, which is the subject of the next remark, whereas a lost effect cannot be recovered at all; the rates show that the choice is not a matter of principle but of which error the effect can absorb.

Remark 16.7. An effect is *idempotent* if applying it twice is the same as applying it once. For an idempotent effect the apply-first order has exactly-once *outcome* even though it has at-least-once *application*, since the state of the world after $N \geq 1$ applications equals the state after one. The distinction between exactly-once application and exactly-once outcome is the distinction between a guarantee about the receiver's steps and a guarantee about the state they leave. It is the second that most users of the system rely on, and it is available without atomicity precisely when the effect has this algebraic property.

Exercises

Exercise 16.8. Suppose duplicates of an identifier can be delivered only within the next W deliveries after its first delivery. Show that a receiver that remembers the last W delivered

identifiers satisfies the specification of Result 16.5, and compare its number of states with $2^{|I|}$. Show that for $|I| = 10$ and $W = 3$ the window store has 1111 states and is larger than 2^{10}, while for $|I| = 100$ it has 1010101 states against 2^{100}, and say what this shows about when a bounded window saves memory.

Exercise 16.9. Prove that if identifiers are not unique, so that two distinct units can carry the same identifier, no receiver can have the exactly-once property for both units. Relate this to the identity criteria of Chapter 15.

Exercise 16.10. The sender retransmits until acknowledged, and the acknowledgement is itself acknowledged, so that the receiver may delete u from D on the second acknowledgement. Show that the second acknowledgement can be lost, leaving the receiver unable to decide whether to delete, and identify the point at which Result 16.1 applies.

Exercise 16.11. An effect consists of two operations, a database write inside the receiver's store and an external notification. Show that exactly-once holds for the write under (A1) to (A4), and give an execution in which the notification is delivered twice or not at all. State the accounting boundary for which the guarantee holds.

Exercise 16.12. Without fairness, give an execution in which the sender retransmits forever, the receiver has applied the effect, and no acknowledgement ever arrives. Explain which part of the exactly-once property fails and which survives.

Exercise 16.13. Verify the formulas of Result 16.6 for $c = \frac{1}{2}$ by summing the distribution of N, and compute the probability that a batch of 1000 units has at least one surplus application in the apply-first order, and at least one lost unit in the record-first order, for $c = 0.001$.

Exercise 16.14. Classify the following effects as idempotent or not: setting a field to a value, incrementing a counter, appending a line to a log, inserting a row with a given key into a table with a uniqueness constraint, and charging a payment with an idempotency token supplied by the sender. For each non-idempotent effect, describe a modification that makes it idempotent and say what record the modification adds.

Exercise 16.15. Modify the crash model so that the crash probability depends on the step, c_1 before applying and c_2 between applying and recording, in the apply-first order, where c_1 is the probability of a crash before the effect is applied. Compute the distribution of N and show that it depends only on c_2. Explain what this says about where in the handler reliability engineering effort is best spent.

Part V

Degradation and distributed systems

Chapter 17

Admissible Degradation

Question. Which shedding of capability preserves the obligations a system must still meet?

Established sources. Dependency graphs and closure operators; Horn clauses and least fixed points.

New results. The admissible shedding sets are exactly those disjoint from some self-supporting set of capabilities (Result 17.3); with a single dependency per capability the admissible sets are those disjoint from one fixed set, and with alternative dependencies no additive capacity measure determines admissibility (Result 17.4).

Proof obligations. None.

Depends on. Chapters 13 and 14.

A system under stress gives something up. What it can afford to give up depends on what it still owes. A hospital may close elective surgery and keep the emergency department; a web service may drop recommendations and keep payments. The decision is easy to make badly when capacity is summarized by a single number, such as the fraction of servers remaining or the fraction of features enabled. This chapter gives the dependency structure that governs which losses are survivable, characterizes the survivable ones exactly, and shows why no single number can do the work.

17.1 Dependencies and shedding

Definition 17.1 (Dependency system)**.** A *dependency system* consists of a finite set C of capabilities and a finite set $\mathcal{R}$ of rules (c, P) with $c \in C$ and $P \subseteq C$, read as: c can be provided if every member of P is provided. A rule with $P = \varnothing$ makes c a primitive. Several rules with the same head are alternatives. For a *shed set* $D \subseteq C$, the capabilities that can still be provided are $\mathrm{Prov}(D)$, the least set X such that, for every rule $(c, P) \in \mathcal{R}$, $c \notin D$ and $P \subseteq X$ imply $c \in X$. An *obligation* j requires a set $N_j \subseteq C$ of capabilities. For a set J of obligations that must still be met, put $\mathrm{Req} = \bigcup_{j \in J} N_j$. A shed set D is *admissible* if $\mathrm{Req} \subseteq \mathrm{Prov}(D)$.

The least set exists because the operator $X \mapsto \{c \notin D : \exists (c, P) \in \mathcal{R},\ P \subseteq X\}$ is monotone on a finite lattice. The definition is a restriction of the usual semantics of Horn clauses, with shedding as the removal of heads.

Definition 17.2 (Supports)**.** A set $S \subseteq C$ is a *support* if $\mathrm{Req} \subseteq S$ and S is self-supporting: the least set X with $(c, P) \in \mathcal{R}$, $c \in S$, $P \subseteq S$, $P \subseteq X$ implying $c \in X$ equals S. A support is *minimal* if no proper subset is a support.

A support is a set of capabilities that together provide the required ones with nothing outside the set: every member has a rule whose premises lie in the set, and the rules can be applied in a well-founded order.

New result 17.3 (Characterization of admissible shedding)**.** *Let* $(C, \mathcal{R})$ *be a dependency system and* $\mathrm{Req} \subseteq C$*.*

(a) $D \subseteq D'$ *implies* $\mathrm{Prov}(D') \subseteq \mathrm{Prov}(D)$*. Consequently the family of admissible shed sets is closed under taking subsets.*

(b) A shed set D *is admissible if and only if* $D \cap S = \varnothing$ *for some support* S*. Equivalently the admissible shed sets are* $\bigcup_S 2^{C \setminus S}$ *over the minimal supports* S*, and the maximal admissible shed sets are the complements of the minimal supports.*

(c) If every capability is the head of exactly one rule and $\mathrm{Req} \subseteq \mathrm{Prov}(\varnothing)$*, then there is a unique minimal support* S^**, and* D *is admissible if and only if* $D \cap S^* = \varnothing$*.*

Assumptions: C and $\mathcal{R}$ finite. Part (c) assumes the unshed system provides the required capabilities.

Proof. (a) The operator defining $\mathrm{Prov}(D')$ is pointwise below that of $\mathrm{Prov}(D)$ since fewer heads are allowed, so its least fixed point is smaller. Admissibility of D' then gives $\mathrm{Req} \subseteq \mathrm{Prov}(D') \subseteq \mathrm{Prov}(D)$.

(b) Suppose D is admissible and put $S = \mathrm{Prov}(D)$. Then $\mathrm{Req} \subseteq S$ and $S \cap D = \varnothing$ because the defining operator never adds a member of D. Every member of S was added at some stage by a rule whose premises were already in S, so the least set built from rules inside S contains S, and it is contained in S by definition; thus S is a support. Conversely let S be a support with $S \cap D = \varnothing$. The rules used to build S have heads and premises in S, hence heads outside D, so each is available when computing $\mathrm{Prov}(D)$, and $S \subseteq \mathrm{Prov}(D)$ by induction on the stage at which a member is built. Hence $\mathrm{Req} \subseteq S \subseteq \mathrm{Prov}(D)$. Since supports contain minimal supports, the union description follows, and so does the description of maximal admissible shed sets.

(c) With one rule per head, a capability $c \in \mathrm{Prov}(\varnothing)$ has a unique derivation tree. Let S^* be the set of capabilities occurring in the derivation trees of members of Req. Then $c \in \mathrm{Prov}(D)$ if and only if $c \notin D$ and its premises lie in $\mathrm{Prov}(D)$, so by induction on the derivation, $c \in \mathrm{Prov}(D)$ if and only if no node of its tree lies in D. Hence $\mathrm{Req} \subseteq \mathrm{Prov}(D)$ if and only if $D \cap S^* = \varnothing$. Every support contains S^*, because it must derive Req with the only available rules. □

17.2 No scalar measure of surviving capacity

A common practice reports the health of a degraded system as a number: the fraction of capabilities retained, or a weighted sum of retained capabilities. The following result states when such a number can decide admissibility.

New result 17.4 (Scalar capacity measures)**.** *Call a* scalar capacity test *a pair* (w, θ) *with* $w : C \to \mathbb{R}$ *and* $\theta \in \mathbb{R}$ *that declares* D *admissible when* $\sum_{c \notin D} w(c) \geq \theta$*.*

(a) In the setting of Result 17.3(c) there is a scalar capacity test that is correct for every shed set, with $w \equiv 1$ on S^*, $w \equiv 0$ elsewhere, and $\theta = |S^*|$.

(b) There is a dependency system with alternatives, and a set of requirements, for which no scalar capacity test is correct for every shed set. In the same system, two shed sets of equal size can differ in admissibility, so the count of surviving capabilities does not decide.

Assumptions: (a) as in Result 17.3(c). (b) is an existence statement.

Proof. (a) The kept weight equals $|S^*|$ if and only if $D \cap S^* = \varnothing$, and is smaller otherwise. (b) Let $C = \{c_1, c_2, c_3, c_4, e\}$ with primitive rules $(c_i, \varnothing)$ and rules $(e, \{c_1, c_2\})$ and $(e, \{c_3, c_4\})$, and $\mathrm{Req} = \{e\}$. The minimal supports are $\{e, c_1, c_2\}$ and $\{e, c_3, c_4\}$. Shedding $\{c_3, c_4\}$ is admissible, and shedding $\{c_2, c_4\}$ is not; both have size 2. Suppose a scalar test existed. Restrict attention to shed sets that avoid e and put $\theta' = \theta - w(e)$. Keeping $\{c_1, c_2\}$ and keeping $\{c_3, c_4\}$ pass: $w_1 + w_2 \geq \theta'$ and $w_3 + w_4 \geq \theta'$. Keeping $\{c_1, c_3\}$ and keeping $\{c_2, c_4\}$ fail: $w_1 + w_3 < \theta'$ and $w_2 + w_4 < \theta'$. Adding the first pair gives $\sum w \geq 2\theta'$ and adding the second gives $\sum w < 2\theta'$, a contradiction. □

The number of surviving capabilities and any weighted sum of them are thus adequate for conjunctive requirements and inadequate once alternatives appear. In the terminology of Part II, the admissibility question is a function of the retained set whose kernel is not the kernel of any additive measure, and a monitor for degradation must retain the set, or at least which supports remain intact.

Remark 17.5. Finding a smallest support is intractable in general. Given a set cover instance with sets $s_1, \ldots, s_n$ over a universe U, take s_i as primitives, $u \in U$ as heads with a rule $(u, \{s_i\})$ for every $s_i \ni u$, and $\mathrm{Req} = U$. A support is a cover together with U, so its size is $|U|$ plus the size of the cover. Choosing what to shed to preserve the most capability is therefore at least as hard as set cover, while checking whether a given shed set is admissible takes linear time in the size of the system by computing $\mathrm{Prov}(D)$.

17.3 Counting admissible shed sets and the error of scalar tests

Result 17.4 shows that no scalar test is correct for every shed set once alternatives are present. The question that matters in practice is how wrong the best scalar test is, and for the parallel-alternatives structure the answer can be computed exactly for small systems. Let the requirement e be provided by any one of m alternatives, the ith alternative consisting of two primitives c_{i1}, c_{i2} that are both needed, so that e has the rules $(e, \{c_{i1}, c_{i2}\})$ for $i = 1, \ldots, m$ and $\mathrm{Req} = \{e\}$. Shed sets are subsets D of the $2m$ primitives.

New result 17.6 (Admissible shed sets of parallel pairs)**.** *Exactly $4^m - 3^m$ of the 4^m shed sets are admissible: a shed set is inadmissible if and only if it contains at least one primitive from each pair, and each pair offers 3 such choices among its 4 subsets. The minimal supports are the m sets $\{e, c_{i1}, c_{i2}\}$, and the maximal admissible shed sets are the m complements of these, each of size $2m - 2$. The fraction of admissible shed sets is $1 - (3/4)^m$.*

Assumptions: As in the construction above; e itself is never shed.

Proof. By Result 17.3(b) a shed set is admissible if and only if it is disjoint from some support, and the supports are the unions of minimal supports; D is disjoint from $\{e, c_{i1}, c_{i2}\}$ exactly

when neither primitive of pair i is shed. So D is inadmissible when each pair has at least one member in D, which gives 3^m choices (three nonempty subsets of a two-element pair) over the m pairs, and $4^m - 3^m$ admissible sets. Complements of minimal supports are the maximal ones by the same theorem. □

The scalar tests of Result 17.4 can now be compared with this structure by exhaustive search. For $m = 2$ there are 16 shed sets of which 7 are admissible. The best equal-weight test, which declares a shed set admissible when at least three primitives survive, misclassifies 2 shed sets, and the best test with weights in $\{0, 1, 2, 3\}$ misclassifies 1, which is also the minimum possible by Result 17.4(b), since no test is perfect. For $m = 3$ there are 64 shed sets of which 37 are admissible. The best equal-weight test, which requires at least three surviving primitives, misclassifies 11 of the 64; the best test with integer weights in $\{0, \ldots, 5\}$ misclassifies 8. The latter figure bounds the optimum from above and the exhaustive search has not been extended to larger weights. In every case the errors are of both kinds: a count test declares admissible a shed set that leaves one primitive from each of several pairs, and declares inadmissible a set that leaves exactly one pair intact when the count is small. The first error is the dangerous one, since it certifies a degraded system as able to deliver the required capability when it cannot, and no choice of threshold removes it without increasing the second.

Exercises

Exercise 17.7. A service has primitives for a database, a cache, a recommender, and a payment gateway. Checkout requires the payment gateway and either the database or the cache, and the recommender requires the database. Write the rules, list the minimal supports for $\mathrm{Req} = \{\text{checkout}\}$, and list the maximal admissible shed sets.

Exercise 17.8. Prove that $\mathrm{Prov}(D \cup D') \subseteq \mathrm{Prov}(D) \cap \mathrm{Prov}(D')$, and show by an example that the inclusion can be strict. Interpret in terms of combining two failures.

Exercise 17.9. Show that the family of admissible shed sets is determined by its maximal members and by Result 17.3(a). Give a dependency system with m alternatives of size 2 each for a single requirement and count the minimal supports.

Exercise 17.10. Extend Definition 17.1 with a cost $\kappa(c) \geq 0$ for keeping each capability and show that choosing a minimum-cost support is a weighted version of the problem in the remark. Describe a case where the minimum-cost support differs from the minimum-cardinality support.

Exercise 17.11. Obligations change over time: after an event some obligations are discharged and others created. Using Result 14.4, describe the sequence of required sets Req_t for a single bounded-response obligation and state when a shed set that was admissible at time t ceases to be admissible at $t + 1$.

Exercise 17.12. For $m = 2$ list the 7 admissible shed sets and the 9 inadmissible ones, evaluate the equal-weight test with threshold 3 on each, and identify the two misclassified sets.

Exercise 17.13. Change the structure so that each alternative has r primitives. Show that the number of admissible shed sets is $2^{rm} - (2^r - 1)^m$, and compute the fraction for $r = 3$, $m = 3$.

Exercise 17.14. Show that an additive test that declares admissible exactly when it is certain, that is, never admits an inadmissible shed set, can be chosen for any dependency system, and find the best such test for $m = 2$ and the number of admissible shed sets it rejects.

Chapter 18

Compensation and Recovery

Question. When does compensation restore the relevant obligations, and when does it only create a different acceptable continuation?

Established sources. Sagas (Garcia-Molina and Salem); reversible computation.

New results. Restoration and replacement are defined relative to an identity criterion (Definition 18.1); no compensation restores a step that has crossed a bad prefix (Result 18.4); aborted sagas are restored exactly when the criterion is a congruence for the compensations, with a counterexample otherwise (Result 18.5).

Proof obligations. None.

Depends on. Chapters 14, 15, and 17.

When something goes wrong after part of a task has been done, the system can try to undo the part. A booking is cancelled, a payment is refunded, a reservation is released. The word compensation covers two quite different outcomes. In one the world returns to a state indistinguishable, for every purpose that matters, from the one before. In the other the world arrives somewhere different but acceptable. The difference is not a property of the compensating action alone; it depends on which distinctions matter. This chapter makes that dependence exact using the identity criteria of Chapter 15.

18.1 Restoration and replacement

Histories are as in Chapter 4. A promise $(\mathcal{Q}, F)$ is as in Chapter 15, and $\approx_{\mathcal{Q}}^{F}$ is the coarsest criterion preserving it.

Definition 18.1 (Compensation, restoration, replacement)**.** A *step* is a word $T \in \mathsf{E}^*$ and a *compensation* for it is a word C. At a history h with $hTC \in \mathcal{H}$, the compensation *restores* T relative to the promise $(\mathcal{Q}, F)$ if $hTC \approx_{\mathcal{Q}}^{F} h$. It is a *replacement* relative to $(\mathcal{Q}, F)$ and a set $\mathcal{A} \subseteq \mathcal{H}$ of acceptable histories if $hTC \in \mathcal{A}$ and $hTC \not\approx_{\mathcal{Q}}^{F} h$.

The history hTC is never equal to h, since it is longer, so restoration is always restoration up to a criterion. This is not a defect: the criterion is exactly the statement of what must be the same afterward.

New result 18.2 (Restoration depends on the promise, and is monotone in it)**.** *(a) If C restores T at h relative to $(\mathcal{Q}', F')$ and $\mathcal{Q} \subseteq \mathcal{Q}'$, $F \subseteq F'$, then C restores T at h relative to $(\mathcal{Q}, F)$.*

(b) For any T and C with $hTC \neq h$ in the sense that some q has $q(hTC) \neq q(h)$, there is a promise relative to which C is not a restoration and, if $hTC \in \mathcal{A}$, is a replacement: take $\mathcal{Q} = \{q\}$.

Assumptions: Observations q are functions on histories.

Proof. (a) Result 15.4(a). (b) The criterion for $(\{q\}, \{\varepsilon\})$ is $\ker q$, which separates hTC from h. □

Example 18.3 (Rebooking)**.** A traveller holds a booking on flight f_1. The step T cancels it and books f_2. Relative to the promise "report whether the traveller arrives at the destination by the deadline", the new history is equivalent to the original, so T and its eventual compensation are a restoration. Relative to the promise "report the flight number" the new history differs, and the action is a replacement. Both descriptions are correct; which one governs depends on which promise the system has made to the traveller.

18.2 Irreversible steps

New result 18.4 (No compensation after a bad prefix)**.** *Let $\mathcal{O} \subseteq \mathcal{H}$ be prefix closed and let $\mathbf{1}_{\mathcal{O}}$ be a promised observation. If $h \in \mathcal{O}$ and $hT \notin \mathcal{O}$, then for every word C the history hTC is not in $\mathcal{O}$, and C does not restore T at h relative to any promise that includes $\mathbf{1}_{\mathcal{O}}$. More generally if q is a promised observation that is monotone ($q(u) = 1$ implies $q(uw) = 1$) and $q(h) = 0$, $q(hT) = 1$, then no compensation restores T.*

Assumptions: Prefix closure of $\mathcal{O}$, which holds for the set of non-violating histories of a safety property.

Proof. If $hTC \in \mathcal{O}$ then $hT \in \mathcal{O}$ by prefix closure, contrary to hypothesis. Then $\mathbf{1}_{\mathcal{O}}(hTC) = 0 \neq 1 = \mathbf{1}_{\mathcal{O}}(h)$, so $hTC \not\approx h$. The monotone case is the same with $q(hTC) = 1 \neq 0 = q(h)$. □

The result is the compensation form of the safety/liveness distinction of Chapter 4: a violation detected on a finite history cannot be repaired by any continuation, so an obligation of that kind can only be restored by not crossing the bad prefix in the first place. In the account of Example 1.3 a withdrawal that overdraws is followed by a deposit that restores the balance and leaves $q_{\text{neg}} = 1$ forever, so the deposit is a restoration relative to the promise $\{\beta\}$ and a replacement relative to $\{\beta, q_{\text{neg}}\}$.

18.3 Sagas

A *saga* is a sequence of steps $T_1, \ldots, T_n$ with compensations $C_1, \ldots, C_n$, executed so that either all T_i occur or, after $T_1 \cdots T_j$, the compensations $C_j, \ldots, C_1$ are executed in that order.

New result 18.5 (Correctness of aborted sagas)**.** *Let $\sim$ be an identity criterion on $\mathcal{H}$ and $h_0 \in \mathcal{H}$, $h_i = h_{i-1}T_i$. Suppose*

(i) (per-step restoration) $h_iC_i \sim h_{i-1}$ for each i, and

(ii) (congruence for compensations) $x \sim y$ implies $xC_i \sim yC_i$ for each i whenever both are histories.

Then for every $j \leq n$, $h_jC_jC_{j-1} \cdots C_1 \sim h_0$. Without (ii) the conclusion can fail.

Assumptions: Histories $h_i C_i C_{i-1} \cdots C_1$ lie in $\mathcal{H}$.

Proof. Induction on j. For $j = 1$ this is (i). For $j > 1$, $h_j C_j \sim h_{j-1}$ by (i). Applying (ii) successively with $C_{j-1}, \ldots, C_1$ gives $h_j C_j C_{j-1} \cdots C_1 \sim h_{j-1} C_{j-1} \cdots C_1$, and the right side is $\sim h_0$ by the induction hypothesis.

For the failure without (ii), let the histories (in a deterministic system with states identified with histories) be given by states s_0, s_1, s_1', s_2, x with $T_1 : s_0 \mapsto s_1$, $T_2 : s_1 \mapsto s_2$, $C_2 : s_2 \mapsto s_1'$, $C_1 : s_1 \mapsto s_0$, $C_1 : s_1' \mapsto x$, and the criterion identifying only s_1 with s_1'. Then $C_2 T_2 s_1 = s_1' \sim s_1$ and $C_1 T_1 s_0 = s_0$, so (i) holds, but $C_1 C_2 T_2 T_1 s_0 = x \not\sim s_0$. □

Condition (ii) is the content of Chapter 15: the criterion must be a congruence for the updates it is used with. A coarse criterion adopted for convenience, such as "same booking status", fails it as soon as a later compensation depends on the particular flight that was booked.

18.4 When the criterion is a congruence for free

Condition (ii) of Result 18.5 is the delicate hypothesis. It holds automatically in the most important case, and the case explains why sagas are correct whenever the promise made to the user already includes the compensating operations among its promised updates.

New result 18.6 (Sagas under a promise that includes the compensations)**.** *Let $\sim = \approx_{\mathcal{Q}}^{F}$ be the coarsest criterion preserving a promise $(\mathcal{Q}, F)$, and suppose every compensation C_i belongs to F. Then condition (ii) of Result 18.5 holds, and the saga is correct, $h_j C_j \cdots C_1 \sim h_0$ for every j, as soon as each step is restored individually, $h_i C_i \sim h_{i-1}$.*

Assumptions: F is a submonoid containing every C_i; histories $h_i C_i \cdots C_1$ lie in $\mathcal{H}$.

Proof. By Result 15.2(c) the relation $\approx_{\mathcal{Q}}^{F}$ preserves the promise, so $x \approx_{\mathcal{Q}}^{F} y$ implies $xw \approx_{\mathcal{Q}}^{F} yw$ for every $w \in F$ with $xw \in \mathcal{H}$. Taking $w = C_i$ is condition (ii). The conclusion follows from Result 18.5. □

The failure mode of the theorem is accordingly the use of a criterion adopted for convenience that is not the continuation equivalence of any promise containing the compensations. The account supplies exact restoration conditions for the individual steps.

New result 18.7 (Restoration of a withdrawal by a deposit)**.** *In the account, with $F = \mathsf{E}^*$ and $\mathcal{Q} = \{\beta, q_{\mathrm{neg}}\}$, the deposit $\mathrm{d}(x)$ restores the withdrawal $\mathrm{w}(x)$ at a history h if and only if $q_{\mathrm{neg}}(h) = 1$ or $x \le \beta(h)$. Relative to $\mathcal{Q} = \{\beta\}$ it restores at every history. In particular, if $q_{\mathrm{neg}}(h) = 0$ and $x > \beta(h)$, the deposit is a replacement and not a restoration.*

Assumptions: Events and balances as in Example 1.3; amounts are positive integers.

Proof. The history $h\,\mathrm{w}(x)\,\mathrm{d}(x)$ has balance $\beta(h)$ and overdraft flag $q_{\mathrm{neg}}(h) \vee [\beta(h) - x < 0]$. By Result 2.13 two histories are continuation equivalent for q_{neg} if both have flag 1, or both have flag 0 and equal balance, and they are equivalent for the pair $(\beta, q_{\mathrm{neg}})$ if the balances agree and the flags agree, because the balance determines the continuation equivalence of β. If $q_{\mathrm{neg}}(h) = 1$ both histories have flag 1 and equal balance. If $q_{\mathrm{neg}}(h) = 0$ the flags agree if and only if $\beta(h) - x \ge 0$. For $\mathcal{Q} = \{\beta\}$ the criterion is $\ker \beta$, and the balances agree. □

The refund of an overdrawing withdrawal is the textbook replacement. The money is back, the balance is what it was, and the account is not the account it was, since the overdraft occurred and remains in its record. A saga that moves money between two accounts is therefore correct for the promise of balances alone at every history, and for the promise that includes the no-overdraft flag only on the histories where each withdrawal is covered by the balance at its moment, a condition that a system can check before it begins and that it cannot restore afterwards.

Exercises

Exercise 18.8. For the account with $F = \mathsf{E}^*$, show that a deposit is restored by the withdrawal of the same amount relative to $\{\beta, q_{\mathrm{neg}}\}$, and that a withdrawal is restored by the deposit of the same amount relative to $\{\beta\}$ but not in general relative to $\{\beta, q_{\mathrm{neg}}\}$. Identify the histories at which the second restoration fails.

Exercise 18.9. Prove that restoration relative to $(\mathcal{Q}, F)$ at h is equivalent to the statement that hTC and h agree on every observation in $\mathcal{Q}$ after every sequence in F. Use it to decide whether a refund is a restoration relative to the promise "report the customer's balance and the list of charges".

Exercise 18.10. Show that a replacement relative to $(\mathcal{Q}, F)$ and $\mathcal{A}$ at h is a restoration relative to some coarser promise, and give the coarsest promise relative to which a given replacement is a restoration.

Exercise 18.11. In Result 18.5, replace the order of compensations by $C_1, \ldots, C_j$ and show with a two-step example that the conclusion fails even with (i) and (ii).

Exercise 18.12. A saga includes a step that sends an email. Argue using Result 18.4 why no compensation restores it relative to a promise that includes "the recipient has not been told", and describe what the compensating message achieves instead.

Exercise 18.13. For the two-account transfer saga with steps $T_1 = \mathrm{w}_A(x)$ and $T_2 = \mathrm{d}_B(x)$ and compensations $C_2 = \mathrm{w}_B(x)$, $C_1 = \mathrm{d}_A(x)$, state exactly the initial histories for which the saga aborted after T_2 is correct relative to the promise $\{\beta_A, \beta_B, q_{\mathrm{neg},A}, q_{\mathrm{neg},B}\}$, using Result 18.7 for each step.

Exercise 18.14. Prove the converse of Result 18.6 in the following form: if $\sim$ is an equivalence relation that preserves the observations $\mathcal{Q}$ and satisfies condition (ii) for every C_i in a submonoid F, then $\sim \subseteq \approx^F_{\mathcal{Q}}$. Conclude that the continuation equivalence is the coarsest criterion for which sagas over F are correct.

Exercise 18.15. Give an example of a criterion that is a congruence for the compensation C_1 and not for C_2 in a two-step saga, and show that the saga's conclusion fails for a particular abort point j but holds for another.

Chapter 19

Distributed Continuation

Question. How does local evidence relate to global obligations?

Established sources. Lamport and vector clocks; conflict-free replicated data types (Shapiro et al.); the CAP theorem (Gilbert and Lynch); invariant confluence (Bailis et al.).

New results. A global property is decidable from a process's causal past exactly when it is insensitive to events concurrent with that past (Result 19.2); for a non-negativity obligation on a replicated counter, the capacities that replicas can spend without communication sum to at most the common balance (Result 19.3).

Proof obligations. None.

Depends on. Chapters 4, 11, and 14.

A distributed system has no single history. Each process sees the events that reach it and the order in which they reach it. A global obligation, such as keeping a shared balance non-negative, is a property of the whole execution, but each process acts on what it knows. This chapter states what a process can decide about a global obligation from its causal past, and what the communication-free actions of several processes can safely add up to.

19.1 Evidence of a process

An *execution* is a finite set $\mathcal{E}$ of events with a partial order $\to$, the happens-before relation of Result 11.2, together with a partition of $\mathcal{E}$ among the processes such that the events of each process are totally ordered. For an event e, its *causal past* is $\downarrow e = \{e' \in \mathcal{E} : e' \to e \text{ or } e' = e\}$ with the induced order and labels. A *global property* is a function Π on executions.

Definition 19.1 (Local decidability)**.** A global property Π is *decidable at e from the causal past* if $\Pi(\mathcal{E}) = \Pi(\mathcal{E}')$ for all executions $\mathcal{E}, \mathcal{E}'$ and events $e \in \mathcal{E}$, $e' \in \mathcal{E}'$ with $\downarrow e \cong \downarrow e'$ (isomorphic as labelled partial orders).

The causal past is the most a process can know under causal delivery: it is carried by a vector clock together with the events themselves, and by Result 11.2 the clock determines the order.

New result 19.2 (Decidability from the causal past)**.** *(a) Π is decidable at e from the causal past if and only if $\Pi(\mathcal{E})$ depends on $\mathcal{E}$ only through $\downarrow e$.*

(b) If there are executions $\mathcal{E} \subseteq \mathcal{E}^+$ with $e \in \mathcal{E}$, $\downarrow_{\mathcal{E}} e = \downarrow_{\mathcal{E}^+} e$, and $\Pi(\mathcal{E}) \neq \Pi(\mathcal{E}^+)$, then Π is not decidable at e. In particular a property that can be changed by an event concurrent with e is not decidable at e.

(c) A property of the form $\Pi(\mathcal{E}) = \mathbf{1}[\Phi(\downarrow e)]$ for a fixed e is decidable at e.

Assumptions: Executions are finite; isomorphism is of labelled partial orders.

Proof. (a) is Definition 19.1 restated as a factorization through the map $\mathcal{E} \mapsto \downarrow e$ (Result 2.5). (b) The two executions have the same causal past at e and different values. (c) The value depends on the execution only through $\downarrow e$. □

The result is the factorization theorem of Chapter 2 for the representation "causal past". It says that the information gap between local and global is the set of concurrent events, and that for a property sensitive to them no amount of cleverness in the local test closes it.

19.2 What communication-free actions can add up to

New result 19.3 (Quota theorem for a non-negative replicated balance)**.** *A balance of $B \geq 0$ is held at n replicas, each of which may execute withdrawals on its own, deciding by a rule that depends only on its causal past. The global obligation is that the total withdrawn never exceeds B. For replica r let q_r be the largest total that r can withdraw when the execution so far consists of the common starting state and the events of r alone. If the rule is safe, in the sense that the obligation holds in every execution consistent with it, then*

$$\sum_{r=1}^{n} q_r \;\leq\; B.$$

Conversely, any assignment of quotas q_r with $\sum_r q_r \leq B$ defines a safe local rule: replica r permits a withdrawal if the total it has withdrawn plus the new amount is at most q_r.

Assumptions: Withdrawals are positive integers; replicas may be isolated from one another for an arbitrary period; the rule at each replica depends only on its own events and the common starting state.

Proof. Consider the execution in which every replica, starting from the common state and hearing from no other replica, performs withdrawals totalling its q_r. Each of these is permitted by the local rule, since the causal past of every event of replica r consists of the common state and its own earlier events. Safety requires that the total withdrawn in this execution, which is $\sum_r q_r$, not exceed B. Conversely, with quotas summing to at most B each replica stays within its quota whatever the others do, so the global total stays at most B. □

The theorem is the quantitative form of the familiar tension between availability and a global invariant under partition, which Gilbert and Lynch state as an impossibility. Here the cost is explicit: the sum of the independent capacities cannot exceed the common balance. Giving a replica more than its share removes it from some other, and the allocation of quotas is the allocation of the right to act without asking. Rebalancing quotas requires communication, and that is where coordination re-enters.

Established result 19.4 (Convergence of replicated data types and invariant confluence)**.** *A state-based replicated data type whose states form a join-semilattice, whose updates are inflationary, and whose merge is the join, converges: replicas that have received the same updates are in the same state. An invariant can be maintained without coordination if and only if it is invariant confluent, meaning that the merge of any two reachable valid states is valid.*

Source: Shapiro, Preguiça, Baquero, and Zawirski (2011); Bailis, Fekete, Franklin, Ghodsi, Hellerstein, and Stoica, "Coordination avoidance in database systems" (2014).

Result 19.3 shows why the non-negativity of a balance is not invariant confluent: two valid states that each remove their share of the balance can merge to a state with a negative balance. It also shows exactly how much non-coordinated action the obligation allows.

19.3 What fixed quotas cost when demand is skewed

Result 19.3 says which allocations of the right to act are safe. It does not say how good they are. The quality of an allocation is measured by how much of the demand it serves, and since demand at each replica is unknown when the quotas are fixed, the measure is the worst case over demands. Let the demand at replica r be a nonnegative integer d_r with $\sum_r d_r \le B$, so that a coordinating system with full knowledge could serve all of it. A quota assignment $q = (q_r)$ with $\sum_r q_r \le B$ serves $\sum_r \min(d_r, q_r)$ of the demand without communication.

New result 19.5 (Worst-case service of fixed quotas)**.** *For n replicas and balance B, every safe fixed quota assignment, and every probability distribution over such assignments, has a demand vector with $\sum_r d_r = B$ on which the expected amount served without communication is at most B/n, that is, at most the fraction $1/n$ of the demand. The equal split $q_r = B/n$ attains the ratio $1/n$ against every demand concentrated at a single replica and does at least that well against every demand vector.*

Assumptions: Integer demands with total B; quotas independent of the realized demand; n divides B for the final statement, or else the floor is taken.

Proof. For a deterministic assignment, $\sum_r q_r \le B$ implies that some replica r_0 has $q_{r_0} \le B/n$. Place the whole demand $d_{r_0} = B$ there; the amount served is $\min(B, q_{r_0}) \le B/n$. For a randomized assignment the same argument applies to the expected quotas, $\sum_r \mathbb{E}q_r \le B$, which gives a replica with $\mathbb{E}q_{r_0} \le B/n$, and by concavity $\mathbb{E}\min(B, q_{r_0}) \le \min(B, \mathbb{E}q_{r_0}) \le B/n$. For the equal split, a demand vector d with $\sum_r d_r = B$ is served to the extent $\sum_r \min(d_r, B/n)$; since $\min(d_r, B/n) \ge d_r/n$ whenever $d_r \le B$ (as $d_r \le B$ gives $d_r/n \le B/n$), the sum is at least B/n. □

The ratio $1/n$ is the price of giving up communication entirely; it is also a statement about how little prior knowledge helps. The bound holds for every allocation rule that fixes quotas before demand is seen, however cleverly it is chosen. A small computation fixes the scale. With $n = 4$ replicas and $B = 100$ the equal split gives each replica 25. A skewed demand $(80, 10, 5, 5)$, which totals exactly the balance, is served $\min(80, 25) + 10 + 5 + 5 = 45$ of 100 without communication, and the remaining 55 must be served by transferring unused quota from the other replicas, which is a coordination step. The worst case, all 100 at one replica, is served 25 of 100. Each replica in the skewed example holds quota it cannot use, and the system cannot know in advance which one.

This is the quantitative content of the remark that rebalancing requires communication. An allocation scheme may reduce the number of coordination rounds, for instance by transferring half of an exhausted replica's shortfall from the richest peer, but the first round of service without communication is always bounded by the fraction above, and the question of how much coordination to buy is the question of Chapter 22.

Exercises

Exercise 19.6. Two replicas hold a balance of 10 and each uses the rule "withdraw if the locally known balance is at least the amount". Give an execution in which the global balance becomes negative. Compute the quotas that make the local rule safe for each split of the 10.

Exercise 19.7. Show that the property "replica r has withdrawn at least as much as replica s" is not decidable at an event of r from its causal past, and identify the condition on communication under which it becomes decidable.

Exercise 19.8. Extend Result 19.3 to deposits that increase the balance. Describe how a deposit at one replica can raise the quota of another only after a message transfers the right, and why the sum of quotas plus rights in transit is conserved.

Exercise 19.9. Prove that the maximum, over executions consistent with the quota rule, of the number of withdrawals executed without any communication is $\sum_r q_r$ when all amounts are 1, and decide whether it is attained.

Exercise 19.10. A vector clock has n coordinates. Show that a representation of the causal past of an event by its vector clock alone is not sufficient for questions that depend on the labels of other processes' events, and say what must be retained in addition.

Exercise 19.11. For $n = 3$ and $B = 60$ list the demand vectors with $\sum_r d_r = 60$ that are served least by the equal split, and compute the served amount for $(30, 20, 10)$, $(60, 0, 0)$ and $(20, 20, 20)$.

Exercise 19.12. Show that if the demand is known to satisfy $d_r \le B/m$ for every r, with $m \le n$, the worst-case served fraction of the equal split improves, and find it as a function of m and n. Interpret the bound as a statement about how much knowledge of the maximum demand of any single replica helps.

Exercise 19.13. One round of rebalancing allows each replica that exhausts its quota to request unused quota from one peer of its choice. Show that for demand concentrated at two replicas this raises the served amount to the full demand if the requests are directed correctly, and construct a demand vector for which a single round of requests to a fixed peer still leaves service below the demand.

Chapter 20

Interfaces and Delegated Authority

Question. Which composition results hold, and under which compatibility conditions?

Established sources. Interface automata (de Alfaro and Henzinger); assume–guarantee contracts; capability systems and the take-grant model.

New results. Refinement preserves compatibility of composition (Result 20.4); the non-circular assume–guarantee rule holds under explicit alphabet conditions (Result 20.6); authority cannot be created by attenuating delegation (Result 20.8).

Proof obligations. None.

Depends on. Chapters 13 and 14.

A component interacts with its surroundings through an interface: the actions it offers, the actions it expects, and the order in which it will accept them. Composition theorems say when components whose interfaces fit can be assembled and when a replacement for one component can be dropped in without disturbing the others. Authority adds a second kind of interface, the rights that a component holds and may pass on. This chapter states the composition results with their hypotheses spelled out, because each of them fails without one.

20.1 Interface automata and compatibility

Definition 20.1 (Interface automaton)**.** An *interface automaton* is a tuple $A = (S, s_0, I, O, \delta)$ with finite state set S, initial state s_0, disjoint finite sets I of input actions and O of output actions, and a partial deterministic transition function $\delta : S \times (I \cup O) \rightharpoonup S$. Write $\mathrm{in}(s) = \{a \in I : \delta(s,a) \text{ defined}\}$ and $\mathrm{out}(s)$ likewise. Two automata A, B are *composable* if $O_A \cap O_B = \varnothing$, $I_A \cap I_B = \varnothing$, and $(I_A \cup O_A) \cap (I_B \cup O_B) = (O_A \cap I_B) \cup (O_B \cap I_A)$.

Definition 20.2 (Product and compatibility)**.** For composable A, B the *product* $A \| B$ has states $S_A \times S_B$, initial state (s_0^A, s_0^B), and transitions: an action a of A that is not shared moves only the first component; an action of B not shared moves only the second; a shared action $a \in (O_A \cap I_B) \cup (O_B \cap I_A)$ moves both, and is defined when it is defined in both. An input of one component that is not shared is supplied by an environment that is not modelled, and is not a transition of the product. A reachable state (s,t) is an *error state* if some shared action is an output of one component at its state and is not accepted by the other: $\mathrm{out}(s) \cap I_B \not\subseteq \mathrm{in}(t)$ or $\mathrm{out}(t) \cap I_A \not\subseteq \mathrm{in}(s)$. The automata are *compatible* if $A \| B$ has no reachable error state.

Compatibility says that in the composed system no component ever emits a message the other cannot accept. It is decided in time proportional to $|S_A||S_B|$ by exploring the product.

Definition 20.3 (Refinement)**.** An automaton A' *refines* A with the same input and output sets if there is a relation $R \subseteq S_{A'} \times S_A$ with $(s'_0, s_0) \in R$ and, whenever $(s', s) \in R$: $\mathrm{in}(s) \subseteq \mathrm{in}(s')$; $\mathrm{out}(s') \subseteq \mathrm{out}(s)$; and for every $a \in \mathrm{in}(s) \cup \mathrm{out}(s')$, $(\delta'(s', a), \delta(s, a)) \in R$.

A refinement accepts at least the inputs the original accepts and offers at most the outputs it offers, in matching states.

New result 20.4 (Refinement preserves compatibility)**.** *Let A and B be compatible and let A' refine A with A' and B composable. Then A' and B are compatible.*

Assumptions: Deterministic automata; the shared actions of A' and B are those of A and B.

Proof. We show by induction on the length of a path in $A' \| B$ that every reachable state (s', t) has a state s with $(s', s) \in R$ and (s, t) reachable in $A \| B$. The initial state is related to (s_0, t_0). Suppose (s', t) is reached with a related state s. A move of B on an unshared output leaves s', s unchanged. A move of A' on an unshared output $a \in \mathrm{out}(s') \subseteq \mathrm{out}(s)$ leads to related states, and $A \| B$ makes the same move. For a shared output a of A' at s', $a \in \mathrm{out}(s)$, and since (s, t) is not an error state, B accepts a at t, so $A \| B$ moves to $(\delta(s, a), \delta_B(t, a))$ and $(\delta'(s', a), \delta(s, a)) \in R$. For a shared output a of B at t, compatibility of A, B gives $a \in \mathrm{in}(s) \subseteq \mathrm{in}(s')$, so A' accepts and the moves correspond. Now suppose (s', t) were an error state of $A' \| B$. If A' outputs a shared a at s' that B does not accept at t, then $a \in \mathrm{out}(s)$ and (s, t) would be an error state of $A \| B$. If B outputs a shared a at t that A' does not accept, then $a \in \mathrm{in}(s) \subseteq \mathrm{in}(s')$, a contradiction. Hence no error state is reachable. □

Established result 20.5 (Interface theories)**.** *For non-deterministic interface automata, refinement defined by alternating simulation is a preorder, is a precongruence for composition of compatible automata, and compatibility is preserved, as above.*

Source: de Alfaro and Henzinger, "Interface automata," ESEC/FSE 2001.

20.2 Assume–guarantee composition

Let a *component* M over an alphabet Σ_M be a prefix-closed set of finite words over Σ_M. The composition $M_1 \| M_2$ is the set of words t over $\Sigma_1 \cup \Sigma_2$ with $t|_{\Sigma_i} \in M_i$, where $t|_\Sigma$ erases letters outside Σ. For a prefix-closed set P over Σ_P write $M \models P$ if $t|_{\Sigma_P} \in P$ for every $t \in M$.

New result 20.6 (A non-circular assume–guarantee rule)**.** *Let A be a prefix-closed set over $\Sigma_A \subseteq \Sigma_1 \cap \Sigma_2$ and P a prefix-closed set over $\Sigma_P \subseteq \Sigma_1$. If*

(i) $M_1 \| A \models P$, and

(ii) $M_2|_{\Sigma_A} \subseteq A$ (every behaviour of M_2 seen on the shared letters is allowed by the assumption),

then $M_1 \| M_2 \models P$.

Assumptions: The alphabet conditions $\Sigma_A \subseteq \Sigma_1 \cap \Sigma_2$ and $\Sigma_P \subseteq \Sigma_1$.

Proof. Let $t \in M_1 \| M_2$ and $u = t|_{\Sigma_1} \in M_1$. Since $\Sigma_A \subseteq \Sigma_1$ and $\Sigma_A \subseteq \Sigma_2$, $u|_{\Sigma_A} = t|_{\Sigma_A} = (t|_{\Sigma_2})|_{\Sigma_A} \in M_2|_{\Sigma_A} \subseteq A$. So $u \in M_1 \| A$ (taking the alphabet Σ_1), and (i) gives $u|_{\Sigma_P} \in P$. Because $\Sigma_P \subseteq \Sigma_1$, $t|_{\Sigma_P} = u|_{\Sigma_P} \in P$. □

Both alphabet conditions are needed. If $\Sigma_P \not\subseteq \Sigma_1$ the guarantee mentions letters that M_1 does not control and the argument breaks at the last step; if the assumption A mentions letters outside Σ_2, then (ii) is a claim about behaviour that M_2 does not exhibit and cannot be discharged by M_2. Rules that assume the properties of each component about the other circularly require an additional argument by induction on the length of words, and are not valid in general for properties that are not safety properties.

20.3 Delegated authority

Definition 20.7 (Delegation system)**.** Let $\mathcal{P}$ be a finite set of principals and Rts a set of rights. The *authority state* is a function $\mathrm{Auth} : \mathcal{P} \to 2^{\mathrm{Rts}}$. A *delegation* $(p \to q, R)$ with $R \subseteq \mathrm{Auth}(p)$ updates $\mathrm{Auth}(q)$ to $\mathrm{Auth}(q) \cup R$. A system is *attenuating* if the only way Auth changes is by delegations of this kind (rights may also be revoked).

New result 20.8 (Authority is not created by delegation)**.** *In an attenuating system, if $r \in \mathrm{Auth}_t(q)$ then there are principals $p_0, \ldots, p_k = q$ with $r \in \mathrm{Auth}_0(p_0)$ and delegations of r from p_{i-1} to p_i occurring in order at times $t_1 \le \cdots \le t_k \le t$. In particular $\mathrm{Auth}_t(q) \subseteq \bigcup_p \mathrm{Auth}_0(p)$, and a principal that is not the target of a chain of delegations from a holder of r never holds r.*

Assumptions: No operation creates a right; revocation only removes.

Proof. Induction on t. At $t = 0$ the claim is trivial with $k = 0$. A step at time t is a delegation $(p \to q', R)$ with $R \subseteq \mathrm{Auth}_{t-1}(p)$ or a revocation. A revocation removes rights and preserves the claim. After a delegation, a right $r \in R$ now held by q' was held by p at $t - 1$, so by hypothesis there is a chain ending at p, and extending it by the delegation at time t gives a chain ending at q'. Rights held by other principals are unchanged. □

The result is a safety property of the delegation graph and can be checked without running the system: whether a principal can ever obtain a right is a reachability question. It is also the reason capability systems are reasoned about by tracing references. A deputy that holds authority of its own and acts on a request from a caller under the caller's choice of target uses authority that it holds and the caller does not; the interface between caller and deputy must therefore specify which rights the request carries, which is a statement about the compatibility of the two interfaces in the sense of Definition 20.2 and not only about the authority state.

20.4 Withholding a right: cuts in the permission graph

In an attenuating system the question whether a principal can ever obtain a right is a reachability question, and the question of how much policy it takes to prevent it is a cut question. Suppose delegation is constrained by a *permission graph* G on the principals, a directed graph in which an edge $p \to q$ means that p is permitted to delegate to q, and that a right may be delegated only along edges of G. Let H be the set of principals that hold the right r initially.

New result 20.9 (The least number of permissions to withhold)**.** *In an attenuating system with permission graph G, a principal $q \notin H$ can obtain r at some time if and only if q is reachable from H in G. The least number of edges whose removal from G prevents q from*

ever obtaining r equals the maximum number of pairwise edge-disjoint chains of permitted delegations from H to q.

Assumptions: Finite graph; rights are only delegated along permitted edges; no principal outside H holds r at the start; revocation is not used.

Proof. If q is reachable by a path, delegating r along the path in order gives q the right. Conversely by Result 20.8 any holder at time t is the end of a chain of delegations from a member of H, and each delegation in the chain uses an edge of G, so q is reachable. A set of edges whose removal makes q unreachable from H is an edge cut separating H from q, and the minimum size of such a cut equals the maximum number of edge-disjoint paths from H to q by Menger's theorem (applied to the graph with a new source joined to every member of H). □

A small instance fixes the numbers. Let the principals be $h_1, h_2, a, b, c, d, e, q$, with $H = \{h_1, h_2\}$ and permitted delegations $h_1 \to a$, $h_1 \to b$, $h_2 \to b$, $h_2 \to c$, $a \to d$, $b \to d$, $b \to e$, $c \to e$, $d \to q$, $e \to q$ and $a \to q$. The target q is reachable, and there are three edge-disjoint chains, namely $h_1 \to a \to q$, $h_2 \to b \to d \to q$ and $h_1 \to b \to e \to q$. No set of two permissions blocks every chain, and the three permissions $h_1 \to a$, $b \to d$ and $e \to q$ do; both facts have been confirmed by exhaustive search over edge subsets. The example shows the sense in which authority is a property of the graph and not of the principals. A policy that withholds the single delegation $d \to q$ removes one chain of the three and leaves q able to obtain r, and no assessment of any individual principal reveals that two further permissions elsewhere are what separates q from the right.

Exercises

Exercise 20.10. Construct two interface automata that are compatible and a refinement of one of them that is composable but loses compatibility because it drops an input, and identify which hypothesis of Definition 20.3 is violated.

Exercise 20.11. Model a client that sends a request and waits for either a reply or an error, and a server that replies or errors after each request. Compute the product, verify compatibility, and then remove the server's error output from the client's inputs to produce an error state.

Exercise 20.12. Give a counterexample to Result 20.6 when the alphabet condition $\Sigma_P \subseteq \Sigma_1$ is dropped, with P a safety property that the composition violates although (i) and (ii) hold.

Exercise 20.13. Show that the compatibility check of Definition 20.2 is a reachability search for an error state in the product, and state the number of states of the product of two automata with m and n states.

Exercise 20.14. In a delegation system with revocation, show that Result 20.8 is no longer sufficient to infer that a principal currently holds a right from the existence of a chain, and state the additional condition on revocations under which the chain characterizes current authority.

Exercise 20.15. In the example above find a second minimum cut of size three that uses only edges entering q or leaving H, or prove that none exists. Determine the minimum cut if q is replaced by e as the target.

Exercise 20.16. Prove the first statement of Result 20.9 when revocation is allowed but is only ever applied to rights held by the revoking principal's own delegates: show that reachability in G is still necessary for obtaining r, and give an example in which it is not sufficient.

Exercise 20.17. Suppose each permitted delegation additionally attenuates: the right delegated along edge e is intersected with a mask $m_e \subseteq \mathrm{Rts}$. Show that the set of rights obtainable by q is the union over chains of the intersection of the masks along the chain together with the initial holdings, and compute it for a chain of three edges with masks $\{r, s\}$, $\{s, t\}$, $\{s\}$.

Part VI

Revision and synthesis

Chapter 21

The Revision Horizon and Revision Sufficiency

Question. Which policies remain implementable from a retained representation, and what does forgetting remove?

Established sources. The data-processing inequality; Bayes decision rules.

New results. The set of policies implementable from a representation determines its kernel and shrinks geometrically under coarsening (Result 21.2); the cases whose revised decision is lost are exactly those in classes on which the revised policy is not constant (Result 21.4); under the uniform distribution a representation with k values has a binary revision of error at least $(n-k)/2n$, while a structured family of revisions can be served with far fewer values (Result 21.6).

Proof obligations. Approximate revision sufficiency for classes of revisions other than the two studied here is open (Chapter 24).

Depends on. Chapters 5 to 7 and 9.

Policies change. A regulation is amended, a threshold is retuned, a new audit question is asked. The representation that was sufficient for yesterday's policy is a bet about which policies will be asked for tomorrow, and forgetting is the loss of the bet. This chapter makes the set of policies that a representation keeps available into an object of study, the revision horizon, shows that it determines the representation, and measures what coarsening removes from it.

21.1 The horizon of a representation

Throughout, X is a finite set of cases, A is a set of actions with $|A| \geq 2$, and $\sigma : X \to R$ is a representation.

Definition 21.1 (Revision horizon)**.** The *revision horizon* of σ is the set $\mathrm{Pol}_A(\sigma) = \{d \circ \sigma : d : \sigma(X) \to A\}$ of policies $X \to A$ that are sufficiently served by σ in the sense of Definition 5.1. A *revision* to a policy π' is *supported* by σ if $\pi' \in \mathrm{Pol}_A(\sigma)$.

New result 21.2 (The horizon determines the representation)**.** *Let $\sigma : X \to R$ and $\sigma' : X \to R'$ be representations.*

(a) $\mathrm{Pol}_A(\sigma') \subseteq \mathrm{Pol}_A(\sigma)$ *if and only if* $\ker\sigma \subseteq \ker\sigma'$*, that is, if and only if* σ' *is a function of* σ*.*
(b) $\mathrm{Pol}_A(\sigma) = \mathrm{Pol}_A(\sigma')$ *if and only if* $\ker\sigma = \ker\sigma'$*.*
(c) $|\mathrm{Pol}_A(\sigma)| = |A|^{|\sigma(X)|}$*. In particular if* $\sigma' = f \circ \sigma$ *merges* k *values into* k' *values, then* $|\mathrm{Pol}_A(\sigma')| = |A|^{k'-k}\,|\mathrm{Pol}_A(\sigma)|$ *and the fraction of the horizon that survives is* $|A|^{-(k-k')}$*.*

Assumptions: X finite, $|A| \geq 2$.

Proof. (a) If $\ker\sigma \subseteq \ker\sigma'$ then $\sigma' = g\circ\sigma$ for some g by Result 2.5, and any $d\circ\sigma' = (d\circ g)\circ\sigma \in \mathrm{Pol}_A(\sigma)$. Conversely suppose $\sigma(x) = \sigma(x')$ but $\sigma'(x) \neq \sigma'(x')$. Fix distinct $a, b \in A$ and let $d : \sigma'(X) \to A$ send $\sigma'(x)$ to a and every other value to b. Then $\pi = d \circ \sigma'$ lies in $\mathrm{Pol}_A(\sigma')$ and has $\pi(x) = a \neq b = \pi(x')$. Every policy in $\mathrm{Pol}_A(\sigma)$ is constant on the class of x, so $\pi \notin \mathrm{Pol}_A(\sigma)$. (b) follows from (a) applied in both directions. (c) The map $d \mapsto d \circ \sigma$ from functions $\sigma(X) \to A$ to policies is injective because σ maps onto $\sigma(X)$, so the count is the number of such functions. □

Part (b) says that the horizon is a complete invariant of the representation up to relabelling. Part (c) says that each distinction discarded divides the horizon by $|A|$.

21.2 Which cases lose their revised decision

For a revision to π', forgetting does not damage every case. A case is damaged only if the representation cannot tell it apart from a case that the new policy treats differently.

Definition 21.3 (Affected cases)**.** For a representation σ and a policy π', the set of *affected cases* is $\mathrm{Aff}(\sigma, \pi') = \{x \in X : \exists x' \text{ with } \sigma(x') = \sigma(x) \text{ and } \pi'(x') \neq \pi'(x)\}$.

New result 21.4 (Affected cases are the non-constant classes)**.** *(a)* $\mathrm{Aff}(\sigma, \pi')$ *is the union of the classes of* $\ker\sigma$ *on which* π' *is not constant.*
(b) $\mathrm{Aff}(\sigma, \pi') = \varnothing$ *if and only if* $\pi' \in \mathrm{Pol}_A(\sigma)$*.*
(c) If $\sigma' = f \circ \sigma$ *then* $\mathrm{Aff}(\sigma, \pi') \subseteq \mathrm{Aff}(\sigma', \pi')$ *for every* π'*.*
(d) Every decision rule applied to σ *disagrees with* π' *on at least one case of each class on which* π' *is not constant, and on at least one case of* $\mathrm{Aff}(\sigma, \pi')$ *whenever it is nonempty.*

Assumptions: X finite.

Proof. (a) By definition x is affected exactly when its class contains a case with a different value of π'. (b) π' is constant on every class if and only if $\ker\sigma \subseteq \ker\pi'$, which is membership in $\mathrm{Pol}_A(\sigma)$ by Result 5.2. (c) Classes of σ' are unions of classes of σ, so a class on which π' is non-constant is contained in a class of σ' on which it is non-constant. (d) A decision rule takes one value on a class; if π' takes two values there, the rule disagrees with π' at some case of that class. □

21.3 Approximate revision sufficiency

Chapter 9 replaced exact sufficiency by small error under a distribution. For revisions the relevant quantity is the worst error over a class $\mathcal{R}$ of revisions. Write $\mathrm{err}_\mu(\sigma, \pi')$ for the Bayes error of Definition 9.1.

Definition 21.5 (Approximate revision sufficiency)**.** The representation σ is $(\varepsilon, \mathcal{R}, \mu)$-*revision sufficient* if $\mathrm{err}_\mu(\sigma, \pi') \le \varepsilon$ for every $\pi' \in \mathcal{R}$.

The first result shows that no compression can be revision sufficient for the class of all binary revisions.

New result 21.6 (Worst-case binary revision under the uniform distribution)**.** *Let $|X| = n$, let μ be uniform, and let σ have classes of sizes $n_1, \dots, n_k$. Let $\mathcal{R}_2$ be the class of all policies $X \to \{0,1\}$.*

(a) $\max_{\pi' \in \mathcal{R}_2} \mathrm{err}_\mu(\sigma, \pi') = \dfrac{1}{n} \displaystyle\sum_{y=1}^{k} \left\lfloor \frac{n_y}{2} \right\rfloor \;\ge\; \frac{n-k}{2n}.$

(b) Consequently σ is $(\varepsilon, \mathcal{R}_2, \mu)$-revision sufficient only if $k \ge n(1-2\varepsilon)$.

(c) Let $X = \{1, \dots, n\}$ with the usual order, let $\mathcal{R}_{\le}$ be the threshold policies $\pi_t(x) = \mathbf{1}[x \ge t]$, and let σ partition X into k consecutive intervals of sizes $n_1, \dots, n_k$. Then $\max_{\pi' \in \mathcal{R}_{\le}} \mathrm{err}_\mu(\sigma, \pi') = \frac{1}{n} \max_y \lfloor n_y/2 \rfloor$, which is at most $\lceil n/k \rceil/(2n)$ for equal intervals. In particular $k \approx 1/(2\varepsilon)$ values suffice for error ε.

Assumptions: (a) and (b) concern binary revisions under the uniform distribution. (c) the threshold family on a line.

Proof. (a) By Definition 9.1 the Bayes error is the sum over classes y of $\min(a_y, n_y - a_y)/n$, where a_y is the number of cases of class y on which π' takes the value 1. Each $a_y \in \{0, \dots, n_y\}$ may be chosen independently, and $\min(a_y, n_y - a_y)$ is maximized at $\lfloor n_y/2 \rfloor$. Then $\sum_y \lfloor n_y/2 \rfloor \ge \sum_y (n_y - 1)/2 = (n-k)/2$. (b) is (a) rearranged. (c) A threshold policy is constant on every interval not containing the threshold, and on the interval containing it a_y can be any integer in $\{0, \dots, n_y\}$ by moving t, so the maximum over t is $\max_y \lfloor n_y/2 \rfloor / n$. For equal intervals $n_y \le \lceil n/k \rceil$. □

The contrast between (b) and (c) is the content of the chapter. If the revisions that might be asked for are arbitrary, a representation must keep nearly everything. If they have structure, such as a threshold on a quantity, a representation that keeps a quantized version of the quantity is nearly sufficient with a number of values independent of n. The problem of characterizing the minimum size for a given class of revisions, in terms of a combinatorial parameter of the class, is stated as a research problem in Chapter 24.

21.4 How many values a threshold family requires

Result 21.6(c) bounds the error of an interval partition for threshold revisions. The bound can be inverted to give the number of values needed for a stated error, and the same family supplies a count of the revisions that are served with no error at all.

New result 21.7 (Exact and approximate service of thresholds)**.** *Let $X = \{1, \dots, n\}$, let μ be uniform, and let σ partition X into k consecutive intervals. The threshold family has $n+1$ members π_t, $t = 1, \dots, n+1$, where π_1 is constant 1 and π_{n+1} is constant 0.*

(a) Exactly $k+1$ of the $n+1$ members lie in $\mathrm{Pol}_{\{0,1\}}(\sigma)$, namely those whose threshold falls on an interval boundary or at an end.

(b) The worst error over the family is at most ε if and only if every interval has at most $2\lfloor \varepsilon n \rfloor + 1$ elements.

(c) *Consequently the least number of intervals achieving worst error at most ε is $\lceil n/(2\lfloor \varepsilon n \rfloor + 1)\rceil$.*

Assumptions: X finite and linearly ordered; μ uniform; σ an interval partition.

Proof. (a) By Result 21.4(b) a threshold policy is served exactly when it is constant on every interval. The policy π_t is non-constant on an interval exactly when $t-1$ and t both lie in it, so it is served exactly when $t = 1$, $t = n+1$ or $t-1$ and t are in different intervals; there are $k-1$ interior boundaries and two ends. (b) By Result 21.6(c) the worst error is $\max_y \lfloor n_y/2 \rfloor / n$, and $\lfloor n_y/2 \rfloor \le \lfloor \varepsilon n \rfloor$ holds if and only if $n_y \le 2\lfloor \varepsilon n \rfloor + 1$. (c) With $s = 2\lfloor \varepsilon n \rfloor + 1$, k intervals of size at most s cover X if and only if $ks \ge n$, and intervals of sizes differing by at most one attain this. □

For $n = 100$ and $\varepsilon = 0.05$ the bound allows intervals of at most 11 elements, so ten intervals suffice and nine do not. The exactly served fraction is $11/101$, about one in nine, while the worst error is 0.05. The gap between these two numbers is the practical content of approximate revision sufficiency: a representation that supports almost no revision exactly can support all of a structured family to within a small error.

Exercises

Exercise 21.8. For the monitor of Example 1.4, compute $|\mathrm{Pol}_A(\sigma)|$ for σ the count capped at 3 and for σ the count capped at 2, with $|A| = 2$, and the fraction of the horizon lost by the lower cap.

Exercise 21.9. Prove that $\mathrm{Aff}(\sigma, \pi') \cap \mathrm{Aff}(\sigma, \pi'')$ need not equal $\mathrm{Aff}(\sigma, \pi' \wedge \pi'')$, and that $\mathrm{Aff}(\sigma, (\pi', \pi'')) = \mathrm{Aff}(\sigma, \pi') \cup \mathrm{Aff}(\sigma, \pi'')$.

Exercise 21.10. Show that Result 21.6(a) is attained by a policy that splits each class as evenly as possible, and compute the worst-case error for $n = 12$ and classes of sizes $5, 4, 3$.

Exercise 21.11. In the account, let X be the histories of length at most 2 and let $\sigma = \beta$. Compute the classes, the affected cases for the revision q_{neg}, and the Bayes error of σ for q_{neg} under the uniform distribution on those histories.

Exercise 21.12. Generalize Result 21.6(a) to $|A| = m$ actions. Show that the worst-case Bayes error under the uniform distribution is $\frac{1}{n}\sum_y (n_y - \lceil n_y/m \rceil)$.

Exercise 21.13. Verify Result 21.7(a) directly for $n = 6$ and intervals $\{1,2\}, \{3,4,5\}, \{6\}$, listing the four exactly served thresholds and the worst error among the remaining three.

Exercise 21.14. Show that part (c) of Result 21.7 can fail to be attained by an interval partition when μ is not uniform, by giving a weighted example in which unequal intervals give a smaller worst error with the same k.

Exercise 21.15. Extend Result 21.7(a) to two-sided interval revisions $\mathbf{1}[s \le x < t]$ and count the exactly served members for an interval partition into k intervals.

Chapter 22

The Cost of Preserving Alternatives

Question. How are storage, update, retrieval, and verification costs traded off when choosing a representation?

Established sources. The minimum test-set problem (Garey and Johnson); weighted set cover and its greedy approximation (Chvátal).

New results. Selecting observations to retain is a weighted set cover over the pairs the policy separates (Result 22.3); it is intractable in general while checking a given choice is cheap; an exact storage cost for updatability (Result 22.5).

Proof obligations. Hardness of approximation for representation selection beyond the set cover bounds.

Depends on. Chapters 6, 7, 12, and 21.

A representation has costs beyond the number of values it takes. It occupies storage, it must be updated when events occur, it must be read when a decision is made, and its adequacy must be checked. Earlier chapters fixed one cost at a time. This chapter states the selection problem for the cheapest adequate set of observations, shows that it is a covering problem, and places the other costs next to it with an exact result for updating.

22.1 Selecting observations

Definition 22.1 (Representation selection)**.** An instance consists of a finite set X, a policy $\pi : X \to A$, a finite family $\mathbb{F}$ of candidate observations $f : X \to B_f$ with costs $c(f) > 0$. A subfamily $F \subseteq \mathbb{F}$ is *sufficient* if the joint observation $(f)_{f \in F}$ is a sufficient representation for π, that is, $\bigcap_{f \in F} \ker f \subseteq \ker \pi$. The problem is to find a sufficient subfamily of minimum total cost.

Let $D(\pi) = \{\{x, x'\} : \pi(x) \neq \pi(x')\}$ be the set of *disagreement pairs* of π. An observation f *covers* the pair $\{x, x'\}$ if $f(x) \neq f(x')$.

Established result 22.2 (Minimum test set and greedy covering)**.** *The minimum test set problem, finding a smallest set of binary tests that separates every pair of elements of a finite set, is NP-complete. The greedy algorithm for weighted set cover, which repeatedly picks the set of least cost per newly covered element, returns a cover of cost at most $H(d)$ times the optimum, where d is the largest size of a set and $H(d) = 1 + \frac{1}{2} + \cdots + \frac{1}{d}$.*

Source: Garey and Johnson, *Computers and Intractability*, problem SP6 (minimum test set); Chvátal, "A greedy heuristic for the set-covering problem," *Mathematics of Operations Research* 4 (1979).

New result 22.3 (Sufficient observation sets are covers of the disagreement pairs)**.** *(a) F is sufficient for π if and only if every pair in $D(\pi)$ is covered by some $f \in F$. Hence representation selection is exactly weighted set cover with universe $D(\pi)$.*
(b) The greedy algorithm returns a sufficient subfamily of cost at most $H(|D(\pi)|)$ times the optimum, and $H(|D(\pi)|) \le 1 + 2\ln|X|$.
(c) The problem is NP-hard, even with unit costs, binary observations, and π injective.
(d) If every function $X \to \{0,1\}$ is available at unit cost, the optimum is $\lceil \log_2 |\pi(X)| \rceil$.

Assumptions: X finite. Parts (b) and (c) use the cited results.

Proof. (a) F is insufficient exactly when some pair $\{x, x'\}$ lies in $\bigcap \ker f$ and not in $\ker \pi$, that is, when some disagreement pair is covered by no member of F. (b) Apply the cited bound with $d \le |D(\pi)| \le n(n-1)/2$ and $H(m) \le 1 + \ln m$. (c) With π injective, $D(\pi)$ is the set of all pairs and a sufficient family is exactly a set of tests separating all pairs, the minimum test set problem. (d) A set of m binary observations takes at most 2^m joint values, so $2^m \ge |\pi(X)|$ is necessary, and $m = \lceil \log_2 |\pi(X)| \rceil$ bits encoding the value of π suffice. □

New result 22.4 (Checking is cheap, choosing is hard)**.** *Given X, π, and a subfamily F, whether F is sufficient can be decided in time $O(|X|\,|F|)$ by hashing the joint observation of each case and testing that π is constant on each hash class.*

Assumptions: Each observation can be evaluated in constant time.

Proof. Compute the tuple $(f(x))_{f \in F}$ for each x and group cases by it; F is sufficient if and only if π is constant on each group, by Result 5.2. □

The asymmetry between Results 22.3 and 22.4 is the one that makes verification (Chapter 10) attractive: a proposed set of observations can be certified quickly even when finding the cheapest one is intractable.

22.2 The cost of being updatable

The cost of updating enters through the maintainable representations of Chapter 7. Recall the property $P_n(h) = \mathbf{1}[c(h) \ge n]$ of Result 7.5, where $c(h)$ counts the alarms since the last reset in the monitor of Example 1.4.

New result 22.5 (The updatable storage frontier for a threshold property)**.** *Every maintainable representation that is sufficient for P_n takes at least $n+1$ values, and the capped count $\min(c, n)$ is one with exactly $n+1$. Any representation sufficient for P_n with fewer than $n+1$ values, in particular the two-valued representation by P_n itself, is not maintainable: the value after the next event is not a function of the value before it and the event.*

Assumptions: Events $\{\text{tick}, \text{alarm}, \text{reset}\}$ as in Example 1.4.

Proof. This is Result 7.5(b): $s_{\text{one}}(P_n) = 2$ and $s_{\text{inc}}(P_n) = n+1$. The capped count is maintainable and $n+1$-valued. A maintainable sufficient representation with fewer than $n+1$ values would contradict $s_{\text{inc}}(P_n) = n+1$. □

The result prices updatability exactly: for this property, from two values to $n+1$. Combined with Result 12.4 the counter also supports retrieval at depth one, since the value is a single stored cell. The three costs (values, update, retrieval) for P_n thus occupy a single point if the counter is chosen, and no design is better on all of them.

Remark 22.6. Result 22.3 treats storage in the number of observations; Result 22.5 treats it in the number of values. The two are related by $\log_2$ of the number of values being a lower bound on the number of binary observations. A complete cost model for Volume 2 will have to fix one convention; the results of this chapter hold in the convention stated in each.

22.3 The price of a restricted vocabulary

Result 22.3(d) shows that when every binary observation is available the optimum is logarithmic in the number of decisions. The available observations in practice are a fixed vocabulary, and the distance between the two optima measures what the vocabulary costs.

New result 22.7 (Parity over a threshold vocabulary)**.** *Let $X = \{1, \dots, n\}$ with $n \geq 2$, let $\pi(x) = x \bmod 2$, and let $\mathbb{F}$ be the threshold observations $\mathbf{1}[x \geq t]$, $t = 2, \dots, n$, at unit cost. Then $|D(\pi)| = \lceil n/2 \rceil \lfloor n/2 \rfloor$, every sufficient subfamily is all of $\mathbb{F}$, and the optimum is $n - 1$, whereas with all binary observations available it is* 1*.*

Assumptions: X finite and linearly ordered.

Proof. The disagreement pairs are the pairs consisting of one odd and one even element, of which there are $\lceil n/2 \rceil \lfloor n/2 \rfloor$. For each t the adjacent pair $\{t-1, t\}$ has opposite parities, and the only threshold observation that separates it is $\mathbf{1}[x \geq t]$. By Result 22.3(a) every threshold must be selected. The unrestricted optimum is $\lceil \log_2 2 \rceil = 1$, by the observation π itself. □

For $n = 8$ the pair set has 16 elements, the vocabulary has seven members, and all seven are needed, so the vocabulary multiplies the cost by seven. The same vocabulary is perfectly adequate for a threshold policy, where a single observation suffices. A vocabulary is therefore not good or bad in itself but adequate or inadequate relative to the policies it must serve, which is the viewpoint of the revision horizon in Chapter 21.

Exercises

Exercise 22.8. Let $X = \{1, \dots, 8\}$, π the parity of x, and $\mathbb{F}$ the threshold observations $\mathbf{1}[x \geq t]$ for $t = 2, \dots, 8$ at unit cost. Compute $D(\pi)$, find a minimum sufficient subfamily, and compare its size with $\lceil \log_2 |\pi(X)| \rceil$.

Exercise 22.9. Show that the greedy algorithm can return a cover that is a factor $\Omega(\log n)$ above the optimum for an explicit family of instances of representation selection, using the standard set cover example.

Exercise 22.10. Prove that if F_1 and F_2 are sufficient for π_1 and π_2 respectively then $F_1 \cup F_2$ is sufficient for (π_1, π_2), and that the minimum cost for the pair can be strictly below the sum of the minimum costs.

Exercise 22.11. For a monitor with m independent obligations (Result 14.6), compute the minimum number of binary observations needed to represent the monitor state, and compare it with m.

Exercise 22.12. Using Result 22.4, describe a procedure that, given a proposed representation and a revision, produces the set of affected cases of Result 21.4 in time linear in $|X|$.

Exercise 22.13. For the vocabulary of Result 22.7 and $n = 8$ find the minimum sufficient subfamily for the policy $\pi(x) = \mathbf{1}[x \bmod 4 \in \{0, 1\}]$, and show that the answer is the number of x with $\pi(x) \neq \pi(x+1)$.

Exercise 22.14. Generalize Result 22.7 to a policy on $\{1, \dots, n\}$ and the threshold vocabulary: prove that the minimum sufficient subfamily has size equal to the number of adjacent disagreements $|\{x : \pi(x) \neq \pi(x+1)\}|$.

Exercise 22.15. Give a vocabulary of $n - 1$ binary observations on n cases and a policy for which some strict subfamily is sufficient although no single observation covers more than one disagreement pair, or prove that no such instance exists.

Chapter 23

Worked Mathematical Systems

Question. What do complete constructions look like?

Established sources. None.

New results. Four constructions, an obligation monitor, a provenance-preserving document system, a degrading service, and a claim ledger, each with its formal assumptions, an implementation sketch, one proposition that instantiates earlier results, and failure cases.

Proof obligations. Each construction lists its own.

Depends on. Parts I to V.

The preceding chapters proved results about representations in isolation. This chapter assembles them. Each of the four systems below states what is assumed, what is retained, what is guaranteed, and how the guarantee fails when an assumption is withdrawn. The sketches are written at the level of state and transition, not of code, because the point is the relation between what is kept and what can be decided.

23.1 An obligation monitor

Assumptions. A service has two obligations over disjoint event components in the sense of Definition 14.5. Invoices must be paid within $k_1 = 3$ events of the billing component $\Sigma_1 = \{\mathrm{inv}, \mathrm{pay}, e_1\}$, and incidents must be reported within $k_2 = 2$ events of the incident component $\Sigma_2 = \{\mathrm{inc}, \mathrm{rep}, e_2\}$. Time for each obligation is counted in the events of its own component.

Construction. The retained state is the pair (ρ_1, ρ_2) with $\rho_1 \in \{0, 1, 2, 3\}$ and $\rho_2 \in \{0, 1, 2\}$, together with the absorbing flag $\bot$. A billing event updates ρ_1 by Definition 14.3; an incident event updates ρ_2; events of the other component leave the coordinate unchanged. An event is *admitted* by the gate if and only if it does not take the state to $\bot$: in a state with $\rho_1 = 1$ the only admitted billing event is pay.

New result 23.1 (Instantiation of the monitor theorem)**.** *The monitor has* $(3+1)(2+1)+1 = 13$ *states. It is a sound and complete gate for compliance. Every maintainable representation of the history with fewer than* 13 *values, used as a sound gate, refuses an event that is admissible at some history.*

Assumptions: As in Result 14.6.

Proof. The count is Result 14.6. Soundness and completeness of the gate hold because $\perp$ is reached exactly by violating histories. For the last statement let σ be maintainable with fewer than 13 values. By Result 14.7 two histories h, h' in distinct monitor states share a value of σ, and by the distinguishing argument in the proof of Result 14.6 there is a word after which exactly one of hw, $h'w$ is violating. Take such a word $w = ua$ of least length. By minimality hu and $h'u$ are both alive, and a is admissible at one and takes the other to $\perp$. Because $\ker \sigma$ is a right congruence, $\sigma(hu) = \sigma(h'u)$. A sound gate consulting σ alone must therefore refuse a at both histories (Result 1.7), so a is refused at a history where it is admissible. □

Failure cases. If the monitor keeps only whether an invoice is pending, it conflates $\rho_1 \in \{1, 2, 3\}$ and the argument above applies: it cannot say whether a payment event is still timely (the second exercise of Chapter 14). If one clock drives both obligations, the system leaves the fragment and the count is the open Research problem 14.9. If the monitor state is lost on restart and re-initialized to $(0, 0)$, a pending obligation is forgotten, and a gate built from the reinitialized state is unsound: it admits events that the true history forbids.

23.2 A provenance-preserving document system

Assumptions. A document has K sections and is revised by a finite set U of authors. An event (u, i, t) is author u replacing the text of section i by t. Events are totally ordered (a single writer or a serializing server). A hash function H is injective on the texts that occur.

Construction. The retained record is $(H(\tau(h)), \Lambda(h))$ where $\tau(h)$ is the current text, $H(\tau(h))$ its content address, and $\Lambda(h) = (\lambda_1, \ldots, \lambda_K)$ the vector of last authors. An event updates both components.

New result 23.2 (Adequacy of the document record)**.** *The record $(H \circ \tau, \Lambda)$ is maintainable and adequate for the family consisting of the content-integrity question "is the text t?" for each t and the questions $\lambda_1, \ldots, \lambda_K$. Neither component alone is adequate for the family if some revision (a, i, t) and (b, i, t) with $a \neq b$ yield the same text from the same lineage.*

Assumptions: As above, with H injective.

Proof. After an event (u, i, t) the new text is a function of the old text and the event, and the new vector is the old vector with coordinate i replaced by u, so each component, and hence the pair, is updated by a function of the current value and the event; its kernel is a right congruence. Content questions are functions of $H \circ \tau$ because H is injective, and λ_i is a coordinate of Λ. For the final statement, the two lineages obtained by the revisions have equal texts and different λ_i, so $H \circ \tau$ is not adequate (Result 11.6); and no function of Λ alone recovers the text, since lineages with equal Λ can hold different texts. □

Failure cases. If two replicas revise the same section concurrently, λ_i is undefined; by Result 19.2 it cannot be computed from either causal past alone, and the record must hold the set of maximal concurrent revisions. If the hash is not injective on occurring texts, content integrity can be falsely affirmed. If the audit family gains the question "has more than one

author revised section i?", the record must be enlarged to the pairs (λ_i, S_i) of Result 11.5, with each section's factor rising from $|U|+1$ to $1+|U|2^{|U|-1}$.

23.3 A degrading service

Assumptions. The capabilities and dependencies are those of the checkout service: $C = \{\text{db}, \text{cache}, \text{pay}, \text{rec}, \text{store}, \text{checkout}\}$ with primitives db, cache, pay, and rules $(\text{store}, \{\text{db}\})$, $(\text{store}, \{\text{cache}\})$, $(\text{rec}, \{\text{db}\})$, $(\text{checkout}, \{\text{pay}, \text{store}\})$. The only required capability is $\text{Req} = \{\text{checkout}\}$.

New result 23.3 (Admissible shedding in the checkout service)**.** *The minimal supports are* $S_1 = \{\text{checkout}, \text{pay}, \text{store}, \text{db}\}$ *and* $S_2 = \{\text{checkout}, \text{pay}, \text{store}, \text{cache}\}$. *The maximal admissible shed sets are* $\{\text{cache}, \text{rec}\}$ *and* $\{\text{db}, \text{rec}\}$. *The shed set* $\{\text{db}, \text{cache}\}$ *has the same size as* $\{\text{db}, \text{rec}\}$ *and is not admissible, so the count of surviving capabilities does not decide admissibility.*

Assumptions: Finite Horn dependency system as above.

Proof. A support must derive checkout, hence contain pay and store, and store needs db or cache; rec is in no support. By Result 17.3(b) the maximal admissible sets are the complements of minimal supports, namely $\{\text{cache}, \text{rec}\}$ and $\{\text{db}, \text{rec}\}$. Shedding $\{\text{db}, \text{cache}\}$ meets both supports, while $\{\text{db}, \text{rec}\}$ meets only S_1; the two sets have the same size. □

The requirement here is $\text{pay} \wedge (\text{db} \vee \text{cache})$, which is a linear threshold function of the primitives (weights 2 for pay, 1 for each of db and cache, threshold 3), so a weighted test exists for this service. The counterexample to additive tests of Result 17.4 needs two disjoint alternative pairs; the service shows only that a count of surviving capabilities is not a test.

Failure cases. A health metric that reports "three of three primitives up" hides that pay is required and rec is not. A dashboard that reports the fraction of capabilities alive treats $\{\text{db}, \text{rec}\}$ and $\{\text{db}, \text{cache}\}$ alike. If a new rule makes store also depend on a logging capability, the supports change and every earlier admissibility judgement must be recomputed; the check takes time linear in the size of the system (Result 17.3).

23.4 A claim ledger

Assumptions. A ledger records claims, each supported by evidence items and other claims. It is a dependency system in the sense of Definition 17.1: an evidence item e has the primitive rule $(e, \varnothing)$, and a claim c has rules (c, P) with P a set of evidence items and claims. Retracting evidence e sheds e. Each claim c also carries a challenge component $\Sigma_c = \{\text{ch}_c, \text{ans}_c, e_c\}$ with the obligation that a challenge be answered within k_c events of its component.

New result 23.4 (Ledger semantics)**.** *(a) After retracting a set D of evidence, the supported claims are* $\text{Prov}(D) \cap \text{Claims}$. *Mutual support does not support: if c and c' have only the rules $(c, \{c'\})$ and $(c', \{c\})$ then neither is supported.*

(b) A retraction set D leaves the required claims Req *supported if and only if D is disjoint from a support of* Req.

(c) The challenge obligations have a minimal compliance monitor with $\prod_c (k_c + 1) + 1$ *states.*

Assumptions: Finite ledger; challenge components disjoint.

Proof. (a) $\mathrm{Prov}(D)$ is the least fixed point, and the set $\{c, c'\}$ with those rules is closed but not derived from the empty set, so it is not in the least fixed point. (b) is Result 17.3(b). (c) is Result 14.6. □

Failure cases. If two evidence items are indistinguishable in role, so that each supports the same claims and nothing else, the ledger holds two records for what the identity criterion of Chapter 15 treats as one; the minimal representation identifies them. If support is read as a greatest fixed point instead of a least one, circular justifications are accepted, which is the characteristic failure of ledgers that allow claims to cite one another as their only basis. If retraction is permitted without the check of part (b), a claim that was required for an obligation can silently lose its support, which is the loss of verification relative to retained evidence described in Chapter 10.

23.5 Counting what the service can lose

The checkout service admits an exact count of its admissible losses, which shows how little of the space of failures an availability fraction can describe. Shed sets here range over subsets of the four capabilities $\mathrm{db}, \mathrm{cache}, \mathrm{pay}, \mathrm{rec}$ that are not derived from others; the derived capabilities store and checkout are lost exactly when their rules fail.

New result 23.5 (Admissible losses in the checkout service)**.** *Of the sixteen subsets of* $\{\mathrm{db}, \mathrm{cache}, \mathrm{pay}, \mathrm{rec}\}$*, exactly six are admissible shed sets: those that omit* pay *and do not contain both* db *and* cache*, with* rec *included or not. The two maximal ones are* $\{\mathrm{cache}, \mathrm{rec}\}$ *and* $\{\mathrm{db}, \mathrm{rec}\}$*, and every admissible shed set is contained in one of them.*

Assumptions: The dependency system and requirement of Result 23.3.

Proof. Shedding D is admissible if and only if checkout remains derivable from the primitives outside D. This requires $\mathrm{pay} \notin D$ and at least one of $\mathrm{db}, \mathrm{cache}$ outside D. The intersection of D with $\{\mathrm{db}, \mathrm{cache}\}$ is then $\varnothing$, $\{\mathrm{db}\}$ or $\{\mathrm{cache}\}$, and rec lies in no support, so it is in D or not freely, giving $3 \cdot 2 = 6$. The sets with one primitive and rec shed are maximal and contain all others. □

A rule that tolerates the loss of any two of the four capabilities admits eleven subsets, five of which are not admissible, namely $\{\mathrm{pay}\}$, $\{\mathrm{db}, \mathrm{cache}\}$, $\{\mathrm{db}, \mathrm{pay}\}$, $\{\mathrm{cache}, \mathrm{pay}\}$ and $\{\mathrm{pay}, \mathrm{rec}\}$. The size rule therefore admits losses that break checkout, and it is not repaired by shrinking the bound: the rule "at most one" refuses the admissible set $\{\mathrm{db}, \mathrm{rec}\}$, so no bound on size alone separates the six admissible sets from the rest.

Exercises

Exercise 23.6. Recompute the count of Result 23.5 after adding the rule $(\mathrm{rec}, \{\mathrm{cache}\})$, and state whether the maximal admissible shed sets change.

Exercise 23.7. Give a dependency system on three primitives and a requirement for which the size rule “shed at most one” is exactly the admissible family, and one for which it is neither sound nor complete.

Exercise 23.8. For the monitor of the first section, compute the number of reachable states if the billing and incident components share a single event e that advances both clocks, and explain why the product count of 13 no longer applies.

Chapter 24

Open Problems in Distinction and Continuation

Question. Which precise questions remain unresolved?

Established sources. None.

New results. None.

Proof obligations. Approximate revision sufficiency, compositional obligation semantics, minimal provenance, and memory allocation under changing policy families, each stated precisely, together with the problems left open in earlier chapters.

Depends on. Everything.

None of the statements below is a theorem or a conjecture with a supporting argument. Each is a question whose answer the preceding chapters leave undetermined, stated so that a result, positive or negative, would be a definite contribution. For each the known partial results are recorded, with their locations, so that the open part is delimited.

Research problem 24.1 (Approximate revision sufficiency)**.** For a finite class $\mathcal{R}$ of binary revisions on a finite set X and the uniform distribution, let $k^*(\mathcal{R}, \varepsilon)$ be the least number of values of a representation σ that is $(\varepsilon, \mathcal{R}, \mu)$-revision sufficient. Result 21.6 gives $k^*(\mathcal{R}_2, \varepsilon) \geq n(1-2\varepsilon)$ for the class of all binary revisions and $k^*(\mathcal{R}_{\leq}, \varepsilon) \leq \lceil 1/(2\varepsilon) \rceil$ up to rounding for thresholds on a line; Result 21.7 makes the threshold value exact, $k^*(\mathcal{R}_{\leq}, \varepsilon) = \lceil n/(2\lfloor \varepsilon n \rfloor + 1) \rceil$ among interval partitions. Determine $k^*(\mathcal{R}, \varepsilon)$, or tight bounds on it, in terms of a combinatorial parameter of $\mathcal{R}$ such as its VC dimension, and decide whether the dependence on ε is $1/\varepsilon$ in general for classes of bounded VC dimension.

Research problem 24.2 (Compositional obligation semantics)**.** Result 14.6 gives the minimal monitor for conjunctions of bounded-response obligations on disjoint alphabets. Research problem 14.9 asks for the shared-clock case; Result 14.11 settles the minimal size for two obligations in terms of the reachable live states, and the case of three or more obligations, where states merge, remains open. More generally, determine the minimal monitor size for a conjunction $\Phi_1 \wedge \Phi_2$ in terms of the minimal monitors of Φ_1 and Φ_2 and their shared alphabet. In particular, characterize the pairs for which it equals the product bound of Result 8.5, and the pairs for which it is strictly smaller.

Research problem 24.3 (Minimal provenance for a query class)**.** Result 11.5 gives the minimal record for last-author audits and for last-author with author sets. For a class $\mathcal{Q}$ of audit queries defined by positive relational algebra over a lineage relation, determine the minimal maintainable record adequate for $\mathcal{Q}$ in terms of the structure of the queries, and compare it with the polynomial annotation of Result 11.3, which is adequate but in general not minimal.

Research problem 24.4 (Memory allocation under changing policy families)**.** A system holds a representation of at most k values, and at each time t a family Φ_t of policies is announced to be required next. Changing the representation costs the number of cases whose representation changes. The offline problem is to choose representations σ_t with $\Phi_t \subseteq \mathrm{Pol}(\sigma_t)$, whenever $|X/\kappa(\Phi_t)| \le k$, minimizing total change cost; the online problem has Φ_t revealed at time t. Determine the competitive ratio of online algorithms for this problem, and decide whether the offline problem is polynomial or NP-hard in the convention of Result 22.3.

Research problem 24.5 (Sharp bounds for policy families)**.** Research problem 6.9 asks for the growth of the joint kernel when one admissible revision is added to a family induced by the states of a monitor, in terms of the transition structure of the monitor.

Research problem 24.6 (Diagnosing joint obstructions)**.** Result 13.3 shows that the pair of emptiness bits for authorization and obligation is insufficient to decide admissibility when the two restrictions act on overlapping realization sets. Determine the minimal representation of the pair of restrictions that decides admissibility of a proposed continuation, as a function of the set system of realizations, and characterize the set systems for which a logarithmic number of bits suffices.

Research problem 24.7 (Synthesis of congruent identity criteria for compensation)**.** Result 18.5 shows that aborted sagas are restored exactly when the identity criterion is a congruence for the compensations. Given steps and compensations, determine the coarsest identity criterion that is a congruence for all of them and under which each per-step restoration holds, and decide the complexity of computing it for finite systems.

Research problem 24.8 (Heterogeneous quotas)**.** Result 19.3 gives the quota condition for a non-negative replicated balance with symmetric replicas. Determine the optimal allocation of quotas, and the rebalancing messages required to change them, when replicas have unequal rates of demand and unequal communication costs, and characterize the tradeoff between the number of rebalancing messages and the fraction of withdrawals refused.

Research problem 24.9 (Interfaces with obligations)**.** Result 20.4 treats compatibility of deterministic interfaces and Chapter 14 treats obligations of single components. Extend the interface theory so that the guarantee of a component includes bounded-response obligations on the actions it exchanges, and determine whether refinement remains a precongruence for composition.

Appendix A

Proof Techniques

The arguments of this volume fall into a small number of patterns. This appendix states each pattern as a result, proves it once, and shows the use that the chapters make of it. A reader who is comfortable with induction and fixed points can skip to the fooling-set method of Section A.3, which is the least familiar.

A.1 Induction on histories

Every history is a finite word, so statements about all histories are proved by induction on length. The shape is always the same: a claim about ε, and a step from a claim about h to claims about ha for the events a with $ha \in \mathcal{H}$.

Established result A.1 (Induction on prefix-closed sets)**.** *Let $\mathcal{H}$ be a history set and Q a predicate on $\mathcal{H}$. If $Q(\varepsilon)$ holds and $Q(h)$ implies $Q(ha)$ for all h, a with $ha \in \mathcal{H}$, then $Q(h)$ holds for every $h \in \mathcal{H}$.*

Source: Induction on word length.

Proof. Let $h \in \mathcal{H}$ have length n. Its prefix of length 0 is ε and each prefix of length $i+1$ is a prefix of length i followed by one event, and all prefixes lie in $\mathcal{H}$ by prefix-closure. Applying the hypothesis n times gives $Q(h)$. □

Example A.2 (An invariant of the account)**.** An account starts at balance 0; events are deposits of $d \geq 1$ units and withdrawals of $w \geq 1$ units, and a withdrawal that exceeds the balance is forbidden. Let $Q(h)$ be "the balance after h equals the total deposited minus the total withdrawn, and is nonnegative". $Q(\varepsilon)$ is clear. If $Q(h)$ holds and ha is allowed, then for a deposit the new balance is the old one plus d, and for a withdrawal the new balance is the old one minus w, nonnegative because the withdrawal was allowed. Hence Q holds on every history, which is the statement that the balance is a function of the history through a sum, the observation behind the quotient of Chapter 2.

A.2 Fixed points

Kernels, closures and reachable sets are fixed points of monotone operators. On a finite set the fixed points are computed by iteration, and the characterization as least or greatest is what

makes them useful in proofs.

Established result A.3 (Knaster–Tarski, finite case)**.** *Let X be finite and $\Phi : 2^X \to 2^X$ monotone ($A \subseteq B$ implies $\Phi(A) \subseteq \Phi(B)$). Then the iteration $\varnothing \subseteq \Phi(\varnothing) \subseteq \Phi^2(\varnothing) \subseteq \cdots$ stabilizes after at most $|X|$ steps at the least fixed point of Φ, and $X \supseteq \Phi(X) \supseteq \Phi^2(X) \supseteq \cdots$ stabilizes at the greatest fixed point. The least fixed point is the intersection of all sets A with $\Phi(A) \subseteq A$, and the greatest is the union of all sets A with $A \subseteq \Phi(A)$.*

Source: Tarski (1955); Knaster (1928).

Proof. By monotonicity and $\varnothing \subseteq \Phi(\varnothing)$, induction gives $\Phi^k(\varnothing) \subseteq \Phi^{k+1}(\varnothing)$. An increasing chain of subsets of an $|X|$-element set is strictly increasing at most $|X|$ times, so it stabilizes at some L with $\Phi(L) = L$. If $\Phi(A) \subseteq A$ then $\varnothing \subseteq A$ gives $\Phi^k(\varnothing) \subseteq A$ for all k by induction, so $L \subseteq A$; since L itself satisfies $\Phi(L) \subseteq L$ it is the intersection of such sets, and in particular the least fixed point. The dual statements follow by the same argument from X downward, with $A \subseteq \Phi(A)$ giving $A \subseteq \Phi^k(X)$. □

Example A.4 (Reading a kernel off the characterization)**.** The viability kernel (Appendix F) is the greatest set $S \subseteq C$ with $S \subseteq \Gamma(S)$, where $\Gamma(S)$ is the set of states with an action into S. To show that a particular set S_0 lies inside the kernel it suffices to show $S_0 \subseteq C$ and $S_0 \subseteq \Gamma(S_0)$: the greatest-fixed-point characterization turns a membership claim into a one-step check. To show a state is outside the kernel, exhibit its removal in the descending iteration.

A.3 Counting and the pigeonhole principle

Lower bounds on the size of a representation follow from exhibiting many histories that no two can share a representation.

New result A.5 (Fooling sets)**.** *Let $P : \mathcal{H} \to B$ be a property of histories and $r : \mathcal{H} \to R$ a representation sufficient for P in the sense that $P(hw)$ depends on h only through $r(h)$ for all continuations w. Call $h_1, \ldots, h_k \in \mathcal{H}$* pairwise separated *if for each $i \neq j$ there is a continuation w_{ij} with $h_i w_{ij}, h_j w_{ij} \in \mathcal{H}$ and $P(h_i w_{ij}) \neq P(h_j w_{ij})$. Then $|R| \geq k$. In particular the number of values of any sufficient representation is at least the size of the largest pairwise separated set, and, by the Myhill–Nerode theorem (Chapter 4), this largest size equals the number of continuation classes.*

Assumptions: None beyond the definitions.

Proof. If $r(h_i) = r(h_j)$ for $i \neq j$ then $P(h_i w) = P(h_j w)$ for the common continuation $w = w_{ij}$ by sufficiency, contradicting separation. So r is injective on $\{h_1, \ldots, h_k\}$ and $|R| \geq k$. For the last statement, continuation classes are by definition pairwise separated when taken one representative each, and conversely a separated set has distinct classes. □

Example A.6 (Counting to n)**.** Let events be a single symbol a and $P(h) = 1$ iff $|h| \equiv 0 \pmod{n}$. The histories $a^0, a^1, \ldots, a^{n-1}$ are pairwise separated: for $i < j$ the continuation a^{n-j} gives $P = 0$ for a^{i+n-j} (since $0 < n - j + i < n$) and $P = 1$ for a^n. Hence every sufficient representation has at least n values, and the counter modulo n achieves it.

A.4 Adversary arguments

A lower bound for an online procedure is often shown by letting an adversary choose each answer after seeing the procedure's questions.

New result A.7 (Adversary for membership queries)**.** *Let a procedure learn which element of a set $\{1, \dots, m\}$ is the hidden one by asking whether the element lies in a chosen subset. Every deterministic procedure needs at least $\lceil \log_2 m \rceil$ queries in the worst case.*

Assumptions: Deterministic procedure; yes/no answers.

Proof. The adversary answers each query so as to keep the larger of the two candidate sets. After q queries at least $m/2^q$ candidates remain, so the procedure cannot stop with a unique candidate before $2^q \geq m$. □

The pattern recurs in the verification chapters: an auditor who queries a record set receives an adversarially chosen answer subject to consistency, and the number of queries needed is the size of the information that separates the candidates.

A.5 Reductions

A hardness claim is proved by transforming instances of a known hard problem into instances of the new one in polynomial time, so that answers correspond.

Established result A.8 (Template for NP-hardness by reduction)**.** *To prove a decision problem Π NP-hard, choose an NP-hard problem Π_0 (SET COVER, VERTEX COVER, 3-SAT) and give a polynomial-time map f from instances I of Π_0 to instances of Π with: I is a yes-instance iff $f(I)$ is. To prove membership in NP give a polynomial-size certificate and a polynomial-time verifier.*

Source: Cook (1971); Karp (1972).

Example A.9 (Reduction used in Chapter 17)**.** Given a set-cover instance with universe $\{1, \dots, q\}$, sets $s_1, \dots, s_p$ and bound k, build a dependency system in which a required capability depends on one capability per universe element, each of which is supported by any set containing it, with load one per set. A self-supporting set of load at most k exists iff a cover of size at most k exists. The map is linear in the size of the instance and preserves answers, which is the whole proof apart from the check of the two directions.

A.6 Exchange arguments

New result A.10 (Exchange for sequencing)**.** *Jobs $1, \dots, n$ have processing times $p_i > 0$ and weights $w_i > 0$, and are processed one at a time with no idle time. The order minimizing $\sum_i w_i C_i$, where C_i is the completion time of job i, sorts jobs by nonincreasing ratio w_i / p_i.*

Assumptions: Single machine, no precedence constraints.

Proof. Suppose in an optimal order two adjacent jobs i then j have $w_i/p_i < w_j/p_j$, that is $w_i p_j < w_j p_i$. Let t be the time at which the pair starts. Swapping them changes the contribution of the pair from $w_i(t+p_i) + w_j(t+p_i+p_j)$ to $w_j(t+p_j) + w_i(t+p_j+p_i)$, a change of $w_i p_j - w_j p_i < 0$, and the other jobs are unaffected. So the swap improves the order, contradicting optimality. Hence an optimal order has no adjacent pair out of ratio order, and a sequence with no adjacent inversion is sorted. □

A.7 Constructing counterexamples

To show that a hypothesis cannot be dropped, build the smallest object that satisfies all other hypotheses and violates the conclusion, and then verify each hypothesis by hand. The examples of Chapters 3 and 14 follow this discipline: two states, two events, a single transition that distinguishes. The test of a counterexample is that deleting any one component restores the theorem.

Exercises

Exercise A.11. Prove by induction on histories that in the account of Example A.2 the balance is at most the total deposited.

Exercise A.12. Compute, by the iteration of Result A.3, the least fixed point of $\Phi(A) = \{1\} \cup \{x+1 : x \in A,\ x < 5\}$ on $\{1, \ldots, 6\}$.

Exercise A.13. Using Result A.5 show that recognizing "the number of a's is at least n" (with a single event a) needs $n+1$ values in any sufficient representation.

Exercise A.14. Show that the adversary argument of Result A.7 is tight by giving a procedure with $\lceil \log_2 m \rceil$ queries.

Exercise A.15. Give a direct exchange argument that sorting by processing time (shortest first) minimizes total completion time when all weights are equal, and state which step of the proof of Result A.10 it specializes.

Appendix B

Category-Theoretic Language

The main text uses the vocabulary of categories sparingly, for three purposes: to say that a construction is universal, to describe quotients and products of representations, and to organize the order structure of representations. This appendix defines the terms and proves the statements used. No result here goes beyond finite sets and relations.

B.1 Categories, functors, and the categories used here

A *category* has objects, morphisms between them, composition that is associative, and identities. A *preorder* is a category with at most one morphism between any two objects, and a *poset* is a preorder that is antisymmetric. The categories used in this volume are:

- **Set**, with functions;
- the poset $\mathrm{Part}(X)$ of equivalence relations on a set X ordered by inclusion (finer $\subseteq$ coarser);
- $\mathbf{Rep}(X)$, whose objects are surjections $r : X \to R$ and whose morphisms $r \to r'$ are functions $g : R \to R'$ with $r' = g \circ r$.

New result B.1 (Representations are partitions)**.** *There is at most one morphism between two objects of* $\mathbf{Rep}(X)$*, and a morphism* $r \to r'$ *exists iff* $\ker r \subseteq \ker r'$*. Two objects are isomorphic iff they have the same kernel. Hence* $\mathbf{Rep}(X)$ *is equivalent to the poset* $\mathrm{Part}(X)$*.*

Assumptions: None.

Proof. If g, g' are morphisms $r \to r'$, then $g(r(x)) = r'(x) = g'(r(x))$ for all x, and r is surjective, so $g = g'$. Existence is the factorization result (Result 2.5). Isomorphic objects have kernels contained in each other, hence equal; conversely equal kernels give mutually inverse factorizations. The assignment $r \mapsto \ker r$ is therefore an equivalence of categories onto $\mathrm{Part}(X)$. □

In this sense "a representation" and "a partition of the cases" are the same notion, which is the identification made in Chapter 2.

B.2 Universal constructions

Quotients as coequalizers. Let $R \subseteq X \times X$ be a relation with projections $p_1, p_2 : R \to X$. The *coequalizer* of p_1, p_2 is a function $q : X \to Q$ with $q \circ p_1 = q \circ p_2$ that is universal: every $f : X \to B$ with $f \circ p_1 = f \circ p_2$ factors uniquely through q.

New result B.2 (Quotient by a relation)**.** *In* **Set** *the coequalizer of $p_1, p_2 : R \to X$ is the quotient map $q : X \to X/{\sim_R}$, where $\sim_R$ is the equivalence relation generated by R (reflexive, symmetric, transitive closure).*

Assumptions: None.

Proof. For $(x, y) \in R$ we have $q(x) = q(y)$, so $q \circ p_1 = q \circ p_2$. If f satisfies $f \circ p_1 = f \circ p_2$ then $f(x) = f(y)$ for $(x, y) \in R$, and the set $\{(x, y) : f(x) = f(y)\}$ is an equivalence relation containing R, hence containing $\sim_R$. So f is constant on classes of $\sim_R$ and factors through q, uniquely since q is surjective. □

Joint representations as products. Given representations $r_1 : X \to R_1$ and $r_2 : X \to R_2$, the pairing $(r_1, r_2) : X \to R_1 \times R_2$ has kernel $\ker r_1 \cap \ker r_2$.

New result B.3 (Meet and join of representations)**.** *In* $\mathrm{Part}(X)$*, ordered by inclusion of kernels (finer below coarser), the pair $\{\ker r_1, \ker r_2\}$ has a greatest lower bound $\ker r_1 \cap \ker r_2$ (the joint representation) and a least upper bound $\ker r_1 \vee \ker r_2$, the transitive closure of $\ker r_1 \cup \ker r_2$ (the finest common coarsening). The joint representation is the categorical product in* $\mathbf{Rep}(X)$ *of r_1 and r_2: it maps to each of them, and any representation mapping to both factors through it. The finest common coarsening is the coproduct.*

Assumptions: None.

Proof. The intersection of two equivalence relations is an equivalence relation, and it is the largest relation contained in both, hence the greatest lower bound. By Result B.1 a representation s maps to r_i iff $\ker s \subseteq \ker r_i$, so s maps to both iff $\ker s \subseteq \ker r_1 \cap \ker r_2$, iff s factors through the pairing. For the join, the transitive closure of the union is the smallest equivalence relation containing both, hence the least upper bound, and the universal property of the coproduct follows dually: r_i maps to s iff $\ker r_i \subseteq \ker s$, and both do iff $\ker r_1 \cup \ker r_2 \subseteq \ker s$ iff $\ker r_1 \vee \ker r_2 \subseteq \ker s$. □

The product is what Chapter 3 calls the joint representation for a family of questions; the coproduct is what the same chapter calls the common coarsening.

B.3 Galois connections

A *Galois connection* between posets P and Q is a pair of order-reversing maps $F : P \to Q$, $G : Q \to P$ with $p \leq G(F(p))$ and $q \leq F(G(q))$. The composites GF and FG are closure operators.

New result B.4 (Questions and representations)**.** *Let $\mathcal{F}$ be a set of functions on X (the questions), and let $\kappa(\Phi) = \bigcap_{f \in \Phi} \ker f$ for $\Phi \subseteq \mathcal{F}$. For an equivalence relation θ on X let $\mathrm{Suff}(\theta) = \{f \in \mathcal{F} : \theta \subseteq \ker f\}$, the questions answerable from a representation with kernel θ.*

Then $\Phi \mapsto \kappa(\Phi)$ and $\theta \mapsto \mathrm{Suff}(\theta)$ form a Galois connection between $(2^{\mathcal{F}}, \subseteq)$ and $(\mathrm{Part}(X), \subseteq)$ (both order-reversing in the sense that larger sets of questions give finer kernels and finer kernels answer more questions), and $\Phi \subseteq \mathrm{Suff}(\kappa(\Phi))$. The closure $\mathrm{Suff}(\kappa(\Phi))$ is the set of all questions answerable by the least representation that answers Φ.

Assumptions: None.

Proof. If $\Phi \subseteq \Psi$ then $\kappa(\Psi) \subseteq \kappa(\Phi)$, and if $\theta \subseteq \theta'$ then $\mathrm{Suff}(\theta') \subseteq \mathrm{Suff}(\theta)$; so both maps reverse inclusion. For $f \in \Phi$, $\kappa(\Phi) \subseteq \ker f$, hence $f \in \mathrm{Suff}(\kappa(\Phi))$. For an equivalence relation θ, every $f \in \mathrm{Suff}(\theta)$ has $\theta \subseteq \ker f$, so $\theta \subseteq \kappa(\mathrm{Suff}(\theta))$. These are the two closure inequalities. By Result B.1, $f \in \mathrm{Suff}(\kappa(\Phi))$ iff f factors through the representation with kernel $\kappa(\Phi)$. $\square$

A consequence is that two families of questions are equivalent for the purposes of representation if they have the same closure, and that a representation designed for Φ automatically answers every other question in the closure at no extra cost.

B.4 Functors and the history construction

A *functor* $F : \mathcal{C} \to \mathcal{D}$ assigns to objects and morphisms of $\mathcal{C}$ objects and morphisms of $\mathcal{D}$ preserving identities and composition. The assignment $\mathcal{H} \mapsto \{\mathrm{Ext}(h) : h \in \mathcal{H}\}$ is not a functor on the nose; what is used in the text is the weaker statement that a simulation (a function between state sets commuting with transitions) induces a map of generated history sets. The reader who wants the full categorical story of minimal automata as terminal coalgebras will find it in Rutten, *Universal coalgebra: a theory of systems*, and in Jacobs, *Introduction to Coalgebra*; it is not needed here.

Exercises

Exercise B.5. Describe $\mathbf{Rep}(X)$ for $|X| = 3$ as a poset and draw its Hasse diagram. How many objects up to isomorphism?

Exercise B.6. Compute the coequalizer in **Set** of the relation $R = \{(1,2),(3,4),(2,3)\}$ on $\{1,2,3,4,5\}$.

Exercise B.7. For $X = \{0,\dots,11\}$, $r_1(x) = x \bmod 3$, $r_2(x) = x \bmod 4$, compute the kernel of the joint representation and of the finest common coarsening. How many classes does each have?

Exercise B.8. Prove that in a Galois connection $F \circ G \circ F = F$.

Exercise B.9. Show that for a family of questions $\Phi \subseteq \mathcal{F}$ the closure $\mathrm{Suff}(\kappa(\Phi))$ contains every question of $\mathcal{F}$ that is computable from the answers to the questions in Φ, and give an example of a question in the closure that is not itself in Φ.

Appendix C

Automata and Temporal Logic

This appendix collects the automata theory and the fragment of temporal logic used in Chapters 4, 14, and 10.

C.1 Deterministic automata and minimization

A *deterministic finite automaton* (DFA) is $A = (Q, q_0, \mathsf{E}, \delta, F)$ with Q finite, $\delta : Q \times \mathsf{E} \to Q$ total, and accepting set $F \subseteq Q$. It accepts w if $\delta^*(q_0, w) \in F$. A state is *reachable* if some word leads to it. Two states p, q are *equivalent* ($p \equiv q$) if for every word w, $\delta^*(p, w) \in F$ iff $\delta^*(q, w) \in F$.

New result C.1 (Moore's algorithm)**.** *Define $\equiv_0$ by $p \equiv_0 q$ iff ($p \in F \Leftrightarrow q \in F$), and $p \equiv_{k+1} q$ iff $p \equiv_k q$ and $\delta(p, a) \equiv_k \delta(q, a)$ for all $a \in \mathsf{E}$. Then $\equiv_k$ is the relation "agree on all words of length $\leq k$"; the sequence is a decreasing chain of equivalence relations that stabilizes after at most $|Q| - 1$ steps at $\equiv$; and the quotient automaton $A/\equiv$ (restricted to reachable states) recognizes the same language and has the least number of states among DFAs for it.*

Assumptions: Finite automaton.

Proof. By induction on k: $p \equiv_k q$ iff the states agree on all words of length at most k (the case $k = 0$ is the definition, and the step splits a word aw into its first event and the rest). The relations decrease, so the number of classes is nondecreasing as k grows; if $\equiv_{k+1} = \equiv_k$ then $\equiv_{k+2} = \equiv_{k+1}$ because the definition of $\equiv_{k+2}$ uses only $\equiv_{k+1}$, so the chain is constant from then on. Each non-final step increases the number of classes by at least one, starting from at least one and ending at most $|Q|$, so it stabilizes within $|Q| - 1$ steps, and the limit is $\equiv$ since two states agreeing on all words of length $\leq k$ for every k are equivalent. Equivalent states have equivalent successors, so δ descends to the quotient, which accepts the same language. Minimality: the language determines the Nerode classes (Result 4.12), and any DFA for it has at least as many reachable states as classes by Result A.5; the quotient has exactly that many. □

The code in Appendix E implements this algorithm.

C.2 Moore machines and outputs

When the object of interest is a property with more than two values, the accepting set is replaced by an output map $\lambda : Q \to B$ and the initial partition by $\ker \lambda$. Everything above goes through unchanged, and the minimal machine has as many states as the property has continuation classes (Chapter 4).

C.3 Temporal logic on finite traces

Definition C.2 (LTL on finite traces)**.** Formulas over atomic propositions AP are built by $\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \mathsf{X}\varphi \mid \varphi \,\mathsf{U}\, \varphi$. On a nonempty finite trace $\tau = \tau_0 \cdots \tau_{n-1}$ with $\tau_i \subseteq \mathrm{AP}$ the satisfaction relation is: $\tau, i \models p$ iff $p \in \tau_i$; Boolean connectives as usual; $\tau, i \models \mathsf{X}\varphi$ iff $i + 1 < n$ and $\tau, i + 1 \models \varphi$; $\tau, i \models \varphi \,\mathsf{U}\, \psi$ iff there is j with $i \le j < n$, $\tau, j \models \psi$, and $\tau, l \models \varphi$ for all $i \le l < j$. Derived: $\mathsf{F}\varphi = \top \,\mathsf{U}\, \varphi$ (eventually), $\mathsf{G}\varphi = \neg\mathsf{F}\neg\varphi$ (always, through the end of the trace), and the bounded forms $\mathsf{F}^{\le k}\varphi = \varphi \vee \mathsf{X}\varphi \vee \cdots \vee \mathsf{X}^k\varphi$.

Established result C.3 (Translation to automata)**.** *Every finite-trace LTL formula φ defines a regular language of finite traces, recognized by a DFA with at most $2^{2^{O(|\varphi|)}}$ states; satisfiability and model checking against a finite transition system are PSPACE-complete.*

Source: De Giacomo and Vardi (2013), *Linear temporal logic and linear dynamic logic on finite traces.*

C.4 Safety, bounded response, and monitors

A property P of infinite behaviours is a *safety property* if every violating behaviour has a finite prefix none of whose extensions satisfies P (the violation is detected in finite time); it is a *liveness property* if every finite prefix has an extension satisfying it.

Established result C.4 (Alpern–Schneider decomposition)**.** *Every property of infinite behaviours is the intersection of a safety property and a liveness property.*

Source: Alpern and Schneider (1985).

The obligations of Chapter 14 are bounded-response properties: every request is followed by a response within k steps. Because the bound makes the violation detectable at a finite time, the property is a safety property, and it has a small monitor.

New result C.5 (Monitor for bounded response)**.** *Let $\mathsf{E} \supseteq \{\mathsf{req}, \mathsf{ack}\}$ and let the property be: after each* req*, the next* ack *occurs within k events, where an* ack *discharges all outstanding requests. The deterministic monitor with states $\{0, 1, \ldots, k\} \cup \{\bot\}$ has $k + 2$ states and is minimal. Here state 0 means no outstanding request, state $i \ge 1$ means the oldest outstanding request is i events old, $\bot$ is the violation sink,* ack *leads to 0, and any other event leads from $i \ge 1$ to $i + 1$ (to $\bot$ if $i = k$), and from 0 to 1 on* req *and to 0 otherwise. The monitor accepts a finite trace iff no violation has occurred.*

Assumptions: Single outstanding-request counter, as in Chapter 14.

Proof. Correctness is by induction on the trace: the state is the age of the oldest unanswered request, and a request of age k followed by a non-ack event is exactly a violation. For minimality, states $i < j$ in $\{0, \ldots, k\}$ are separated: if $i \ge 1$, the continuation $(\text{idle})^{k+1-j}$ leads j to

$\perp$ (age $k+1$) while it leads i to a state of age $i+k+1-j \le k$ and so not to $\perp$; if $i=0$, idling keeps 0 at 0 and sends $j \ge 1$ to $\perp$ after $k+1-j$ steps. State $\perp$ is separated from the others by the empty continuation under the accepting set $\{0,\dots,k\}$. So there are $k+2$ distinct classes, and Result C.1 shows the automaton is minimal. □

The runnable check in Appendix E confirms $5 = k+2$ states for $k=3$. For m independent obligations with bounds $k_1,\dots,k_m$ the product of the separate monitors has $\prod(k_i+2)$ states, but the violation states of the factors merge into one sink and the minimal monitor has $\prod(k_i+1)+1$; Chapter 14 proves this count and explains why it is the right measure of the cost of independence.

Exercises

Exercise C.6. Run Moore's algorithm by hand on the DFA over $\{a,b\}$ with states $\{1,2,3,4\}$, initial state 1, accepting $\{4\}$, and transitions $1 \xrightarrow{a} 2$, $1 \xrightarrow{b} 3$, $2 \xrightarrow{a} 4$, $2 \xrightarrow{b} 3$, $3 \xrightarrow{a} 2$, $3 \xrightarrow{b} 4$, and 4 looping on both. Report the partitions $\equiv_0, \equiv_1, \dots$.

Exercise C.7. Write the bounded-response property of Result C.5 with $k=2$ in the notation of Definition C.2.

Exercise C.8. Show that "a request is eventually answered" ($\mathsf{G}(\mathsf{req} \to \mathsf{F}\,\mathsf{ack})$) is not detectable by a finite monitor on infinite behaviours, by exhibiting for any finite prefix an extension that satisfies it.

Exercise C.9. Compute the number of states of the product of the two separate monitors for two bounded-response obligations with bounds 2 and 3 on disjoint events, and the number of states of the minimal monitor for their conjunction. Then compute the number of minimal states if both are triggered by the same request and answered by the same acknowledgement, and explain why it is the smaller of the two counts.

Exercise C.10. Prove that the complement of a regular language of finite words is regular by swapping accepting and non-accepting states of a complete DFA, and say what this means for violation monitors versus compliance monitors.

Appendix D

Information Theory

The text counts the values of a representation; this appendix relates the count to the entropy of the representation and to what it preserves about a target quantity. All random variables are discrete with finite support, and logarithms are to base 2.

D.1 Entropy and mutual information

For a random variable X with law p, the *entropy* is $H(X) = -\sum_x p(x) \log p(x)$, and for two variables $H(X, Y)$ is the entropy of the pair, $H(Y \mid X) = H(X, Y) - H(X)$ the conditional entropy, and $I(X; Y) = H(Y) - H(Y \mid X)$ the mutual information.

Established result D.1 (Basic properties)**.** *(a)* $0 \le H(X) \le \log |\mathrm{supp} X|$*, with equality on the right iff* X *is uniform on its support. (b)* $H(Y \mid X) = 0$ *iff* Y *is almost surely a function of* X*. (c)* $I(X; Y) = I(Y; X) \ge 0$*, with equality iff* X, Y *are independent. (d) Chain rule:* $H(X, Y) = H(X) + H(Y \mid X)$ *and* $I(X, Z; Y) = I(Z; Y) + I(X; Y \mid Z)$*.*

Source: Cover and Thomas, *Elements of Information Theory*, Chapter 2.

D.2 Data processing and sufficiency

New result D.2 (Data processing and equality)**.** *For a function* r *and random variables* X, Y*:* $I(r(X); Y) \le I(X; Y)$*, with equality iff* $I(X; Y \mid r(X)) = 0$*, that is, iff* Y *and* X *are conditionally independent given* $r(X)$*.*

Assumptions: Finite supports.

Proof. Since $r(X)$ is a function of X, $I(X; Y) = I(X, r(X); Y)$. By the chain rule, $I(X, r(X); Y) = I(r(X); Y) + I(X; Y \mid r(X))$. The second term is nonnegative, which gives the inequality, and it vanishes iff equality holds; it vanishes iff X and Y are conditionally independent given $r(X)$ by Result D.1(c) applied conditionally. □

In the statistical sense, r is sufficient for Y when Y and X are conditionally independent given $r(X)$; the result says this is exactly when r loses no information about Y.

New result D.3 (Deterministic sufficiency)**.** *If the target is a deterministic function $Y = f(X)$ then r is sufficient in the sense of Chapter 5 ($\ker r \subseteq \ker f$ on the support of X) iff $H(Y \mid r(X)) = 0$ iff $I(r(X); Y) = H(Y)$.*

Assumptions: $\Pr(X = x) > 0$ for all x in the support.

Proof. By Result D.1(b), $H(Y \mid r(X)) = 0$ iff Y is a function of $r(X)$ almost surely; with full support this holds iff f is constant on each fibre of r, that is, $\ker r \subseteq \ker f$. And $I(r(X); Y) = H(Y) - H(Y \mid r(X))$. □

D.3 Counting and entropy

New result D.4 (Bits of a representation)**.** *If a representation r takes m values then $H(r(X)) \leq \log m$ for every law of X, and the number of bits of any fixed-length code for $r(X)$ is at least $\lceil \log m \rceil$ for the worst-case law. For a variable-length prefix code, the expected length L satisfies $H(r(X)) \leq L < H(r(X)) + 1$ for the best code (Shannon).*

Assumptions: Finite support; the last statement is Shannon's source coding theorem.

Proof. The first inequality is Result D.1(a). A fixed-length binary code with ℓ bits has at most 2^ℓ codewords, so $2^\ell \geq m$. The last statement is Shannon's theorem for symbol codes (Cover and Thomas, Theorem 5.4.1). □

The count of values and the entropy therefore measure different things: the first is the worst case, the second is the average under a particular law of usage. A system whose events have a skewed law needs far fewer bits on average than its worst case, and the first volume's counts are conservative in this sense.

D.4 Fano's inequality and limits on recovery

Established result D.5 (Fano)**.** *If $\hat{Y} = g(Z)$ is an estimate of Y from Z, with error probability $P_e = \Pr(\hat{Y} \neq Y)$ and Y taking M values, then $H(Y \mid Z) \leq h(P_e) + P_e \log(M - 1)$, where h is the binary entropy.*

Source: Fano (1961); Cover and Thomas, Theorem 2.10.1.

New result D.6 (A discarded distinction cannot be recovered)**.** *Suppose $r(X)$ is a coarsening that identifies $M_0 \geq 2$ equally likely values of a target Y (that is, Y is uniform on M_0 values given $r(X)$). Then every estimator of Y from $r(X)$ has error probability at least $1 - 1/M_0$.*

Assumptions: Uniform conditional law.

Proof. The error probability of any estimator is at least $1 - \max_y \Pr(Y = y \mid r(X)) = 1 - 1/M_0$, since a guess from a uniform distribution on M_0 values is correct with probability at most $1/M_0$. (For $M_0 \geq 3$ Fano's inequality gives the weaker bound $P_e \geq (\log M_0 - 1)/\log(M_0 - 1)$, obtained from $\log M_0 = H(Y \mid r(X)) \leq h(P_e) + P_e \log(M_0 - 1)$ and $h \leq 1$; for $M_0 = 2$ it gives $h(P_e) \geq 1$, hence $P_e = \frac{1}{2}$, which agrees with the exact value.) □

This is the quantitative form of the corollary on non-recovery in Chapter 2: once the representation merges two cases, no computation can tell them apart better than chance.

Exercises

Exercise D.7. Compute the entropy of the uniform law on four values and of the law $(1/2, 1/4, 1/8, 1/8)$, and give an optimal prefix code for the second.

Exercise D.8. Let X be uniform on $\{0, \dots, 7\}$ and $r(x) = x \bmod 2$, $Y = \lfloor x/4 \rfloor$. Compute $I(r(X); Y)$ and $I(X; Y)$ and decide whether r is sufficient for Y.

Exercise D.9. Prove from Result D.2 that if $r' = g \circ r$ then $I(r'(X); Y) \le I(r(X); Y)$.

Exercise D.10. Find a law of X on $\{1, \dots, m\}$ for which a representation with m values has entropy as small as $\epsilon > 0$, and explain why this does not contradict the lower bound of Chapter 3 on the number of values.

Exercise D.11. In Result D.6 take $M_0 = 4$ and compute the Fano bound and the exact error probability. Which is larger?

Appendix E

Executable Examples

Three short programs reproduce computations of the text. Each is deterministic, uses only the Python standard library, and prints the numbers quoted below. They are meant to be read and modified, not used as software; the point of each is that a definition of the text becomes a few lines of code whose output can be compared with a hand calculation.

E.1 Continuation classes by partition refinement

The program `continuation.py` implements Moore's algorithm (Result C.1). It starts from the partition by the property and repeatedly separates states whose successors lie in different classes, until nothing changes. The first example is the bounded balance with an overdraft sink; the second is the family of two parities of Chapter 3.

Listing E.1: `continuation.py`

```
"""Continuation equivalence by bounded-depth refinement.

A deterministic transition system with a state property is given; two states
are k-equivalent if they agree on the property after every word of length <= k.
Iterating the one-step refinement computes the classes (Moore's algorithm).
"""
from itertools import product

def refine(states, events, delta, prop, rounds):
    cls = {s: prop(s) for s in states}              # round 0: the property itself
    for r in range(rounds):
        sig = {s: (cls[s],) + tuple(cls[delta(s, a)] for a in events) for s in states}
        ids = {}
        new = {}
        for s in states:
            new[s] = ids.setdefault(sig[s], len(ids))
        if len(set(new.values())) == len(set(cls.values())) and r > 0:
            return new, r + 1
        cls = new
    return cls, rounds

# Example 1: bounded counter with an overdraft sink.  State: balance 0..N or 'X'
N = 8
states = list(range(N + 1)) + ['X']
```

```
def delta(s, a):
    if s == 'X':
        return 'X'
    if a == 'dep':
        return min(s + 1, N)
    return s - 1 if s > 0 else 'X'                # withdrawing from 0 overdraws
prop = lambda s: s != 'X' and s >= 2              # "at least two units on hand"
cls, rounds = refine(states, ['dep', 'wd'], delta, prop, 20)
print("balances 0..%d plus overdrawn: %d states" % (N, len(states)))
print("classes after refinement: %d (stabilised in %d rounds)" % (len(set(cls.values())),
    rounds))
groups = {}
for s in states:
    groups.setdefault(cls[s], []).append(s)
for g in groups.values():
    print("  class:", g)

# Example 2: the parity family.  Two properties, one per event, four joint classes.
ps = [(i, j) for i in (0, 1) for j in (0, 1)]
def d2(s, a):
    i, j = s
    return ((i + 1) % 2, j) if a == 'a' else (i, (j + 1) % 2)
for name, pr in [("parity of a", lambda s: s[0]), ("parity of b", lambda s: s[1])]:
    c, r = refine(ps, ['a', 'b'], d2, pr, 10)
    print(name, "-> classes:", len(set(c.values())))
c, r = refine(ps, ['a', 'b'], d2, lambda s: s, 10)
print("both properties jointly -> classes:", len(set(c.values())))
```

Running it prints

Listing E.2: Output of `continuation.py`

```
balances 0..8 plus overdrawn: 10 states
classes after refinement: 10 (stabilised in 7 rounds)
  class: [0]
  class: [1]
  class: [2]
  class: [3]
  class: [4]
  class: [5]
  class: [6]
  class: [7]
  class: [8]
  class: ['X']
parity of a -> classes: 2
parity of b -> classes: 2
both properties jointly -> classes: 4
```

Every balance is its own class, as in the quotient of the account (Chapter 2): withdrawing from a balance n leads to an overdraft after $n+1$ withdrawals, so balances differ in how many withdrawals they survive, and the property "at least two on hand" separates them. The sink is a class of its own. The two parities have two classes each but four jointly, which is the upper bound of the product in Result B.3 attained.

E.2 The viability kernel by the counter algorithm

The program `kernel.py` implements the linear-time algorithm of Result F.3 of Appendix F: counters of alive actions at each state, and a queue of states whose counter reaches zero. The system is the vehicle of the second volume, positions $0, \ldots, 10$ with a charger at 0 and a battery of 6 units.

Listing E.3: `kernel.py`

```
"""Linear-time viability kernel by the counter algorithm (Appendix C of the second volume).

System: a vehicle on positions 0..L with a charger at 0 and battery 0..B.
"""
from collections import deque

L, B = 10, 6
X = [(x, b) for x in range(L + 1) for b in range(B + 1)]

def actions(s):
    x, b = s
    out = []
    if x == 0:
        out.append(('wait', (0, B)))
    if b >= 1:
        if x > 0:
            out.append(('left', (x - 1, b - 1)))
            out.append(('stay', (x, b - 1)))
        if x < L:
            out.append(('right', (x + 1, b - 1)))
    return out

C = set(X)
alive = {}                      # (state, action) -> bool
count = {s: 0 for s in X}
preds = {s: [] for s in X}      # successor -> list of (state, action)
for s in X:
    for a, t in actions(s):
        alive[(s, a)] = t in C
        count[s] += alive[(s, a)]
        preds[t].append((s, a))
queue = deque(s for s in X if count[s] == 0)
dead = set(queue)
while queue:
    t = queue.popleft()
    for (s, a) in preds[t]:
        if alive[(s, a)]:
            alive[(s, a)] = False
            count[s] -= 1
            if count[s] == 0 and s not in dead:
                dead.add(s)
                queue.append(s)
kernel = set(X) - dead
print("states: %d, kernel: %d" % (len(X), len(kernel)))
print("kernel equals {b >= x}:", kernel == {(x, b) for (x, b) in X if b >= x})
reach, frontier = {(0, B)}, [(0, B)]
while frontier:
    s = frontier.pop()
```

```
    for a, t in actions(s):
        if t in kernel and t not in reach:
            reach.add(t); frontier.append(t)
print("positions␣reachable␣inside␣the␣kernel␣from␣(0,%d):" % B, sorted({x for x, b in reach}))
```

Listing E.4: Output of `kernel.py`

```
states: 77, kernel: 28
kernel equals {b >= x}: True
positions reachable inside the kernel from (0,6): [0, 1, 2, 3]
```

Of 77 states, 28 are viable, and they are exactly the states with battery at least the distance to the charger. From the full battery at the charger, only positions up to $3 = 6/2$ can be reached without leaving the kernel.

E.3 A bounded-response monitor

The program `monitor.py` builds the monitor of Result C.5 for $k = 3$ and minimizes it by the same refinement; the minimal automaton has $k + 2 = 5$ states, so the construction is already minimal.

Listing E.5: `monitor.py`

```
"""Bounded-response monitor: every request must be answered within K steps.

The monitor state is the age of the oldest unanswered request (0 = none).
We build the monitor as a deterministic automaton and minimise it by
partition refinement, checking the (K+2)-state count of the main text, one state for each age
    of the oldest request from 0 to K and one violation sink.
"""
K = 3
Ev = ['req', 'ack', 'idle']
states = list(range(K + 1)) + ['viol']

def delta(s, a):
    if s == 'viol':
        return 'viol'
    if a == 'ack':
        return 0                                    # an acknowledgement clears the oldest
    request
    age = (1 if a == 'req' else 0) if s == 0 else s + 1
    return 'viol' if age > K else age

accept = lambda s: s != 'viol'
cls = {s: accept(s) for s in states}
while True:
    sig = {s: (cls[s],) + tuple(cls[delta(s, a)] for a in Ev) for s in states}
    ids = {}
    new = {s: ids.setdefault(sig[s], len(ids)) for s in states}
    if len(set(new.values())) == len(set(cls.values())):
        cls = new
        break
    cls = new
print("monitor␣states␣(including␣the␣violation␣sink):", len(states))
print("minimal␣automaton␣states:", len(set(cls.values())))
```

```
def run(word):
    s = 0
    for a in word:
        s = delta(s, a)
    return s
for w in (['req', 'idle', 'idle', 'ack'], ['req', 'idle', 'idle', 'idle', 'ack'], ['req', 'req
    ', 'ack']):
    print('␣'.join(w), '->', 'violation' if run(w) == 'viol' else 'ok')
```

Listing E.6: Output of `monitor.py`

```
monitor states (including the violation sink): 5
minimal automaton states: 5
req idle idle ack -> ok
req idle idle idle ack -> violation
req req ack -> ok
```

The three test traces show the intended behaviour: an acknowledgement within three events is accepted, a fourth event without one is a violation, and a second request while one is outstanding does not reset the clock of the oldest.

Exercises

Exercise E.1. Change the property in `continuation.py` to "the balance is even" and predict the number of classes before running it.

Exercise E.2. Modify `kernel.py` so that staying put costs no battery away from the charger. Show by hand that every one of the 77 states is then in the kernel, and check that the program agrees. Then restore the cost of staying and instead let moving left cost no battery. Predict the kernel size, list the states that are lost relative to the original, and check the prediction (the answer is 67).

Exercise E.3. Extend `monitor.py` to two independent obligations with bounds 2 and 3 and verify that the minimal product monitor has $(2+1)(3+1)+1 = 13$ states, and explain why this is smaller than the $4 \cdot 5 = 20$ pairs of states of the two separate monitors of Result C.5: the two violation states merge into one sink, which removes the pairs in which either coordinate is a violation but one is not.

Exercise E.4. Write a program that computes the joint kernel of a family of functions on a finite set and reports the number of classes; test it on the parities of Result B.3.

Appendix F

Recap of Reachability and Viability for Readers of This Volume First

The second volume develops reachability and viability in full. This appendix gives the definitions and the results of that development that the present volume uses, with proofs, so that this volume can be read first. Statuses follow the main text.

F.1 Controlled systems and reachability

A *controlled system* consists of a finite set X of states, a finite set U of actions, and a transition relation $E \subseteq X \times U \times X$; write $x \xrightarrow{u} y$ for $(x, u, y) \in E$. A *trajectory* is a finite or infinite sequence $x_0 \xrightarrow{u_0} x_1 \xrightarrow{u_1} \cdots$. The system is *deterministic* if for each (x, u) there is at most one y. The set of states *reachable* from x is $\mathrm{Reach}(x)$, the least set containing x and closed under successors.

New result F.1 (Reachability is a preorder; components)**.** *The relation $x \to^* y$ ($y \in \mathrm{Reach}(x)$) is a preorder on X. Its symmetric part is an equivalence relation whose classes are the strongly connected components, and the preorder induces a partial order on the set of components (the condensation), which is acyclic. A state from which no component other than its own is reachable lies in a* terminal *component, and every state reaches a terminal component.*

Assumptions: Finite state set.

Proof. Reflexivity is the empty trajectory and transitivity is concatenation. The symmetric part is the mutual reachability relation, an equivalence relation. On components the induced relation is reflexive and transitive, and antisymmetric because mutual reachability of two components makes them the same. A longest chain of components must end, by finiteness, at a component with no outgoing edge to another; every state reaches one by following a chain upward. □

Components and the condensation are computed in time linear in the size of the system (Tarjan's algorithm, Appendix C gives the same quotient construction for automata). In a history system the components are the sets of states between which the system can move in

both directions; a state in a terminal component of size one is *absorbing*, and a history that reaches it can no longer change the future except by waiting.

F.2 Viability

Fix a *constraint set* $C \subseteq X$. A set $S \subseteq C$ is a *viability domain* if every $x \in S$ has an action u and a successor y with $x \xrightarrow{u} y$ in the deterministic case, and in the nondeterministic case (a disturbance chooses the successor) an action u all of whose successors lie in S and at least one exists.

New result F.2 (The viability kernel)**.** *The union of all viability domains contained in C is a viability domain, the* viability kernel $\mathrm{Viab}(C)$*; it is the greatest fixed point of $\Gamma_C(S) = \{x \in C : \exists u\ (\varnothing \neq \mathrm{succ}(x,u) \subseteq S)\}$, and is obtained by the iteration $S_0 = C$, $S_{k+1} = \Gamma_C(S_k)$, which stabilizes in at most $|C|$ steps. A state lies in the kernel iff there is a strategy that keeps the trajectory in C forever against any choice of successor. If $x \notin \mathrm{Viab}(C)$, every strategy is forced out of C within $k_x \leq |C|$ steps, where k_x is the step at which x leaves the iteration.*

Assumptions: Finite system.

Proof. A union of viability domains is a viability domain: a state in one of them has its action into that one, hence into the union. So the union K of all viability domains inside C is the greatest one. Γ_C is monotone, and $S \subseteq \Gamma_C(S)$ with $S \subseteq C$ exactly states that S is a viability domain, so by Result A.3 the greatest fixed point is K and the descending iteration reaches it in at most $|C|$ steps. If $x \in K$, the strategy "choose an action into K" keeps the trajectory in $K \subseteq C$ forever. If $x \notin K$ then x is removed at some stage k_x; by induction on k_x, from x every action has either no successor at all or a successor outside C (when $k_x = 1$), or a successor removed earlier (when $k_x > 1$), so the adversary can force the trajectory to stop or leave C within k_x steps. □

New result F.3 (A linear-time algorithm)**.** *Let $|E|$ be the number of transitions. The kernel is computed in time $O(|X| + |E|)$ by maintaining, for each state-action pair, a flag that records whether every successor lies in the current candidate set, and for each state the number of actions whose flag is true; a state is removed when its count reaches zero, and removing a state clears the flags of the pairs having it as a successor.*

Assumptions: Finite system.

Proof. The invariant is that a pair's flag is true iff all its successors are in the candidate set S and in C, and the count of a state equals the number of its true pairs. A state is removed exactly when it has no true pair, which is the condition $x \notin \Gamma_C(S)$, so the final set is a fixed point reached by removing only states outside the greatest fixed point (each removal is justified by the previous ones, as in the iteration). Each pair's flag is cleared at most once and each state is removed at most once, and each transition is examined when one of its endpoints is removed, giving the linear bound. □

Example F.4 (The account with a reserve)**.** States $\{0, 1, \ldots, N\}$ are balances and the constraint is balance $\geq r$ for a reserve $0 < r < N$. At each step the controller chooses to deposit one unit or not, and a disturbance withdraws one unit (the withdrawal is applied after the

deposit, and the balance is capped at N and floored at 0). With a deposit the balance changes by 0 from any $b < N$ and by -1 from $b = N$; without it, by -1. The iteration of Result F.2 starts from $[r, N]$. Every $b \in [r, N-1]$ has the action "deposit", whose successor is b itself, and N has the successor $N - 1 \geq r$. So no state is removed at the first step and $[r, N]$ is a fixed point: the kernel is $[r, N]$, and states below r are outside the constraint set to begin with.

F.3 Connection to histories and obligations

For a history system with representation r (Chapter 2), viability is a property of the representation: a history is viable iff the state it reaches is in the kernel of the transition system generated by the Nerode quotient (Result 4.17). The monitor states of an obligation (Appendix C) are appended to the physical state to obtain the product system in which obligations are constraints, and the kernel of the product is the set of *compliantly admissible* histories studied in Chapter 14.

F.4 Robust viability

When a disturbance chooses the successor among a set $D(x, u)$, the nondeterministic definition above gives the *robust* kernel: a state is robustly viable iff the controller has an action all of whose successors are viable. The two statements of Result F.2 and Result F.3 cover it verbatim, and it is monotone in the disturbance: enlarging D can only shrink the kernel. The scalar system $x^+ = ax + u$, $|u| \leq 1$, constraint $[-k, k]$, has kernel $[-c^*, c^*]$ with $c^* = k$ if $|a| \leq 1$ and $c^* = \min(k, 1/(|a| - 1))$ otherwise, as is checked by solving $|a|c \leq c + 1$; this is the standard example used throughout the second volume.

Exercises

Exercise F.5. Compute the condensation of the system on $\{1, \ldots, 6\}$ with transitions $1 \to 2$, $2 \to 3$, $3 \to 1$, $3 \to 4$, $4 \to 5$, $5 \to 4$, $5 \to 6$. Which components are terminal?

Exercise F.6. Run the iteration of Result F.2 on the same system with $C = \{1, 2, 3, 4, 5\}$, taking each state to have a single action whose successors are the states listed in the transitions, so that a disturbance chooses among them. What is the kernel, and what is the exit step of state 3? Then let the controller choose the successor instead and explain why the kernel changes.

Exercise F.7. Show that the kernel of the intersection of two constraint sets is contained in the intersection of their kernels, with equality failing in general. Give an example.

Exercise F.8. For $x^+ = 2x + u$, $|u| \leq 1$ and $C = [-3, 3]$, show that $x = 1.5$ is not in the kernel and compute the step at which it is forced out.

Exercise F.9. Verify the counts in Result F.3 by tracing the algorithm on the vehicle system of Appendix E for $B = 2$, $L = 3$.

Appendix G

Hints and Solutions to Selected Exercises

This appendix gives hints, and in several cases complete solutions, for a selection of exercises of the first eight chapters. Each numerical claim was checked by enumeration before it was written down. Exercises are cited by reference so that the numbering of the main text can change without affecting the appendix.

Chapter 1

Exercise 1.17. With $k = 3$ and $m = 1$ the pair $\kappa =$ (content, count), where the count is the number of distinct contents held, has the following values. When one content has been held the current content is c_0, which gives one value. When two have been held the current content is any of the three, since the history may have returned to c_0 or stopped on the second content, which gives three values; the same holds when all three have been held. The total is $1 + 3 + 3 = 7$, against 8 for the pair (τ, ν) of Result 1.9. The representation answers "has c_0 ever been held?" trivially, since the answer is always yes, and answers no question about c_1 or c_2 in general. For a separating pair take the history that edits to c_1 and back to c_0 and the history that edits to c_2 and back to c_0. Both have $\kappa = (c_0, 2)$, and they differ on whether c_1 has been held.

Chapter 2

Exercise 2.16. Let $x \equiv y$ mean that there is a chain $x = z_0, \dots, z_k = y$ with consecutive elements related by $\sim$ or by $\approx$. This relation is reflexive (take $k = 0$), symmetric (reverse the chain), and transitive (concatenate), and it contains both relations, so it contains their join. Any equivalence relation containing both must relate consecutive chain elements and hence, by transitivity, the endpoints, so the relation is contained in every upper bound and is the join. For the union, take $X = \{1, 2, 3\}$ with $\sim$ identifying $1, 2$ and $\approx$ identifying $2, 3$: the union relates 1 to 2 and 2 to 3 and does not relate 1 to 3.

Exercise 2.17. If r answers q then $q = g \circ r$ for some g, so $q(x)$ depends only on $r(x)$ and $\ker q$ has at most $|r(X)| = k$ classes. For the failure of the converse take $X = \{1, 2, 3\}$,

let r send $1, 2$ to a and 3 to b, and let q separate 2 from $\{1, 3\}$. Both have two classes, yet $r(1) = r(2)$ and $q(1) \neq q(2)$, so r does not answer q.

Exercise 2.21. The classes of $\ker(f, g)$ refine those of $\ker f$ and of $\ker g$, so there are at least a and at least b of them. Each class is determined by a pair of values, so there are at most ab. On the six cases $\{0, \ldots, 5\}$ let $f(i) = i \bmod 2$. With $g(i) = i \bmod 3$ the pair determines i by the Chinese remainder theorem, so the joint kernel has $6 = 2 \cdot 3$ classes, the maximum. With $g(i) = i \bmod 4$ the value of g determines that of f, so the joint kernel has $4 = \max(2, 4)$ classes, the minimum.

Chapter 3

Exercise 3.18. The elements of $\mathcal{R}_2$ on five cases are the partitions into two blocks, of which there are $S(5, 2) = 15$. Five have blocks of sizes 1 and 4 and ten have blocks of sizes 2 and 3. A question that separates one named case a from the other four has kernel $\{a\} \mid \text{rest}$, and a two-block partition answers it only if that kernel is coarser than the partition, which forces equality. Exactly one of the fifteen answers the question.

Exercise 3.19. Classify partitions of $\{1, \ldots, n\}$ into k blocks by the last element. Either it forms a block alone, leaving a partition of the first $n - 1$ elements into $k - 1$ blocks, or it joins one of the k blocks of a partition of the first $n - 1$ elements into k blocks. Hence $S(n, k) = kS(n - 1, k) + S(n - 1, k - 1)$. With $S(5, 3) = 25$ and $S(5, 2) = 15$ one gets $S(6, 3) = 3 \cdot 25 + 15 = 90$.

Chapter 4

Exercise 4.32. Suppose Π is live, so that $\mathrm{sat}_\Pi(u)$ holds for every finite word. Then every infinite word has all of its prefixes satisfying sat_Π, and so lies in Π_{cl}, which therefore equals E^ω. Conversely, suppose $\Pi_{\mathrm{cl}} = \mathsf{E}^\omega$ and let u be a finite word. Choose any infinite word σ with prefix u. It lies in Π_{cl}, so every prefix of it, in particular u, satisfies sat_Π. If Π is also safe then $\Pi = \Pi_{\mathrm{cl}}$ by Result 4.15, which is E^ω.

Chapter 5

Exercise 5.17. A representation whose classes are intervals and which has k values is determined by $k - 1$ cut points in $\{1, \ldots, n - 1\}$, a cut at c meaning that $c - 1$ and c lie in different classes. The window $w_{s,t}$ is constant on every class exactly when no class straddles s or t, that is, when both s and t are cut points. The number of served windows is therefore at most $\binom{k-1}{2}$, and it equals this when the cut points are chosen as any $k - 1$ values and every pair among them is used. For three values this gives one window. Without the interval restriction take $s_1 < s_2 < s_3$ in $\{1, \ldots, n - 1\}$ and the three classes $A = [s_1, s_2)$, $B = [s_2, s_3)$ and the rest. The representation serves w_{s_1,s_2}, w_{s_2,s_3} and w_{s_1,s_3}, the last being the indicator of $A \cup B$.

Chapter 6

Exercise 6.20. The eight listed thresholds separate every pair of the $6 \cdot 4 = 24$ cases, so the joint kernel is the identity relation, every policy is sufficient for the family's representation,

and the marginal cost of $[b + a \geq 4]$ is 0.

Exercise 6.22. The pair of residues modulo 2 and modulo 3 determines the residue modulo 6, so $X/\kappa(\Phi)$ has 6 classes. Adding x mod 4 gives the residue modulo 12, which is 12 classes, so the marginal cost is 6. Adding x mod 6 gives no new classes, so its marginal cost is 0. In general, if the moduli present have least common multiple L and $|X| \geq \operatorname{lcm}(L, p)$, the marginal cost of x mod p is $\operatorname{lcm}(L, p) - L$.

Chapter 7

Exercise 7.20. Two histories are continuation equivalent exactly when they have used the same number of the m permitted uses, so there are $m + 1$ classes. The question "is another use permitted?" has two answers, so its one-shot size is 2.

Exercise 7.21. Count the classes by the number u of uses. At $u = 0$ no clock is running, which gives one class. At each u with $1 \leq u \leq m-1$ the remaining wait takes w distinguishable values, which gives $(m - 1)w$ classes. At $u = m$ no further use is ever permitted, so the clock is irrelevant and there is one class. The total is $(m-1)w+2$, which is 5 for $w = 3$ and $m = 2$. The product $w(m + 1) = 9$ is larger because the clock carries no information when no use has occurred and when the cap is spent.

Chapter 8

Exercise 8.23. For the moduli 3 and 4, which are coprime, the pair has 12 classes, equal to the product. For the moduli 4 and 6 the pair has 12 classes, the least common multiple, against the product 24. The deficit arises because the reachable pairs of residues are constrained by $\gcd(4, 6) = 2$.

Chapter 9

Exercise 9.16. For $q = 0.8$, $B = 6$ and $j = 0$ the errors for $t = 1, \ldots, 5$ are 0.200, 0.360, 0.250, 0.147 and 0.066 (the minimum of $1 - q^s$ and $q^s - q^6$ with $s = t$). The worst threshold is $t = 2$. The continuous maximizer solves $q^s = (1+q^6)/2$, that is $s \approx 2.06$, with value $(1-q^6)/2 \approx 0.369$. It is not an integer because the equation is solved by a real number determined by q and B, while thresholds are integers; the integer maximizer is whichever neighbour of s gives the larger minimum, and here it is the lower one.

Chapter 10

Exercise 10.14. Enumerating the 31 words of length at most 4 gives the twelve values (b, flag) with $(-4, 1), (-3, 1), (-2, 1), (-1, 1), (0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1), (3, 0), (4, 0)$. Negative balances occur only with the flag set, since reaching a negative balance is overdrawing, and balances 3 and 4 occur only with the flag unset, since a history of length at most 4 with balance 3 or 4 has no room for a withdrawal that precedes it. This removes six of the eighteen pairs. The five values with flag 0, namely $b = 0, 1, 2, 3, 4$, lie in the interior of φ, because the evidence determines φ and each of these values is realized.

Chapter 11

Exercise 11.13. The sixteen lists are the empty list, three of length one, six of length two and six of length three. The pair (λ, S) identifies two lists exactly when they begin with the same author and have the same set of three authors, so the lists (a, b, c) and (a, c, b) share a value, and there are three such values, one for each leading author. The lists of length at most two have one preimage each. This gives $16 - 3 = 13$ values of (λ, S), in agreement with $1 + 3 \cdot 2^2 = 13$.

Exercise 11.14. The record (ϕ, λ) takes the value of the empty record and one value for each ordered pair of authors, in all $|U|^2 + 1$ values, and it is maintainable because a further revision leaves ϕ unchanged and replaces λ. The family consisting of ϕ_i and S_i is served by pairs (first author, set of authors) with the first author in the set, which has $1 + |U|2^{|U|-1}$ values, so the count $|U|^2 + 1$ does not hold; for $|U| = 3$ the two counts are 10 and 13.

Chapter 12

Exercise 12.12. Each pair of the three parities is a pair of independent unbiased bits, so a binary cell serves at most one of them. Three binary derived cells are therefore needed for depth one if the cells are binary. The three parities satisfy $(x_1 \oplus x_2) \oplus (x_2 \oplus x_3) = x_1 \oplus x_3$, so their joint kernel has four classes, and a single four-valued cell storing the pair $(x_1 \oplus x_2, x_2 \oplus x_3)$ serves all three at depth one. The packing bound counts $\lfloor \log_2 s \rfloor = 2$ questions per four-valued cell and would suggest two cells for three questions; the true requirement is one, because the proof of Result 12.6 uses the independence of the block parities to say that $|T|$ questions have $2^{|T|}$ joint classes, and here three questions have only 4.

Exercise 12.14. A write to an original cell changes the parity of the single block containing it, so it changes exactly one stored parity. In the packed index that parity lives in one derived cell, and in the unpacked index in one derived cell as well, so each write causes one derived-cell update in both. Packing groups parities that are written independently, which lowers the number of cells and does not lower the number of parities that a write must refresh.

Chapter 13

Exercise 13.13. There are $3^3 = 27$ ordered pairs of disjoint subsets of a three-element set. The pairs with $A = \varnothing$ number 8, those with $B = \varnothing$ number 8, and the pair with both empty is counted twice, so 15 pairs have an empty set. The remaining $27 - 15 = 12$ pairs have both sets non-empty and disjoint, and these are the joint obstructions; the formula gives $3^3 - 2^4 + 1 = 12$.

Chapter 14

Exercise 14.17. Simulating the shared-clock semantics for deadlines $(2, 2, 2)$ from the all-zero vector gives 13 reachable live vectors. They are the all-zero vector, the six vectors with one nonzero coordinate, and the six vectors with two nonzero coordinates that are different. The invariant is that the nonzero coordinates are pairwise distinct, because obligations opened at different events have been advanced by different numbers of events, and an event that opens or discharges one coordinate cannot make two pending coordinates equal. Three nonzero

coordinates would need three distinct values in $\{1, 2\}$, which is impossible. The 14 unreachable vectors of $\{0, 1, 2\}^3$ are exactly those with two equal nonzero coordinates.

Exercises 14.18 and 14.19. For deadlines $(2, 1)$ the doomed state is $(1, 1)$: every event either advances a coordinate with one step left to violation or discharges only one of the two, so some obligation is violated on the next event. It is reached by $a_1 a_2$, and any single event from it violates. A monitor for realized violation signals on the third event of $a_1 a_2 e$, and a monitor for inevitable violation signals after the second, one event earlier. For deadlines $(3, 2)$ the word a_1 gives $(3, 0)$, then a_2 gives $(2, 2)$ and e gives $(1, 1)$, which is doomed. The pairs $(r_1, 0)$ with $1 \le r_1 \le k_1$ are reached by a_1 followed by $k_1 - r_1$ copies of e, and the pairs $(0, r_2)$ symmetrically.

Chapter 15

Exercise 15.12. Computing continuation classes by Moore's algorithm for undo stacks of depth d gives 2^{d+1} classes for $k = 2$, namely $4, 8, 16$ for $d = 1, 2, 3$, and 3^{d+1} for $k = 3$, namely $9, 27, 81$. The general pattern is k^{d+1} classes: the current content and the d stored contents, with trivial positions identified with the empty slots as in the proof of Result 15.6.

Exercise 15.13. With redo and undo of depth one, the reachable configurations for $k = 2$ number 9 and the continuation classes number 6. Classes that differ only in whether an undone content can be restored by redo are separated by the word redo followed by a report, and classes that differ in the previous content are separated by undo followed by a report, as before.

Chapter 16

Exercise 16.13. For $c = \frac{1}{2}$ the law is $\Pr[N = n] = 2^{-n}$, which sums to 1, with $\mathbb{E}[N] = \sum n 2^{-n} = 2$ and expected surplus $\mathbb{E}[N - 1] = 1$. For a batch of 1000 units with $c = 0.001$, the probability of at least one surplus application in the apply-first order is $1 - (1 - c)^{1000} = 1 - 0.999^{1000} \approx 0.632$, and the probability of at least one lost unit in the record-first order is the same number, since each unit independently has probability c of each fault.

Exercise 16.15. Each delivery either crashes before the effect, with probability c_1, which changes nothing, or applies the effect and then crashes with probability c_2. Conditioned on reaching the application step, the delivery ends the process with probability $1 - c_2$, so $\Pr[N = n] = c_2^{n-1}(1 - c_2)$ whenever $c_1 < 1$. The crash before the effect only delays the process. Reliability effort belongs in the gap between applying and recording.

Chapter 17

Exercise 17.12. For $m = 2$ the primitives are $c_{11}, c_{12}, c_{21}, c_{22}$ and a shed set is admissible when some pair survives intact. The seven admissible sets are $\varnothing$, the four singletons, $\{c_{11}, c_{12}\}$ and $\{c_{21}, c_{22}\}$. The equal-weight test with threshold 3 admits the shed sets that leave at least three primitives, which are the five sets of size at most one. It rejects the two admissible two-element sets, and these are the two misclassified sets.

Exercise 17.13. A shed set is inadmissible if and only if it contains at least one primitive from each of the m alternatives, and for each alternative there are $2^r - 1$ subsets that do. So

$(2^r-1)^m$ shed sets are inadmissible and $2^{rm}-(2^r-1)^m$ are admissible. For $r=3$ and $m=3$ this is $512-343=169$, a fraction $169/512\approx 0.330$.

Exercise 17.14. A test that admits only the empty shed set is sound for every dependency system, since the empty set is admissible whenever the requirement is satisfiable. For $m=2$ the best sound additive test admits six of the seven admissible sets and rejects one. Give weight 1 to c_{11}, c_{12}, weight 2 to c_{21}, c_{22} and threshold 2 on the total shed weight: this admits $\varnothing$, the four singletons and $\{c_{11}, c_{12}\}$ and rejects every inadmissible set. No additive sound test admits both two-element admissible sets, since adding the two inequalities $w_{11}+w_{12}\le\tau$ and $w_{21}+w_{22}\le\tau$ contradicts the sum of $w_{11}+w_{21}>\tau$ and $w_{12}+w_{22}>\tau$.

Chapter 18

Exercise 18.8. Write $\beta=\beta(h)$. After $\mathrm{d}(x)\,\mathrm{w}(x)$ the balance is β again. The overdraft flag could only change if an intermediate prefix were below zero, but the intermediate balance is $\beta+x$, which is negative only if $\beta<-x<0$, and then h was already flagged. So the flag is unchanged and the pair is restored for every continuation. After $\mathrm{w}(x)\,\mathrm{d}(x)$ the balance is β again, so relative to $\{\beta\}$ the withdrawal is restored at every history. The flag becomes $q_{\mathrm{neg}}(h)\vee[\beta-x<0]$, so restoration relative to $\{\beta, q_{\mathrm{neg}}\}$ fails exactly at the histories with $q_{\mathrm{neg}}(h)=0$ and $x>\beta$, where the history had never been below zero and the withdrawal overdraws.

Chapter 19

Exercise 19.11. With $n=3$ and $B=60$ the equal split gives each replica 20, and the served amount is $\sum_r \min(d_r, 20)$. It is 50 on $(30,20,10)$, 20 on $(60,0,0)$ and 60 on $(20,20,20)$. The demands served least are those concentrated at a single replica, with served amount $B/n=20$.

Exercise 19.12. The served amount $\sum_r \min(d_r, B/n)$ is concave in d, so its minimum over the polytope $\{0\le d_r\le B/m,\ \sum_r d_r=B\}$ is at an extreme point, and for integer m the extreme points have m coordinates equal to B/m and the rest 0. Each of the m coordinates is served B/n since $B/m\ge B/n$, so the worst served fraction is m/n. Knowing that no replica carries more than a $1/m$ share improves the guarantee from $1/n$ to m/n.

Chapter 20

Exercise 20.17. A right reaches q along a chain only if every edge passes it, so the rights obtained along a chain are the intersection of the masks along it, and the obtainable set is the union over chains of these intersections together with the initial holdings. For the three masks $\{r,s\}$, $\{s,t\}$, $\{s\}$ the intersection is $\{s\}$.

Chapter 21

Exercise 21.13. For $n=6$ the thresholds are $\pi_1,\dots,\pi_7$. The exactly served thresholds are $t=1$, $t=7$, $t=3$ (between 2 and 3) and $t=6$ (between 5 and 6), which is four, equal to $k+1$. The remaining thresholds $t=2,4,5$ each split one interval into two nonconstant parts

and have error $1/6$, since the interval $\{1, 2\}$ or $\{3, 4, 5\}$ contributes the smaller of two masses, $1/6$ in each case. The worst error is $1/6 = \lfloor 3/2 \rfloor / 6$, as the formula of part (b) states.

Chapter 22

Exercise 22.13. For $x \in \{1, \dots, 8\}$ the policy takes the values $1, 0, 0, 1, 1, 0, 0, 1$. Adjacent disagreements occur between 1 and 2, 3 and 4, 5 and 6, and 7 and 8, which is four. Each adjacent pair $\{t - 1, t\}$ is separated only by the threshold observation at t, so thresholds $t = 2, 4, 6, 8$ must be selected, and these four suffice. The minimum sufficient subfamily has four members, equal to the number of x with $\pi(x) \neq \pi(x + 1)$.